Deep Learning: Making It Learnable

A course companion in native Python and PyTorch

Heman Shakeri

2026-07-13

Table of contents

Preface

> ⚠ Warning
>
> **Draft.** This book is being written in the open as the companion text for DS 6050 Deep Learning at UVA's School of Data Science. Chapters appear as they mature; everything here runs — every figure and result is produced by the code on the page.

Deep learning did not appear out of nowhere. Nearly every idea that powers modern AI is a classical idea — a line of best fit, a hand-designed image filter, a kernel-weighted average — that someone dared to ask one question about:

> *What if we made this* ***learnable****?*

That question is the spine of this book. We will ask it over and over:

- Take **linear regression**, let its features be learned, and you get the multilayer perceptron.
- Watch the MLP **fail to generalize** on images — see the failure in pictures — and the cure (respecting the structure of the data) becomes obvious: an **inductive bias**.
- Take the **fixed filters** of classical computer vision, make them learnable, and you get the convolutional network.
- Take **kernel regression** — a century-old way to average nearby evidence — make the similarity function learnable, and you get **attention**, then self-attention, then the Transformer.
- Then comes the **BERT moment**: stop training from scratch at all, and adapt what has already been learned.

Each part of the book replays this same move at a larger scale. By the end, the modern stack should feel less like a zoo of architectures and more like one idea, compounding.

How to read this book

Every chapter follows the course's learning loop: **read → run → check**. The code is native Python and PyTorch, written to run on an ordinary laptop CPU — small data, honest experiments. Watch the lecture videos, read the chapter, run the cells, and test yourself against the self-checks on the course site.

Acknowledgments

To the students of DS 6050, whose questions shaped every explanation here.

Part I.

I · From Lines to Networks

1. Linear Regression, the Mother Model

The core task of everything we will do in this course is to teach a computer to learn a function from examples. Before we can appreciate what is *deep* about deep learning, we need one complete, honest example of learning — a model, a loss, and a way to improve. Linear regression is that example. It is the smallest model that contains the entire supervised learning story, and by the end of this chapter we will have built it three times: once in closed form, once by gradient descent, and once the way you will build everything else in this book — with PyTorch modules. All three will agree, and you will know exactly why.

1.1. Learning from examples

We start with a dataset of examples,

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n},$$

where each $\mathbf{x}_i \in \mathbb{R}^d$ is an input with d features and y_i is the desired output — the *supervision signal*. Stack the inputs as rows and you get the data matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$: n samples down, d features across, with the targets collected in a vector $\mathbf{y}$.

Our goal is a flexible function f such that

$$f(\mathbf{x}_i) \approx y_i \quad \text{and, crucially, } f \text{ generalizes to unseen data.}$$

That second clause is the whole game. We do not care how well f recites the training examples; we care how it behaves on a new $\mathbf{x}$ it has never seen — one drawn from the same distribution as the training data.

Supervised learning, three flavors

The range of y decides the task and, as we will see, the natural loss:

Task	Range of y	Typical loss
Regression	$\mathbb{R}$	Mean squared error
Classification	$\{0, \dots, C-1\}$	Cross-entropy
Structured prediction	sequences, images, graphs	task-specific

This chapter is regression; Chapter 2 handles classification with the same machinery.

1.1.1. Why is this hard?

This task sounds simple, but it is fundamentally hard: for any finite set of examples, there are *infinitely many* functions that fit them perfectly. We want the one that is most suited to our problem → the learning algorithm needs a nudge in the right direction. That guiding assumption is called its **inductive bias**, and it is a thread we will pull on for the rest of the book.

- A **linear model** has a *strong* bias: it assumes the world is linear. Simple and stable, but systematically wrong when the data is not linear (high bias, low variance).
- A **deep neural network** has a *weak* bias. Its flexibility lets it learn complex patterns, but it can memorize the training data instead of the underlying rule (low bias, high variance). Some networks can memorize an entire training set — and then perform poorly the day you deploy them.

The key to modern deep learning is to use a flexible model and fight overfitting with data, regularization, and (the deepest idea of all) inductive biases matched to the task. That last idea gets its own chapter (Chapter 6).

To make any of this concrete, we parameterize a family of functions $f_{\mathbf{w}}$ by a set of tuning parameters $\mathbf{w}$ — think of each parameter as a knob. *Learning* is finding a good setting $\hat{\mathbf{w}}$ of the knobs; *inference* is using $f_{\hat{\mathbf{w}}}$ to predict on unseen data. That is the vocabulary we will use all course.

1.2. The linear model and its geometry

Linear regression assumes a linear relationship:

$$y = \mathbf{w}^\top \mathbf{x} + b.$$

Geometrically, $\mathbf{w}$ sets the orientation (the slope) of a hyperplane, and the bias b shifts it away from the origin. In two dimensions this is the familiar line through a scatter of points; in d dimensions it is a d-dimensional plane, but nothing about the algebra changes.

There is a second way to read $\mathbf{w}^\top \mathbf{x}$, and it may be the most important sentence in this chapter: **a dot product is a similarity score.** More precisely,

$$\mathbf{w}^\top \mathbf{x} = \|\mathbf{w}\|_2 \, \|\mathbf{x}\|_2 \cos\theta.$$

For normalized vectors it measures directional similarity directly: positive when $\mathbf{x}$ points along $\mathbf{w}$, zero when they are orthogonal, and negative when they oppose. Without normalization, the two norms also control the score, so a large dot product can mean strong alignment, large magnitude, or both. A linear model is therefore a learned *pattern scorer*: $\mathbf{w}$ is a template, and its direction and scale both matter. Keep this reading close at hand. In Part II, an image filter's response will be this same dot product slid across an image; in Part IV, *attention* will be built out of learned similarity scores between queries and keys. Both are this one primitive, asked the question we will ask all book long: *what if the template were learnable?*

1.2.1. The augmentation trick

Carrying b separately clutters the algebra, so we will often fold it into the weights: append a column of ones to the data matrix, making $\mathbf{X} \in \mathbb{R}^{n\times(d+1)}$, and append b to the weight vector, making $\mathbf{w} \in \mathbb{R}^{d+1}$. Then

$$y = \mathbf{w}^\top \mathbf{x} \qquad \text{(bias included)}.$$

Whenever *we use this augmented notation*, a missing b is living inside $\mathbf{w}$. Do not assume that every bias-free formula or software layer is augmented, however: a model can deliberately constrain $b = 0$ (for example, PyTorch layers created with `bias=False`).

1.3. The loss: what makes a hyperplane "bad"?

To find the best hyperplane we must first quantify error. The standard choice for regression is the **mean squared error (MSE)**:

$$\mathcal{L}(\mathbf{w}) \;=\; \frac{1}{n}\sum_{i=1}^{n}(y_i - \mathbf{w}^\top \mathbf{x}_i)^2 \;=\; \frac{1}{n}\,\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2\,. \tag{1.1}$$

The loss is the guiding principle of training: it is how we tell the model *you are a bad model, and here is by how much*; the model then has to find a way to improve. The choice of loss is not arbitrary, and we will justify this one honestly at the end of the chapter.

> The first "make it learnable" seed
>
> Look closely at $y = \mathbf{w}^\top \mathbf{x} + b$. A single neuron in a neural network computes *exactly this*, followed by a nonlinear squashing function $\sigma(\mathbf{w}^\top \mathbf{x} + b)$. Linear regression **is** a neuron with the identity activation. Everything we learn here — loss, gradients, optimization — transfers directly. The rest of this book is, in a precise sense, the story of what happens when we take this fixed, simple recipe and progressively let more of it be *learned*.

1.4. Finding the best weights, method 1: solve it exactly

For this one special problem we can find the exact minimizer. Set the gradient of Equation 1.1 to zero and you get the **normal equations**:

$$\hat{\mathbf{w}}_{\text{OLS}} = (\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top \mathbf{y}. \tag{1.2}$$

There is a picture behind this formula worth carrying for the rest of your career. Our predictions $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ can only ever live in the **column space** of $\mathbf{X}$ — the span of its feature columns. If the true $\mathbf{y}$ lies outside that space (it almost always does), the best we can do is **project** $\mathbf{y}$ onto

it. The leftover error $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$ is what our model cannot explain, and at the optimum it is *perpendicular* to the column space — no adjustment of the knobs can reduce it further.

One more observation to file away: substituting Equation 1.2 back in gives $\hat{\mathbf{y}} = \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$, so the fitted predictions are a *weighted combination of the training targets* $\mathbf{y}$. Linear regression predicts by mixing the answers it has already seen. Hold that thought: in Part IV an entire architecture — attention — grows out of choosing such mixing weights by *similarity*.

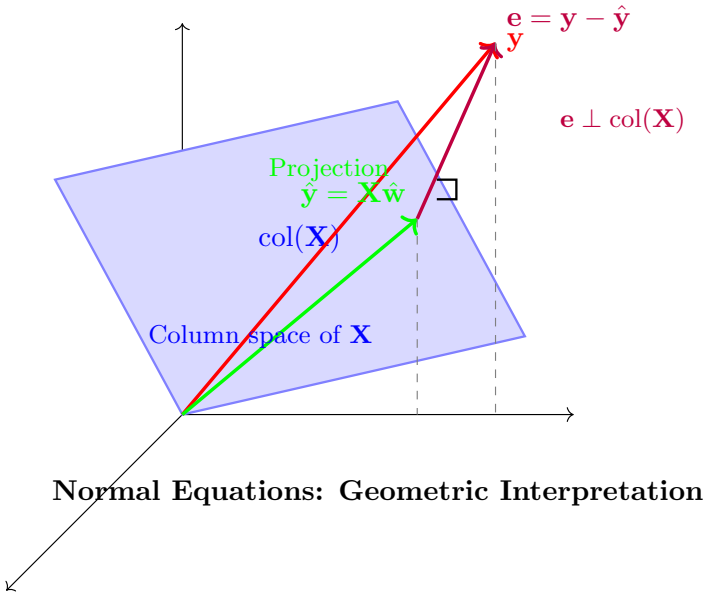

Figure 1.1.: The normal equations, geometrically: the optimal prediction $\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}$ is the projection of $\mathbf{y}$ onto the column space of $\mathbf{X}$; the residual $\mathbf{e}$ is orthogonal to it.

Elegant. So why is this formula almost absent from deep learning? Two reasons. Solving it costs $O(d^3)$, which is hopeless when d runs to millions or billions. And it only exists because the model is linear; the moment we add a nonlinearity, there is no closed form to solve. Our datasets are large and our models are not linear → we have neither luxury. We need another way.

1.5. Finding the best weights, method 2: walk downhill

Here is the geometric intuition. Picture the loss as a landscape over the parameter space, and imagine standing on it *blindfolded*, trying to find the lowest valley. You cannot see the valley, but you can feel the slope under your feet. So you feel the slope, take a small step downhill, and repeat.

The mathematical form of this hill-descending intuition is **gradient descent**:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta\,\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}^{(t)}), \tag{1.3}$$

where the *learning rate* η sets the step size. For the MSE loss the gradient has a clean closed form — with $\mathbf{X} \in \mathbb{R}^{n \times d}$, $\mathbf{w} \in \mathbb{R}^d$, and $\mathbf{y} \in \mathbb{R}^n$:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \frac{2}{n} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) \ \in \ \mathbb{R}^d. \tag{1.4}$$

```
import torch
import numpy as np
import matplotlib.pyplot as plt

torch.manual_seed(6050)
x1 = torch.randn(60)
y1 = 2.5 * x1 - 1.0 + 0.3 * torch.randn(60)          # truth: w = 2.5, b = -1.0

def mse(w, b):                                        # loss surface over (w, b)
    return ((y1 - (w * x1 + b)) ** 2).mean()

w, b, path = -0.5, 2.0, []                            # start far from the valley
for _ in range(20):
    path.append((w, b))
    err = (w * x1 + b) - y1                           # feel the slope...
    w -= 0.25 * float(2 * (err * x1).mean())          # ...step downhill
    b -= 0.25 * float(2 * err.mean())

W, B = np.meshgrid(np.linspace(-1.5, 5.5, 100), np.linspace(-4, 3, 100))
L = np.vectorize(lambda wi, bi: float(mse(wi, bi)))(W, B)

plt.figure(figsize=(5.8, 3.8))
plt.contour(W, B, L, levels=25, cmap="Blues", alpha=0.8)
pw, pb = zip(*path)
plt.plot(pw, pb, "o-", color="#E57200", ms=4, lw=1.5, label="descent path")
plt.plot(2.5, -1.0, "k*", ms=12, label="truth $(w, b)$")
plt.xlabel("$w$"); plt.ylabel("$b$"); plt.legend()
plt.tight_layout()
plt.show()
```

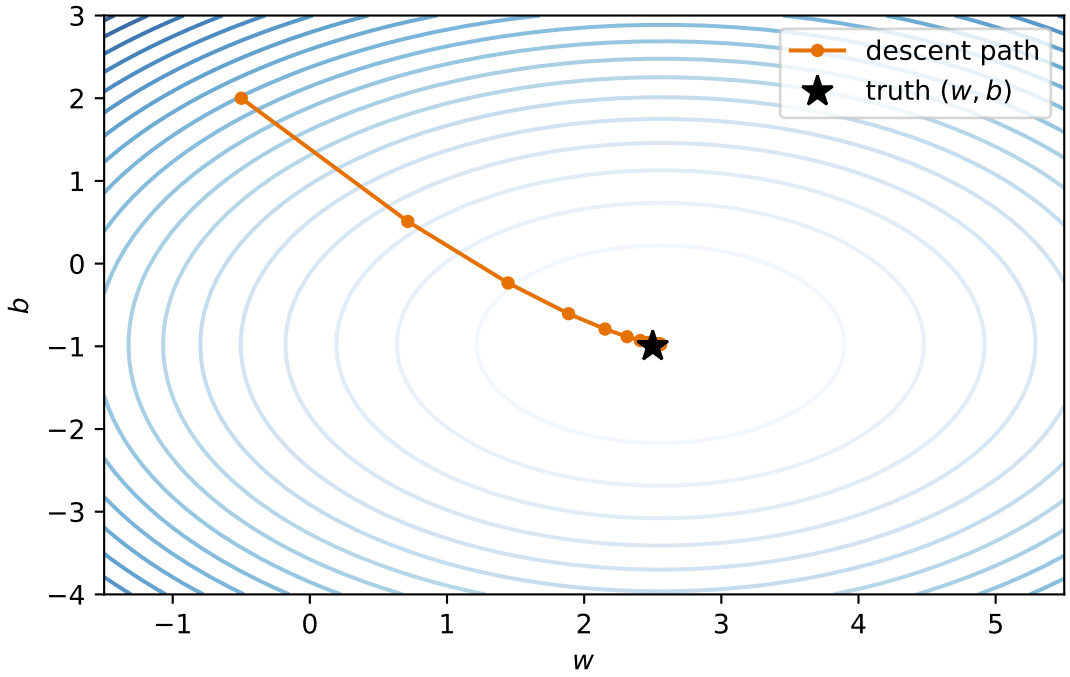

Figure 1.2.: The blindfolded walk, made visible: gradient descent on the MSE landscape of a tiny one-feature problem. Each step follows the slope felt at the current point; steps shrink as the valley floor flattens.

This iterative loop (predict → measure the loss → feel the gradient → step) is exactly what modern deep learning frameworks automate, at billion-parameter scale. When the dataset itself is huge we will not even use all of it per step: a small random *batch* gives a good-enough gradient. That refinement (stochastic gradient descent) matters enough to get its own treatment in Chapter 4.

1.6. Build it three times

Enough theory: let us implement linear regression, and let us do it in PyTorch. You have already built this in NumPy in earlier courses, so we will use it as an excuse to get comfortable with tensors, while still doing everything from scratch.

> **Coding hygiene**
>
> Two habits worth forming from the very first cell: **fix your seeds** so runs are reproducible, and **annotate your functions with types** — not critical for Python to run, but it makes code readable and debugging far easier.

```
import torch
import matplotlib.pyplot as plt
```

```
torch.manual_seed(6050)  # reproducible runs, always
```

```
<torch._C.Generator at 0x11053c7d0>
```

First, synthetic data. We control the ground truth (the true weights and bias), so we can check whether each method actually recovers them.

```
def make_synthetic_data(
    weights: torch.Tensor, bias: float, n_samples: int, noise: float = 0.1
) -> tuple[torch.Tensor, torch.Tensor]:
    """y = Xw + b + noise.  Returns X: (n, d) and y: (n,)."""
    X = torch.randn(n_samples, len(weights))
    y = X @ weights + bias + noise * torch.randn(n_samples)
    return X, y

true_w, true_b = torch.tensor([2.0, -3.4]), 4.2
X, y = make_synthetic_data(true_w, true_b, n_samples=200)
X.shape, y.shape        # always check your shapes
```

```
(torch.Size([200, 2]), torch.Size([200]))
```

1.6.1. Take 1: the closed form

The normal equations, Equation 1.2, on the *augmented* matrix (the column of ones carries the bias):

```
X_aug = torch.cat([X, torch.ones(X.shape[0], 1)], dim=1)   # (n, d+1)
w_ols = torch.linalg.solve(X_aug.T @ X_aug, X_aug.T @ y)   # solve, don't invert
print(f"closed form:      w = {w_ols[:2].numpy().round(3)},  b = {w_ols[2]:.3f}")
print(f"ground truth:     w = {true_w.numpy()},  b = {true_b}")
```

```
closed form:      w = [ 2.01  -3.402],  b = 4.196
ground truth:     w = [ 2.  -3.4],  b = 4.2
```

Two lines, and we recover the truth up to the noise floor. Remember what this cost us: $O(d^3)$, and the special grace of a linear model. Now the method that scales.

1.6.2. Take 2: gradient descent, from scratch

We write the model as a small class: a forward pass, manually derived gradients (Equation 1.4), and an update step. No autograd yet; we want to feel the mechanics once with our own hands.

```python
class LinearRegressionScratch:
    """Linear regression trained with manually derived gradients."""

    def __init__(self, input_size: int, lr: float = 0.1):
        self.w = 0.01 * torch.randn(input_size)
        self.b = torch.zeros(1)
        self.lr = lr

    def forward(self, X: torch.Tensor) -> torch.Tensor:
        return X @ self.w + self.b

    def step(self, X: torch.Tensor, y: torch.Tensor) -> float:
        err = self.forward(X) - y            # 1. predict, 2. measure   (n,)
        self.w -= self.lr * 2 * X.T @ err / len(y)   # 3. feel the slope,
        self.b -= self.lr * 2 * err.mean()           # 4. step downhill
        return float((err ** 2).mean())

model = LinearRegressionScratch(input_size=2)
losses = [model.step(X, y) for _ in range(100)]
print(f"gradient descent: w = {model.w.numpy().round(3)},  b = {model.b.item():.3f}")
```

```
gradient descent: w = [ 2.01  -3.402],  b = 4.196
```

```python
plt.figure(figsize=(5.5, 3.2))
plt.plot(losses)
plt.xlabel("step")
plt.ylabel("MSE loss")
plt.yscale("log")
plt.tight_layout()
plt.show()
```

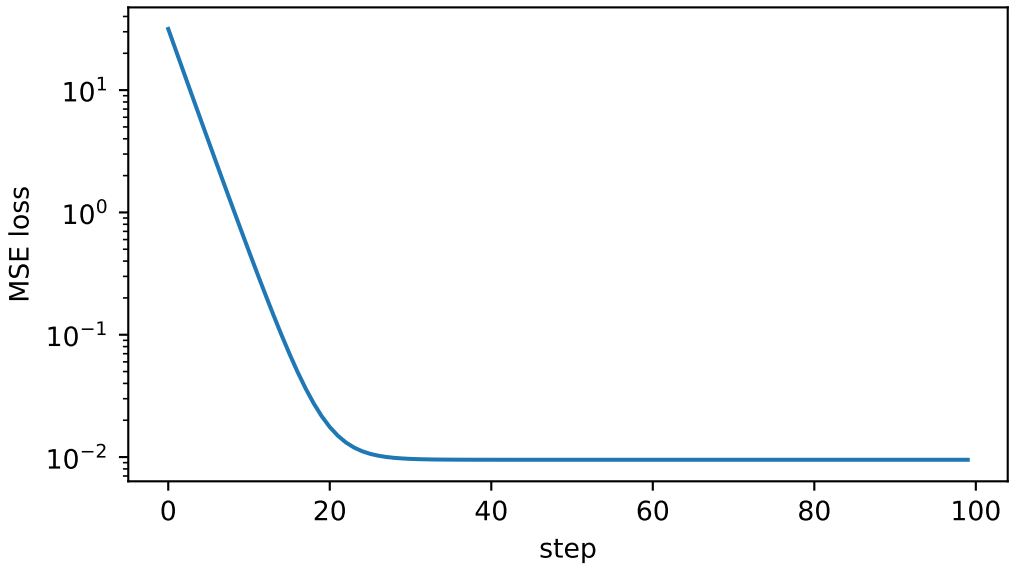

Figure 1.3.: The loss falls fast at first — steep slope underfoot — then flattens as we approach the valley floor set by the irreducible noise.

1.6.3. Take 3: the framework way

Finally, the idiom you will use for every model from here on: `nn.Linear` holds the knobs, autograd feels the slope for us, and the optimizer takes the step.

> **i** A deliberate framework preview
>
> This cell shows the complete PyTorch training loop so you can recognize its shape. We use the three framework calls as black boxes here: Chapter 3 introduces modules, Chapter 4 opens the optimizer, and Chapter 5 explains what `backward()` computes. Until then, the manual implementation above is the chapter's working toolbox.

```
from torch import nn

net = nn.Linear(in_features=2, out_features=1)
optimizer = torch.optim.SGD(net.parameters(), lr=0.1)
loss_fn = nn.MSELoss()

for _ in range(100):
    optimizer.zero_grad()                # clear old gradients: they accumulate!
    loss = loss_fn(net(X).squeeze(-1), y)   # (n, 1) -> (n)
    loss.backward()                      # autograd feels the slope for us
    optimizer.step()

w_nn = net.weight.detach().squeeze()
```

```
b_nn = net.bias.item()
print(f"nn.Linear:        w = {w_nn.numpy().round(3)},  b = {b_nn:.3f}")
```

```
nn.Linear:        w = [ 2.01  -3.402],  b = 4.196
```

Three implementations, one answer:

```
grid = torch.linspace(X[:, 0].min(), X[:, 0].max(), 50)
x2_mean = X[:, 1].mean()

def prediction_slice(weights: torch.Tensor, bias: float) -> torch.Tensor:
    """Predictions along x1 while x2 is held at its sample mean."""
    return weights[0] * grid + weights[1] * x2_mean + bias

y_on_slice = y - true_w[1] * (X[:, 1] - x2_mean)

plt.figure(figsize=(5.5, 3.6))
plt.scatter(X[:, 0], y_on_slice, s=12, alpha=0.4, label="data adjusted to slice")
plt.plot(grid, prediction_slice(w_ols, float(w_ols[2])), lw=3, label="closed form")
plt.plot(grid, prediction_slice(model.w, model.b.item()), "--", lw=2,
         label="gradient descent")
plt.plot(grid, prediction_slice(w_nn, b_nn), ":", lw=2, label="nn.Linear")
plt.xlabel("$x_1$"); plt.ylabel("$y$"); plt.legend()
plt.tight_layout()
plt.show()
```

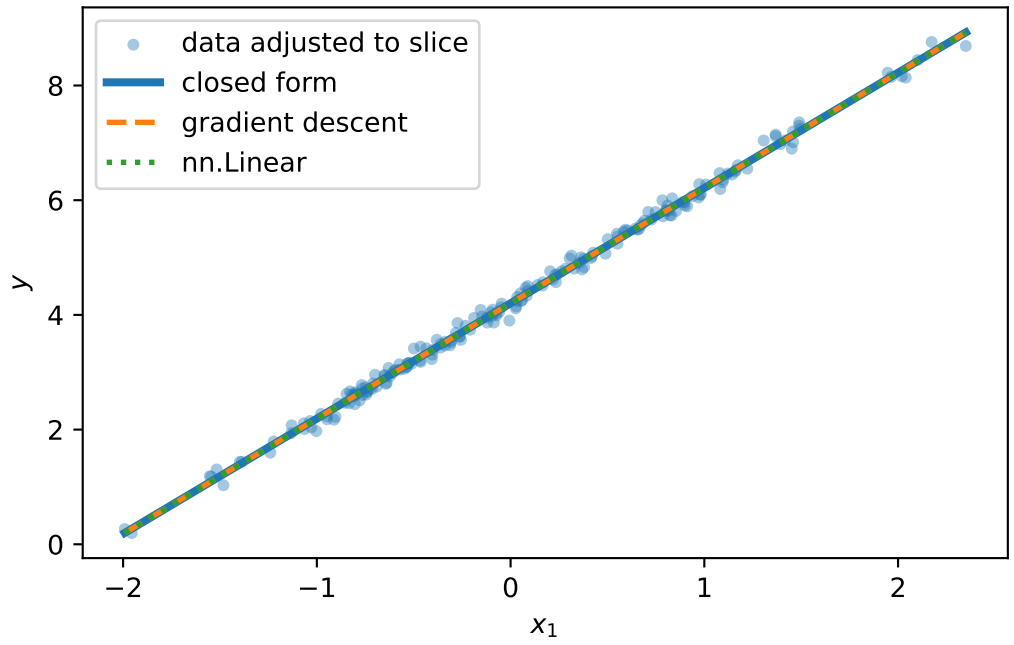

Figure 1.4.: All three methods find the same line on the slice $x_2 = \bar{x}_2$. Because this is synthetic data, the points can be adjusted to that same slice using the known true coefficient; line and points now show the same question.

1.7. Why squared error? The maximum-likelihood view

The MSE is not an arbitrary choice. Assume the data really is generated by a linear process plus Gaussian noise:

$$y = \mathbf{w}^\top \mathbf{x} + b + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \sigma^2).$$

Then the probability of seeing output y given input $\mathbf{x}$ is a Gaussian centered on the model's prediction. **Maximum likelihood estimation** asks: which parameters make the observed data most likely? Take the log of the likelihood over all n samples and the Gaussian's exponent comes down:

$$\log \mathcal{L}(\mathbf{w}, b) = C - \frac{1}{2\sigma^2} \sum_{i=1}^{n} \left(y_i - (\mathbf{w}^\top \mathbf{x}_i + b)\right)^2.$$

Maximizing the log-likelihood is *exactly* minimizing the sum of squared errors. The loss encodes an assumption about the noise. This is a pattern to remember: when we meet cross-entropy in Chapter 2, it will be the same story with a different distribution.

1.8. A first look at bias and variance

Our learned predictor $\hat{f}_{\mathcal{D}}$ depends on the random training set $\mathcal{D}$. Collect a fresh dataset and you get a different prediction at the same input $\mathbf{x}$. Let $f^*(\mathbf{x}) = \mathbb{E}[Y \mid X = \mathbf{x}]$ be the noise-free regression function. Then the expected squared error on a fresh outcome at this fixed input decomposes into three parts:

$$\mathbb{E}_{\mathcal{D},Y}\left[(\hat{f}_{\mathcal{D}}(\mathbf{x}) - Y)^2 \mid X = \mathbf{x}\right] = \underbrace{\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] - f^*(\mathbf{x})\right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}\left[(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})])^2\right]}_{\text{Variance}} + \underbrace{\text{Var}(Y \mid X = \mathbf{x})}_{\text{Irreducible noise}}. \tag{1.5}$$

Bias measures how far the *average* prediction is from the truth, because of limiting assumptions like linearity: a simple model on complex data is systematically wrong (underfitting). **Variance** measures how much the prediction would change if we collected a fresh training set (the model's sensitivity to sampling noise); a complex model can fit the noise itself (overfitting). The **irreducible noise** is inherent to the data-generating process and cannot be learned away.

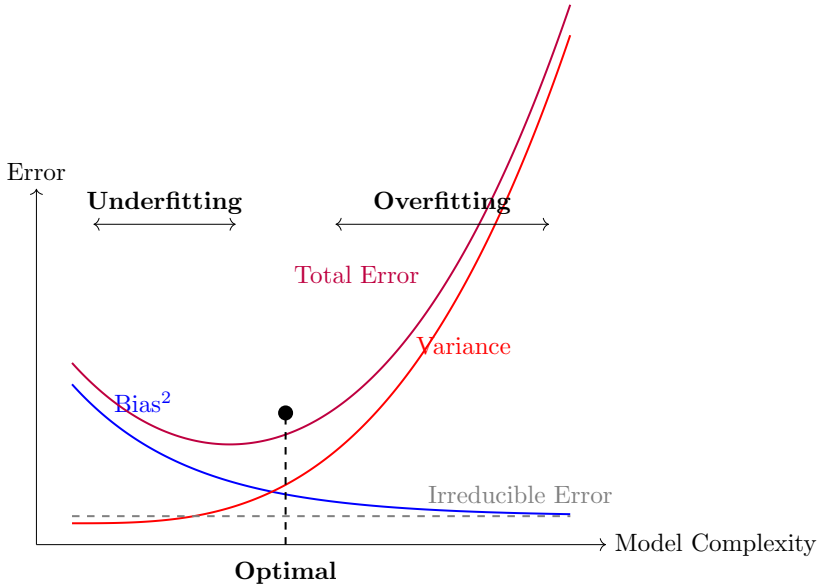

Figure 1.5.: The classical picture: as model complexity grows, bias falls while variance rises; total error is U-shaped, with sweet spots between under- and overfitting.

This is why we split data into training, validation, and test: the model sees the training set, the validation set referees the bias–variance trade-off, and the test set stays untouched until the end.

> ⚠ The U-shaped curve is a cartoon, not a law of nature
>
> The textbook picture, validation error falling then rising as complexity grows, is a helpful first mental model, and you should have it. But bias and variance are not a mechanical

see-saw. In classical fixed-dimensional, well-specified settings, variance often falls roughly like $1/n$ as data grows; that rate is not a universal law for modern models. Regularization and inductive biases matched to the task can buy capacity without exploding variance. Modern over-parameterized networks live in regimes where this cartoon bends in surprising ways; we will look at those pictures properly in Chapter 6.

1.8.1. Regularization, briefly

One lever we can pull right now: penalize complexity in the loss itself. **Ridge regression** adds an L_2 penalty,

$$\mathcal{L}_\lambda(\mathbf{w}) = \frac{1}{n}\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_2^2, \qquad \hat{\mathbf{w}}_{\text{ridge}} = (\mathbf{X}^\top\mathbf{X} + n\lambda I)^{-1}\mathbf{X}^\top\mathbf{y},$$

nudging the model toward smaller, more stable weights: a little more bias for a lot less variance. The factor n is there because our data-fit term is a *mean*. This displayed solution applies when there is no intercept or when features and targets have been centered so the intercept is recovered separately. We normally do not penalize that intercept. In augmented-matrix notation, replace I by $\text{diag}(1, \ldots, 1, 0)$ so the final bias coordinate remains free.

Let us see that, not just assert it. We build a problem designed to punish an unregularized model: 20 features of which only 5 matter, noisy targets, and (the cruel part) barely more samples than parameters. This is exactly the regime where variance explodes and regularization shines:

```
n, d = 25, 20                              # barely more samples than knobs!
w_true = torch.zeros(d)
w_true[:5] = torch.tensor([3.0, -2.0, 1.5, 2.5, -1.0])   # only 5 real features
Xh, yh = make_synthetic_data(w_true, bias=0.0, n_samples=n, noise=2.0)

w_ols_h = torch.linalg.solve(Xh.T @ Xh, Xh.T @ yh)        # plain OLS
lam = 0.4
w_ridge = torch.linalg.solve(Xh.T @ Xh + n * lam * torch.eye(d), Xh.T @ yh)

def report(name: str, w_hat: torch.Tensor) -> None:
    err = ((w_hat - w_true) ** 2).mean()
    print(f"{name:>6}:  weight MSE = {err:.4f}")

report("OLS", w_ols_h)
report("ridge", w_ridge)
```

```
   OLS:  weight MSE = 1.2347
 ridge:  weight MSE = 0.4400
```

OLS, with all its freedom and almost no data to discipline it, assigns large, confident, *wrong* weights to the 15 noise features — pure variance. Ridge shrinks everything and cuts the damage

several-fold: a little bias, traded for a lot of variance. (Rerun this cell with `n = 100` and watch OLS become more stable, as classical fixed-dimensional theory predicts.) Ridge's L_1 cousin, **lasso**, can go further and zero out the irrelevant features entirely; we leave it to the exercises, because on the road ahead it is weight decay, the L_2 idea, that you will meet again and again.

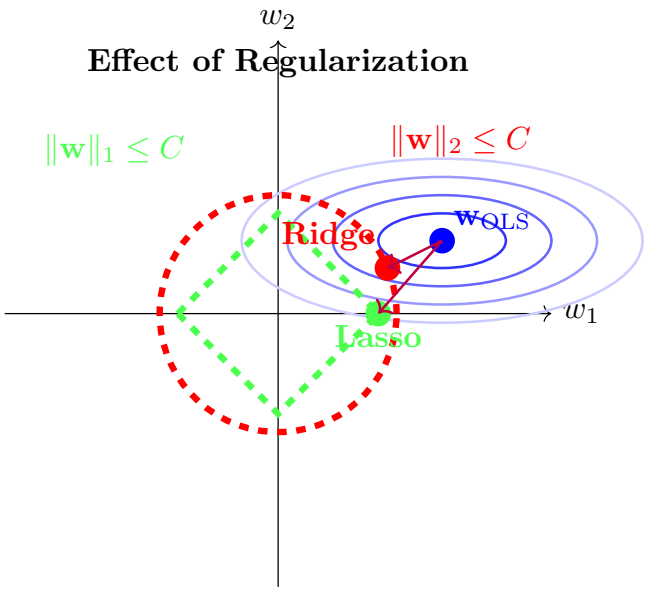

Figure 1.6.: Why the penalties behave differently: minimizing the loss subject to a weight budget. The spherical L_2 budget shrinks the solution toward the origin; the diamond-shaped L_1 budget has corners on the axes, so the solution tends to land where some coordinates are exactly zero (Exercise 5).

1.9. Okay, so — what did we just build?

We taught a computer a function from examples, completely:

1. **A model family** — hyperplanes, parameterized by knobs $\mathbf{w}, b$ (with the augmentation trick to fold b into $\mathbf{w}$).
2. **A loss** — MSE, which is maximum likelihood under Gaussian noise; the loss is how we tell the model how bad it is.
3. **An optimizer** — the exact projection when linearity permits, and gradient descent (feel the slope, step downhill) when it does not, which is always from here on.
4. **The generalization lens** — bias vs. variance, the data splits that referee them, and regularization as a first counter-measure to overfitting.

And one reframing that will carry the whole book: linear regression is a single neuron with the identity activation,

$$\underbrace{y = \mathbf{w}^\top \mathbf{x} + b}_{\text{this chapter}} \xrightarrow{\text{add a nonlinearity}} \underbrace{y = \sigma(\mathbf{w}^\top \mathbf{x} + b)}_{\text{a neuron}}.$$

First, in Chapter 2, that squashing function turns our regressor into a classifier. Then, in Chapter 3, we stack neurons — and discover why the nonlinearity is not optional.

Sources and further reading

- Hoerl and Kennard, "Ridge Regression: Biased Estimation for Nonorthogonal Problems" (1970) introduces the biased estimator behind this chapter's stability trade-off and regularization geometry.
- Belkin et al., "Reconciling Modern Machine-Learning Practice and the Classical Bias–Variance Trade-Off" (2019) is the promised next picture: why the classical U-curve can become double descent in interpolating models.

Exercises

1. **(Pencil.)** Derive the normal equations Equation 1.2 by expanding $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$, taking the gradient with respect to $\mathbf{w}$, and setting it to zero. Where exactly does the assumption that $\mathbf{X}^\top\mathbf{X}$ is invertible enter?
2. **(Pencil.)** Repeat the derivation with the ridge penalty $\lambda\|\mathbf{w}\|_2^2$ and show the solution becomes $(\mathbf{X}^\top\mathbf{X} + n\lambda I)^{-1}\mathbf{X}^\top\mathbf{y}$ when the data-fit term is the MSE. Why does the $n\lambda I$ term guarantee invertibility for any $\lambda > 0$? If you augment $\mathbf{X}$ with a bias column, which diagonal entry should be zero so the intercept is not penalized?
3. **(Code.)** In `make_synthetic_data`, raise `noise` to 1.0 and rerun all three implementations 10 times with different seeds. How much do the learned weights vary across runs? Which term of Equation 1.5 are you watching?
4. **(Code.)** Set the learning rate in `LinearRegressionScratch` to 1.0 and rerun. Describe what the loss curve does, and explain it using the blindfolded-descent picture.
5. **(Code.)** In the ridge experiment, sweep $\lambda \in \{0.01, 0.1, 1, 10, 100\}$ and plot weight MSE against λ. Where is the sweet spot, and what happens at the extremes? Connect your answer to bias and variance. Then replace ridge with `sklearn.linear_model.Lasso` at the best scale you found and count how many weights land exactly at zero. Why can an L_1 diamond zero coordinates when an L_2 sphere usually only shrinks them?

2. Linear Models that Classify: Logistic and Softmax

Regression predicts numbers; most of what the world wants predicted is *categories* — cat or dog, spam or not, which of ten digits. This chapter converts the linear machinery of Chapter 1 into a classifier without discarding a single idea: the same weighted sums, the same maximum-likelihood recipe for the loss, the same gradient descent. What changes is one squashing function at the output, and it changes less than you might think.

2.1. What actually changes

In regression, $y \in \mathbb{R}$ and the Gaussian noise model handed us the MSE (Chapter 1). In classification, y is a label from $\{0, \dots, K-1\}$. Could we just regress onto the labels with MSE anyway? We could, and it fails for a principled reason: **the loss is a noise model, and Gaussian is the wrong model for a coin flip.** A binary outcome deviates from its probability like a Bernoulli variable, not like additive Gaussian noise → maximum likelihood demands a different loss. Everything in this chapter falls out of taking that seriously.

What we need is two things:

1. a way to turn the model's raw score (its **logit**, $o \in \mathbb{R}$) into a probability,
2. a loss that measures how wrong a *probability* is.

2.2. Binary classification

2.2.1. From score to probability: the sigmoid

Our linear model produces $o = \mathbf{w}^\top \mathbf{x} + b$, an unbounded score. The **sigmoid** squashes it into a probability:

$$\sigma(o) = \frac{1}{1 + e^{-o}}. \tag{2.1}$$

```
import torch
import matplotlib.pyplot as plt

o = torch.linspace(-6, 6, 300)
plt.figure(figsize=(5.8, 3))
plt.plot(o, torch.sigmoid(o), color="#232D4B", lw=2.2)
plt.axhline(0.5, ls=":", color="#B8B8A8"); plt.axvline(0, ls=":", color="#B8B8A8")
plt.annotate("decision boundary\n$o = 0$", xy=(0, 0.5), xytext=(1.6, 0.32),
             color="#E57200", arrowprops=dict(arrowstyle="-|>", color="#E57200"))
plt.xlabel(r"logit $o = \mathbf{w}^\top \mathbf{x} + b$")
plt.ylabel(r"$\hat{p} = \sigma(o)$")
plt.tight_layout(); plt.show()
```

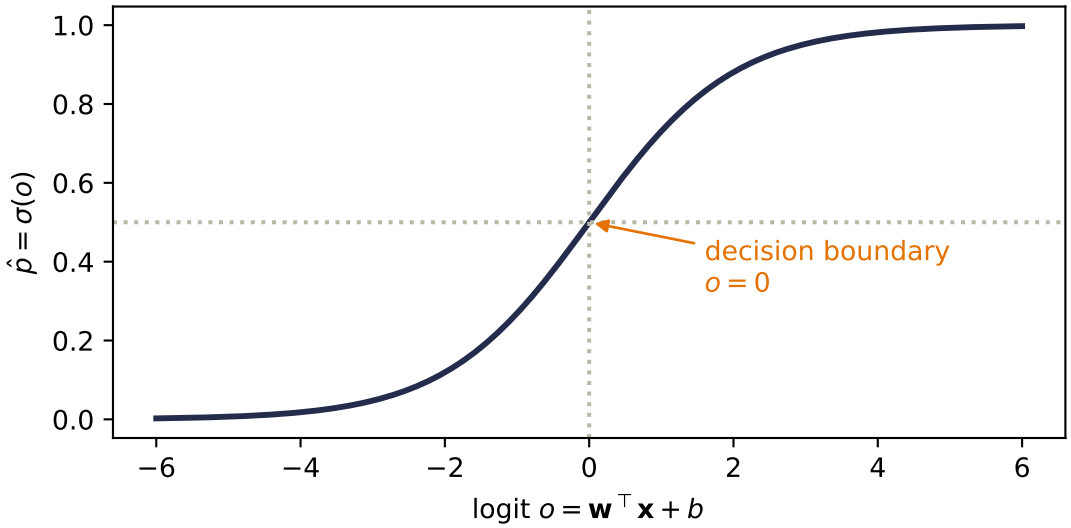

Figure 2.1.: The sigmoid maps any real score to $(0, 1)$, crossing $\frac{1}{2}$ exactly at $o = 0$. The model's uncertainty lives in the middle; its confident regions saturate at the tails.

Notice what did *not* change: the decision boundary $\hat{p} = \frac{1}{2}$ is exactly $o = 0$, i.e. $\mathbf{w}^\top \mathbf{x} + b = 0$ — the same hyperplane geometry from Chapter 1, with $\mathbf{w}$ still the template the input is scored against. The sigmoid does not bend the boundary; it grades our confidence on either side of it.

2.2.2. The loss: cross-entropy from maximum likelihood

Run the maximum-likelihood recipe with the correct noise model. If the label is a coin flip with $P(y{=}1 \mid \mathbf{x}) = \hat{p}$, the likelihood of the dataset is $\prod_i \hat{p}_i^{y_i}(1-\hat{p}_i)^{1-y_i}$; taking the negative log gives the **binary cross-entropy**:

$$\mathcal{L}_{\text{BCE}} = -\big[\, y \log \hat{p} + (1-y) \log(1-\hat{p}) \,\big]. \tag{2.2}$$

Read it as *surprise*: if the true label is 1, the loss is $-\log \hat{p}$ — small when the model believed in the outcome, exploding as the model's belief goes to zero. A confident wrong answer is punished without mercy, which is exactly the pressure a probability deserves.

2.2.3. The beautiful gradient

Chain the sigmoid into the cross-entropy and differentiate with respect to the logit, and something remarkable drops out (Exercise 1 walks you through it):

$$\frac{\partial \mathcal{L}}{\partial o} = \hat{p} - y. \tag{2.3}$$

The gradient is *literally the prediction error*, the same form MSE gave us for regression. All the logs and exponentials annihilate each other. This is not luck, and it is not the last time you will see it.

2.3. Multiclass classification: softmax

With K classes, run K templates at once: $\mathbf{W} \in \mathbb{R}^{K \times d}$ produces a score vector $\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$, one logit per class. To turn K scores into a probability distribution, exponentiate (making everything positive) and normalize:

$$\hat{y}_j = \mathrm{softmax}(\mathbf{o})_j = \frac{e^{o_j}}{\sum_{k=1}^{K} e^{o_k}}. \tag{2.4}$$

Softmax has three properties worth committing to memory: every output is positive, the outputs sum to one, and — less obviously — it is **invariant to constant shifts**: adding the same c to every logit changes nothing, because $e^{o_j + c} = e^c e^{o_j}$ and the e^c cancels top and bottom. Only the *differences* between scores matter.

```
logits = torch.tensor([2.0, 0.5, -1.0, 1.0])
fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.9), sharey=True)
for ax, shift in [(axes[0], 0.0), (axes[1], 100.0)]:
    p = torch.softmax(logits + shift, dim=0)
    ax.bar(range(4), p, color="#5379AA", width=0.55)
    for j, v in enumerate(p):
        ax.text(j, v + 0.015, f"{v:.2f}", ha="center", fontsize=9)
    ax.set_xticks(range(4))
    ax.set_xticklabels([f"$o_{j}$ = {v:+.1f}" for j, v in enumerate(logits + shift)],
                       fontsize=8)
    ax.set_title("original logits" if shift == 0 else "all logits + 100")
axes[0].set_ylabel("probability")
plt.tight_layout(); plt.show()
```

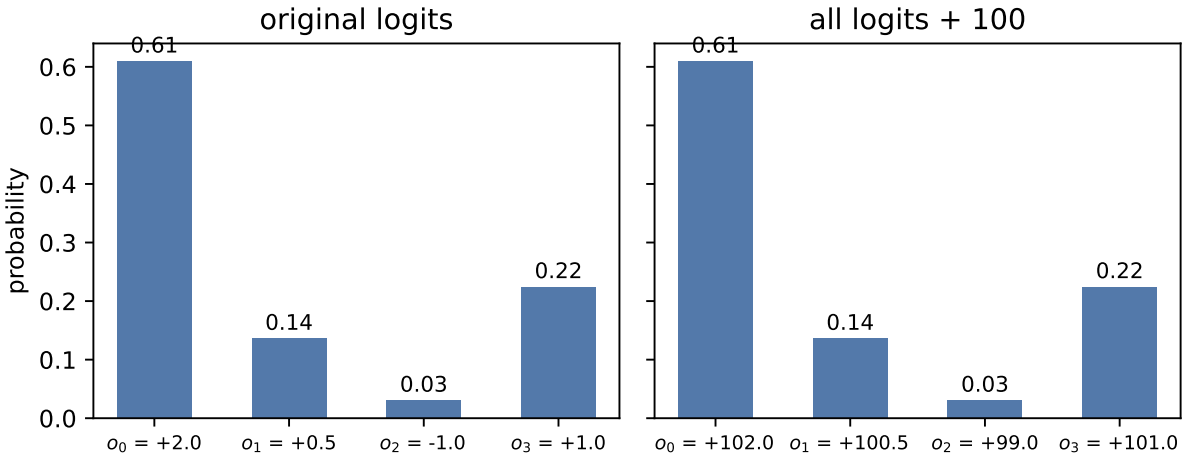

Figure 2.2.: Softmax turns scores into a distribution. Shifting every logit by the same constant (right panel) leaves the probabilities unchanged: only the differences between scores carry information.

Remember this machine: scores → weights

Step back from classification for a moment. What softmax really is: a differentiable device that takes any list of scores and returns **positive weights that sum to one**, favoring the large scores without silencing the small ones. Today the scores are class logits. In Part IV, the scores will be *similarities between a query and a set of keys*, and this exact same machine will turn them into **attention weights** (Chapter 13). One normalizer, the whole book long.

What "differentiable" buys us — softening the hard

That word *differentiable* deserves a moment in the light, because it names one of the great design patterns of this field.

Consider the obvious way to pick a class: take the **argmax**, then output a one-hot vector with 1 at the winning index and 0 elsewhere. (The scalar `max` operation is different and does have a gradient through its winning input.) Argmax-plus-one-hot is *hard*: nudge a losing logit slightly and the output does not move at all; nudge it past the winner and the output jumps all at once. Its derivative is zero almost everywhere → gradient descent receives no useful signal. In the language we will formalize in Chapter 5, the hard selection is a wall that blame cannot flow through.

Softmax is a smooth relaxation of that one-hot selection; the name is literal: a **soft max**. Instead of crowning one winner, it distributes belief according to the scores, and now every logit can affect the loss. We gave up a little decisiveness and bought trainability.

Watch for this move; it is everywhere once you see it. Whenever a useful operation is hard — a lookup, a yes/no choice, even *sorting a list* (yes, researchers have built differentiable sorting!) — the play is the same: replace it with a smooth version that gradients can flow through, train, and sharpen later if needed. The one to remember for this book: fetching

a value from memory is a hard lookup, and in Part IV we will soften it. A *differentiable lookup*, it turns out, is precisely attention.

2.3.1. Cross-entropy, multiclass edition

With one-hot labels $\mathbf{y}$ (all zeros except a 1 at the true class), the cross-entropy is

$$\mathcal{L}_{\text{CE}} = -\sum_{j=1}^{K} y_j \log \hat{y}_j = -\log \hat{y}_{\text{true class}}, \tag{2.5}$$

the surprise at the true class, again. And the gradient repeats its trick:

$$\frac{\partial \mathcal{L}}{\partial o_j} = \hat{y}_j - y_j, \tag{2.6}$$

prediction minus target, one more time. This recurrence is no coincidence — sigmoid/BCE and softmax/CE are both members of the exponential family paired with their natural losses, and that pairing always collapses the gradient to the error.

i The information-theory reading

Entropy $H[P] = -\sum_j p_j \log p_j$ measures the uncertainty in a distribution; cross-entropy $H(P, Q) = -\sum_j p_j \log q_j$ is the cost of encoding outcomes from P using codes built for Q. Minimizing our loss is minimizing that encoding cost — equivalently, maximizing likelihood, equivalently driving the KL divergence from truth to prediction toward zero. Three vocabularies, one objective.

2.3.2. One numerical landmine

Computing e^{o_j} overflows for logits as small as a few hundred. The fix exploits shift invariance: subtract the max logit before exponentiating (the *log-sum-exp trick*), which changes nothing mathematically and everything numerically. We will use it in the code below; the deeper story of floating-point limits lives in Appendix D.

2.4. Build it: three classes, from scratch

Now we assemble the full classifier on three Gaussian blobs. It has four pieces, all of which you have already met: three templates (one per class), the stable softmax, the cross-entropy loss, and the gradient. And since Equation 2.6 reduced that gradient to $\hat{y} - y$, we can implement it by hand → no autograd needed:

```
torch.manual_seed(6050)
centers = torch.tensor([[-2.0, 0.0], [2.0, 0.0], [0.0, 2.5]])
X = torch.cat([c + 0.7 * torch.randn(150, 2) for c in centers])   # (450, 2)
y = torch.arange(3).repeat_interleave(150)                        # (450,)
Y = torch.nn.functional.one_hot(y, 3).float()                     # (450, 3)

def stable_softmax(logits: torch.Tensor) -> torch.Tensor:
    z = logits - logits.max(dim=-1, keepdim=True).values   # shift invariance at work
    e = torch.exp(z)
    return e / e.sum(dim=-1, keepdim=True)

W, b = torch.zeros(3, 2), torch.zeros(3)
for step in range(400):
    P = stable_softmax(X @ W.T + b)          # (450, 3) predicted probabilities
    G = (P - Y) / len(X)                     # the beautiful gradient, eq. (6)
    W -= 1.0 * G.T @ X                       # blame out x signal in
    b -= 1.0 * G.sum(0)

P = stable_softmax(X @ W.T + b)              # evaluate the final updated parameters
ce = -(Y * torch.log(P)).sum(1).mean()
acc = (P.argmax(1) == y).float().mean()
print(f"cross-entropy {ce:.3f}   accuracy {acc:.1%}")
```

```
cross-entropy 0.033   accuracy 98.9%
```

```
import numpy as np

gx, gy = np.meshgrid(np.linspace(-4.5, 4.5, 300), np.linspace(-2.5, 5, 300))
grid = torch.tensor(np.stack([gx.ravel(), gy.ravel()], 1), dtype=torch.float32)
Pg = stable_softmax(grid @ W.T + b)
cls = Pg.argmax(1).reshape(gx.shape).numpy()
conf = Pg.max(1).values.reshape(gx.shape).numpy()

plt.figure(figsize=(6, 4.4))
plt.contourf(gx, gy, cls, levels=[-0.5, 0.5, 1.5, 2.5],
             colors=["#DCE6F2", "#FDE3C8", "#E3EEE3"])
plt.contourf(gx, gy, 1 - conf, levels=12, cmap="Greys",
             alpha=0.25, antialiased=True)
for k, color in enumerate(["#232D4B", "#E57200", "#6B8E6B"]):
    plt.scatter(X[y == k, 0], X[y == k, 1], s=8, c=color, label=f"class {k}")
plt.xlabel("$x_1$"); plt.ylabel("$x_2$"); plt.legend(loc="lower right")
plt.tight_layout(); plt.show()
```

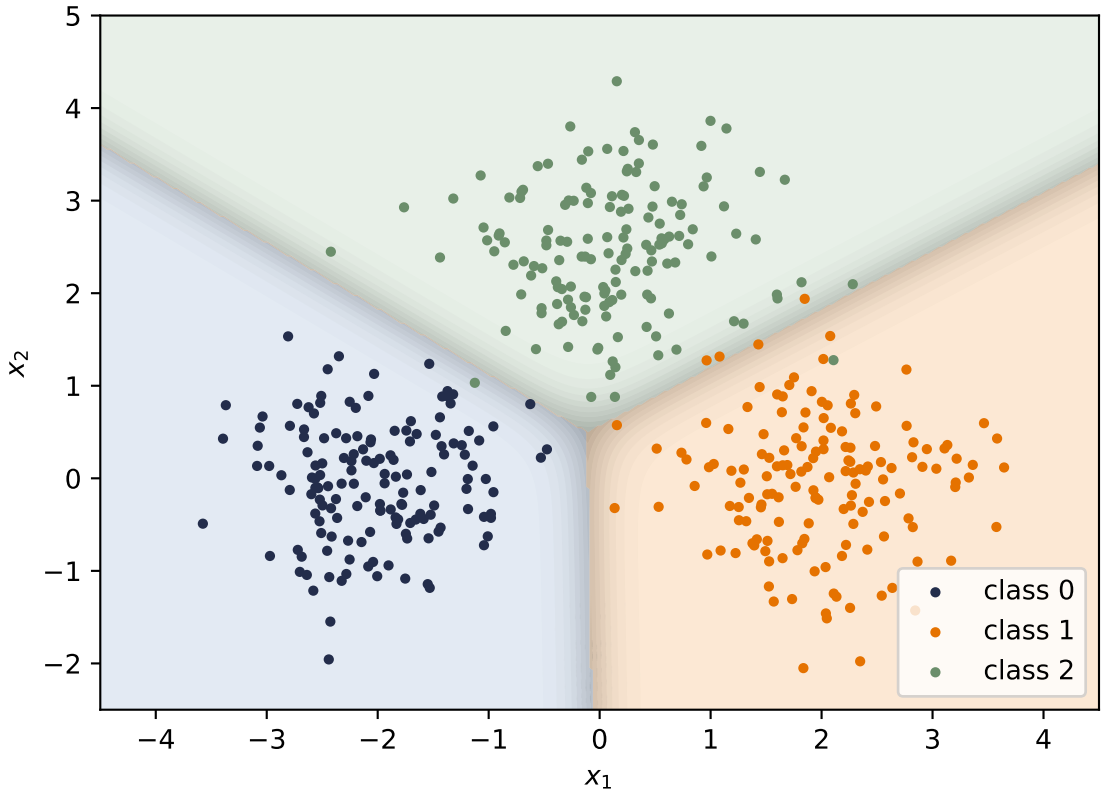

Figure 2.3.: Three templates, three linear boundaries. Color shows the predicted class; shading shows confidence (the max softmax probability) — crisp deep in each region, washed out along the boundaries where classes compete.

The framework version is two lines, with one trap worth a warning. This loss object is the only new framework tool introduced here: it evaluates logits and labels, while the manual loop above still supplies the gradient. Optimizers and automatic backpropagation remain the previews marked in Chapter 1 until Chapters 4 and 5.

```
loss_fn = torch.nn.CrossEntropyLoss()
print(f"ours: {ce:.6f}   torch: {loss_fn(X @ W.T + b, y):.6f}")
```

```
ours: 0.033425   torch: 0.033425
```

⚠ Let PyTorch do the logits-to-probabilities conversion

`nn.CrossEntropyLoss` expects **raw logits**, and that is a feature, not a quirk: it fuses the softmax and the logarithm internally using the log-sum-exp trick, which is more numerically stable than any softmax-then-log you would write yourself. So the advice is: have your model output logits, hand them straight to the loss, and only apply softmax when you actually need probabilities to report. (The same goes for binary problems: prefer

`binary_cross_entropy_with_logits` over sigmoid-then-BCE.) The classic bug is feeding softmax outputs into `CrossEntropyLoss` — the model still trains, just badly and silently. If a classifier is mysteriously mediocre, check this first.

2.5. Okay, so — classification is regression plus a normalizer

1. **The geometry survived**: logits are still template similarities $\mathbf{w}^\top \mathbf{x} + b$, boundaries are still hyperplanes. Sigmoid and softmax grade confidence; they do not bend anything.
2. **The loss came from the noise model**, exactly as in Chapter 1: Bernoulli → BCE, categorical → cross-entropy, both read as *surprise at the truth*.
3. **The gradient is the error**, $\hat{y} - y$, both times — the exponential-family pairing at work.
4. **Softmax is a scores-to-weights machine**, shift-invariant, numerically tamed by log-sum-exp — and it will return, unchanged, as the normalizer of attention.

Next, we let the templates themselves be built out of other templates: Chapter 3 stacks these linear units — and discovers why a bend between them is not optional.

Sources and further reading

- Cox, "The Regression Analysis of Binary Sequences" (1958) develops the logistic model as a principled tool for binary outcomes.
- Bridle, "Probabilistic Interpretation of Feedforward Classification Network Outputs" (1990) connects normalized exponential outputs to probabilistic multiclass classification.
- Shannon, "A Mathematical Theory of Communication" (1948) is the original source behind the entropy and information vocabulary used to read cross-entropy.

Exercises

1. **(Pencil.)** Derive Equation 2.3: write $\mathcal{L}_{\text{BCE}}$ as a function of o through $\hat{p} = \sigma(o)$, use $\sigma'(o) = \sigma(o)(1-\sigma(o))$, and simplify. Which factors cancel, and why is the result independent of the sigmoid's saturation?
2. **(Pencil.)** Prove softmax's shift invariance from Equation 2.4, then explain why subtracting the max logit makes the computation overflow-proof: what is the largest value ever exponentiated?
3. **(Code.)** Temperature: replace `logits` with `logits / T` in the softmax figure for $T \in \{0.5, 1, 2, 10\}$. Describe what happens to the distribution at both extremes. (Keep this dial in mind — it returns when we scale attention scores in Part IV.)
4. **(Code.)** Train the blobs classifier with MSE on one-hot targets instead of cross-entropy (same manual loop; the gradient changes). Compare accuracy and the confidence shading near boundaries. What does the wrong noise model cost?

5. **(Code.)** Move the three blob centers closer together (`0.7 → 1.2` noise, or centers at distance 1) and retrain. Where does the confidence shading collapse, and what does the cross-entropy value tell you compared to the accuracy?

3. Nonlinearity and the MLP

Our linear models can regress (Chapter 1) and classify (Chapter 2), and it is tempting to think we could get more power by simply stacking them: feed the output of one linear layer into another. Watch what happens:

$$\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\,\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2) = \mathbf{W}_{\text{new}}\,\mathbf{x} + \mathbf{b}_{\text{new}}. \tag{3.1}$$

The stack *collapses*. Two linear layers, or twenty, are exactly one linear layer with different knobs $\rightarrow$ no amount of stacking buys any new expressive power. To learn the patterns of the real world we need a magic ingredient: **nonlinearity**. This chapter is about what one small bend does to everything.

3.1. The tiny dataset that embarrassed a field

You do not need real-world data to expose the linear ceiling. Four points suffice. The XOR function outputs 1 exactly when its two binary inputs *differ*:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Plot the four points and try to separate the 1s from the 0s with a single line. The positive examples sit on one diagonal, the negatives on the other; any line you draw puts at least one point on the wrong side. A linear classifier cannot represent this function, no matter how it is trained.

> **i** A historical scar
>
> This is not a toy observation. The limitations of single-layer perceptrons, including XOR, were publicized in 1969 and became emblematic of the case against that research program. They contributed to reduced enthusiasm and funding, but they were not a single-cause explanation for the first "AI winter"; limited compute, data, and unmet promises mattered too. Multi-layer networks and practical backpropagation (Chapter 5) later removed the representational obstacle exposed here.

Let us confirm the ceiling empirically. We reuse the framework loop previewed in Chapter 1 as a measuring instrument; Chapter 4 and Chapter 5 will explain the optimizer and `backward()` rather than asking you to treat them as magic.

```
import torch
from torch import nn

torch.manual_seed(6050)
X_xor = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y_xor = torch.tensor([0., 1., 1., 0.])

def train(
    model: nn.Module, X: torch.Tensor, y: torch.Tensor,
    steps: int = 4000, lr: float = 0.5
) -> nn.Module:
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    for _ in range(steps):
        opt.zero_grad()
        loss = nn.functional.binary_cross_entropy_with_logits(model(X).squeeze(-1), y)
        loss.backward()
        opt.step()
    return model

linear = train(nn.Linear(2, 1), X_xor, y_xor)
mlp = train(nn.Sequential(nn.Linear(2, 4), nn.ReLU(), nn.Linear(4, 1)),
            X_xor, y_xor)

with torch.no_grad():
    linear_p = torch.sigmoid(linear(X_xor).squeeze(-1))
    linear_bce = nn.functional.binary_cross_entropy(linear_p, y_xor)
    mlp_acc = ((mlp(X_xor).squeeze(-1) > 0).float() == y_xor).float().mean()

print(f"trained linear: probabilities {linear_p.numpy().round(3)}, "
      f"BCE {linear_bce:.3f}")
print(f" MLP (4 hidden): accuracy {mlp_acc:.0%}")
```

```
trained linear: probabilities [0.5 0.5 0.5 0.5], BCE 0.693
 MLP (4 hidden): accuracy 100%
```

The cross-entropy optimum for this symmetric XOR dataset assigns probability 0.5 to all four points, with loss $\log 2 \approx 0.693$. Turning those exact ties into hard labels with a floating-point threshold can report 25%, 50%, or 75% because of tiny rounding differences; that number is not evidence about the classifier. The honest geometric statement is that a hard linear boundary can classify at most three of the four points (75%), while the little network with a bend is perfect.

```
import numpy as np
import matplotlib.pyplot as plt

gx, gy = np.meshgrid(np.linspace(-0.6, 1.6, 220), np.linspace(-0.6, 1.6, 220))
grid = torch.tensor(np.stack([gx.ravel(), gy.ravel()], 1), dtype=torch.float32)

fig, axes = plt.subplots(1, 2, figsize=(7.4, 3.4), sharey=True)
for ax, model, title in [(axes[0], None, "best linear rule (75%)"),
                         (axes[1], mlp, "trained MLP (100%)")]:
    with torch.no_grad():
        if model is None:
            Z = (grid.sum(1) - 0.5 > 0).float().reshape(gx.shape)
        else:
            Z = (model(grid).squeeze(-1) > 0).float().reshape(gx.shape)
    ax.contourf(gx, gy, Z, levels=[-0.5, 0.5, 1.5],
                colors=["#DCE6F2", "#FDE3C8"], alpha=0.9)
    for cls, marker, color in [(0, "o", "#232D4B"), (1, "s", "#E57200")]:
        pts = X_xor[y_xor == cls]
        ax.scatter(pts[:, 0], pts[:, 1], marker=marker, s=120, c=color,
                   edgecolors="white", linewidths=1.5, zorder=3,
                   label=f"class {cls}")
    ax.set_title(title); ax.set_xlabel("$x_1$")
axes[0].set_ylabel("$x_2$"); axes[0].legend(loc="center right")
plt.tight_layout(); plt.show()
```

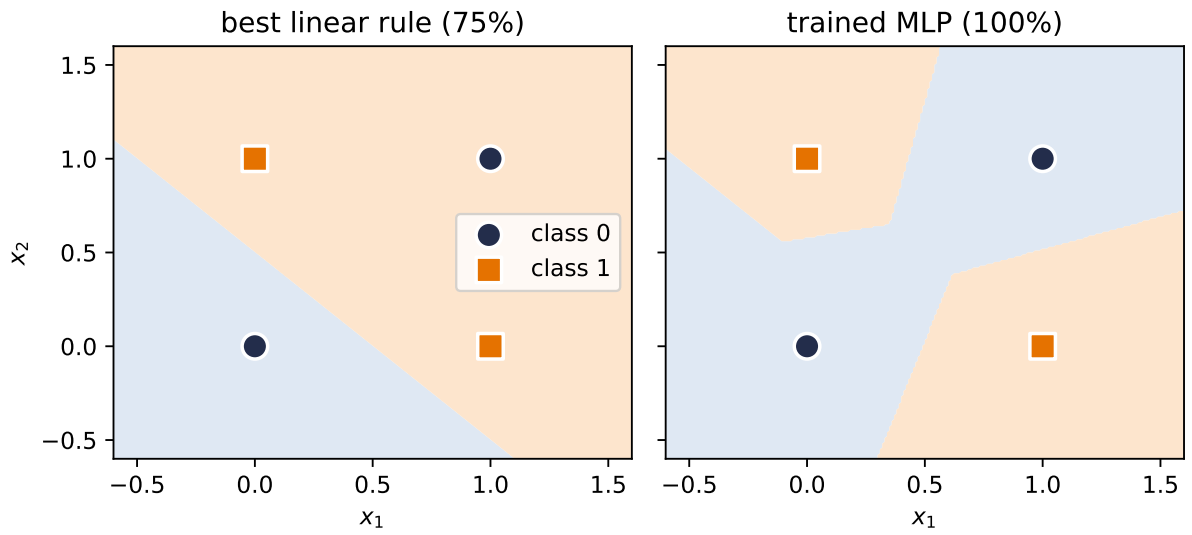

Figure 3.1.: Left: a representative best linear boundary gets three of four XOR points right, but one point must remain on the wrong side. Right: the trained 4-hidden-unit MLP carves separate regions for the two positive corners.

3.2. The neuron: a threshold with a bend

The artificial neuron takes its inspiration from biology: a neuron receives signals, and it *fires* only when the accumulated stimulus crosses a threshold. The modern, computationally friendly version of that thresholding is the **rectified linear unit**:

$$\mathrm{ReLU}(z) = \max(0, z), \qquad f(\mathbf{x}) = \mathrm{ReLU}(\mathbf{w}^\top \mathbf{x} + b). \tag{3.2}$$

The weighted input $\mathbf{w}^\top \mathbf{x} + b$ is the stimulus — and recall from Chapter 1 that it is a *similarity score* against the template $\mathbf{w}$. Below threshold, the neuron is silent (output 0); above it, the neuron fires with intensity proportional to the stimulus.

In one dimension a single neuron turns a line into a **hinge**, and in two dimensions it draws a line through the plane: an OFF region and an ON region, with $\mathbf{w}$ perpendicular to the boundary. That boundary is the fundamental unit of computation in everything that follows.

(What about the sharp corner at $z = 0$, where the derivative is undefined? Libraries choose a subgradient convention, commonly 0. Exact zeros can occur systematically with zero biases, sparse activations, or symmetric inputs, so the convention is not merely academic. The gradient story of this open-or-shut gate — including when it helps and when a unit goes dead — is told in Chapter 5.)

3.3. Layers of hinges: bumps and polytopes

One neuron makes one boundary. A layer of k neurons makes k boundaries, and their intersections partition the input space into convex **polyhedral regions** (bounded ones are polytopes), and within each cell the layer computes a plain linear function. For fixed input dimension d, the maximum region count of one generic hyperplane arrangement grows on the order of k^d.

The simplest compact constructive example lives in 1D. Combine three hinges so the slopes cancel again after the third breakpoint:

```
xs = torch.linspace(-3, 5, 400)
h1 = torch.relu(xs + 1.5)
h2 = torch.relu(xs - 0.5)
h3 = torch.relu(xs - 2.5)
bump = h1 - 2 * h2 + h3

fig, axes = plt.subplots(1, 2, figsize=(7.4, 3))
axes[0].plot(xs, h1, color="#5379AA", label=r"$h_1 = \mathrm{ReLU}(x + 1.5)$")
axes[0].plot(xs, h2, color="#6B8E6B", label=r"$h_2 = \mathrm{ReLU}(x - 0.5)$")
axes[0].plot(xs, h3, color="#722F37", label=r"$h_3 = \mathrm{ReLU}(x - 2.5)$")
axes[0].set_title("three hidden units"); axes[0].legend(fontsize=7)
axes[1].plot(xs, bump, color="#E57200", lw=2.5,
             label=r"$y = h_1 - 2h_2 + h_3$")
axes[1].set_title("their combination: a bump"); axes[1].legend(fontsize=9)
```

```
for ax in axes: ax.set_xlabel("$x$")
plt.tight_layout(); plt.show()
```

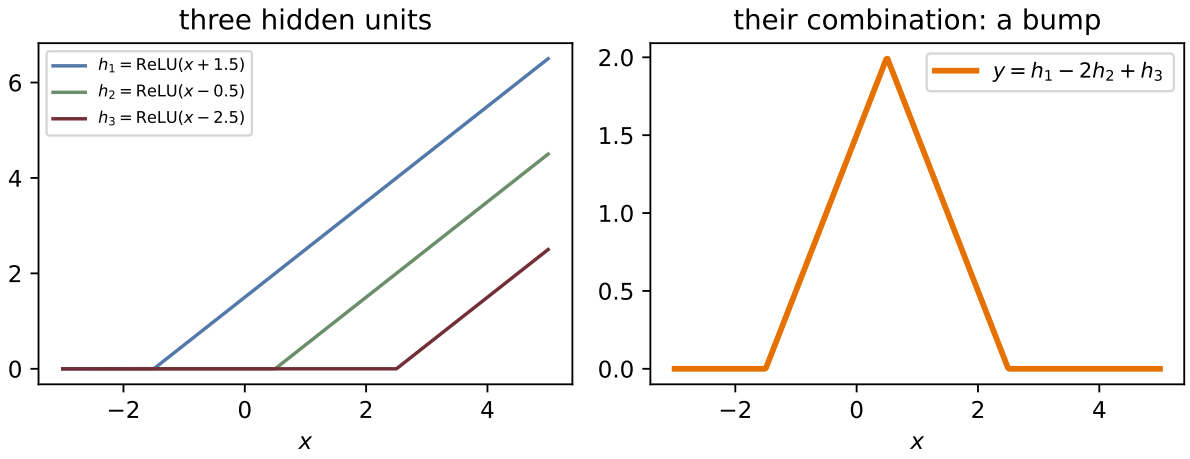

Figure 3.2.: Three hinges make a compact triangular bump: the first starts the rise, the second reverses the slope, and the third cancels it back to zero. Bumps are the brush strokes of function approximation.

Follow the pieces: before the first hinge, all units are silent and the output is zero. After the first breakpoint, h_1 starts the climb. The coefficient -2 on h_2 reverses the slope at the peak, and h_3 cancels that downward slope at the final breakpoint. Four linear pieces → one compact bump.

3.4. The universal approximation theorem

Bumps are the whole secret. If you can make one bump, you can make many, at any position and height → with enough hidden units you can *tile* any continuous function, the way an artist paints a shape with brush strokes. That is the intuition behind the **universal approximation theorem**: a network with a single hidden layer can, in principle, approximate any continuous function on a bounded region to any desired accuracy.

Watch the theorem materialize as width grows:

```
xs1 = torch.linspace(-3, 3, 200)[:, None]
target = torch.sin(1.5 * xs1) + 0.08 * xs1 ** 2

def fit(width: int, steps: int = 3000, lr: float = 0.05) -> torch.Tensor:
    torch.manual_seed(6050)
    net = nn.Sequential(nn.Linear(1, width), nn.ReLU(), nn.Linear(width, 1))
    opt = torch.optim.SGD(net.parameters(), lr=lr)
    for _ in range(steps):
        opt.zero_grad()
```

```
        ((net(xs1) - target) ** 2).mean().backward()
        opt.step()
    with torch.no_grad():
        return net(xs1).squeeze(-1)

plt.figure(figsize=(6.6, 3.6))
plt.plot(xs1, target, color="#232D4B", lw=2.5, label="target")
for width, color in [(2, "#B8B8A8"), (8, "#5379AA"), (64, "#E57200")]:
    plt.plot(xs1, fit(width), color=color, lw=1.6, label=f"width {width}")
plt.xlabel("$x$"); plt.legend(); plt.tight_layout(); plt.show()
```

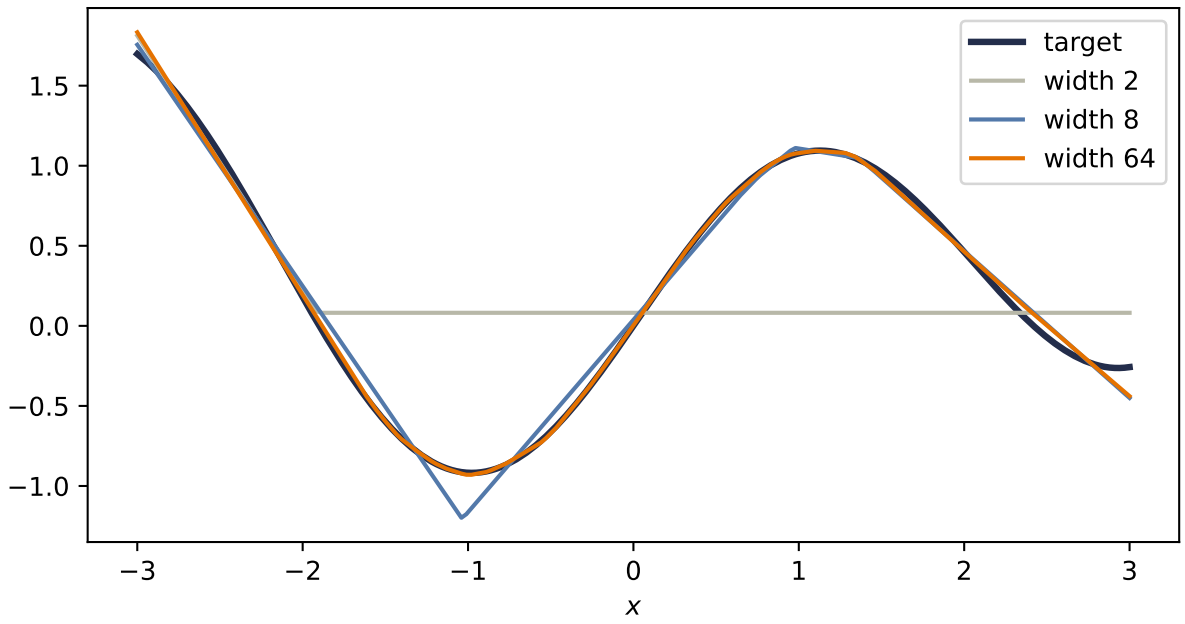

Figure 3.3.: One hidden ReLU layer approximating a continuous curved target. Width 2 is too rigid; width 8 catches the shape; width 64 follows it closely. Width buys more hinge locations, but the theorem itself does not tell the optimizer where to place them.

Now read the theorem's fine print, because it is where beginners over-promise:

⚠ What the theorem does *not* say

1. It does not tell you **how to find the weights** — existence of a good network is not a training algorithm. Finding the weights is the job of the loss and the optimizer, and nothing guarantees gradient descent lands on the theorem's solution.
2. It does not say the network is **small**. Tiling a complicated function with one-layer bumps can require astronomically many units.
3. In its usual uniform-approximation form, it speaks of **continuous functions on bounded regions** and says nothing about how the fit behaves *between* or *beyond*

your samples. A continuous ReLU network cannot uniformly approximate a jump to arbitrary accuracy: near the jump its worst-case error stays at least half the jump size. Under a mean-square (L_2) criterion, however, it can squeeze the transition into a narrow interval and make the *average* error arbitrarily small. The norm and the target class matter. Approximation is still not generalization; that distinction gets its own chapter (Chapter 6).

3.5. Why depth: boundaries that bend

If one hidden layer suffices in principle, why is the field called *deep* learning? Efficiency, and geometry. A second hidden layer receives, as its inputs, the piecewise-linear outputs of the first → its boundaries are no longer straight lines. They *bend* wherever the first layer's boundaries cut the space. Boundaries composed of boundaries: each layer folds the space the previous layer drew.

```
torch.manual_seed(6050)
net2 = nn.Sequential(nn.Linear(2, 8), nn.ReLU(), nn.Linear(8, 8), nn.ReLU())

gx2, gy2 = np.meshgrid(np.linspace(-2, 2, 500), np.linspace(-2, 2, 500))
G = torch.tensor(np.stack([gx2.ravel(), gy2.ravel()], 1), dtype=torch.float32)
with torch.no_grad():
    h1_ = torch.relu(net2[0](G))
    h2_ = torch.relu(net2[2](h1_))
    pattern = torch.cat([(h1_ > 0), (h2_ > 0)], 1).to(torch.int64)
    raw_code = pattern @ (2 ** torch.arange(16, dtype=torch.int64))
    _, region_id = torch.unique(raw_code, return_inverse=True)
    region_id = region_id.reshape(gx2.shape)

plt.figure(figsize=(5.4, 4.6))
plt.imshow(region_id, extent=[-2, 2, -2, 2], origin="lower",
           cmap="turbo", alpha=0.85)
plt.xlabel("$x_1$"); plt.ylabel("$x_2$")
plt.tight_layout(); plt.show()
```

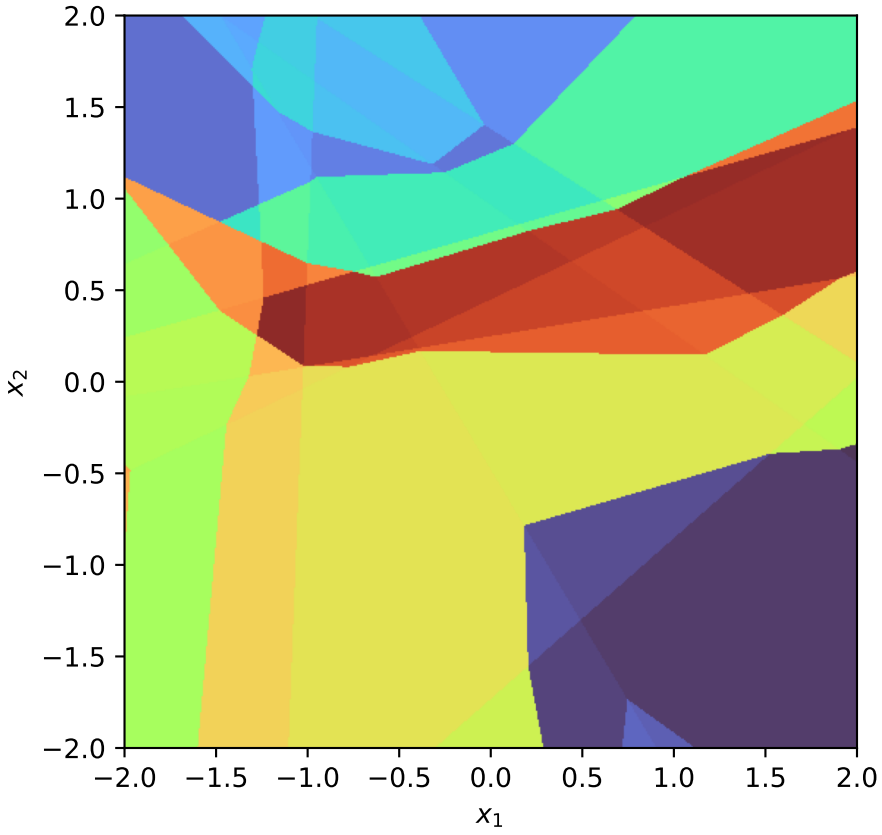

Figure 3.4.: The plane, colored by which ReLUs are on/off in a random 2-layer network ($2 \to 8 \to 8$): every cell is a region where the network is exactly linear. First-layer boundaries are straight; second-layer boundaries visibly bend where they cross them.

This bending can compound dramatically. For suitable ReLU constructions at fixed input dimension, achievable region counts grow exponentially with depth, while a single hidden layer's count grows polynomially with width. This is an existence and architecture-dependent comparison, not a promise that every trained deep network realizes all those regions. It gives the economic argument for depth: some partitions represented by a deep, narrow network require far more units in a shallow one.

3.6. Your first real MLP

Time to build the thing properly, the way the live session does: as a class.

> Coding hygiene: the house and its foundation
>
> Every model you write from now on subclasses `nn.Module`, and the first line of your `__init__` is always `super().__init__()`. Think of it as pouring the foundation before building the house: it is what wires up parameter tracking, `.parameters()`, and everything

> the optimizer and autograd need. Forget it and PyTorch fails loudly — the one favor a framework can do you.

```
class MLP(nn.Module):
    """A multilayer perceptron: linear layers with bends between them."""

    def __init__(self, input_dim: int, hidden_dim: int, output_dim: int):
        super().__init__()                      # the foundation
        self.hidden = nn.Linear(input_dim, hidden_dim)
        self.out = nn.Linear(hidden_dim, output_dim)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.out(torch.relu(self.hidden(x)))
```

And a problem worthy of it: two interleaved moons, the classic shape no line can separate well. Same training loop as always — the model is the only thing that changed:

```
torch.manual_seed(6050)
n = 200
t = torch.rand(n) * torch.pi
moon1 = torch.stack([torch.cos(t), torch.sin(t)], 1) + 0.12 * torch.randn(n, 2)
moon2 = torch.stack([1 - torch.cos(t), 0.4 - torch.sin(t)], 1) + 0.12 * torch.randn(n, 2)
X_m = torch.cat([moon1, moon2])
y_m = torch.cat([torch.zeros(n), torch.ones(n)])

torch.manual_seed(6050)
lin_m = train(nn.Linear(2, 1), X_m, y_m, steps=3000, lr=0.8)
torch.manual_seed(6050)
mlp_m = train(MLP(2, 16, 1), X_m, y_m, steps=3000, lr=0.8)

gx3, gy3 = np.meshgrid(np.linspace(-1.6, 2.6, 240), np.linspace(-1.1, 1.6, 240))
grid3 = torch.tensor(np.stack([gx3.ravel(), gy3.ravel()], 1), dtype=torch.float32)

fig, axes = plt.subplots(1, 2, figsize=(7.6, 3.3), sharey=True)
for ax, model, title in [(axes[0], lin_m, "linear"), (axes[1], mlp_m, "MLP (16 hidden)")]:
    with torch.no_grad():
        acc = ((model(X_m).squeeze(-1) > 0).float() == y_m).float().mean()
        Z = (model(grid3).squeeze(-1) > 0).float().reshape(gx3.shape)
    ax.contourf(gx3, gy3, Z, levels=[-0.5, 0.5, 1.5],
                colors=["#DCE6F2", "#FDE3C8"], alpha=0.9)
    ax.scatter(X_m[y_m == 0, 0], X_m[y_m == 0, 1], s=8, c="#232D4B")
    ax.scatter(X_m[y_m == 1, 0], X_m[y_m == 1, 1], s=8, c="#E57200")
    ax.set_title(f"{title}: {acc:.1%}"); ax.set_xlabel("$x_1$")
axes[0].set_ylabel("$x_2$")
plt.tight_layout(); plt.show()
```

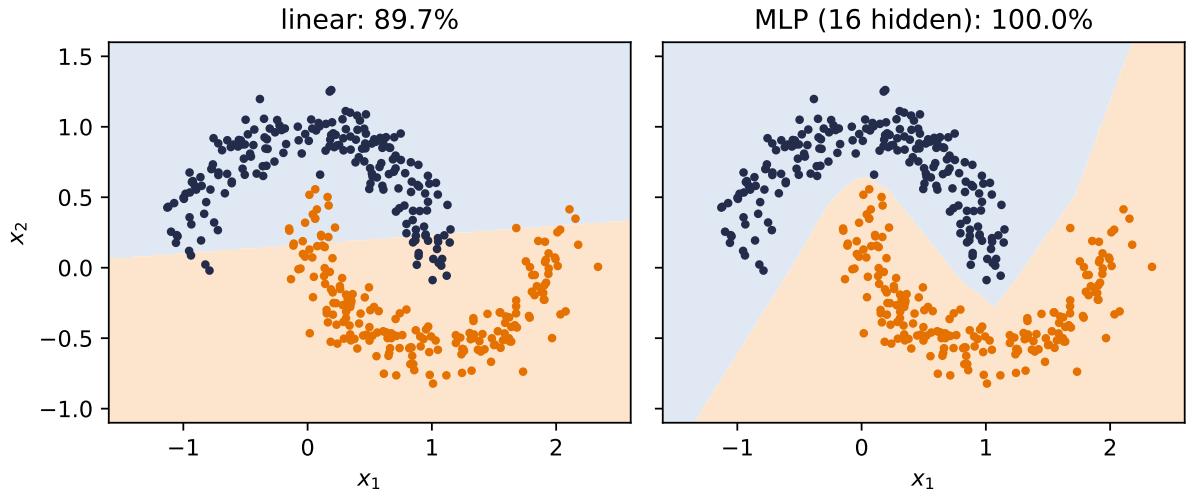

Figure 3.5.: Two moons: the linear model does what it can with one line (about 90% here); the MLP bends its boundary through the gap and separates the classes completely.

One honest footnote on the XOR network: in principle *two* hidden units suffice, but such minimal networks are temperamental; across random seeds they frequently get stuck at 50% accuracy with silent units (you will verify this in Exercise 3). A few spare units cost little and improve reliability. Chapter 4 will examine when optimization noise helps exploration, without treating it as a guaranteed escape.

3.7. A cautionary tale, and a look ahead

There is a seductive story about what the layers of an MLP learn: the first layer detects edges, the next combines them into shapes, the next into objects — a tidy hierarchy of increasingly abstract features. It is the *idealized* story, and for fully-connected networks on raw images it is mostly **not what happens**. A fully-connected layer sees its input as one long, structureless vector; it has no notion that pixel potentially relates to its neighbor. Nothing in the architecture encourages the tidy hierarchy, and with every pixel connected to every unit, the parameter count explodes long before the hierarchy emerges.

We now hold real expressive power — networks that can, in principle, represent almost anything. Chapter 4 trains them at scale and Chapter 5 supplies their gradients. And then Chapter 6 asks this chapter's uncomfortable question: what happens when all that flexibility meets data it does not understand the *structure* of? The answer — watch an MLP fail, in pictures, then fix it by building knowledge into the architecture — is the turn the whole book pivots on.

3.8. Okay, so — one bend changed everything

1. **Linear stacks collapse** (Equation 3.1); XOR proves four points can defeat any line.

2. **A neuron is a thresholded similarity score**: below the boundary silent, above it proportional. One neuron → one boundary; a layer → a polyhedral partition; three hinges → a compact bump.
3. **Bumps tile functions** (universal approximation) — but the theorem promises existence, not learnability, economy, or generalization.
4. **Depth bends boundaries**: suitable deep constructions can create region counts that grow exponentially with depth, yielding large efficiency gaps relative to comparable shallow constructions.
5. **The MLP is a class now** (`nn.Module`, foundation first), and it separates what no line can.

Sources and further reading

- Cybenko, "Approximation by Superpositions of a Sigmoidal Function" (1989) gives a foundational uniform universal- approximation result on compact domains.
- Hornik, "Approximation Capabilities of Multilayer Feedforward Networks" (1991) makes the distinction between uniform and L_p approximation explicit, which is exactly the jump-function nuance in this chapter.
- Nair and Hinton, "Rectified Linear Units Improve Restricted Boltzmann Machines" (2010) is an early empirical account of rectifiers preserving useful signal through multiple layers.
- Montúfar et al., "On the Number of Linear Regions of Deep Neural Networks" (2014) develops the architecture-dependent region-count argument behind the chapter's case for depth.

Exercises

1. **(Pencil.)** Generalize Equation 3.1: show by induction that any stack of L linear layers equals a single linear layer. Where exactly does inserting ReLU between layers break your proof?
2. **(Pencil.)** Using the bump construction, give explicit weights for a one-hidden-layer network that outputs approximately 1 on $[0, 1]$ and 0 outside $[-0.2, 1.2]$. How many hidden units did you need?
3. **(Code.)** Retrain the XOR network with `hidden_dim = 2` across ten different seeds. How often does it reach 100%? Inspect a failed run: what happened to its hidden units? (Hint: check how many are permanently silent.)
4. **(Code.)** In the polytope figure, count distinct activation patterns as you vary width at depth 1 versus depth at width 4 (keep total units comparable). Which grows the region count faster?
5. **(Code.)** Replace the moons MLP's ReLU with `nn.Identity()` and retrain. Explain the resulting boundary in one sentence using Equation 3.1.

4. Training: Loss and Stochastic Gradient Descent

We now have models worth training: linear regressors (Chapter 1), classifiers (Chapter 2), and multilayer perceptrons (Chapter 3). We have losses that tell each of them how bad they are, and we know the direction of improvement is the negative gradient. What remains is the question every practitioner faces daily: *how, exactly, do you run the descent* when the dataset has a million examples, the model has millions of knobs, and the loss landscape is no longer a friendly bowl?

This chapter is about the workhorse answer, stochastic gradient descent, and the small set of refinements (learning rates, batch sizes, momentum, Adam) that turn it from a theoretical idea into the algorithm that trains essentially everything in this book.

4.1. Where losses come from, one more time

Before optimizing a loss, remember why it is *that* loss. In Chapter 1 we saw that assuming Gaussian noise on a linear process makes maximum likelihood collapse into least squares; in Chapter 2 the same argument with a categorical distribution produced cross-entropy. This is the general pattern: **a loss function is a noise model in disguise.** Choose how you believe the data deviates from your model, and maximum likelihood hands you the loss. The loss is how we tell the model it is doing badly → choosing it well is not a detail, it is the supervision itself.

With the loss fixed, training is one line:

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} \; \mathcal{L}(\mathbf{w}) = \arg\min_{\mathbf{w}} \; \frac{1}{n} \sum_{i=1}^{n} \ell_i(\mathbf{w}), \tag{4.1}$$

where ℓ_i is the loss on example i. Everything that follows is about minimizing this average when n is enormous.

4.2. The computational bottleneck

Gradient descent, as we left it in Chapter 1, updates with the *full* gradient:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\mathbf{w}^{(t)}). \tag{4.2}$$

Look at the cost. One step touches every example; modern datasets have millions to billions of them, modern models have millions to billions of parameters, and training needs thousands of steps. One exact gradient per step is a luxury we cannot afford → we need to change strategy to scale up.

4.3. Stochastic gradient descent

The idea is almost cheeky: we do not need the exact gradient. A good approximation is enough. Instead of the expensive sum over all n examples, average over a small random **mini-batch** $\mathcal{B}$ — think 32 examples against a million:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla \ell_i(\mathbf{w}^{(t)}). \tag{4.3}$$

Why does this work? At a *fixed* parameter vector, a uniformly sampled mini-batch gradient is **unbiased**: the expected contribution of example i to a batch equals its contribution to the full average, so

$$\mathbb{E}\big[\nabla \mathcal{L}_{\mathcal{B}}(\mathbf{w})\big] = \nabla \mathcal{L}(\mathbf{w}). \tag{4.4}$$

This statement holds for independent draws with replacement and also for one uniformly chosen subset without replacement. In expectation, that cheap gradient points where the expensive gradient points. Each estimate is noisy; its average direction is right.

In practice the algorithm is:

1. Shuffle the training data.
2. Split it into mini-batches of size B.
3. For each batch: compute the average gradient, update the parameters.
4. One pass through all batches is an **epoch**; repeat for multiple epochs.

> **Honest fine print**
>
> There are two different facts hiding under the word "without replacement." A single uniform subset, evaluated at a fixed $\mathbf{w}$, is still unbiased and has *less* variance than with-replacement sampling. Its variance includes the finite-population correction $(n - B)/(n - 1)$, which reaches zero at $B = n$.
> Random reshuffling is subtler. After the first batch changes $\mathbf{w}$, the next batch comes from the remaining examples and is statistically tied to the earlier path. It is not an independent unbiased draw of the full gradient at this *new* iterate. Dedicated random-reshuffling theory handles that dependence; the one-line proof in Equation 4.4 does not. Keep the caveat in mind; keep using the shuffle.

4.4. The two zones of SGD

The noise gives SGD a characteristic two-phase behavior, and it is worth seeing rather than just hearing about. Far from the optimum, almost any batch tells you roughly the same thing → rapid, consistent progress. Close to the optimum, different batches start to disagree ("go left" — "no, go right"), and the iterate bounces around in what we will call the **region of confusion**:

```
import torch
import numpy as np
import matplotlib.pyplot as plt

torch.manual_seed(6050)
x1 = torch.randn(80)
y1 = 2.5 * x1 - 1.0 + 0.4 * torch.randn(80)

def descend(
    batch: int | None, lr: float = 0.12, steps: int = 60
) -> np.ndarray:
    w, b, path = torch.tensor(-0.5), torch.tensor(2.0), []
    for _ in range(steps):
        idx = torch.randperm(80)[:batch] if batch else slice(None)
        err = (w * x1[idx] + b) - y1[idx]
        path.append((float(w), float(b)))
        w = w - lr * 2 * (err * x1[idx]).mean()
        b = b - lr * 2 * err.mean()
    return np.array(path)

W, B = np.meshgrid(np.linspace(-1.5, 5.5, 90), np.linspace(-4, 3, 90))
L = ((y1.numpy()[None, None, :] -
      (W[..., None] * x1.numpy() + B[..., None])) ** 2).mean(-1)

plt.figure(figsize=(6.2, 4))
plt.contour(W, B, L, levels=25, cmap="Blues", alpha=0.7)
for batch, color, label in [(None, "#232D4B", "full batch"),
                            (4, "#E57200", "SGD, B = 4")]:
    p = descend(batch)
    plt.plot(p[:, 0], p[:, 1], "o-", ms=2.5, lw=1.2, color=color, label=label)
plt.plot(2.5, -1.0, "k*", ms=13, label="truth")
plt.xlabel("$w$"); plt.ylabel("$b$"); plt.legend(loc="upper right")
plt.tight_layout(); plt.show()
```

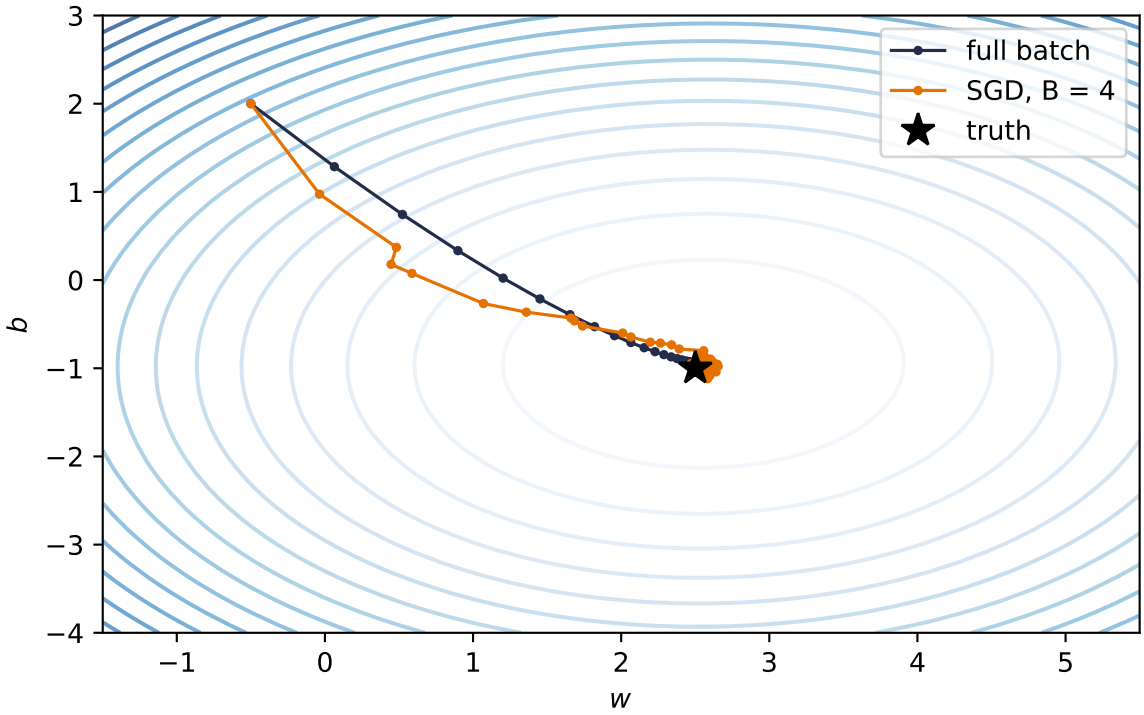

Figure 4.1.: Full-batch descent (blue) against mini-batch SGD (orange, $B = 4$) on the same landscape. Far out, the noisy steps agree with the true direction; near the optimum they scatter — the region of confusion.

Here is the surprise: the bouncing is not merely tolerable; in some regimes it is a **feature**. A noisy step may perturb the iterate away from a saddle, a narrow basin, or a shallow trap that exact descent would follow more predictably. Mini-batch noise can also bias training toward different solutions, and smaller batches sometimes improve generalization. None of these outcomes is guaranteed: the effect depends on the model, data, learning rate, and batch size. We will give that evidence its proper treatment in Chapter 6. For now, treat noise as part of the algorithm, not automatically as a defect or a cure.

4.5. The learning rate

One knob dominates all others. The learning rate α scales every step, and its failure modes are asymmetric: too small wastes your compute budget crawling; too large overshoots the valley and diverges.

```
def losses_for(lr: float, steps: int = 60) -> list[float]:
    w, b, out = torch.tensor(-0.5), torch.tensor(2.0), []
    for _ in range(steps):
        err = (w * x1 + b) - y1
        out.append(float((err ** 2).mean()))
```

```
        w = w - lr * 2 * (err * x1).mean()
        b = b - lr * 2 * err.mean()
    return out

plt.figure(figsize=(6.2, 3.4))
for lr, style, color in [(0.005, "-", "#5379AA"), (0.12, "-", "#E57200"),
                         (1.1, "--", "#722F37")]:
    plt.semilogy(losses_for(lr), style, color=color, label=f"$\\alpha = {lr}$")
plt.xlabel("step"); plt.ylabel("loss (log scale)")
plt.legend(); plt.tight_layout(); plt.show()
```

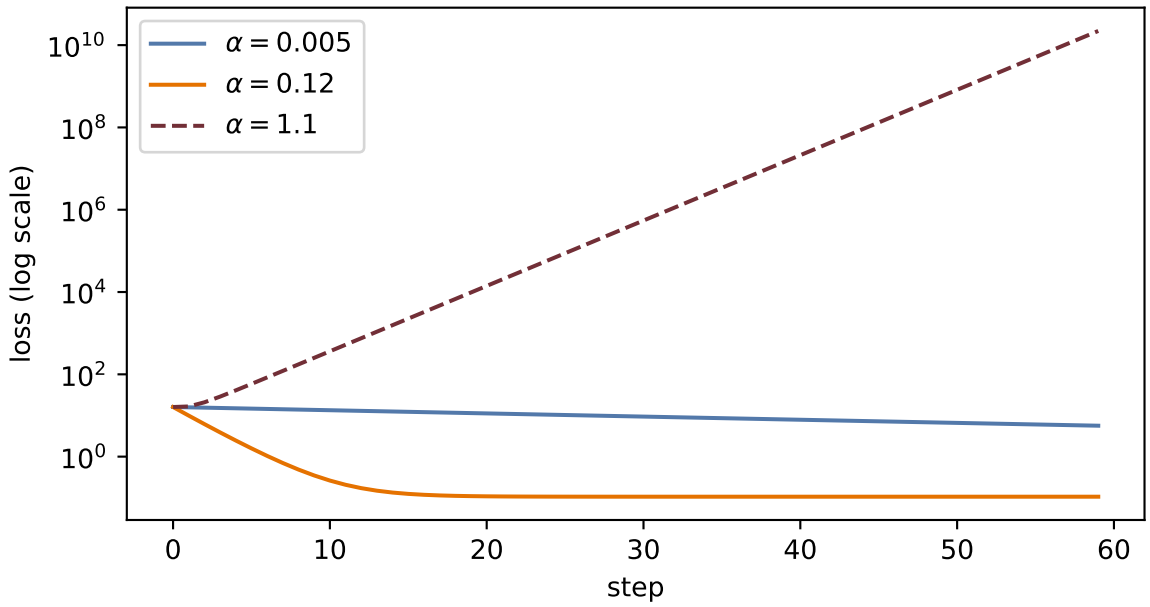

Figure 4.2.: Same problem, same steps, three learning rates. Too small crawls; too large overshoots back and forth and climbs; the middle one converges quickly.

Rule of thumb from the lectures

For the normalized toy problems in these lectures, plain SGD often starts around $\alpha = 0.1$. That is a scale-calibrated starting point, not a universal constant; Adam, unnormalized features, and very deep models can require very different values. Read the training curves: crawling → consider raising it; oscillating or exploding → lower it. Later in training it often pays to *decay* the learning rate so the fine-tuning steps get smaller; that is a **learning-rate schedule**. One exotic-sounding relative, *warmup* (starting tiny and ramping up), will matter enormously when we train Transformers in Chapter 14.

4.6. The batch size

The batch size B sets where you live on the noise–efficiency trade-off:

Batch size	Character	Trade-off
Small (1–32)	Noisy, exploratory	Sometimes a generalization benefit; slow and less hardware-efficient
Medium (32–256)	Balanced	The usual default; hardware-efficient
Large (256+)	Smooth, high throughput	Less gradient noise; often needs learning-rate and schedule retuning

With independent sampling, gradient standard deviation scales like $1/\sqrt{B}$: quadrupling the batch roughly halves the jitter. For a uniform subset without replacement, multiply by $\sqrt{(n-B)/(n-1)}$; when $B = n$, the noise is exactly zero. For $B \ll n$ the correction is near one, which is why the simpler rule is useful. There is no universal best setting here, only a dial linking statistics and hardware.

4.7. Momentum: give the ball some mass

Vanilla SGD has exactly one control, α, and in narrow, curved valleys that is not enough: the iterate zigzags across the steep direction while inching along the shallow one. The fix is to give the update a memory. Keep a running **velocity** that accumulates gradients, and move along the velocity instead of the raw gradient:

$$\mathbf{v}^{(t)} = \beta\,\mathbf{v}^{(t-1)} + \nabla\mathcal{L}(\mathbf{w}^{(t)}), \qquad \mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha\,\mathbf{v}^{(t)}, \tag{4.5}$$

with $\beta \in [0.9, 0.99]$. This is the ball rolling downhill: in directions where gradients keep agreeing, speed builds; in directions where they flip sign every step, they cancel. The zigzag damps itself, and small bumps in the landscape get rolled through rather than obeyed.

4.8. Adam: adaptive steps per knob

Momentum treats every parameter alike, but some knobs sit on steep cliffs while others sit on plains. **Adam** (adaptive moment estimation) combines three ideas from the lecture: momentum (accumulate past gradients), per-parameter adaptive learning rates (scale each knob's step by its own gradient history), and bias correction (account for both accumulators starting at zero). Formally, with elementwise operations:

$$\mathbf{m}^{(t)} = \beta_1 \mathbf{m}^{(t-1)} + (1-\beta_1)\,\mathbf{g}^{(t)}, \quad \mathbf{s}^{(t)} = \beta_2 \mathbf{s}^{(t-1)} + (1-\beta_2)\,\mathbf{g}^{(t)\,2}, \quad \mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha\,\frac{\hat{\mathbf{m}}^{(t)}}{\sqrt{\hat{\mathbf{s}}^{(t)}} + \epsilon}, \tag{4.6}$$

where $\hat{\mathbf{m}}, \hat{\mathbf{s}}$ are the bias-corrected accumulators. When to use what, per the lecture: **SGD with momentum** is simple, reliable, and strong for large models; **Adam** is the good default that works out of the box.

4.9. The race, on a problem where we know the truth

Claims about optimizers deserve a test you can rerun. We build a least-squares problem with a deliberately *ill-conditioned* landscape; the second feature is ten times the scale of the first, so the loss surface is a long narrow valley. We give all three optimizers the same 120-step, 16-example-batch budget, while using learning rates chosen for this toy: 0.003 for SGD and momentum, 0.1 for Adam. This is a mechanism comparison, not a controlled claim that one optimizer wins at a shared learning rate.

The cell also uses `backward()` as the framework instrument previewed in Chapter 1. It supplies gradients for the race; Chapter 5 opens that box next.

```
torch.manual_seed(6050)
n = 256
Xr = torch.randn(n, 2) * torch.tensor([1.0, 10.0])   # ill-conditioned by design
w_true = torch.tensor([3.0, 0.3])
yr = Xr @ w_true + 0.1 * torch.randn(n)

def race(kind: str, steps: int = 120, B: int = 16) -> list[float]:
    torch.manual_seed(1)                              # same batches for everyone
    w = torch.zeros(2, requires_grad=True)
    opt = {"SGD":      torch.optim.SGD([w], lr=3e-3),
           "momentum": torch.optim.SGD([w], lr=3e-3, momentum=0.9),
           "Adam":     torch.optim.Adam([w], lr=0.1)}[kind]
    hist = []
    for _ in range(steps):
        idx = torch.randint(0, n, (B,))
        opt.zero_grad()
        ((Xr[idx] @ w - yr[idx]) ** 2).mean().backward()
        opt.step()
        hist.append(((Xr @ w.detach() - yr) ** 2).mean().item())
    return hist

plt.figure(figsize=(6.2, 3.6))
for kind, color in [("SGD", "#5379AA"), ("momentum", "#232D4B"),
                    ("Adam", "#E57200")]:
    plt.semilogy(race(kind), color=color, label=kind)
```

```
plt.xlabel("step"); plt.ylabel("full-batch loss (log scale)")
plt.legend(); plt.tight_layout(); plt.show()
```

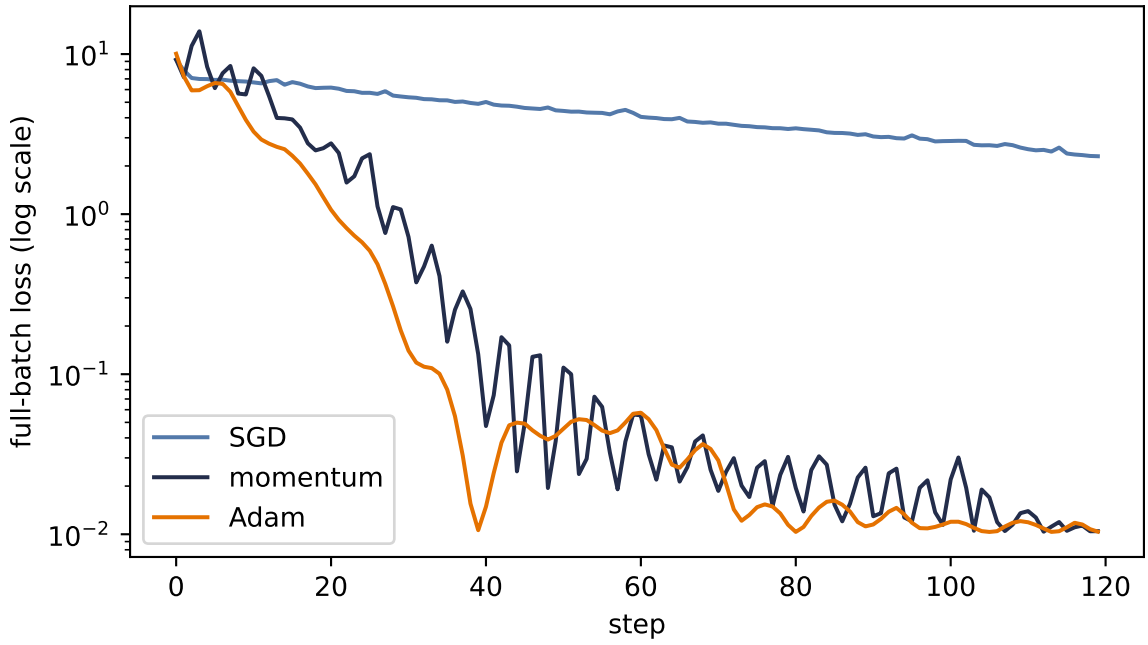

Figure 4.3.: The optimizer race on an ill-conditioned valley (feature scales 1 and 10), with an equal step budget and optimizer-appropriate learning rates (0.003 for SGD/momentum, 0.1 for Adam). Plain SGD crawls along the shallow direction; momentum damps the zigzag; Adam's per-parameter scaling largely neutralizes the conditioning.

Read the result honestly. This is one problem, chosen to expose conditioning; it is not proof that Adam always wins (it does not — plain SGD with momentum, well tuned, still trains many of the best vision models). What the race does show is *mechanism*: when different directions of the landscape have wildly different steepness, per-direction memory (momentum) and per-parameter scaling (Adam) buy you orders of magnitude. It also whispers a practical lesson from the live session: much of this valley's narrowness came from unscaled features → *normalize your inputs* and you fight the optimizer less (Exercise 5).

4.10. Two regularizers that live inside training

Two ideas from Chapter 1 reappear here as training-loop citizens rather than loss-function algebra.

Weight decay. Under plain SGD, adding an L_2 penalty is equivalent (up to whether the coefficient is written as λ or $\lambda/2$) to one extra shrinkage term: $\mathbf{w} \leftarrow (1 - \alpha\lambda)\mathbf{w} - \alpha\nabla\mathcal{L}$. This is the ridge connection from Chapter 1. Many PyTorch optimizers expose a `weight_decay` argument, but the equivalence needs a boundary: coupled L_2 inside an adaptive optimizer such

as Adam is rescaled by that optimizer and is not the same trajectory as multiplicative shrinkage. **AdamW** decouples the decay step explicitly. Also, training recipes often exempt bias and normalization parameters rather than shrinking every parameter indiscriminately.

Early stopping. Track the validation loss during training and stop when it turns upward, even though the training loss is still falling:

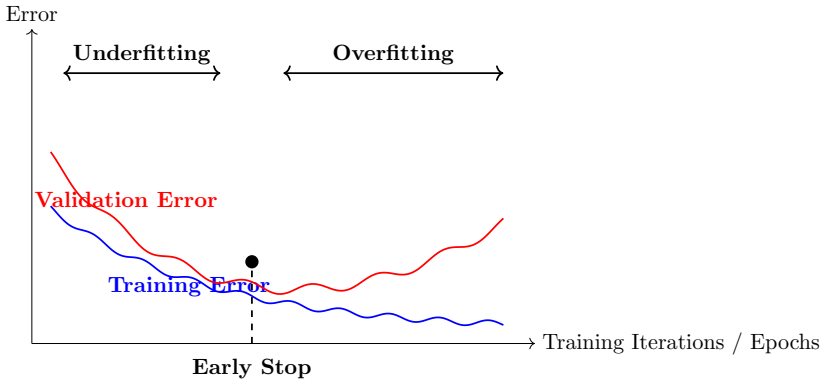

Figure 4.4.: Training error keeps falling; validation error turns. Stopping at the turn is regularization by clock — no penalty term required.

Both are cheap, both are everywhere, and both are previews: the full story of *why* constraining training improves generalization is the business of Chapter 6.

4.11. Okay, so — the training loop

Assemble the chapter into the loop you will run, in some form, for the rest of the book:

1. Start from (well-initialized) random parameters.
2. For each epoch: shuffle, split into mini-batches of size B.
3. For each batch: forward pass → loss (your noise model in disguise) → backward pass for the gradients → optimizer step (α, momentum or Adam, weight decay).
4. Watch the validation curve; schedule the learning rate; stop early when it turns.

One box in that loop is still magic: "backward pass for the gradients." Computing millions of partial derivatives at the cost of roughly one extra forward pass is the subject of Chapter 5.

Sources and further reading

- Robbins and Monro, "A Stochastic Approximation Method" (1951) is the classical starting point for replacing exact quantities with noisy observations in an iterative procedure.
- Sutskever et al., "On the Importance of Initialization and Momentum in Deep Learning" (2013) shows how initialization and momentum interact with curvature in deep-network training.

- Kingma and Ba, "Adam: A Method for Stochastic Optimization" (2015) gives the algorithm, bias corrections, and motivation for adaptive moment estimates.
- Loshchilov and Hutter, "Decoupled Weight Decay Regularization" (2019) explains precisely why coupled L_2 regularization and weight decay separate under adaptive optimizers.

Exercises

1. **(Pencil.)** Prove Equation 4.4 for sampling with replacement: if i is drawn uniformly from $\{1, \dots, n\}$, show $\mathbb{E}[\nabla \ell_i(\mathbf{w})] = \frac{1}{n} \sum_j \nabla \ell_j(\mathbf{w})$. Where exactly does the argument use uniformity?
2. **(Code.)** In the two-zones figure, sweep $B \in \{1, 4, 16, 80\}$ and plot the final 30 steps of each path. How does the radius of the region of confusion scale with B? Compare against $1/\sqrt{B}$ first, then include the finite-population correction. Why must the noise vanish at $B = 80$?
3. **(Code.)** Add a learning-rate schedule to the race: halve α every 30 steps for plain SGD. How much of the gap to momentum does scheduling close? What does that tell you about what momentum is *really* fixing here?
4. **(Code.)** Sweep momentum's $\beta \in \{0, 0.5, 0.9, 0.99\}$ in the race. Explain the failure mode at the top end using the ball analogy.
5. **(Code.)** Standardize the race's features (divide each column of `Xr` by its standard deviation) and rerun all three optimizers. How much of Adam's advantage evaporates? State the practical moral in one sentence.

5. Backpropagation

Recall the update rule that ended Chapter 1: new parameters come from old parameters by stepping against the gradient, $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} \mathcal{L}$. For linear regression we could write that gradient down by hand. But a modern network has millions, sometimes billions, of knobs, tangled through layer after layer of composed functions → computing each partial derivative naively, one at a time, is hopeless.

Backpropagation is the algorithm that computes *all* of them, exactly, in one backward sweep whose cost is a small constant multiple of the graph's elementary operations. It is not a new kind of calculus. It is the chain rule, *organized*. By the end of this chapter you will have derived it, implemented it by hand, rebuilt it as a tiny scalar autograd engine, and then checked that `torch.autograd` agrees with you to machine precision.

5.1. One neuron, one chain

Start with the smallest possible case: one neuron on one training example. The previous activation $a^{(l-1)}$ is multiplied by a weight w, giving the pre-activation $z = w\,a^{(l-1)} + b$; the activation function produces $a = \sigma(z)$; and a feeds a loss L.

Now turn the knob w a little. How much does L change? Follow the chain: turning w changes z; the change in z changes a; the change in a changes L. The chain rule multiplies these local sensitivities:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}. \tag{5.1}$$

```
import matplotlib.pyplot as plt
import matplotlib.patches as mp

fig, ax = plt.subplots(figsize=(7.2, 2.4))
nodes = {r"$w$": 0, r"$z = w a^{(l-1)} + b$": 1, r"$a=\sigma(z)$": 2, r"$L$": 3}
for label, i in nodes.items():
    ax.add_patch(mp.FancyBboxPatch((i * 2.1, 0), 1.5, 0.7, boxstyle="round,pad=0.08",
                                   fc="#E8EEF7", ec="#232D4B", lw=1.4))
    ax.text(i * 2.1 + 0.75, 0.35, label, ha="center", va="center", fontsize=11)
fwd = dict(arrowstyle="-|>", color="#5379AA", lw=1.8)
bwd = dict(arrowstyle="-|>", color="#E57200", lw=1.8)
lbl = [r"$\frac{\partial z}{\partial w}$", r"$\frac{\partial a}{\partial z}$",
       r"$\frac{\partial L}{\partial a}$"]
```

```
for i in range(3):
    ax.annotate("", xy=(2.1 * (i + 1) - 0.1, 0.55), xytext=(2.1 * i + 1.6, 0.55),
                arrowprops=fwd)
    ax.annotate("", xy=(2.1 * i + 1.6, 0.12), xytext=(2.1 * (i + 1) - 0.1, 0.12),
                arrowprops=bwd)
    ax.text(2.1 * i + 1.85, -0.28, lbl[i], ha="center", fontsize=12, color="#E57200")
ax.set_xlim(-0.3, 8.3); ax.set_ylim(-0.7, 1.1); ax.axis("off")
plt.tight_layout(); plt.show()
```

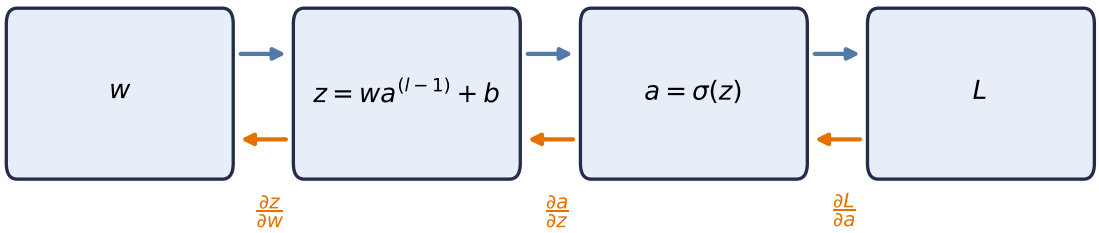

Figure 5.1.: Computation as a graph: the forward pass (top, blue) computes and caches values; the backward pass (bottom, orange) multiplies local derivatives along the same edges, in reverse.

Two things about this picture generalize to any network, however deep:

1. **The forward pass caches.** Computing L produces the intermediate values (z, a) along the way; we keep them, because the backward pass will need them.
2. **The backward pass reuses.** Each edge contributes one *local* derivative, and the derivative of L with respect to anything is the product of local derivatives along the path back to it. Nothing is recomputed → the whole sweep costs about as much as one forward pass.

5.2. The game of blame

For a full layer of neurons the bookkeeping threatens to become a tangled mess of sums. The cure is to pick the right intermediate quantity and track only that. At the output of the network the error is obvious: it is just the gap to the target. What is *not* obvious is how much of that error is the fault of a weight buried ten layers deep. Backpropagation sends this blame backward, from where it is obvious to where it is not.

The blame signal

Define, for each layer l, the **blame signal**

$$\delta^{(l)} := \frac{\partial L}{\partial \mathbf{z}^{(l)}},$$

the sensitivity of the loss to that layer's *pre-activations*. It answers: how much did each neuron's z contribute to the final loss?

Everything now unfolds in four short equations. With $\odot$ denoting elementwise (Hadamard) product:

1. Start where blame is obvious (output layer L):

$$\delta^{(L)} = \frac{\partial L}{\partial \mathbf{a}^{(L)}} \odot \sigma'(\mathbf{z}^{(L)}). \tag{5.2}$$

For the half-squared-error convention $L = \frac{1}{2}\|\mathbf{a}^{(L)} - \mathbf{y}\|^2$, $\partial L/\partial \mathbf{a}^{(L)}$ is exactly the prediction error $\mathbf{a}^{(L)} - \mathbf{y}$. Other conventions contribute a known scale: mean squared error, for example, contributes $2/n$ for n scalar outputs. In every case the output error is then gated by how sensitive the activation was to its input.

2. Propagate blame one layer back:

$$\delta^{(l)} = \left((\mathbf{W}^{(l+1)})^\top \delta^{(l+1)}\right) \odot \sigma'(\mathbf{z}^{(l)}). \tag{5.3}$$

Blame flows backward through the same weights the signal flowed forward through, hence the transpose; then the local gate $\sigma'(\mathbf{z}^{(l)})$ scales it. This recursion runs from the last layer to the first.

```
import torch
import matplotlib.pyplot as plt

torch.manual_seed(6050)
sizes = [3, 4, 4, 1]
Ws = [torch.randn(sizes[i + 1], sizes[i]) for i in range(3)]

x = torch.randn(3)                          # forward pass, cached
zs, a = [], x
for W in Ws[:-1]:                           # hidden layers: sigmoid
    zs.append(W @ a)
    a = torch.sigmoid(zs[-1])
zs.append(Ws[-1] @ a)                       # identity output, as in our 2-layer build

delta = [None] * 3                          # backward pass: eqs. 1 and 2
sp = lambda z: torch.sigmoid(z) * (1 - torch.sigmoid(z))
delta[2] = zs[2] - 1.0                                  # eq. 1 (identity output, y = 1)
```

```
delta[1] = (Ws[2].T @ delta[2]) * sp(zs[1])          # eq. 2
delta[0] = (Ws[1].T @ delta[1]) * sp(zs[0])          # eq. 2 again

fig, ax = plt.subplots(figsize=(7, 3.2))
blames = [torch.full((3,), float("nan"))] + delta    # input layer: no blame needed
vmax = max(d.abs().max() for d in delta)
for col, (n_units, bl) in enumerate(zip(sizes, blames)):
    ys = torch.linspace(-1, 1, n_units + 2)[1:-1]
    mag = bl.abs() if col > 0 else torch.zeros(n_units)
    ax.scatter([col] * n_units, ys, s=200 + 2200 * mag / vmax,
               c=mag, cmap="Oranges", vmin=0, vmax=float(vmax),
               edgecolors="#232D4B", zorder=3)
    if col > 0:                                       # backward arrows
        prev = torch.linspace(-1, 1, sizes[col - 1] + 2)[1:-1]
        for yj in ys:
            for yi in prev:
                ax.plot([col, col - 1], [yj, yi], color="#B8B8B8",
                        lw=0.5, zorder=1)
ax.annotate("blame flows this way", xy=(0.4, 1.25), xytext=(2.2, 1.25),
            arrowprops=dict(arrowstyle="-|>", color="#E57200", lw=2),
            color="#E57200", va="center")
ax.set_xticks(range(4))
ax.set_xticklabels(["input", r"$\delta^{(1)}$", r"$\delta^{(2)}$", r"$\delta^{(3)}$"])
ax.set_yticks([]); ax.set_ylim(-1.4, 1.6)
for s in ax.spines.values(): s.set_visible(False)
plt.tight_layout(); plt.show()
```

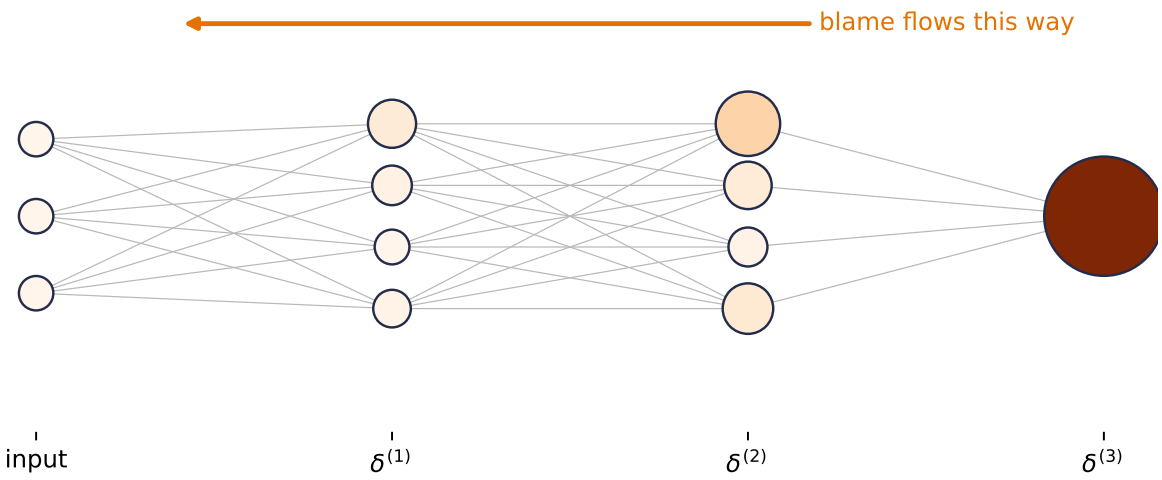

Figure 5.2.: Equations 1 and 2 in action on a real network (sizes 3–4–4–1, random weights, one example). Node size and color show each neuron's blame $|\delta_j|$: it starts at the output, where the error is obvious, and flows backward through $W^\top$, shrinking wherever a sigmoid gate is nearly closed.

3. Convert blame into weight gradients:

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} \, (\mathbf{a}^{(l-1)})^\top. \tag{5.4}$$

4. And into bias gradients:

$$\frac{\partial L}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}. \tag{5.5}$$

Equation 5.4 deserves a pause, because its shapes tell the story. With m neurons in layer l and n in layer $l-1$: $\delta^{(l)}$ is $(m \times 1)$, and $(\mathbf{a}^{(l-1)})^\top$ is $(1 \times n)$. Their **outer product** is an $m \times n$ matrix, exactly the shape of $\mathbf{W}^{(l)}$, and entry (i, j) is the blame of neuron i times the activation of neuron j that fed it. The update for a weight is *blame out times signal in*:

```
d2 = delta[1]                                # layer-2 blame        (4,)
a1 = torch.sigmoid(zs[0])                    # layer-1 activations  (4,)
G = torch.outer(d2, a1)                      # eq. 3

fig, axes = plt.subplots(1, 3, figsize=(7.6, 2.6),
                         gridspec_kw={"width_ratios": [1, 4, 4.6]})
panels = [(d2[:, None], r"$\delta^{(2)}$"), (a1[None, :], r"$(a^{(1)})^\top$"),
          (G, r"$\partial L/\partial W^{(2)} = \delta^{(2)}(a^{(1)})^\top$")]
for ax, (M, title) in zip(axes, panels):
    ax.imshow(M, cmap="Oranges", vmin=-G.abs().max(), vmax=G.abs().max())
```

```
    for (i, j), v in [((i, j), M[i, j]) for i in range(M.shape[0])
                      for j in range(M.shape[1])]:
        ax.text(j, i, f"{v:.2f}", ha="center", va="center", fontsize=8)
    ax.set_title(title, fontsize=11); ax.set_xticks([]); ax.set_yticks([])
plt.tight_layout(); plt.show()
```

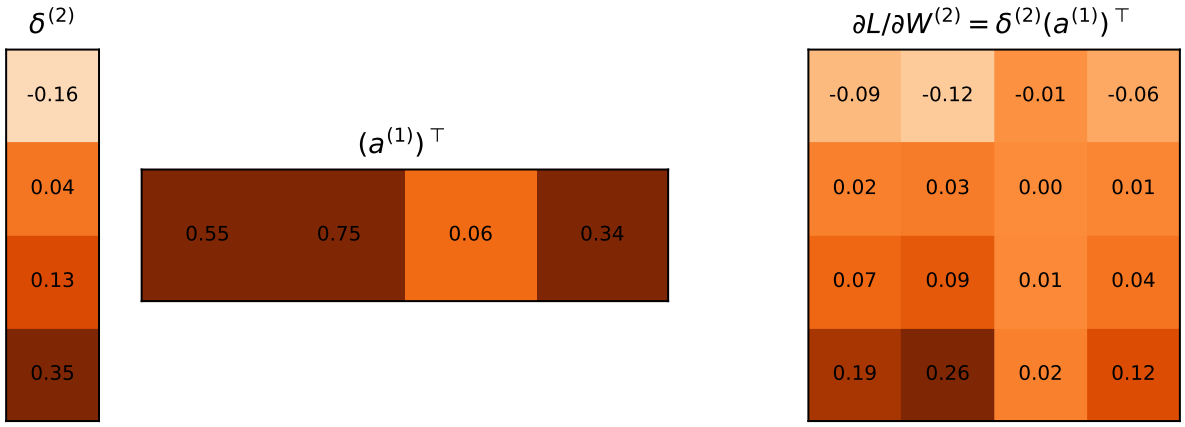

Figure 5.3.: Equation 3, pictured with the blame and activations from the network above: a (4×1) blame column times a (1×4) activation row yields the full (4×4) gradient matrix in one stroke — entry (i, j) is blame out $\times$ signal in.

No messy summations: the matrix form performs them all at once, and it maps directly onto the parallel matrix multiplies GPUs are built for. The same four equations govern a toy network and a billion-parameter model; only the matrix sizes change.

5.3. Build it, then check it against autograd

Let us implement the four equations on a two-layer network, with no autograd anywhere, and then ask `torch.autograd` to differentiate the same network. If our blame algebra is right, the two answers must agree to machine precision.

```
import torch

_ = torch.manual_seed(6050)
```

```
n, d, h = 32, 5, 8                                  # batch, input dim, hidden dim
X = torch.randn(n, d)
y = torch.randn(n)

W1, b1 = torch.randn(h, d) * 0.3, torch.zeros(h)
```

```
W2, b2 = torch.randn(1, h) * 0.3, torch.zeros(1)

# forward pass -- cache everything the backward pass will need
Z1 = X @ W1.T + b1                              # (n, h)
A1 = torch.sigmoid(Z1)                          # (n, h)
Z2 = A1 @ W2.T + b2                             # (n, 1)
L = ((Z2.squeeze() - y) ** 2).mean()

# backward pass -- the four equations (batch-averaged)
sig_prime = A1 * (1 - A1)                       # sigma'(z) = sigma(z)(1 - sigma(z))
delta2 = 2 * (Z2.squeeze() - y)[:, None] / n  # eq. 1 (identity output: no gate)
delta1 = (delta2 @ W2) * sig_prime              # eq. 2: blame through W^T, gated
grads = {                                       # eqs. 3 & 4: blame out x signal in
    "W2": delta2.T @ A1, "b2": delta2.sum(0),
    "W1": delta1.T @ X,  "b1": delta1.sum(0),
}

# same network, autograd's turn
params = {k: v.clone().requires_grad_(True) for k, v in
          {"W1": W1, "b1": b1, "W2": W2, "b2": b2}.items()}
A1_ = torch.sigmoid(X @ params["W1"].T + params["b1"])
L_ = (((A1_ @ params["W2"].T + params["b2"]).squeeze() - y) ** 2).mean()
L_.backward()

for k in grads:
    gap = (grads[k] - params[k].grad).abs().max()
    print(f"{k}: max |ours - autograd| = {gap:.2e}")
```

```
W2: max |ours - autograd| = 0.00e+00
b2: max |ours - autograd| = 0.00e+00
W1: max |ours - autograd| = 3.73e-09
b1: max |ours - autograd| = 1.86e-09
```

The largest discrepancy is 3.73×10^{-9}, consistent with float32 rounding. The blame equations *are* what autograd computes; the framework simply builds the computation graph for us as we go and then walks it backward, caching and reusing exactly as we did.

5.4. A tiny scalar autograd engine

To remove the last trace of magic, let us build the graph ourselves. Each `Value` node remembers how it was made (its parents and the local derivative rule); calling `backward()` walks the graph in reverse topological order, accumulating blame into every node's `grad`.

This teaching implementation is adapted from Andrej Karpathy's micrograd, released under the MIT License. We use only the scalar reverse-mode pattern needed for this derivation; the upstream copyright and license notice are preserved in the repository's third-party notices.

```
class Value:
    """A scalar that remembers its history -- a node in the computation graph."""

    def __init__(self, data: float, parents=(), backward_rule=lambda: None):
        self.data = data
        self.grad = 0.0
        self._parents = parents
        self._backward_rule = backward_rule

    def __add__(self, other: "Value | float") -> "Value":
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other))
        def rule():                         # d(out)/d(self) = d(out)/d(other) = 1
            self.grad += out.grad
            other.grad += out.grad
        out._backward_rule = rule
        return out

    def __mul__(self, other: "Value | float") -> "Value":
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other))
        def rule():                         # product rule, one side at a time
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
        out._backward_rule = rule
        return out

    def sigmoid(self) -> "Value":
        s = 1 / (1 + 2.718281828459045 ** (-self.data))
        out = Value(s, (self,))
        def rule():                         # sigma' = s (1 - s), the gate
            self.grad += s * (1 - s) * out.grad
        out._backward_rule = rule
        return out

    def backward(self) -> None:
        order, seen = [], set()
        def topo(v):                        # children before parents
            if v not in seen:
                seen.add(v)
                for p in v._parents:
                    topo(p)
                order.append(v)
```

```
        topo(self)
        self.grad = 1.0                  # dL/dL = 1: blame starts here
        for v in reversed(order):
            v._backward_rule()
```

```
w, x, b = Value(0.7), Value(2.0), Value(-0.5)
a = ((w * x) + b).sigmoid()              # one neuron, forward
loss = (a + (-0.3)) * (a + (-0.3))       # squared error vs target 0.3
loss.backward()

# analytic check: dL/dw = 2(a - y) * sigma'(z) * x
z = 0.7 * 2.0 - 0.5
s = 1 / (1 + 2.718281828459045 ** (-z))
print(f"micro-autograd: dL/dw = {w.grad:.6f}")
print(f"by hand:        dL/dw = {2 * (s - 0.3) * s * (1 - s) * 2.0:.6f}")
```

```
micro-autograd: dL/dw = 0.337801
by hand:        dL/dw = 0.337801
```

In a few dozen lines, it is the real thing: PyTorch's autograd is this idea with tensors instead of scalars, a compiled graph walker, and thousands of derivative rules.

5.5. `torch.autograd` in practice: five rules

Autograd is doing exactly what we just built by hand, and knowing that mechanics tells you why each of its rules exists. These five cover nearly every autograd bug you will meet; the live-session notebook walks through each in more depth.

Rule 1 — by default, only leaf tensors retain gradients. A *leaf* is a tensor you created directly (your parameters); a *non-leaf* is the result of an operation (activations, losses). After `backward()`, leaves have `.grad` populated while intermediate blame is used and released. Call `.retain_grad()` on a non-leaf before `backward()` only when you explicitly need its gradient for inspection.

Rule 2 — gradients accumulate. `backward()` *adds* into `.grad`. Zero them before each step (`optimizer.zero_grad()`), or every update uses the sum of all past gradients.

```
w = torch.randn(3, requires_grad=True)      # leaf: our parameter
y_hat = (w * 2).sum()                       # non-leaf: result of an operation

y_hat.backward()
print(f"leaf grad:      {w.grad.tolist()}")          # populated
print(f"non-leaf grad:  {y_hat.grad}")               # None -- rule 1
```

```
(w * 2).sum().backward()                    # backward again, WITHOUT zeroing
print(f"after 2nd pass: {w.grad.tolist()}")           # doubled -- rule 2
```

```
leaf grad:      [2.0, 2.0, 2.0]
non-leaf grad:  None
after 2nd pass: [4.0, 4.0, 4.0]
```

Rule 3 — parameter updates happen outside the graph. For a raw leaf tensor, the update `w = w - lr * w.grad` inside autograd's view records the update and rebinds `w` to a non-leaf, so its `.grad` will not be retained by default on the next pass. Assigning a plain tensor over a registered `nn.Parameter` may instead fail loudly. The safe idiom from the live session covers both cases:

```
with torch.no_grad():                 # temporarily stop recording
    w -= lr * w.grad                  # update in place; w stays a leaf
```

Rule 4 — in-place operations can corrupt the tape. Methods ending in `_` (`add_`, `relu_`) overwrite values that the backward pass may still need from the forward cache. Inside `no_grad()` parameter updates they are safe; anywhere else, prefer creating a new tensor.

Rule 5 — the graph lives until you let it go. Every forward pass builds a fresh graph, freed after `backward()`. But any tensor you keep (say, appending `loss` to a list for plotting) keeps its whole graph alive with it. Store `loss.item()` (a plain float) or `loss.detach()` instead: `detach()` returns the same value with the history cut.

> ⚠ Keep updates outside the graph
>
> Rules 1 and 3 combine into a classic raw-tensor bug: rebinding `w = w - lr * w.grad` creates a **new non-leaf**. Its `.grad` then comes back `None` by default on the next pass. A registered `nn.Parameter` assignment can raise an error instead. In either case, update the existing parameter under `torch.no_grad()`.

5.6. The gate, and the problem with deep chains

Look again at Equation 5.3. Every backward step multiplies by $\sigma'(\mathbf{z}^{(l)})$. That derivative acts as a **gate** on the blame: wide open, and the signal passes; nearly closed, and it dies.

Now examine the sigmoid's gate. From $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, its maximum value, at $z = 0$, is exactly 0.25; away from zero the neuron *saturates* and the gate approaches 0. The sigmoid gate is at best a quarter open, and often nearly closed.

```
import matplotlib.pyplot as plt

z = torch.linspace(-6, 6, 300)
sig = torch.sigmoid(z)
fig, axes = plt.subplots(1, 2, figsize=(7.4, 2.9), sharey=True)
axes[0].plot(z, sig * (1 - sig), color="#232D4B")
axes[0].axhline(0.25, ls="--", color="#E57200", lw=1)
axes[0].text(2.1, 0.26, "ceiling: 0.25", color="#E57200", fontsize=9)
axes[0].set_title(r"sigmoid gate $\sigma'(z)$"); axes[0].set_xlabel("$z$")
axes[1].plot(z, (z > 0).float(), color="#232D4B")
axes[1].set_title(r"ReLU gate"); axes[1].set_xlabel("$z$")
plt.tight_layout(); plt.show()
```

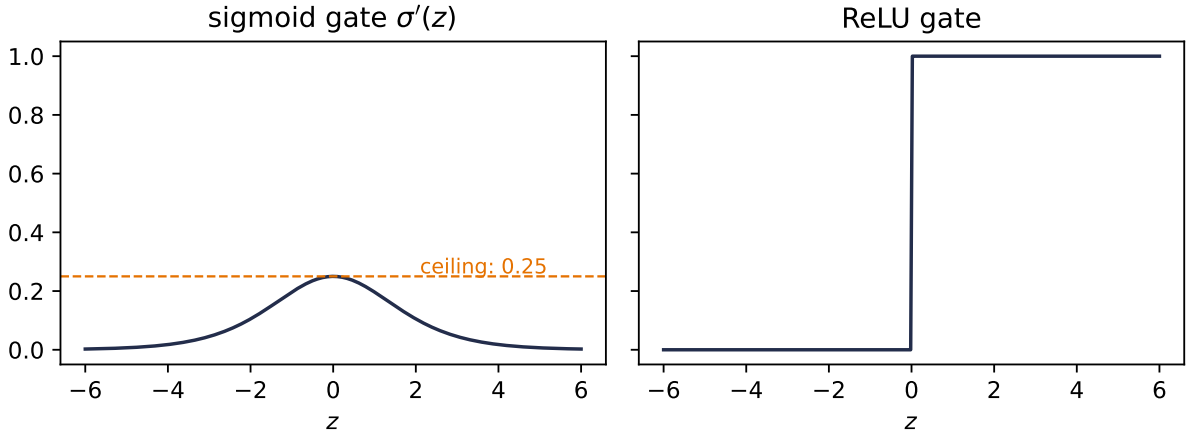

Figure 5.4.: Two gates. The sigmoid's derivative never exceeds 0.25 and vanishes under saturation; ReLU's is fully open (exactly 1) for every active neuron.

Here is the consequence, and it is one of deep learning's most important failure modes. The chain rule *multiplies* local Jacobians across layers. The activation-derivative part of a sigmoid path contributes at most 0.25 per layer, so that part is bounded by 0.25^k after k gates. Ten gates in, its ceiling is below one in a million. The weight matrices and branching paths also scale the full network Jacobian, so 0.25^k is not a bound on the complete gradient. It isolates why repeated sigmoid gates strongly favor vanishing signals.

The fix is almost embarrassingly simple. $\mathrm{ReLU}(z) = \max(0, z)$ has derivative 1 for $z > 0$ and 0 for $z < 0$. At the kink $z = 0$, PyTorch uses derivative 0; exact zeros can occur with zero biases or sparse activations. The gate is either fully open or fully closed. For every neuron that was active in the forward pass, the activation derivative contributes a factor of exactly one. The surrounding weight matrices can still scale the full signal, but the active ReLU does not shrink it: a **gradient superhighway** through the network. This, more than anything, is why ReLU and its variants are the default in deep networks.

Do not take the argument on faith; measure it. We build the same 30-layer network twice (sigmoid versus ReLU), give both copies the same He-scaled weights and input batch, and record

how much gradient reaches each layer. We compute the diagnostic in float64 so the norm itself does not underflow; then we ask the practical float32 question: would a learning-rate-0.1 update actually change the first layer's stored weights?

```
from torch import nn

def gradient_diagnostic(
    activation: type[nn.Module], depth: int = 30, width: int = 64
) -> dict[str, object]:
    torch.manual_seed(6050)
    layers = []
    for _ in range(depth):
        layers += [nn.Linear(width, width), activation()]
    net = nn.Sequential(*layers, nn.Linear(width, 1)).double()
    with torch.no_grad():
        for layer in net:
            if isinstance(layer, nn.Linear):
                nn.init.normal_(layer.weight, std=(2 / layer.in_features) ** 0.5)
                nn.init.zeros_(layer.bias)

    out = net(torch.randn(128, width, dtype=torch.float64)).mean()
    out.backward()
    linears = [layer for layer in net if isinstance(layer, nn.Linear)][:-1]
    first = linears[0]
    grad32 = first.weight.grad.float()
    weight32 = first.weight.detach().float()
    changed = ((weight32 - 0.1 * grad32) != weight32).sum().item()
    return {
        "norms": [layer.weight.grad.norm().item() for layer in linears],
        "first_max": first.weight.grad.abs().max().item(),
        "first_nonzero": grad32.count_nonzero().item(),
        "first_changed": changed,
        "first_size": grad32.numel(),
    }

diagnostics = {}
plt.figure(figsize=(6.2, 3.4))
for act, color in [(nn.Sigmoid, "#232D4B"), (nn.ReLU, "#E57200")]:
    diagnostics[act.__name__] = gradient_diagnostic(act)
    plt.semilogy(diagnostics[act.__name__]["norms"], "o-", ms=3,
                 color=color, label=act.__name__)
plt.xlabel("layer (0 = closest to input)")
plt.ylabel(r"$\|\partial L / \partial W^{(l)}\|$")
plt.legend(); plt.tight_layout(); plt.show()

for name, result in diagnostics.items():
    print(f"{name:7s}: max|g0|={result['first_max']:.2e}; "
```

```
        f"nonzero={result['first_nonzero']}/{result['first_size']}; "
        f"changed={result['first_changed']}/{result['first_size']}")
```

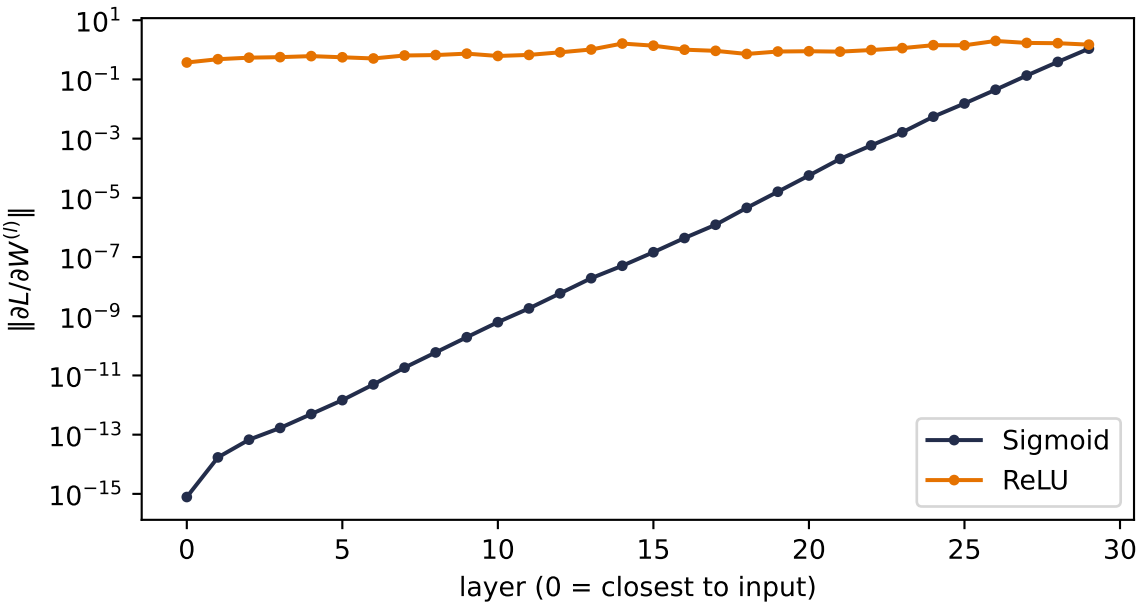

Figure 5.5.: Gradient reaching each layer of two matched 30-layer MLPs (float64 norm, log scale). With the same He-scaled weights and inputs, repeated sigmoid gates leave a nonzero but unusably small first-layer signal; ReLU preserves enough signal for a float32 update to change every first-layer weight.

```
Sigmoid: max|g0|=9.47e-17; nonzero=4096/4096; changed=0/4096
ReLU   : max|g0|=2.54e-02; nonzero=4096/4096; changed=4096/4096
```

Read the slopes, then read the update audit. The sigmoid first-layer entries are not mathematically or representationally zero: all 4,096 survive the float32 cast. They are so small, however, that subtracting $0.1g$ changes none of the stored float32 weights. The ReLU copy changes all 4,096. That distinction matters: a displayed float32 norm can underflow while individual entries remain nonzero, and nonzero gradients can still be too small to move a parameter at its current magnitude. ReLU is not magic, but its open gates make the signal usable in this matched experiment.

5.7. Initialization: setting the signal's energy

One more lever lives in this chapter, because it falls out of the same variance bookkeeping. Think of the network as a pipeline and the variance of the signal as its *energy*. If each layer multiplies the variance by more than one, activations explode after enough layers; by less than one, they vanish, and by Equation 5.3 the gradients follow. A good initialization keeps the energy roughly constant in both directions.

The analysis is one line. For $z = \sum_{i=1}^{n_{\text{in}}} w_i x_i$ with independent, zero-mean inputs and weights, $\operatorname{Var}(z) = n_{\text{in}} \operatorname{Var}(w) \operatorname{Var}(x)$. Keeping $\operatorname{Var}(z) = \operatorname{Var}(x)$ demands

$$\operatorname{Var}(w) = \frac{1}{n_{\text{in}}} \quad \text{(forward)}, \qquad \operatorname{Var}(w) = \frac{1}{n_{\text{out}}} \quad \text{(backward)}.$$

Xavier initialization splits the difference for symmetric activations like tanh and sigmoid, $\operatorname{Var}(w) = 2/(n_{\text{in}} + n_{\text{out}})$. **He initialization** accounts for ReLU discarding half its input distribution by doubling the forward recipe, $\operatorname{Var}(w) = 2/n_{\text{in}}$. PyTorch applies sensible defaults for you; the difference between a good and a careless choice is smooth training versus a dead or exploding network:

```
from collections.abc import Callable

def signal_std(
    scale_fn: Callable[[torch.Tensor, int], torch.Tensor],
    depth: int = 40, width: int = 256
) -> list[float]:
    x = torch.randn(512, width)
    stds = []
    for _ in range(depth):
        W = scale_fn(torch.randn(width, width), width)
        x = torch.relu(x @ W)
        stds.append(x.std().item())
    return stds

torch.manual_seed(6050)
schemes = {
    "naive  (std = 0.05)": lambda W, n: W * 0.05,
    "naive  (std = 0.12)": lambda W, n: W * 0.12,
    "He  (std = sqrt(2/n))": lambda W, n: W * (2 / n) ** 0.5,
}
plt.figure(figsize=(6.2, 3.4))
for (name, fn), color in zip(schemes.items(), ["#5379AA", "#722F37", "#E57200"]):
    plt.semilogy(signal_std(fn), color=color, label=name)
plt.xlabel("layer"); plt.ylabel("std of activations")
plt.legend(); plt.tight_layout(); plt.show()
```

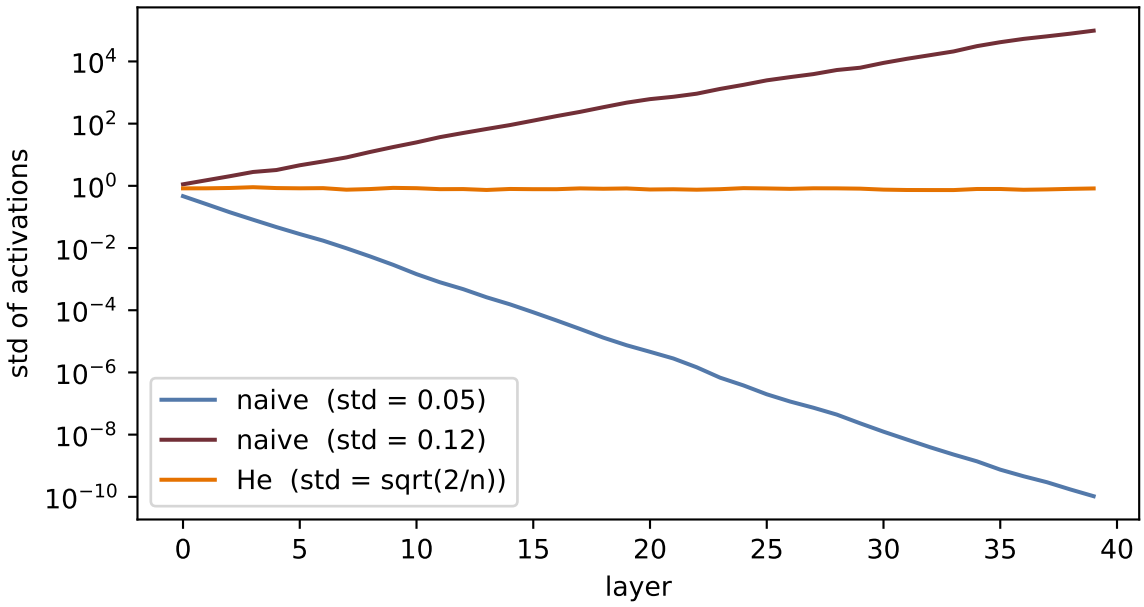

Figure 5.6.: Signal energy through 40 ReLU layers. Careless scales lose or amplify the signal exponentially; He initialization holds it steady.

Initialization gives a stable *start*. Keeping the signal stable *during* training, as the weights move, is the job of normalization layers; and for very deep networks even that is not enough, and gradients get an architectural bypass: skip connections that add a block's input straight to its output. Both belong to later chapters (normalization and the residual idea in Chapter 9, and layer normalization where it becomes essential, inside the Transformer of Chapter 14). What you should carry from here is the invariant they all serve: *keep the signal, forward and backward, at constant energy.*

5.8. Okay, so — the chain rule, organized

1. **The graph.** Any network is a computation graph; the forward pass computes and caches, the backward pass multiplies local derivatives in reverse.
2. **The blame signal.** $\delta^{(l)} = \partial L/\partial \mathbf{z}^{(l)}$ turns the tangle into four equations: start at the output (Equation 5.2), propagate through $\mathbf{W}^\top$ and the gate (Equation 5.3), then read off weight and bias gradients (Equation 5.4, Equation 5.5). The backward sweep costs a small constant multiple of the forward graph's operations, for *every* gradient at once.
3. **We verified it**: our hand-built equations match `torch.autograd` to 10^{-8}, and a tiny scalar `Value` class reproduces the machinery end to end.
4. **The gate rules the depth.** The sigmoid-derivative product along a path is bounded above by 0.25^k after k gates; ReLU's open gate is a gradient superhighway. Initialization sets the signal's energy so neither direction explodes or dies at the start.

Backpropagation is the engine under every remaining chapter. The next time a deep model fails to learn, your first questions now have names: *is the gradient reaching the early layers? what is*

gating it? what is its energy?

Sources and further reading

- Rumelhart, Hinton, and Williams, "Learning representations by back-propagating errors" (1986): the landmark short account of learning internal representations by reverse error propagation.
- Baydin et al., "Automatic Differentiation in Machine Learning: a Survey" (2018): separates reverse-mode automatic differentiation from symbolic and numerical differentiation.
- Glorot and Bengio, "Understanding the difficulty of training deep feedforward neural networks" (2010): the variance argument behind Xavier initialization.
- He et al., "Delving Deep into Rectifiers" (2015): derives the rectifier-aware initialization used in the matched depth experiment.

Exercises

1. **(Pencil.)** Derive the four blame equations for a network whose output layer uses the identity activation and squared-error loss (the network of our verification cell). Confirm that your $\delta^{(2)}$ matches the `delta2` line in the code.
2. **(Pencil.)** In Equation 5.3, why does the blame flow through $(\mathbf{W}^{(l+1)})^\top$ rather than $\mathbf{W}^{(l+1)}$? Argue from shapes: what are the dimensions of $\delta^{(l)}$, $\delta^{(l+1)}$, and $\mathbf{W}^{(l+1)}$?
3. **(Code.)** Extend the `Value` class with `tanh` and use it to train the one-neuron model from the check cell for 50 steps (write the update loop yourself). Verify the loss decreases.
4. **(Code.)** In the depth experiment, give the sigmoid network Xavier initialization (`nn.init.xavier_normal_` on each linear layer). How much of the vanishing does good initialization recover at depth 30? What does that tell you about why both fixes, initialization *and* ReLU, became standard?
5. **(Code.)** Comment out `optimizer.zero_grad()` in any training loop from Chapter 1 and describe what happens to the loss curve. Explain it with one sentence about `.grad` accumulation.

6. When It Fails: Generalization in Pictures, and Inductive Bias

Part I has given us everything a course could promise: models with universal expressive power (Chapter 3), losses grounded in maximum likelihood (Chapter 2), an optimizer that scales (Chapter 4), and gradients for anything (Chapter 5). This chapter takes all of it, aims it at real images, and succeeds completely.

Then we will look a little closer, and the wheels will come off. Watching *how* they come off — in pictures rather than theorems — is the most important lesson of Part I, because the cure is the idea the entire rest of this book is built on.

6.1. Victory

The data is a subset of Fashion-MNIST, the classic benchmark from our live sessions: 28×28 grayscale images of clothing in ten classes. The model and training loop hold no surprises: an MLP from Chapter 3, trained exactly as in Chapter 4:

```
import torch
from torch import nn

torch.manual_seed(6050)
development = torch.load("../../data/fashion-train.pt")
holdout = torch.load("../../data/fashion-test.pt")
X_dev = development["X"].float().reshape(-1, 784) / 255.0  # (1200, 784)
y_dev, classes = development["y"], development["classes"]
X_test = holdout["X"].float().reshape(-1, 784) / 255.0     # (600, 784)
y_test = holdout["y"]

split = torch.randperm(len(X_dev), generator=torch.Generator().manual_seed(6050))
fit_idx, val_idx = split[:1000], split[1000:]
X_tr, y_tr = X_dev[fit_idx], y_dev[fit_idx]                # (1000, 784), (1000,)
X_val, y_val = X_dev[val_idx], y_dev[val_idx]              # (200, 784), (200,)

def train_mlp(
    X: torch.Tensor, y: torch.Tensor, hidden: int = 256,
    epochs: int = 40, seed: int = 6050
) -> nn.Sequential:
    torch.manual_seed(seed)
```

```
    net = nn.Sequential(nn.Linear(784, hidden), nn.ReLU(), nn.Linear(hidden, 10))
    opt = torch.optim.Adam(net.parameters(), lr=1e-3)
    for _ in range(epochs):
        perm = torch.randperm(len(X))
        for i in range(0, len(X), 64):
            idx = perm[i:i + 64]
            opt.zero_grad()
            nn.functional.cross_entropy(net(X[idx]), y[idx]).backward()
            opt.step()
    return net

def accuracy(net: nn.Module, X: torch.Tensor, y: torch.Tensor) -> float:
    net.eval()
    with torch.no_grad():
        return (net(X).argmax(1) == y).float().mean().item()

net = train_mlp(X_tr, y_tr)
print(f"train accuracy: {accuracy(net, X_tr, y_tr):.1%}")
print(f"validation accuracy: {accuracy(net, X_val, y_val):.1%}")
```

```
train accuracy: 95.0%
validation accuracy: 76.0%
```

Seventy-six percent on held-out validation images, from one thousand fitting examples and a few seconds of CPU. The split was fixed before any experiment, and the separate test set remains sealed until the end of the chapter. By every metric we have learned to compute, we might be done.

The rest of this chapter is about why we are not.

6.2. Two experiments that should worry you

6.2.1. Experiment 1: move everything two pixels

Take the validation images and shift each one two pixels to the right. Nothing about any garment changes: a coat is a coat, wherever it hangs in the frame. Any human would call the original and shifted batches equivalent for classification.

```
def shift_right(X: torch.Tensor, px: int) -> torch.Tensor:
    img = X.reshape(-1, 28, 28)
    out = torch.zeros_like(img)
    if px > 0:
        out[:, :, px:] = img[:, :, :-px]
    else:
```

```
        out = img.clone()
    return out.reshape(-1, 784)

for px in [0, 2]:
    print(f"shift {px}px: validation accuracy "
          f"{accuracy(net, shift_right(X_val, px), y_val):.1%}")
```

```
shift 0px: validation accuracy 76.0%
shift 2px: validation accuracy 46.5%
```

Two pixels, and the accuracy is nearly cut in half. Push further and the collapse completes:

```
import matplotlib.pyplot as plt

shifts = list(range(5))
accs = [accuracy(net, shift_right(X_val, s), y_val) for s in shifts]
plt.figure(figsize=(5.6, 3.3))
plt.plot(shifts, accs, "o-", color="#E57200", lw=2, ms=7)
plt.axhline(0.1, ls=":", color="#B8B8A8")
plt.text(0.1, 0.115, "chance (10 classes)", color="#8A8A7A", fontsize=9)
plt.xlabel("shift (pixels right)"); plt.ylabel("validation accuracy")
plt.ylim(0, 0.9); plt.xticks(shifts)
plt.tight_layout(); plt.show()
```

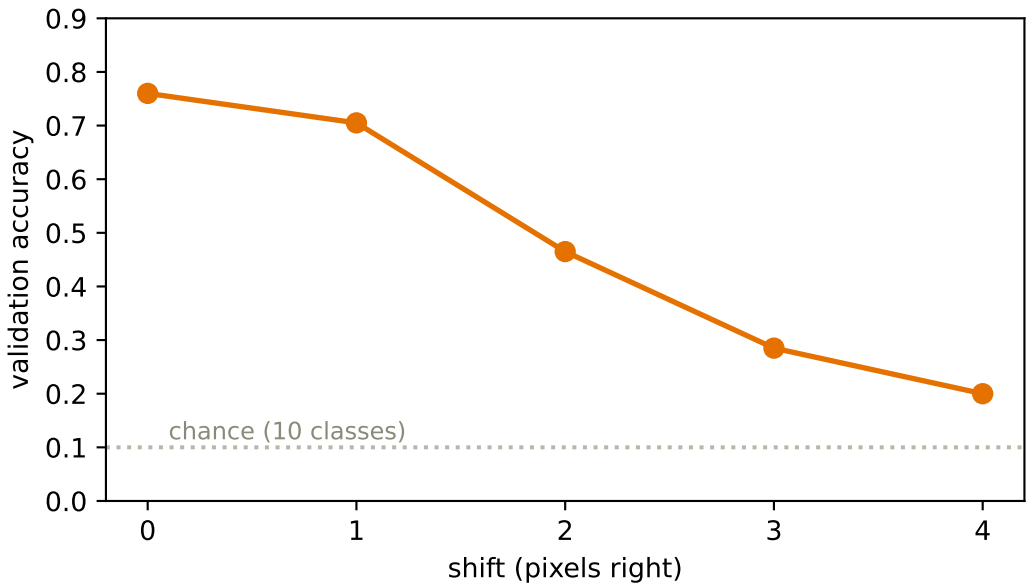

Figure 6.1.: The validation cliff. Each pixel of horizontal shift destroys accuracy although the garments themselves never changed. By four pixels the model is approaching guesswork.

6.2.2. Experiment 2: destroy the images entirely

Now the stranger experiment, from the lecture's "fatal flaw" argument. Choose one fixed random permutation of the 784 pixel positions and apply it to *every* image, fit and validation alike. The displayed arrays are not recognizable pictures anymore — the ordinary grid adjacency has been destroyed. Retrain the same MLP on this scrambled world:

```
torch.manual_seed(0)
pixel_perm = torch.randperm(784)          # one fixed scrambling for all images

net_shuffled = train_mlp(X_tr[:, pixel_perm], y_tr)
print(f"original images:  validation accuracy {accuracy(net, X_val, y_val):.1%}")
print(f"shuffled pixels:  validation accuracy "
      f"{accuracy(net_shuffled, X_val[:, pixel_perm], y_val):.1%}")
```

```
original images:  validation accuracy 76.0%
shuffled pixels:  validation accuracy 77.0%
```

The two validation accuracies are close in this one deterministic run. That is not an uncertainty estimate, but it is enough for the structural point: after retraining, an invertible relabeling of pixel coordinates costs this MLP no visible accuracy.

Hold the two results side by side and the diagnosis writes itself:

> The MLP hypothesis class and training procedure show no preference for the structure that matters, including pixel adjacency and image geometry, yet the fitted model is **hypersensitive** to a nuisance that should not matter: where the garment happens to sit. It learned the dataset without needing an image-aware representation.

6.3. The autopsy, in pictures

Why exactly does shifting hurt a model that shuffling cannot? Look at what the model actually learned. Each row of the first layer's weight matrix is a 784-vector — which means we can reshape it into a 28×28 image and look at it. And we know from Chapter 1 what such a row *is*: a **template**, scored against the input by dot product.

```
W1 = net[0].weight.detach()               # (256, 784)
order = W1.norm(dim=1).argsort(descending=True)[:16]

fig, axes = plt.subplots(2, 8, figsize=(7.6, 2.2))
for ax, i in zip(axes.ravel(), order):
    ax.imshow(W1[i].reshape(28, 28), cmap="RdBu_r")
    ax.set_xticks([]); ax.set_yticks([])
plt.tight_layout(); plt.show()
```

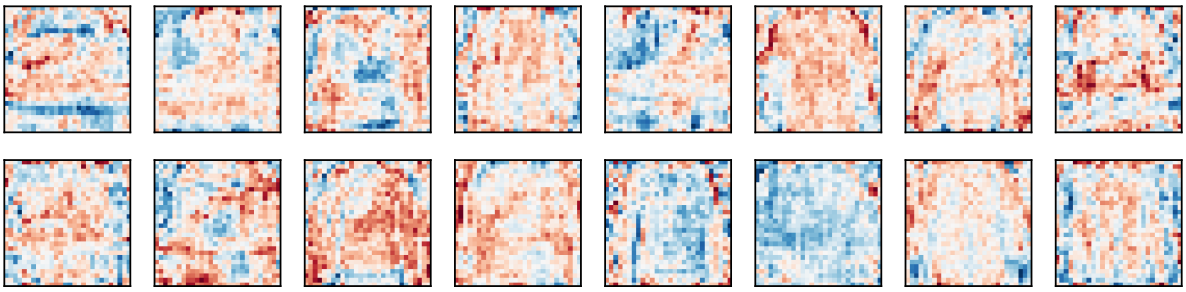

Figure 6.2.: Sixteen first-layer templates, reshaped into images. Each is a global, full-frame pattern; many are diffuse rather than recognizable garment detectors. A template that spans the whole frame changes its score when the input pattern moves.

There is the whole story in one figure. Every template is **global** — a pattern stretched across the entire frame, tuned to where garments sat in the training images. Shift the input two pixels and every pixel of every template now scores against the wrong part of the garment; hundreds of learned template scores change at once. Conversely, the fixed permutation is merely an invertible relabeling of coordinates. Retraining can permute the first-layer weights in the opposite direction, so the same functions remain available even though adjacency has been destroyed. A model trained on ordinary images is not itself invariant to arbitrary pixel permutations.

Capacity was never the issue. By Chapter 3's universal approximation theorem, this network *can represent* a perfectly shift-tolerant classifier. It has no reason to find one: nothing in its architecture, loss, or data ever told it that position is a nuisance and adjacency is meaningful.

6.4. Naming the disease

With the failure in hand, the classical concepts of Part I snap into focus.

6.4.1. Hunting the U-curve

The textbook story (Chapter 1's cartoon) predicts that as capacity grows, validation error should fall and then *rise* as the model overfits. We now have everything needed to test the cartoon on real data: a width sweep at two dataset sizes:

```
widths = [2, 4, 8, 16, 64, 256]
plt.figure(figsize=(6, 3.5))
for n, color in [(300, "#5379AA"), (1000, "#232D4B")]:
    errs = [1 - accuracy(train_mlp(X_tr[:n], y_tr[:n], hidden=h, epochs=60),
                         X_val, y_val)
            for h in widths]
    plt.semilogx(widths, errs, "o-", color=color, label=f"n = {n}", base=2)
plt.xlabel("hidden width (capacity)"); plt.ylabel("validation error")
plt.legend(); plt.tight_layout(); plt.show()
```

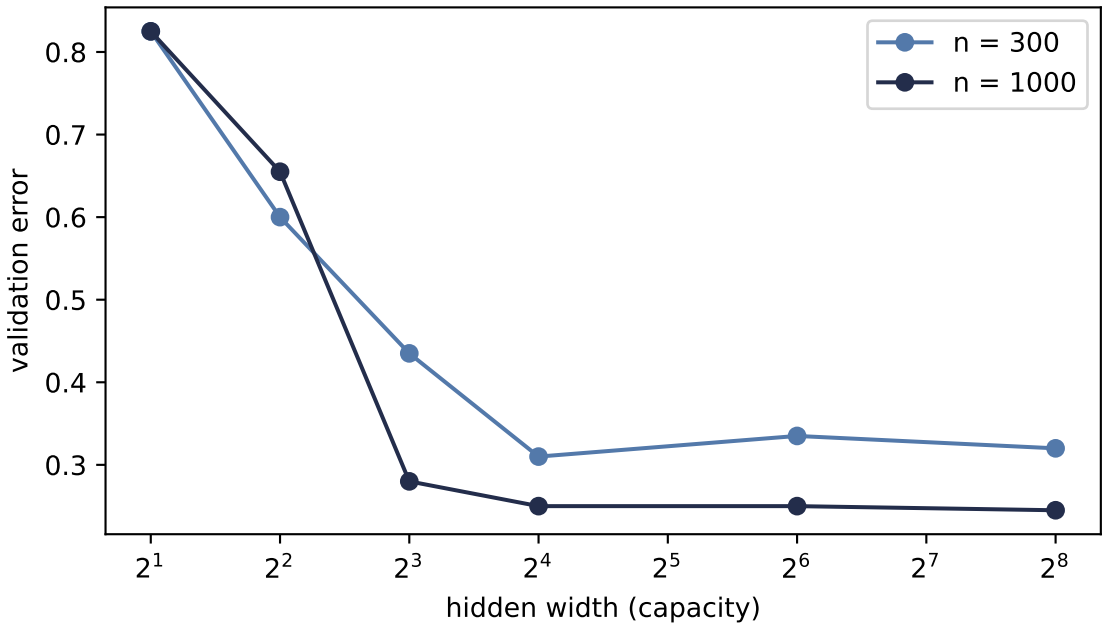

Figure 6.3.: Hunting the U-curve: validation error vs. hidden width at two fitting-set sizes. Error falls steeply as capacity grows, then flattens. In this one seeded sweep, n=300 shows a small late upturn while n=1000 does not. The result describes this regime; it does not estimate a scaling law.

Read it honestly. The left side of the cartoon is emphatically real: too little capacity underfits badly. But the dreaded right side barely materializes, with a whisper of an upturn at the small dataset and nothing at the larger one. This is exactly the caveat planted in Chapter 1: the U is a helpful cartoon, not a law of nature. The optimizer, sample size, and data structure can all shape the curve. This one seeded Adam sweep does not identify their separate effects, and it does not establish a $1/n$ variance law.

Which sharpens our problem instead of solving it: **our model does not fail for having too much capacity. It fails for having no knowledge.** No point on either curve survives a two-pixel shift.

6.4.2. The curse of dimensionality

Could we fix everything with data alone, just by showing the model garments at every position? Count what that costs. Our images live in 784 dimensions; a small color image (32×32×3) needs 3,072 numbers, a one-megapixel photo three million, an ImageNet input 150,528. And covering a d-dimensional space densely requires exponentially many examples: if 9 points cover a 1-D interval at some resolution, matching that density takes 9^2 points in 2-D and 9^d in d dimensions. For $d = 784$, there are not enough atoms in the universe, at any exchange rate.

High dimensionality also breaks the very idea of "similar examples":

```
torch.manual_seed(6050)
dims = [2, 10, 100, 784, 5000]
ratios = []
for d in dims:
    P = torch.rand(300, d)
    D = torch.cdist(P, P)
    ratios.append(((D + torch.eye(300) * 1e9).min(1).values / D.max(1).values).mean())

plt.figure(figsize=(5.6, 3.2))
plt.semilogx(dims, ratios, "o-", color="#722F37", lw=2)
plt.axvline(784, ls=":", color="#B8B8A8")
plt.text(830, 0.25, "Fashion-MNIST\n(d = 784)", fontsize=9, color="#8A8A7A")
plt.xlabel("dimension $d$"); plt.ylabel("nearest / farthest distance")
plt.ylim(0, 1); plt.tight_layout(); plt.show()
```

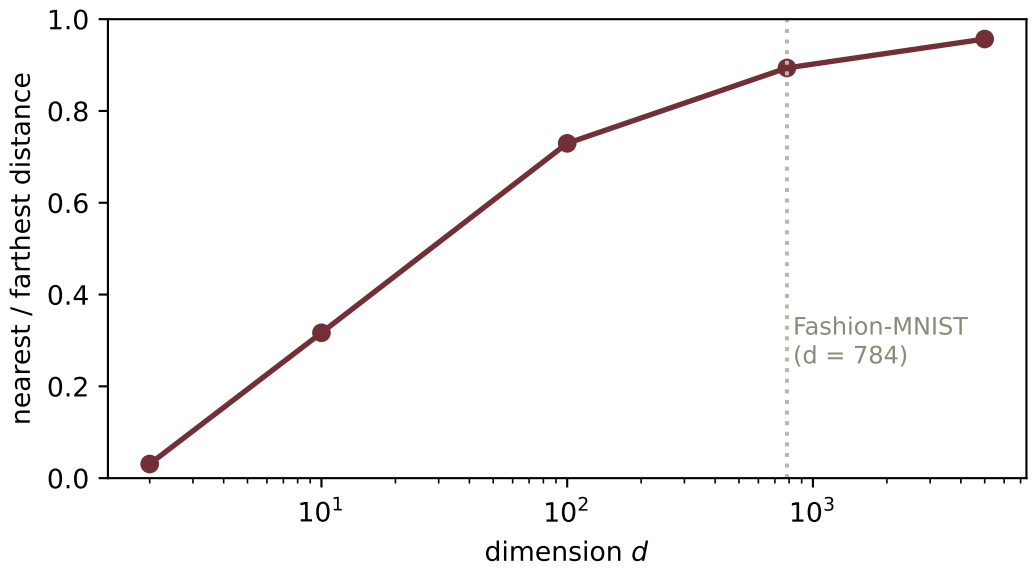

Figure 6.4.: Distance concentration for 300 points sampled uniformly from a unit cube: the average ratio of nearest-neighbor to farthest-neighbor distance, by ambient dimension. At d=784 the ratio reaches 0.89. This is a controlled geometry example, not a claim that natural images fill the 784-dimensional cube uniformly.

In the uniform cube, brute-force interpolation loses genuinely near neighbors as the ambient dimension grows. Natural images are highly structured and may concentrate near a much lower-dimensional set, so this experiment is not their empirical distance distribution. It shows the price of trying to cover an *unconstrained* 784-dimensional space and why useful structure must enter through data, representation, or architecture.

6.5. The cure has a name

Return to the word planted on the very first pages of this book (Chapter 1): **inductive bias** — the nudge that steers a learner toward the functions we want, among the infinitely many that fit. Our MLP's failure is not a capacity problem or (fixably) a data problem. It is a *knowledge* problem: we knew things about images that we never told the model.

Say them out loud — the two principles from the lecture:

1. **Locality.** Nearby pixels are related; a pixel and its neighbor jointly form edges, textures, parts. Distant pixels, far less so. (The shuffle experiment proved our MLP holds no such belief.)
2. **Translation structure.** A local detector's response should move with the feature (**equivariance**), while a classifier's final verdict should tolerate or ignore small changes in position (**invariance**). The shift experiment showed that our fitted MLP did neither reliably.

Here is the move that powers the rest of this book: **build these beliefs into the architecture itself** — as constraints. Constrain each template to look at a small local patch instead of the whole frame. Then *share* that template across every position, so the detection rule itself does not depend on where. Two constraints, and both pathologies are attacked at their root. The parameter count falls too: our MLP has 203,530 parameters, while the LeNet model in Chapter 8 will have 61,706, less than a third as many. Sharing is a massive economy.

> **Constraints are knowledge**
>
> A constraint looks like a *reduction* in power: the constrained model is a strict subset of the MLP. That is exactly the point. By Chapter 1's bias–variance lens, we are spending bias — assumptions we are confident of — to collapse the search space onto functions that respect the structure of images, slashing variance and sample complexity. Inductive bias is not a limitation of a model; done right, it is everything you know about the world, cast in architecture. And it is a dial, not a dogma: give a weakly-biased model enough data and it can win again — a trade we will meet at the frontier in Chapter 16.

6.6. The pivot

Look once more at the templates figure. The fix is almost visible in it: the templates should not span the frame. They should be **small and local** — and they should **slide**, scoring every neighborhood of the image with the same little pattern, so that a sleeve found anywhere is the same sleeve.

A local template, slid across the image, scoring each position by dot product. That machine has a name, it predates deep learning by decades, and Part II opens by building it with our bare hands — then asking this book's favorite question about it:

What if the template were learnable?

6.7. One sealed test check

The architecture, optimizer, shift size, and reported metrics are now fixed. Only now do we fit the same MLP on all 1,200 development examples and open the 600-example test set once:

```
net_final = train_mlp(X_dev, y_dev)
print(f"clean test accuracy:   {accuracy(net_final, X_test, y_test):.1%}")
print(f"shift-2 test accuracy: "
      f"{accuracy(net_final, shift_right(X_test, 2), y_test):.1%}")
```

```
clean test accuracy:   80.8%
shift-2 test accuracy: 42.0%
```

The sealed check confirms the diagnostic rather than creating it: 80.8% on clean test images becomes 42.0% after the predeclared two-pixel shift.

6.8. Okay, so — the lesson of Part I

1. **Every metric can say yes while the model has learned the wrong thing.** Our MLP scored well on clean held-out data while its hypothesis class showed no adjacency preference (fixed shuffle) and its fitted predictions were hypersensitive to a nuisance shift.
2. **The autopsy is visual**: global templates, tuned to position, blind to adjacency.
3. **Capacity is not the culprit in this experiment.** The U-curve barely appears in one seeded sweep, while the uniform-cube example shows why unconstrained ambient coverage becomes expensive.
4. **The cure is inductive bias**: locality and translation structure, built in as constraints. Knowledge, cast in architecture.

Deeper dive

> **i** For the curious: how deep this rabbit hole goes
>
> **Networks can memorize random labels.** Zhang et al. reported 100% training accuracy on randomly labeled CIFAR-10 and 95.20% on randomly labeled ImageNet under their finite training budget, not 100% on ImageNet. Capacity is abundant enough that fitting noise is easy (Exercise 4 reproduces the phenomenon on our subset). Capacity alone therefore does not explain generalization; the data, optimizer, and architecture must enter the account.
> **The bias–variance picture, updated.** Our flattened U-curve is a mild case of a broader phenomenon: with modern training, test error can even *descend again* beyond the interpolation point — "double descent." The cartoon survives as intuition for the underfit regime; the overparameterized regime plays by subtler rules.
> **Further reading**

- Zhang, Bengio, Hardt, Recht, Vinyals, "Understanding deep learning requires rethinking generalization" (ICLR 2017), the primary source for the random-label experiment.
- Neal, "On the Bias-Variance Tradeoff: Textbooks Need an Update" (2020), the source of this book's "cartoon, not a law" stance.
- Belkin et al., "Reconciling modern machine-learning practice and the classical bias–variance trade-off" (PNAS 2019), a primary account of double descent.
- 3Blue1Brown, *Gradient descent, how neural networks learn*, the visual companion to this chapter's pacing, including the demonstration that a trained MLP can classify pure noise confidently.

Exercises

1. **(Pencil.)** Explain algebraically why the pixel shuffle costs the MLP nothing: if $\mathbf{P}$ is a permutation matrix and we train on $\mathbf{Px}$ instead of $\mathbf{x}$, exhibit the weight matrix that makes the two networks identical. Why does no analogous argument work for the shift?
2. **(Code.)** Augment the training set with random shifts of ± 3 pixels and retrain. Re-plot the cliff. How much robustness did augmentation buy, what did it cost in clean accuracy and compute, and — the real question — what *knowledge* did augmentation encode, compared to building invariance into the architecture?
3. **(Code.)** The U-curve hunt used widths up to 256. Push to 1024 and 4096 at $n = 300$, repeating each width across several seeds. Then compare Adam with minibatch SGD from Chapter 4. Which patterns are stable enough to interpret?
4. **(Code.)** Randomly permute the training *labels* and train a wide MLP for long enough. Report train and test accuracy, and state in one sentence what this proves about capacity as an explanation of generalization.
5. **(Pencil.)** Estimate the first-layer parameter count of an MLP with 1,000 hidden units on an ImageNet-scale input ($224 \times 224 \times 3$). At one float32 per parameter, how much memory is that — and how does the number change if each unit instead sees only a $7 \times 7 \times 3$ patch?

Part II.

II · Vision: Learning the Filters

7. Filters and Convolution (Fixed Kernels)

Part I ended with a diagnosis and a prescription. The diagnosis: our MLP's templates span the whole frame, so they are blind to adjacency and brittle to position (Chapter 6). The prescription, in one sentence: *make the template local, and slide it.*

This chapter builds that machine with our bare hands — no learning anywhere. It is a piece of classical engineering, decades older than deep learning, and computer-vision experts spent careers designing its templates manually. Understanding the machine in its fixed, hand-crafted form is the whole point of the chapter, because Part II's revolution (Chapter 8) will consist of exactly one change to it.

Recall the primitive from Chapter 1: a dot product scores an input against a template. When their norms are controlled, its angle component measures similarity. Everything below is that one primitive, applied locally, everywhere.

7.1. Two principles, one strategy

The failures of Chapter 6 tell us what knowledge to build in, the two principles from the lecture:

1. **Spatial locality.** Nearby pixels are more related than distant ones. Edges, textures, and patterns emerge from *local neighborhoods*, not global pixel arrangements.
2. **Translation equivariance.** The same feature detector should work regardless of position — a cat's ear is a cat's ear whether it appears top-left or bottom-right.

The strategy they suggest: instead of one massive network staring at the entire image, use a *small* detector that analyzes one patch at a time, and **slide it across the image** to look for its feature everywhere. To understand what such detectors do, we first look at how vision experts used to design them by hand.

7.2. Start in 1D: the moving average

The core operation appears in its friendliest form when smoothing a noisy signal. A moving average replaces each point by the mean of its neighborhood: locality, with *uniform* weights:

```
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt
```

```
torch.manual_seed(6050)
t = torch.linspace(0, 4 * torch.pi, 300)
noisy = torch.sin(t) + 0.35 * torch.randn(300)
kernel = torch.full((9,), 1 / 9)                          # uniform local weights
smooth = F.conv1d(noisy[None, None], kernel[None, None]).squeeze()

plt.figure(figsize=(6.4, 3))
plt.plot(t, noisy, color="#B8B8A8", lw=0.8, label="noisy signal")
plt.plot(t[4:-4], smooth, color="#E57200", lw=2, label="moving average (9-wide)")
plt.xlabel("$t$"); plt.legend()
plt.tight_layout(); plt.show()
```

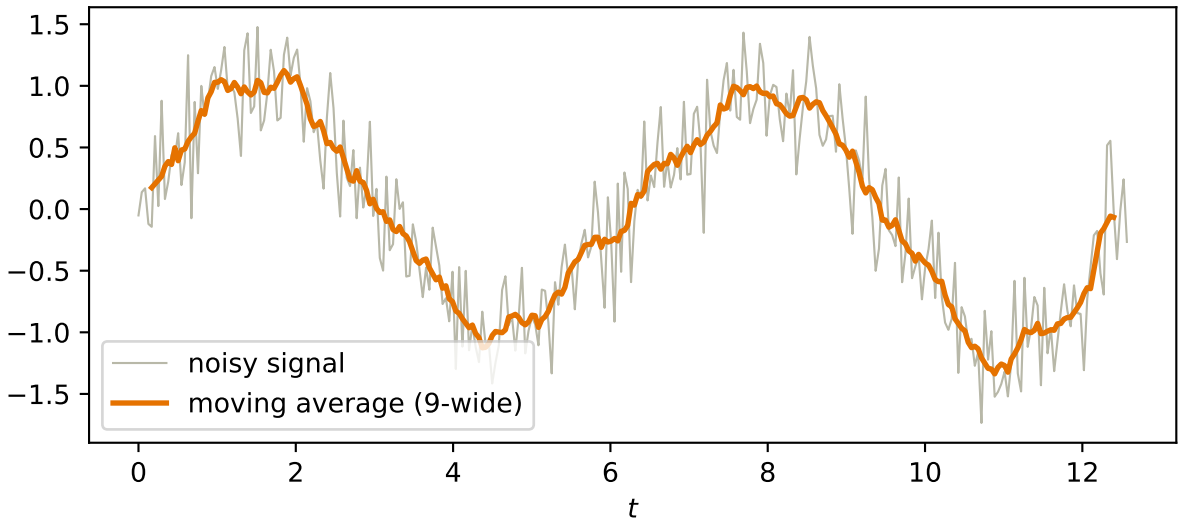

Figure 7.1.: A window of uniform weights, slid along a noisy signal, averaging as it goes: the simplest sliding-window operation, and already recognizably a denoiser.

7.3. To 2D: the recipe

The image version is the same idea with a small 2-D window, and it is worth stating as the five-step recipe from the lecture:

1. Define a small window of weights, the **kernel** (for a blur: all positive, summing to 1).
2. Place the kernel over a patch of the image.
3. Multiply elementwise: kernel weights against the pixels underneath.
4. Sum the products into a single output pixel. *(Steps 3–4 are one dot product: the patch, scored against a small template.)*
5. **Slide the window** to the next location and repeat, producing a new image.

Let us expose every sliding patch at once, multiply each by the kernel, and sum. Then we check the result against the framework's version, following the build-then-verify habit of this book:

```
def manual_conv2d(x: torch.Tensor, k: torch.Tensor) -> torch.Tensor:
    """Cross-correlate one 2-D image and kernel via a view of sliding patches."""
    kh, kw = k.shape
    patches = x.unfold(0, kh, 1).unfold(1, kw, 1)          # (Hout, Wout, kh, kw)
    return (patches * k).sum(dim=(-1, -2))                 # one score per patch

def conv2d(x: torch.Tensor, k: torch.Tensor) -> torch.Tensor:
    return F.conv2d(x[None, None], k[None, None]).squeeze()

torch.manual_seed(6050)
x_test = torch.rand(10, 12)
k_test = torch.randn(3, 3)
gap = (manual_conv2d(x_test, k_test) - conv2d(x_test, k_test)).abs().max()
print(f"recipe vs. torch.conv2d: max |diff| = {gap:.1e}")
```

```
recipe vs. torch.conv2d: max |diff| = 4.8e-07
```

Also worth noticing before we move on: the output is smaller than the input. A $k \times k$ kernel on an $n \times n$ image produces $(n-k+1) \times (n-k+1)$ outputs, because the window must stay inside the frame. What to do about that (and about sliding in bigger steps) is Chapter 8's business.

7.4. The filter zoo

Here is the remarkable part: the recipe never changes — *only the numbers in the kernel do.* Different numbers produce completely different specialists. Three classics, applied to a synthetic test scene and to a boot from our Fashion-MNIST subset:

- **Blur**: uniform positive weights summing to 1, the 2-D moving average.
- **Sharpen**: amplify each pixel's difference from its neighbors.
- **Edge detection** (Sobel): positive and negative weights summing to *zero* — over any flat region the products cancel and the output is silent; over an edge, a sharp change in values, the sum swings large. A detector that answers one question: *does intensity change here, in my direction?*

```
def make_shapes(n: int = 64) -> torch.Tensor:
    img = torch.zeros(n, n)
    img[10:26, 8:30] = 0.9                                 # rectangle
    yy, xx = torch.meshgrid(torch.arange(n), torch.arange(n), indexing="ij")
    img[((yy - 44) ** 2 + (xx - 20) ** 2) < 100] = 0.6     # disk
    stripes = ((xx + yy) % 12 < 5).float() * 0.8
    img[8:56, 40:60] = stripes[8:56, 40:60]                # diagonal stripes
    return img

development = torch.load("../../data/fashion-train.pt")
```

```
boot = development["X"][(development["y"] == 9).nonzero()[0, 0]].float() / 255.0

kernels = {
    "original": torch.tensor([[0., 0., 0.], [0., 1., 0.], [0., 0., 0.]]),
    "blur": torch.full((3, 3), 1 / 9),
    "sharpen": torch.tensor([[0., -1., 0.], [-1., 5., -1.], [0., -1., 0.]]),
    "Sobel (vert.)": torch.tensor([[-1., 0., 1.], [-2., 0., 2.], [-1., 0., 1.]]),
    "Sobel (horiz.)": torch.tensor([[-1., -2., -1.], [0., 0., 0.], [1., 2., 1.]]),
}

fig, axes = plt.subplots(2, 5, figsize=(7.8, 3.4))
for row, image in enumerate([make_shapes(), boot]):
    for col, (name, k) in enumerate(kernels.items()):
        out = conv2d(image, k)
        axes[row, col].imshow(out, cmap="gray")
        axes[row, col].set_xticks([]); axes[row, col].set_yticks([])
        if row == 0:
            axes[row, col].set_title(name, fontsize=9)
plt.tight_layout(); plt.show()
```

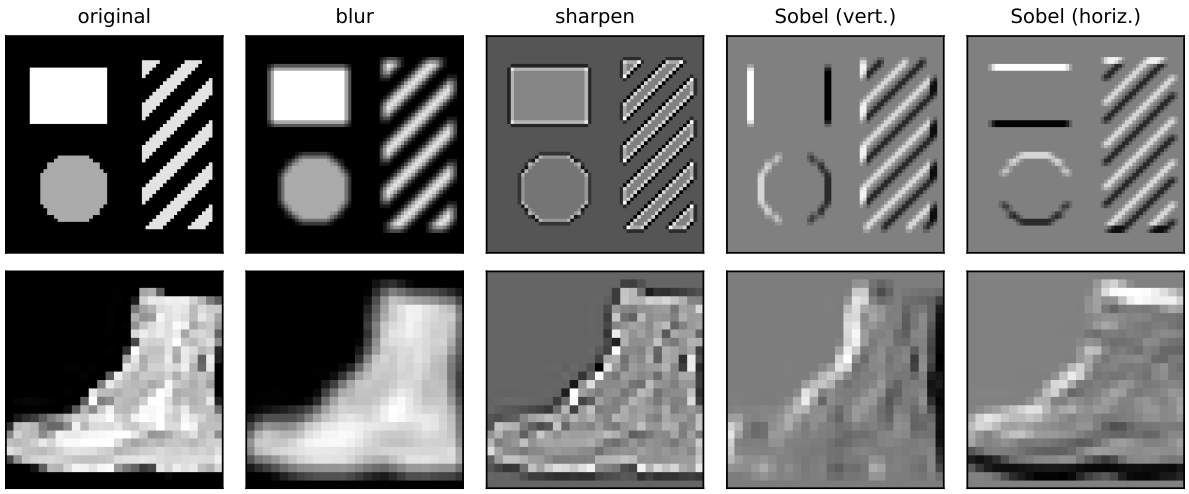

Figure 7.2.: One recipe, five kernels. Each column applies a different 3×3 kernel to the same inputs: identity, blur (local average), sharpen, and two Sobel edge detectors. The vertical-edge kernel fires on vertical boundaries; the horizontal one fires on horizontal boundaries.

Study the two Sobel columns for a moment. The vertical-edge detector lights up on the rectangle's sides and stays silent on its top and bottom; the horizontal detector does the reverse; the diagonal stripes excite both. Nine numbers, and we have built a primitive *feature detector* — precisely the "small network analyzing one patch" of our strategy, except nobody has learned anything yet.

7.5. Naming the operation

This sliding sum-of-products has a proper name: **cross-correlation**. With kernel V of half-width Δ and image X:

$$[H]_{i,j} = \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} [V]_{a,b} [X]_{i+a,\, j+b}. \tag{7.1}$$

> **i** "Convolution", with a small asterisk
>
> The deep learning community universally calls this operation a **convolution**, and so will we. A mathematician's convolution flips the kernel before sliding; since our kernels will soon be *learned* parameters, a flipped kernel is just another kernel, and the distinction is inconsequential in practice (Exercise 4 makes you confirm this).

7.6. The property we paid for: equivariance

Now collect the reward that Chapter 6's MLP could never have. Away from boundaries, slide-and-score treats every position identically → if the input shifts, the output shifts *with it*, unchanged in content. That is **translation equivariance**, and it need not be taken on faith:

```
img = make_shapes()
sobel = kernels["Sobel (vert.)"]
s = 5

shifted = torch.zeros_like(img)
shifted[:, s:] = img[:, :-s]                  # input shifted right by 5

A = conv2d(shifted, sobel)                    # filter the shifted image
B = conv2d(img, sobel)                        # filter, THEN shift the result
B_shifted = torch.zeros_like(B)
B_shifted[:, s:] = B[:, :-s]

interior = (A - B_shifted)[:, s + 2:]         # compare away from the blank border
print(f"filter(shift(x)) vs shift(filter(x)): max |diff| = {interior.abs().max():.1f}")
```

```
filter(shift(x)) vs shift(filter(x)): max |diff| = 0.0
```

Exactly zero on the compared interior. The detector's report moves with the garment. Contrast Chapter 6: the MLP's global templates degraded under a 2-pixel shift because position was baked into every weight. Here, position cannot matter to *what* is detected on the interior, only to *where* the detection lands. At the frame boundary, cropping, padding, or fill rules can break exact equality; that is why the code excludes the blank-border region.

One precision worth keeping sharp, because the two words will both matter: **equivariant** means the output moves along with the input (what convolution gives us); **invariant** means the output does not change at all (what a classifier ultimately wants — "boot", regardless of position). Equivariance is the raw material; in Chapter 8, pooling will spend some position information to buy local shift tolerance. Exact invariance is a stronger property and must be tested, not presumed.

7.7. The matrix view: a structured, weight-shared linear map

One more pair of glasses, from the lecture's preview box. Every output pixel is a dot product → the whole operation is *linear* → cross-correlation could be written as one enormous matrix multiplying the flattened image. But it is a matrix with a rigid structure: almost everywhere zero (locality: each row touches only one patch), and the same nine numbers repeating along its diagonals (sharing: every row is the same template, relocated).

That view makes the economics explicit. A dense layer from our 28×28 images to an equal-sized output would own $784 \times 784 \approx 615{,}000$ weights. The Sobel detector owns **nine** — reused at every position. This is Chapter 6's constraints-are-knowledge callout made concrete: locality zeroes out most of the matrix, equivariance ties the survivors together, and what remains is a linear map with almost nothing left to specify.

```
from matplotlib.colors import ListedColormap

weight_ids = torch.zeros(4, 6)
for row in range(4):
    weight_ids[row, row:row + 3] = torch.tensor([1, 2, 3])

fig, ax = plt.subplots(figsize=(5.7, 2.8))
ax.imshow(weight_ids, cmap=ListedColormap(["white", "#DCE6F2", "#F8D9B0", "#E7D4E8"]),
          vmin=0, vmax=3)
symbols = {1: r"$v_1$", 2: r"$v_2$", 3: r"$v_3$"}
for i in range(weight_ids.shape[0]):
    for j in range(weight_ids.shape[1]):
        value = int(weight_ids[i, j])
        ax.text(j, i, symbols.get(value, "0"), ha="center", va="center")
ax.set_xticks(range(6), [rf"$x_{j + 1}$" for j in range(6)])
ax.set_yticks(range(4), [rf"$h_{i + 1}$" for i in range(4)])
ax.set_xticks(torch.arange(-0.5, 6, 1), minor=True)
ax.set_yticks(torch.arange(-0.5, 4, 1), minor=True)
ax.grid(which="minor", color="#B8B8A8", linewidth=0.8)
ax.tick_params(which="minor", bottom=False, left=False)
ax.set_title(r"$\mathbf{h}=\mathbf{T}(\mathbf{v})\mathbf{x}$: local and weight-shared")
plt.tight_layout(); plt.show()
```

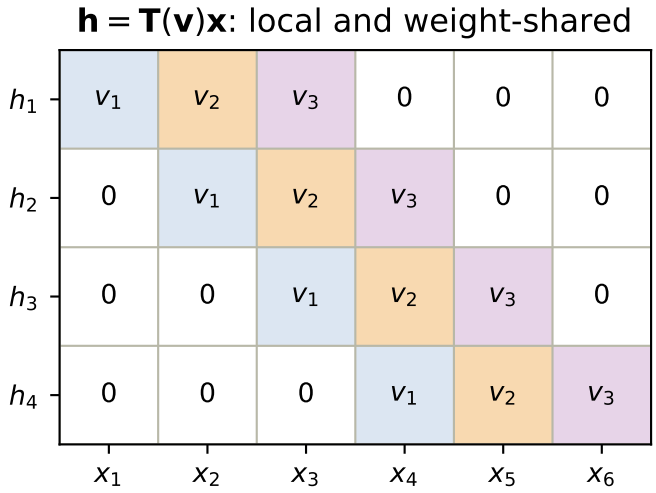

Figure 7.3.: The 1-D version of convolution as a structured matrix (shown in 1-D only so the pattern is visible). Each output row touches one local three-value window, so most entries are zero. The same three kernel weights repeat along diagonals: locality creates sparsity; sharing ties the nonzero entries together. A 2-D image produces the analogous block-structured matrix.

7.8. The bottleneck: someone has to design the kernel

So why did this classical machinery not solve vision decades ago? Because the kernels are only as good as their designers, and hand-crafted kernels have three limitations the lecture names:

1. **Limited expressiveness.** They detect simple, predefined patterns — gradients, blobs, corners. Nobody can write down nine numbers for "cat ear" or "car wheel."
2. **Manual feature engineering.** Choosing which kernels to use, and how to combine them, takes deep domain expertise and endless experimentation — for every new task.
3. **No adaptation.** A fixed kernel cannot adjust to a dataset. What works for one task fails for another.

An entire era of computer vision lived inside these constraints, stacking hand-designed detectors into elaborate pipelines. The machine was right; the templates were the bottleneck.

You can hear the question coming. We have a device that is local, weight-shared, and equivariant by construction — with a handful of numbers sitting inside it, currently chosen by hand. Nine numbers. We have spent four chapters building tools that tune numbers against a loss.

What if the template were learnable? That question is Chapter 8, and its answer has a name you already know.

7.9. Okay, so — the machine before the learning

1. **One operation, many specialists**: slide a small template, take a dot product at every position (Equation 7.1). Blur, sharpen, and edge detection differ only in the kernel's numbers.
2. **Locality and sharing are built in**: as a matrix, convolution is almost-all-zero with nine numbers repeating — 615,000 weights collapsed to 9.
3. **Equivariance is exact on the interior**, not approximate: filter-then-shift equals shift-then-filter wherever both operations see the same pixels. Boundary handling must be specified separately. The next chapter asks how much local shift tolerance pooling can purchase from that equivariance.
4. **Hand-crafting is the bottleneck** — expressiveness, engineering cost, no adaptation — and it is exactly the kind of bottleneck this book knows how to break.

Sources and further reading

- Fukushima, "Neocognitron: A Self-organizing Neural Network Model" (1980): an early hierarchy of local, shift-tolerant feature detectors.
- LeCun et al., "Gradient-Based Learning Applied to Document Recognition" (1998): connects shared local kernels to the trainable CNN built next.
- Dumoulin and Visin, "A Guide to Convolution Arithmetic for Deep Learning" (2016): a visual reference for padding, stride, and output-shape arithmetic.

Exercises

1. **(Pencil.)** Compute by hand the full output of cross-correlating this 4×4 image with the vertical Sobel kernel: rows $(0,0,1,1)$, $(0,0,1,1)$, $(0,0,1,1)$, $(0,0,1,1)$. Before computing: where should the detector fire?
2. **(Pencil + code.)** Design a 3×3 kernel that detects *diagonal* edges (bottom-left to top-right). State your design rule (what must the weights sum to, and why?), then run it on `make_shapes()` and check it fires on the stripes.
3. **(Pencil.)** Prove translation equivariance from Equation 7.1: substitute the shifted image $X'_{i,j} = X_{i-s,j}$ and show $H'_{i,j} = H_{i-s,j}$. Why does the same argument fail for the MLP of Chapter 6? State where the proof applies at a finite image boundary.
4. **(Code.)** True convolution flips the kernel: run `conv2d` with `k` and with `k.flip(0, 1)` for the blur and for Sobel. Which outputs differ, which do not, and what property of the kernel decides it?
5. **(Code.)** The box blur is *separable*: show that convolving with the 3×3 uniform kernel equals convolving with a 3×1 column of $\frac{1}{3}$s and then a 1×3 row of $\frac{1}{3}$s. Count multiplications per output pixel for each route — this trick mattered enormously in the hand-crafted era.

8. CNNs: Making the Filters Learnable

Chapter 7 closed on a question it had spent a whole chapter earning: we built a machine that is local, weight-shared, and translation-equivariant away from boundaries, with nine hand-picked numbers sitting inside it. Nine numbers → nine knobs → we have spent half a book tuning knobs against a loss.

So: declare the kernel a **parameter**. That single change is this chapter, and it is the whole revolution. Everything else here — channels, padding, stride, pooling — is the supporting cast that turns one learnable filter into a working network. By the end we will have assembled LeNet, the first great convolutional architecture, trained it on our garments, and re-run the cruelest experiment of Chapter 6 to see what the inductive bias actually bought.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

torch.manual_seed(6050)
development = torch.load("../../data/fashion-train.pt")
holdout = torch.load("../../data/fashion-test.pt")
X_dev = development["X"].float().unsqueeze(1) / 255.0  # (1200, 1, 28, 28)
y_dev, classes = development["y"], development["classes"]
X_test = holdout["X"].float().unsqueeze(1) / 255.0     # (600, 1, 28, 28)
y_test = holdout["y"]

split = torch.randperm(len(X_dev), generator=torch.Generator().manual_seed(6050))
fit_idx, val_idx = split[:1000], split[1000:]
X_tr, y_tr = X_dev[fit_idx], y_dev[fit_idx]             # (1000, 1, 28, 28), (1000,)
X_val, y_val = X_dev[val_idx], y_dev[val_idx]           # (200, 1, 28, 28), (200,)
```

Note the shapes. In Part I we flattened every image into a 784-vector before the model ever saw it. That stops now: the images stay rectangular, and they pick up a *channel* dimension, for reasons that will occupy us shortly.

> ⚠ The book's batch convention: NCHW
>
> Throughout this book, every image **batch** uses PyTorch's **NCHW** order: (batch, channels, height, width). A single grayscale batch is therefore shaped `(1, 1, 28, 28)`. `Conv2d` can also accept an unbatched `(C, H, W)` input, but keeping the batch dimension explicit makes downstream shapes predictable. The kernels of a conv layer live in a 4-D tensor too:

(out-channels, in-channels, height, width). This bookkeeping is the single most common source of errors in convolutional code: when a shape error greets you, count dimensions before touching anything else. Chapter 7's `conv2d` helper hid this with `x[None, None]`; from here on we handle it in the open.

8.1. Nine numbers, learned

Here is a game. I have a feature detector — one of Chapter 7's zoo, but I won't tell you which. I will only show you what it does: you hand it images, it hands back feature maps. Your job is to build a detector that matches it.

With the tools of Part I, this is not even hard. A kernel applied by Equation 7.1 is just nine numbers entering a dot product → the output is differentiable with respect to those numbers → gradient descent applies. Start from a random kernel and run the loop we have run since Chapter 1: predict → measure → step.

```
mystery = torch.tensor([[-1., 0., 1.],
                        [-2., 0., 2.],
                        [-1., 0., 1.]])            # (don't peek)
imgs = X_tr[:256]                                  # (256, 1, 28, 28)
with torch.no_grad():
    target = F.conv2d(imgs, mystery[None, None])   # what the detector does

torch.manual_seed(0)
K = 0.1 * torch.randn(1, 1, 3, 3)
K_init = K.clone().squeeze()
K.requires_grad_(True)

lr = 0.5
for step in range(601):
    pred = F.conv2d(imgs, K)                       # predict
    loss = F.mse_loss(pred, target)                # measure
    loss.backward()
    with torch.no_grad():                          # step
        K -= lr * K.grad
        K.grad.zero_()
    if step % 200 == 0:
        print(f"step {step:3d}   loss {loss.item():.2e}")

print(f"\nlearned kernel, rounded:\n{K.detach().squeeze().round(decimals=2)}")
print(f"max |learned - mystery| = {(K.detach().squeeze() - mystery).abs().max():.3f}")
```

```
step   0   loss 1.37e+00
step 200   loss 5.53e-04
```

```
step 400   loss 4.80e-05
step 600   loss 4.42e-06

learned kernel, rounded:
tensor([[-1.0100, -0.0000,  1.0100],
        [-1.9800, -0.0100,  1.9900],
        [-1.0100,  0.0100,  1.0000]])
max |learned - mystery| = 0.018
```

The loss falls more than five orders of magnitude, and the learned kernel is the vertical Sobel detector to within 0.02 in every entry. Gradient descent, given nothing but input–output pairs, has *reinvented* a filter that took computer-vision researchers years of squinting at image gradients to design.

```python
fig, axes = plt.subplots(1, 3, figsize=(7, 2.6))
lim = 2.2
for ax, k, title in zip(axes, [K_init, K.detach().squeeze(), mystery],
                        ["random start", "learned", "Sobel (truth)"]):
    im = ax.imshow(k, cmap="RdBu_r", vmin=-lim, vmax=lim)
    ax.set_title(title, fontsize=10); ax.set_xticks([]); ax.set_yticks([])
fig.colorbar(im, ax=axes, shrink=0.8)
plt.show()
```

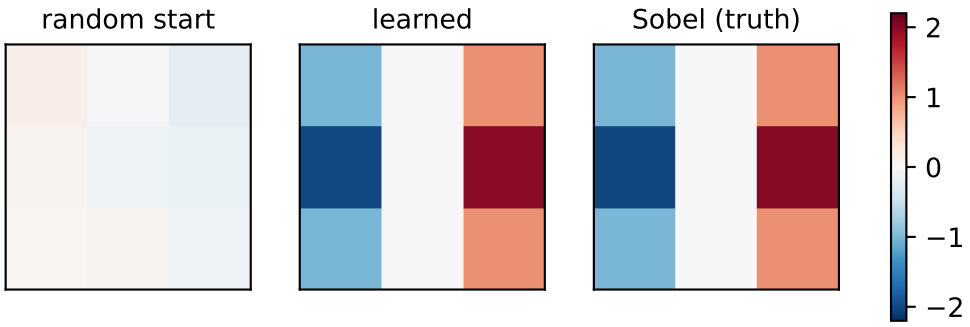

Figure 8.1.: The mystery-detector game. Left: the random starting kernel. Middle: the kernel gradient descent found from input–output pairs alone. Right: the hand-crafted vertical Sobel detector from Chapter 7's zoo. Nobody told the middle panel what an edge is.

The pivot of Part II

Read that cell again, because the entire deep-learning revolution in vision is that cell writ large. Chapter 7 ended with the bottleneck: hand-crafted kernels are limited to patterns a human can articulate in nine numbers — nobody can write down "cat ear." The fix is not better articulation. It is to stop articulating: put the numbers on the gradient tape and let

the task pull the detector it needs out of the data. *What if the template were learnable?* It is, and the machinery has been in our hands since Chapter 4.

Two honest disclaimers about the game, both of which make the real thing *more* remarkable, not less. First, the game was rigged: the target came from a known kernel, so a perfect answer existed and the loss surface was a clean convex bowl (the output is linear in V, and squared error in a linear map is Chapter 1's setting all over again). Second, and more important: **in a real CNN, nobody supplies the target feature maps.** The only supervision is the classification loss at the far end of the network. Blame flows backward through every layer — exactly the mechanics of Chapter 5 — and the kernels that emerge are whatever detectors the task itself demands. We chose Sobel here; a trained network might discover it needs something no one has named.

One backpropagation detail deserves a sentence, because it is the same fan-out rule we met in Chapter 5. The *same* nine numbers are used at every position of the slide → every position contributes blame to the shared kernel. Those contributions accumulate, then the loss reduction supplies its scale; `F.mse_loss` with the default `reduction="mean"` averages over batch, channel, and spatial elements. Weight sharing is not just a parameter economy. At training time every patch of every image teaches the one shared template.

8.2. From one detector to a layer of detectives

One learnable kernel gives one feature map: one specialist, sweeping the image with one magnifying glass. But Chapter 7's zoo already told us one specialist is not enough — vertical edges, horizontal edges, blobs, textures all carry information. So we hire a *team*. The lecture's image is worth keeping: employ six detectives, each with a different expertise, each producing their own annotated map of the crime scene.

That is all the **channel** machinery is:

- **Out-channels: more detectives.** A conv layer with 6 output channels owns 6 independent kernels and produces a stack of 6 feature maps.
- **In-channels: detectives read the whole stack.** The *next* layer's input is that 6-deep stack, so each of its kernels grows a third dimension: a $6 \times 5 \times 5$ volume that looks at all six incoming maps *simultaneously* and sums the evidence into one output map, one bias per output channel at the end.

For an input stack X with C_{in} channels, output map o is

$$[H_o]_{i,j} \;=\; b_o \;+\; \sum_{c=1}^{C_{\text{in}}} \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} [V_{o,c}]_{a,b}\, [X_c]_{i+a,\, j+b}, \tag{8.1}$$

which is Equation 7.1 run once per input channel and summed — followed, as always since Chapter 3, by a nonlinearity. A convolutional layer is Chapter 3's recipe with a structured linear map: dot products → add bias → ReLU. Nothing in the recipe changed; only the wiring of the linear part did.

```
conv1 = nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, padding=2)
conv2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5)

x = X_tr[:1]                                    # (1, 1, 28, 28): one garment
h1 = F.relu(conv1(x))
h2 = F.relu(conv2(h1))
print(f"input   {tuple(x.shape)}")
print(f"conv1   {tuple(h1.shape)}   kernels: {tuple(conv1.weight.shape)}")
print(f"conv2   {tuple(h2.shape)}  kernels: {tuple(conv2.weight.shape)}")
```

```
input   (1, 1, 28, 28)
conv1   (1, 6, 28, 28)   kernels: (6, 1, 5, 5)
conv2   (1, 16, 24, 24)  kernels: (16, 6, 5, 5)
```

Look at `conv2.weight`: sixteen kernels, each $6 \times 5 \times 5$. This is where features start *composing*. Suppose detective 3 in the first layer is a vertical-edge expert and detective 5 hunts horizontal edges. A second-layer kernel that weights both maps positively in the same neighborhood fires precisely when both experts agree → a **corner detector**, built from evidence no single first-layer kernel could see. The cross-talk between channels is how edges become corners, corners become textures, and textures become parts. We promised this compositional ladder in Chapter 3; here is the hardware it runs on.

Chapter 7's zoo can stage the story before any learning enters. Give the first layer two hand-crafted experts — vertical and horizontal edge energy — then hand-build one second-layer kernel that reads both reports at once:

```
square = torch.zeros(28, 28)
square[7:21, 7:21] = 1.0
sobel_v = torch.tensor([[-1., 0., 1.], [-2., 0., 2.], [-1., 0., 1.]])
v = F.conv2d(square[None, None], sobel_v[None, None], padding=1).abs()
h = F.conv2d(square[None, None], sobel_v.T.contiguous()[None, None], padding=1).abs()

experts = torch.cat([v, h], dim=1)              # (1, 2, 28, 28): two reports
corner_k = torch.full((1, 2, 5, 5), 1 / 50)     # one kernel, spanning both
corners = F.conv2d(experts, corner_k, padding=2).squeeze()

fig, axes = plt.subplots(1, 4, figsize=(9, 2.5))
panels = [(square, "input"), (v.squeeze(), "vertical-edge expert"),
          (h.squeeze(), "horizontal-edge expert"), (corners, "their reader: corners")]
for ax, (t, title) in zip(axes, panels):
    ax.imshow(t, cmap="gray"); ax.set_title(title, fontsize=9); ax.axis("off")
plt.tight_layout(); plt.show()
```

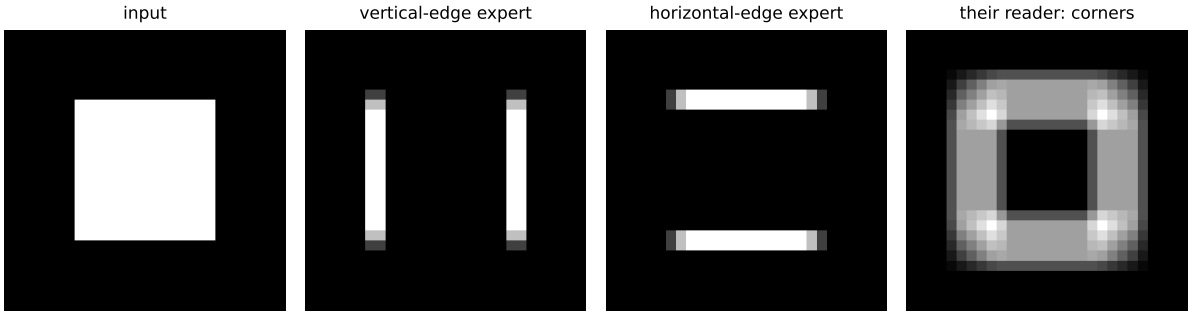

Figure 8.2.: Cross-talk staged by hand. Two first-layer experts report edge energy (absolute Sobel response) on a square. One second-layer kernel spanning both channels (all-positive weights; Equation 8.1 with two input channels) fires hardest exactly where both experts agree — the four corners, a feature neither channel could see alone.

The four brightest cells of the right panel are exactly the square's four corners: a corner detector, assembled from experts that individually know nothing about corners. In a trained CNN, gradient descent builds combinations like this — and far stranger ones — on its own.

> **`nn` versus `F`, appliances versus recipes**
>
> The lecture's analogy: `nn` modules are *appliances* — they have knobs and memory, and PyTorch carries their state around for you (`nn.Conv2d` owns its kernels and biases as `nn.Parameter`s, registered for autograd and visible to the optimizer). `F` functions are *recipes* — stateless, everything passed in by hand. In Chapter 7 the kernel was ours, so `F.conv2d` was the honest choice; now the kernel is the layer's business, so `nn.Conv2d` is. Use appliances for anything with parameters, recipes for everything else (`F.relu`, `F.max_pool2d`).

8.3. Padding and stride: the bookkeeping with a payoff

Chapter 7 left an administrative debt: a $k \times k$ kernel on an $n \times n$ image yields only $(n-k+1) \times (n-k+1)$ outputs, because the window must stay inside the frame. For one layer, a nuisance. For a *deep* stack, fatal: $28 \to 24 \to 20 \to \cdots$ and the feature map shrinks toward nothing, with edge pixels contributing to ever fewer windows along the way.

The fix is blunt: **padding**. Add a border of zeros around the input so the window can center itself on every real pixel. For a 5×5 kernel, a 2-pixel border returns exactly the input size → that is why `conv1` above says `padding=2`. Padding decouples "how deep can the network be" from "how fast do the maps shrink," and that decoupling is what makes deep stacks possible at all.

Sometimes, though, we *want* to shrink — coarser maps mean less computation and a wider view per pixel. Then we shrink on purpose with **stride**: slide the window s pixels at a time instead of

one. Both knobs live in one formula, worth memorizing because you will run it in your head for every architecture you ever read:

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1. \tag{8.2}$$

The numerator is the padded length minus the window, i.e. how far the window can travel; dividing by the step and flooring counts the stops; +1 counts the starting position. The lecture's three worked cases, verified:

```
x8 = torch.randn(1, 1, 8, 8)
for p, s in [(0, 1), (1, 1), (1, 2)]:
    out = F.conv2d(x8, torch.randn(1, 1, 3, 3), padding=p, stride=s)
    n_out = (8 + 2 * p - 3) // s + 1
    print(f"n=8, k=3, p={p}, s={s}:  formula {n_out}  torch {tuple(out.shape[2:])}")
```

```
n=8, k=3, p=0, s=1:  formula 6  torch (6, 6)
n=8, k=3, p=1, s=1:  formula 8  torch (8, 8)
n=8, k=3, p=1, s=2:  formula 4  torch (4, 4)
```

8.4. Pooling: trading exact location for local tolerance

Now for the deepest design decision in the network, and it starts from the asset Chapter 7 banked. Under matched boundary rules, convolution is equivariant on the interior: shift the garment, and every feature map shifts in lockstep. The detective's map of clues faithfully tracks the scene. But the network's final job is not mapping clues; it is a verdict. *Is this a trouser?* The answer must not depend on where the trouser stands. The feature-finding stage wants **equivariance** (where things are matters); the decision stage wants **invariance** (only what was found matters). Equivariance is what we have; invariance is what the classifier head needs → something must reduce sensitivity to location.

That converter is **pooling**. Slide a window over each feature map (no parameters this time) and summarize each neighborhood by one number:

- **Max pooling** asks: *what is the strongest clue here?* Keep the loudest activation, discard the rest.
- **Average pooling** asks: *what is the general vibe of this neighborhood?* Smoother, but a single sharp clue gets diluted by quiet neighbors.

```
t = torch.arange(16.).reshape(1, 1, 4, 4)
print(f"input:\n{t.squeeze()}")
print(f"max pool 2x2:\n{F.max_pool2d(t, 2).squeeze()}")
print(f"avg pool 2x2:\n{F.avg_pool2d(t, 2).squeeze()}")
```

```
input:
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11.],
        [12., 13., 14., 15.]])
max pool 2x2:
tensor([[ 5.,  7.],
        [13., 15.]])
avg pool 2x2:
tensor([[ 2.5000,  4.5000],
        [10.5000, 12.5000]])
```

The common configuration is the one above: 2×2 windows with stride 2, non-overlapping, halving each spatial dimension. Max is the default choice in practice, precisely because feature detection is about the strongest evidence, not the committee average.

How can this buy local tolerance? Start with a deliberately clean construction in which a shift becomes invisible:

```python
scene = torch.zeros(1, 1, 4, 4)
scene[0, 0, 1, 0] = 9.0        # a strong clue, upper-left region
scene[0, 0, 2, 2] = 3.0        # a weaker clue, lower-right region

shifted = torch.zeros_like(scene)
shifted[0, 0, 1, 1] = 9.0      # both clues moved one pixel right
shifted[0, 0, 2, 3] = 3.0

print(f"pooled original: {F.max_pool2d(scene,   2).squeeze().tolist()}")
print(f"pooled shifted:  {F.max_pool2d(shifted, 2).squeeze().tolist()}")
```

```
pooled original: [[9.0, 0.0], [0.0, 3.0]]
pooled shifted:  [[9.0, 0.0], [0.0, 3.0]]
```

Both clues moved, yet these particular pooled maps are *identical*: each isolated maximum stayed inside its 2×2 window. That equality belongs to this construction, not to max pooling in general. A shift can change which values share a window, which maximum wins, or what crosses the boundary. Pooling can therefore buy local tolerance to jitter, but the amount depends on the activations and alignment; the end of this chapter measures it rather than assuming it.

> ⚠ Tolerance has a price tag
>
> Pooling can buy tolerance by *throwing away position*. After two rounds, the network knows a strong vertical edge exists in a region; it no longer knows exactly where. For classification this is the right trade — but it is a trade, and it will come due twice in this book. Deeper in Part II, tasks that need precise locations will have to be more careful with resolution. And in Part IV the tables turn completely: self-attention (Chapter 14) is so thoroughly

position-agnostic that we will have to *pay to put position back in*. Remember, when we get there, that position was something we once discarded on purpose.

8.5. How far a deep neuron sees

There is one more consequence of stacking, and it quietly resolves the tension Chapter 6 left us with. Every kernel here is tiny — 5×5, resolutely local. Yet recognizing a garment is a *global* judgment. Where does globality come from?

From depth. A neuron in the first feature map sees a 5×5 patch of the image: its **receptive field**. A neuron one layer up sees a 5×5 patch *of feature maps*, each pixel of which already summarizes a patch below → its receptive field on the original image is wider. Pooling accelerates this: after a 2×2/stride-2 pool, neighboring pixels in the pooled map stand two original pixels apart, so every later window covers twice the ground. For our upcoming network the ledger reads: conv1 sees 5×5 → pooling nudges it to 6×6 → conv2's 5×5 window, at stride-2 spacing, expands it to 14×14 → the final pool reaches 16×16 — over half the frame in every direction, from nothing but 5×5 looks.

```
import matplotlib.patches as mpatches

fig, ax = plt.subplots(figsize=(4.2, 3.6))
ax.imshow(X_val[1, 0], cmap="gray")
for size, color, lab in [(5, "#E57200", "conv1: 5×5"),
                         (14, "#5379AA", "conv2: 14×14"),
                         (16, "#DDA0DD", "pool2: 16×16")]:
    lo = 14 - size / 2
    ax.add_patch(mpatches.Rectangle((lo, lo), size, size, fill=False,
                                    lw=2, edgecolor=color, label=lab))
ax.legend(fontsize=8, loc="center left", bbox_to_anchor=(1.02, 0.5))
ax.axis("off")
plt.tight_layout(); plt.show()
```

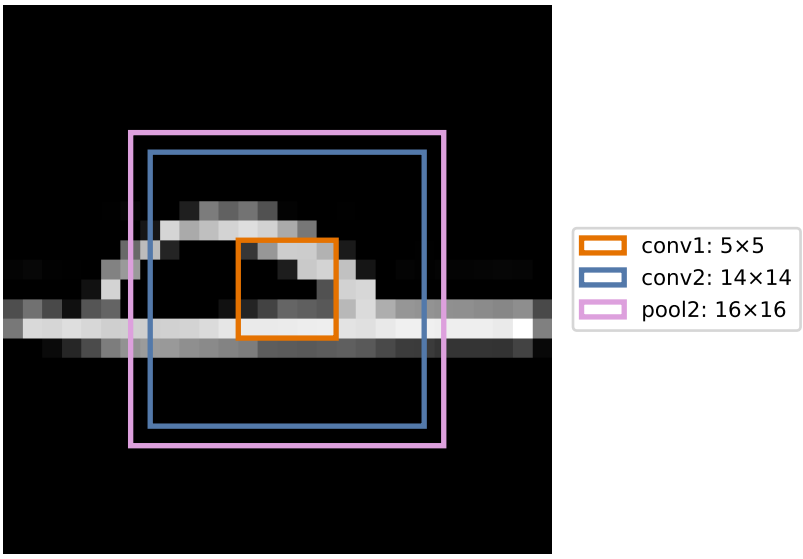

Figure 8.3.: What one neuron sees, by depth. Boxes mark the receptive field of a single unit in each stage of our network, centered on the same spot: conv1 sees a 5×5 patch, conv2 sees 14×14, the final pooled cell 16×16. Local wiring, increasingly global sight; the dense head then reads all 25 such overlapping views at once.

This is the compositional hierarchy of Chapter 3 given a strong architectural invitation. The MLP *could* represent features-of-features but nothing encouraged it to; locality and growing receptive fields encourage later layers to compose earlier responses. Learned channels often progress from edge-like patterns to textures and parts, but the architecture does not force those human labels. The CNN buys global sight gradually, paying for it with depth; hold that thought until Part IV, where attention will buy global sight in a single step and pay for it differently (Chapter 13).

8.6. LeNet: the whole machine

Time to assemble. The canonical blueprint is **LeNet**, Yann LeCun's digit-reading network, and its rhythm is the CNN design pattern to this day: *convolve, shrink, deepen; repeat; then decide.* Spatial size falls ($28 \to 14 \to 5$) while feature depth grows ($1 \to 6 \to 16$): the network trades "where" for "what" stage by stage, until a small dense head reads the final $16 \times 5 \times 5$ summary and votes.

```
class LeNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, 5, padding=2)   # (N,1,28,28) -> (N,6,28,28)
        self.conv2 = nn.Conv2d(6, 16, 5)             # (N,6,14,14) -> (N,16,10,10)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
```

```
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = F.max_pool2d(F.relu(self.conv1(x)), 2)    # -> (N, 6, 14, 14)
        x = F.max_pool2d(F.relu(self.conv2(x)), 2)    # -> (N, 16, 5, 5)
        x = x.flatten(1)                              # -> (N, 400)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x)                            # logits, (N, 10)
```

Every line is a tool we already own: Equation 8.1 twice, Equation 8.2 silently fixing `padding=2`, pooling twice, then Chapter 3's MLP as the decision head — and, per Chapter 2's advice, the model returns raw logits and lets `F.cross_entropy` handle the probabilities.

> **i** LeNet, then and now
>
> The LeNet family (LeCun and colleagues, 1989–1998) began by reading handwritten ZIP codes for the US Postal Service, and its mature form, LeNet-5, went on to read a substantial share of America's bank checks — convolutional networks were literally cashing checks decades before they were fashionable. Our variant keeps the classic skeleton but upgrades the internals with Part I's verdicts: ReLU where the original used sigmoid-family activations (Chapter 3, Chapter 5), max pooling where it used average-flavored subsampling, Adam as the optimizer. The lecture's live session adds one more modern stabilizer, batch normalization, which we meet properly in Chapter 9.

Before training, the parameter audit — because parameter *economy* was half of Chapter 7's sales pitch, and Chapter 6's MLP is the yardstick:

```
def make_mlp() -> nn.Sequential:
    return nn.Sequential(nn.Flatten(),
                         nn.Linear(784, 256), nn.ReLU(), nn.Linear(256, 10))

lenet, mlp = LeNet(), make_mlp()
for name, p in lenet.named_parameters():
    print(f"{name:14s} {str(tuple(p.shape)):16s} {p.numel():>7,}")
n_conv = sum(p.numel() for n, p in lenet.named_parameters() if "conv" in n)
n_lenet = sum(p.numel() for p in lenet.parameters())
n_mlp = sum(p.numel() for p in mlp.parameters())
print(f"\nLeNet: {n_lenet:,} parameters ({n_conv:,} of them convolutional)")
print(f"MLP:   {n_mlp:,} parameters")
```

```
conv1.weight   (6, 1, 5, 5)         150
conv1.bias     (6,)                   6
conv2.weight   (16, 6, 5, 5)      2,400
conv2.bias     (16,)                 16
```

```
fc1.weight      (120, 400)        48,000
fc1.bias        (120,)               120
fc2.weight      (84, 120)         10,080
fc2.bias        (84,)                 84
fc3.weight      (10, 84)             840
fc3.bias        (10,)                 10

LeNet: 61,706 parameters (2,572 of them convolutional)
MLP:   203,530 parameters
```

Two readings of this table. Against the MLP: LeNet carries 61,706 parameters to the MLP's 203,530 (less than a third) while performing far richer spatial computation. Weight sharing is doing exactly what Chapter 7's matrix view promised. Within LeNet: look where the parameters live. The two conv layers, the part that actually *sees*, own 2,572 parameters (about 4%) while the dense head hoards the rest. That imbalance is a design smell we will fix in Chapter 9, where modern architectures go deeper in convolution and lighter in the head.

Now train both models under one shared optimization recipe: same fitting split, optimizer, minibatch size, number of epochs, and therefore number of parameter updates. The comparison does **not** match parameter count or floating-point work; a convolution and a dense layer spend different computation per update. Architecture is the intended difference, compute is not controlled:

```python
def train_model(
    model_fn, X_fit: torch.Tensor = X_tr, y_fit: torch.Tensor = y_tr,
    epochs: int = 150, batch: int = 128, seed: int = 6050
) -> nn.Module:
    torch.manual_seed(seed)
    model = model_fn()
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    n = len(X_fit)
    for _ in range(epochs):
        perm = torch.randperm(n)
        for i in range(0, n, batch):
            idx = perm[i:i + batch]
            loss = F.cross_entropy(model(X_fit[idx]), y_fit[idx])
            opt.zero_grad(); loss.backward(); opt.step()
    return model

@torch.no_grad()
def accuracy(model: nn.Module, X: torch.Tensor, y: torch.Tensor) -> float:
    model.eval()
    return (model(X).argmax(1) == y).float().mean().item()

mlp = train_model(make_mlp)
lenet = train_model(LeNet)
for name, model in [("MLP  ", mlp), ("LeNet", lenet)]:
```

```
    print(f"{name}  train {accuracy(model, X_tr, y_tr):.1%}   "
          f"validation {accuracy(model, X_val, y_val):.1%}")
```

```
MLP    train 100.0%   validation 75.5%
LeNet  train 99.5%   validation 74.5%
```

Read the clean-validation column honestly: the MLP scores 75.5% and LeNet 74.5% in this single seeded run. A one-point difference is descriptive, not an uncertainty estimate, so it does not establish which architecture has higher expected clean accuracy. Chapter 6 already told us why a landslide is unlikely here: centered garments do not expose the MLP's main weakness. Both models nearly interpolate the fitting split. Our small-data regime makes a narrower point: LeNet reaches comparable clean accuracy with less than a third of the parameters. The shift experiment now asks whether it learned a different kind of representation.

8.7. The rematch

Chapter 6 convicted our MLP with one experiment: shift every validation garment two pixels right, and accuracy fell off a cliff — the full-frame templates kept scoring sleeves against background. We then promised that an architecture built on locality and weight sharing would face the same trial. The rematch:

```
def shift_right(X: torch.Tensor, px: int) -> torch.Tensor:
    out = torch.zeros_like(X)
    if px == 0:
        return X.clone()
    out[..., px:] = X[..., :-px]
    return out

shifts = list(range(5))
acc_mlp = [accuracy(mlp, shift_right(X_val, s), y_val) for s in shifts]
acc_cnn = [accuracy(lenet, shift_right(X_val, s), y_val) for s in shifts]
for s in shifts:
    print(f"shift {s}px:   MLP {acc_mlp[s]:.1%}   LeNet {acc_cnn[s]:.1%}")

plt.figure(figsize=(5.5, 3.2))
plt.plot(shifts, acc_mlp, "o-", color="#E57200", lw=2, ms=7, label="MLP (Ch. 6)")
plt.plot(shifts, acc_cnn, "s-", color="#232D4B", lw=2, ms=7, label="LeNet")
plt.axhline(0.1, ls=":", color="#B8B8A8")
plt.xlabel("shift (pixels right)"); plt.ylabel("validation accuracy")
plt.ylim(0, 0.9); plt.xticks(shifts); plt.legend()
plt.tight_layout(); plt.show()
```

```
shift 0px:   MLP 75.5%   LeNet 74.5%
```

```
shift 1px:   MLP 64.0%   LeNet 66.0%
shift 2px:   MLP 42.0%   LeNet 57.5%
shift 3px:   MLP 27.0%   LeNet 42.0%
shift 4px:   MLP 18.5%   LeNet 23.0%
```

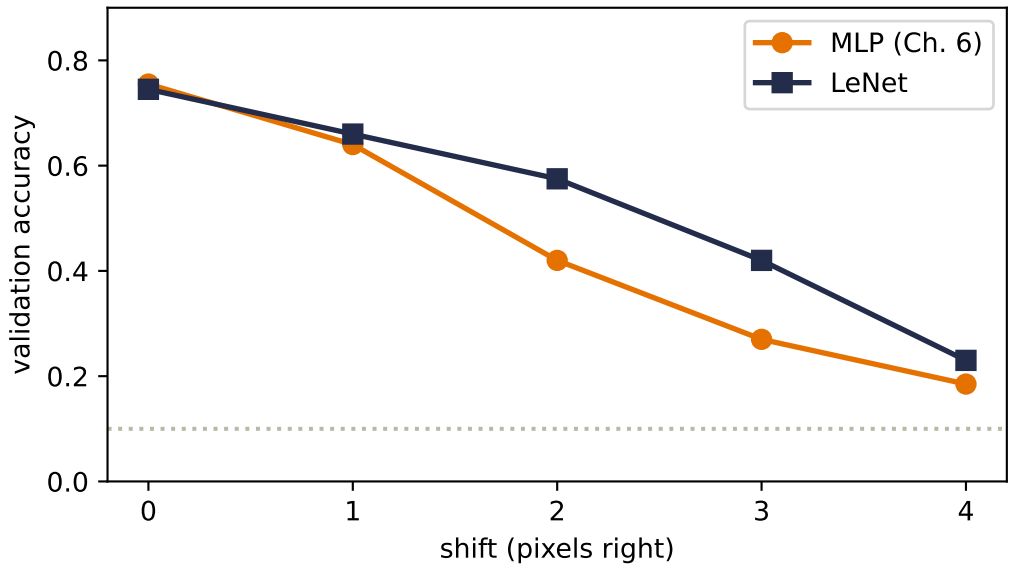

Figure 8.4.: The validation rematch. Both architectures use the same optimizer schedule, though not matched compute. At two pixels the MLP falls from 75.5% to 42.0%, while LeNet falls from 74.5% to 57.5%. The dotted line is chance.

There is the payoff, and it is worth stating precisely. At two pixels the MLP falls from 75.5% to 42.0%, while LeNet falls from 74.5% to 57.5%. That is one deterministic validation split, not a sampling distribution, but the within-run contrast is large. The mechanism is everything this chapter built: the kernels travel with the garment (equivariance), so the *features are still found*; pooling forgives some sub-window jitter (local insensitivity); only the coarse layout entering the head changes at all.

And yet the navy curve falls too. Also worth stating precisely, because it teaches the architecture's fine print. Two rounds of 2× pooling buy tolerance measured in a few pixels, not unlimited; and after `flatten(1)`, the dense head reads the final 5×5 grid *positionally* — a four-pixel input shift moves features roughly one full cell in that grid, and the head is as brittle to cell-level shifts as Chapter 6's MLP was to pixel-level ones. The inductive bias did not abolish the cliff; it moved it outward and flattened it into a slope. Reducing the remaining sensitivity is the next chapter's business, where we revisit depth and the position-sensitive head.

One last look inside, because Chapter 6 also showed us what the MLP's first layer *looked like* — global, smeared, garment-shaped templates (Chapter 6) — and promised that structure would change:

```
img = X_val[1:2]                                        # one validation garment
with torch.no_grad():
    fmaps = F.relu(lenet.conv1(img)).squeeze(0)         # (6, 28, 28)
kernels = lenet.conv1.weight.detach().squeeze(1)        # (6, 5, 5)

fig, axes = plt.subplots(2, 6, figsize=(9, 3.2))
lim = kernels.abs().max()
for i in range(6):
    axes[0, i].imshow(kernels[i], cmap="RdBu_r", vmin=-lim, vmax=lim)
    axes[1, i].imshow(fmaps[i], cmap="gray")
    axes[0, i].axis("off"); axes[1, i].axis("off")
plt.tight_layout(); plt.show()
```

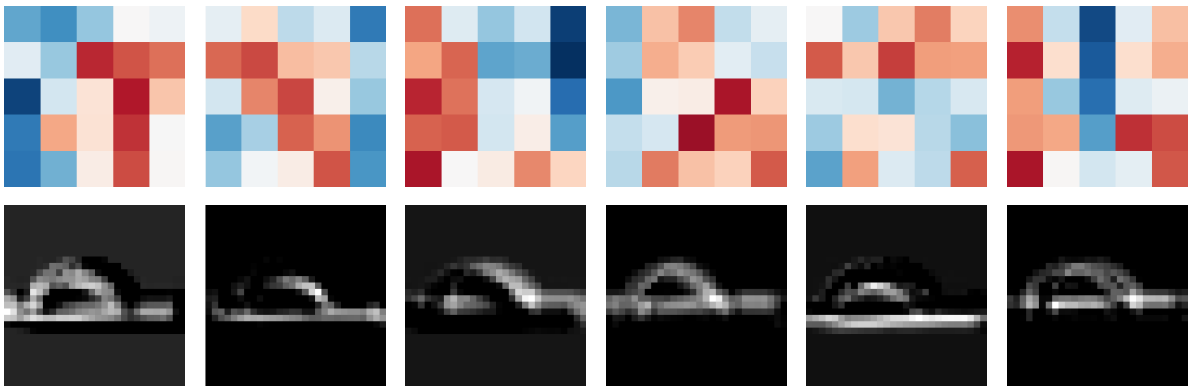

Figure 8.5.: What LeNet learned to look for. Top: six first-layer 5×5 kernels (red positive, blue negative). Bottom: their feature maps on one validation garment. The maps respond differently, but at this toy scale several kernels are diffuse and their semantics should not be over-interpreted. Compare their local support with Chapter 6's full-frame templates.

Six small, local kernels produce distinct response maps on the same garment. Some show directional structure; others remain noisy enough that naming a detector would tell more story than this one run supports. Nobody designed them. The classification loss, pulling blame backward through Equation 8.1 for roughly 1,200 optimizer steps, chose these local measurements.

8.8. One sealed test check

We have now fixed both architectures, the optimizer schedule, the shift size, and the two reported metrics. We refit each model on all 1,200 development images, then open the 600-image test set once:

```
mlp_final = train_model(make_mlp, X_dev, y_dev)
lenet_final = train_model(LeNet, X_dev, y_dev)
for name, model in [("MLP  ", mlp_final), ("LeNet", lenet_final)]:
    print(f"{name}  clean {accuracy(model, X_test, y_test):.1%}   "
          f"shift-2 {accuracy(model, shift_right(X_test, 2), y_test):.1%}")
```

```
MLP    clean 82.2%   shift-2 41.8%
LeNet  clean 82.5%   shift-2 62.3%
```

The sealed check confirms the validation diagnosis. Clean accuracy is 82.2% for the MLP and 82.5% for LeNet, a descriptive 0.3-point gap from one seed. Under the predeclared two-pixel shift, the gap becomes 41.8% versus 62.3%.

8.9. Okay, so —

1. **One change made the revolution**: the kernel's numbers became parameters. Gradient descent can recover a hand-crafted detector from data (our rigged game) and, more importantly, discover detectors nobody designed, supervised only by the task loss at the far end.
2. **Channels turn one detector into a team**: out-channels stack specialists' feature maps; in-channels let the next layer read all maps at once (Equation 8.1), which is how edges compose into corners and parts.
3. **Padding and stride are the shape budget**: $n_{\text{out}} = \lfloor (n + 2p - k)/s \rfloor + 1$ (Equation 8.2). Padding lets networks go deep without shrinking to nothing; stride shrinks on purpose.
4. **Pooling can turn equivariance into local shift tolerance** — the strongest clue per window, some position discarded. It is a purchase, and position is the currency.
5. **Receptive fields grow with depth**: $5 \to 6 \to 14 \to 16$ in our LeNet. Local wiring earns global sight gradually; the hierarchy Chapter 3 could only hope for is now encouraged by architecture.
6. **The verdict**: on the sealed test set, one seed puts clean accuracy at 82.2% for the MLP and 82.5% for LeNet; this does not establish an expected clean-accuracy difference. Under the predeclared two-pixel shift, the same models score 41.8% and 62.3%. The bias did not add capacity; it added the *right constraints*, exactly Chapter 6's prescription.

Sources and further reading

- LeCun et al., "Gradient-Based Learning Applied to Document Recognition" (1998): the primary LeNet account, including the architecture's document-recognition setting.
- Fukushima, "Neocognitron: A Self-organizing Neural Network Model" (1980): historical context for alternating feature extraction and spatial tolerance.
- Zeiler and Fergus, "Visualizing and Understanding Convolutional Networks" (2014): methods and evidence for inspecting learned convolutional features without over-reading a single filter image.

- Dumoulin and Visin, "A Guide to Convolution Arithmetic for Deep Learning" (2016): a compact companion for checking convolution, pooling, and receptive-field shapes.

Exercises

1. **(Pencil.)** Run Equation 8.2 through LeNet layer by layer, starting from 28×28: verify every shape comment in the `LeNet` class, then verify the parameter counts of `conv1` and `conv2` from the table (weights *and* biases).
2. **(Pencil.)** Verify the receptive-field ledger $5 \to 6 \to 14 \to 16$. (Track two numbers per stage: the field size, and the *jump* — the spacing in input pixels between neighboring units — which each stride-2 pool doubles.) Then show that no convolutional or pooling unit in LeNet ever sees the full 28×28 frame. Where does globality finally enter the network?
3. **(Code.)** Play the mystery-detector game with the 3×3 blur kernel as the target. Does gradient descent recover it as cleanly as Sobel? Now rerun with only 8 training images instead of 256. What happens to the recovered kernel, and what does that say about data volume versus parameter identifiability?
4. **(Code.)** Swap LeNet's max pooling for average pooling and retrain. Compare clean accuracy and the shift curve. Which changes more, and does the result match the "strongest clue vs. general vibe" story?
5. **(Pencil.)** The final dense head receives 25 spatial positions per feature map. Explain why this can reintroduce shift sensitivity after equivariant convolutions. Describe what information a position-insensitive head would keep and discard; we will build one in Chapter 9.

9. Modern CNNs and Transfer Learning

Chapter 8 ended with a working machine and a report card. LeNet reads garments at 82.5% with 61,706 parameters; graceful under shift where the MLP cliff-dived. But the card has two demerits we wrote down explicitly: 96% of those parameters sit in the dense head rather than in the convolutions that do the seeing, and the shift cliff was softened, not abolished. And one IOU: the live session's batch normalization, promised for this chapter.

What happened after LeNet is a decade of architecture research, and it can drown you in named networks. The lecture compresses it into **four design questions**, and we will let them drive:

1. **Why 5×5?** Could smaller filters do the same job with fewer parameters?
2. **How do we go deeper** without gradients dying on the way down?
3. **Where do the parameters actually live**, and how do we evict them?
4. **Can we design reusable blocks**: optimized layer-combinations we stack, keeping a bird's-eye view instead of re-designing what already works?

Each question has a famous answer (VGG, ResNet, NiN's 1×1 + global pooling, and the block habit itself), and each answer is a constraint-shaped idea we can test on our own data. At the end, the practical superpower the lecture builds to, reusing a pretrained backbone, gets the most honest experiment in this book.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
from collections.abc import Callable

torch.manual_seed(6050)
train = torch.load("../../data/fashion-train.pt")
test = torch.load("../../data/fashion-test.pt")
X_tr = train["X"].float().unsqueeze(1) / 255.0    # (1200, 1, 28, 28)
X_te = test["X"].float().unsqueeze(1) / 255.0     # (600, 1, 28, 28)
y_tr, y_te, classes = train["y"], test["y"], train["classes"]

def train_existing(
    model: nn.Module, opt: torch.optim.Optimizer, epochs: int,
    batch: int = 128,
    data: tuple[torch.Tensor, torch.Tensor] | None = None,
) -> nn.Module:
    X, y = data if data is not None else (X_tr, y_tr)
    n = len(X)
```

```
    for _ in range(epochs):
        perm = torch.randperm(n)
        for i in range(0, n, batch):
            idx = perm[i:i + batch]
            model.train()
            loss = F.cross_entropy(model(X[idx]), y[idx])
            opt.zero_grad(); loss.backward(); opt.step()
    model.eval()
    return model

def train_model(model_fn: Callable[[], nn.Module], epochs: int,
                batch: int = 128, seed: int = 6050, lr: float = 1e-3,
                data: tuple[torch.Tensor, torch.Tensor] | None = None) -> nn.Module:
    torch.manual_seed(seed)
    model = model_fn()
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    return train_existing(model, opt, epochs, batch, data)

@torch.no_grad()
def accuracy(model: nn.Module, X: torch.Tensor, y: torch.Tensor) -> float:
    model.eval()
    return (model(X).argmax(1) == y).float().mean().item()

def count(model: nn.Module) -> int:
    return sum(p.numel() for p in model.parameters())
```

One evaluation label needs an honesty note. Chapter 8 already opened the 600-image holdout, and this chapter queries it repeatedly while comparing architectures. We keep the familiar `X_te` name, but treat it as the book's fixed **benchmark**. These comparisons are descriptive, not an unbiased estimate after model selection; a final claim would require a fresh, untouched test set.

Two small novelties in the recipe, both because this chapter's networks contain batch normalization: `model.train()` before each step and `model.eval()` before each evaluation. Why that matters is the subject of the second section.

9.1. Question 1: why 5×5? Stack small kernels instead

VGG's answer (Simonyan & Zisserman, 2014) is the cleanest idea in this chapter. Recall the receptive-field ledger of Chapter 8: stacking grows the field. Two stacked 3×3 convolutions let the second layer see 5×5 of the input, the *same* receptive field as one 5×5 kernel. But compare the bills, for a layer with C channels in and C out:

$$\underbrace{25\,C^2}_{\text{one } 5\times5} \qquad \text{versus} \qquad \underbrace{2 \times 9\,C^2 = 18\,C^2}_{\text{two } 3\times3\text{s}},$$

a 28% saving. The stacked pair also fires a ReLU *twice* where the big kernel fires once. Same sight, fewer parameters, more nonlinearity. Three 3×3s reach a 7×7 field for $27C^2$ against $49C^2$: the deeper you take the idea, the better the deal gets.

```
C = 32
one_5x5 = nn.Conv2d(C, C, 5, padding=2)
two_3x3 = nn.Sequential(nn.Conv2d(C, C, 3, padding=1), nn.ReLU(),
                        nn.Conv2d(C, C, 3, padding=1))
print(f"one 5x5:  {count(one_5x5):,} parameters, 1 ReLU after")
print(f"two 3x3s: {count(two_3x3):,} parameters, 2 ReLUs")
```

```
one 5x5:  25,632 parameters, 1 ReLU after
two 3x3s: 18,496 parameters, 2 ReLUs
```

One honest caveat before you re-derive the field equations of vision from this: the $18C^2 < 25C^2$ arithmetic assumes the channel width stays C through the stack. When a layer *grows* channels (as LeNet's $6 \to 16$ did), splitting it into two growing 3×3s can cost more, not less. VGG's design sidesteps this by keeping width constant inside each block and changing it only between blocks, which is exactly the shape our code will take. (Exercise 1 makes you find the break-even point.)

9.2. The stabilizer we owe you: batch normalization

Before we stack deeper than ever, we need the tool Chapter 8's live session used and we deferred. Here is the problem it solves. Watch what happens to the *scale* of activations as a signal crosses twelve freshly initialized 3×3 conv layers:

```
def stack12(with_bn: bool, ch: int = 16) -> nn.Sequential:
    layers = [nn.Conv2d(1, ch, 3, padding=1)]
    for _ in range(11):
        if with_bn:
            layers.append(nn.BatchNorm2d(ch))
        layers += [nn.ReLU(), nn.Conv2d(ch, ch, 3, padding=1)]
    return nn.Sequential(*layers)

torch.manual_seed(0)
x = X_tr[:256]
plain_stack = stack12(False)
bn_stack = stack12(True)
plain_convs = [m for m in plain_stack if isinstance(m, nn.Conv2d)]
bn_convs = [m for m in bn_stack if isinstance(m, nn.Conv2d)]
for source, target in zip(plain_convs, bn_convs):
    target.load_state_dict(source.state_dict())       # identical convolution weights
```

```
for tag, network in [("without BN", plain_stack), ("with BN   ", bn_stack)]:
    h, stds = x, []
    with torch.no_grad():
        for layer in network:
            h = layer(h)
            if isinstance(layer, nn.Conv2d):
                stds.append(h.std().item())
    print(f"{tag} layer stds: " + "  ".join(f"{s:.3f}" for s in stds[::2]))
```

```
without BN layer stds: 0.344  0.059  0.043  0.046  0.049  0.061
with BN    layer stds: 0.344  0.404  0.395  0.385  0.432  0.384
```

Without normalization the signal's spread collapses by an order of magnitude within a few layers and keeps sagging. Forward activation scales and backward gradients are not the same quantity, but both are shaped by the stack's weights and nonlinearities; poorly scaled activations are therefore a warning to inspect gradient flow directly, as we do below.

Batch normalization (Ioffe & Szegedy, 2015) re-standardizes at every layer. For each channel of an NCHW tensor, `BatchNorm2d` computes the mean and variance across the current minibatch **and both spatial axes** (N, H, and W), normalizes the activations, then lets the layer undo that transformation through two learnable knobs:

$$\hat{x} = \frac{x - \mu_{\text{batch}}}{\sqrt{\sigma^2_{\text{batch}} + \epsilon}}, \qquad y = \gamma\,\hat{x} + \beta. \tag{9.1}$$

The γ, β pair matters: normalization is a *reset to a healthy scale*, not a straitjacket. The network can learn to restore any mean and spread that helps, but it starts every layer from sane numbers. In the printout above, the BN column holds near a constant spread through all twelve layers: every layer trains from day one. The standard placement, which the live session uses and we adopt, is conv → BN → ReLU, with `bias=False` on the convolution, since β already provides the shift.

> ⚠ BN is two different machines: tell PyTorch which one you're running
>
> At training time BN normalizes by the *current batch's* statistics. At evaluation time there may be no batch (one image!), so it uses running averages collected during training. `model.train()` and `model.eval()` switch between the two. Forgetting the switch is the classic BN bug: evaluate in train mode and your predictions depend on whatever else happens to be in the batch; train in eval mode and BN never learns its statistics. Our `train_model` recipe flips the switch in both directions; look for it. Corollary: BN needs real batches to estimate statistics, so it gets unreliable at tiny batch sizes.

> **i** Normalize across *what*? A seed for Part IV
>
> Batch statistics are one choice among several. When we build transformers (Chapter 14), batches of variable-length sequences will make per-batch statistics awkward, and the same standardize-then-rescale idea will reappear computed per *token* instead: **layer normalization**. Same equation, different axis. Remember Equation 9.1 when you meet it.

9.3. Building with blocks: a small VGG

Now assemble Question 1's answer with Question 4's habit. Define the *atom* — conv, BN, ReLU — once; a block stacks atoms and pools; a network stacks blocks. This is the lecture's point about VGG versus its hand-tuned predecessors: you stop re-designing and start repeating.

```
def atom(c_in: int, c_out: int) -> list[nn.Module]:
    return [nn.Conv2d(c_in, c_out, 3, padding=1, bias=False),
            nn.BatchNorm2d(c_out), nn.ReLU()]

class VGGSmall(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            *atom(1, 16), *atom(16, 16), nn.MaxPool2d(2),    # -> (16, 14, 14)
            *atom(16, 32), *atom(32, 32), nn.MaxPool2d(2),   # -> (32, 7, 7)
        )
        self.head = nn.Sequential(nn.Flatten(),
                                  nn.Linear(32 * 7 * 7, 128), nn.ReLU(),
                                  nn.Linear(128, 10))
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.head(self.features(x))

vgg = train_model(VGGSmall, epochs=60)
n_head = count(vgg.head)
print(f"VGGSmall: {count(vgg):,} parameters ({n_head:,} in the head, "
      f"{n_head / count(vgg):.0%})")
print(f"test accuracy {accuracy(vgg, X_te, y_te):.1%}")
```

```
VGGSmall: 218,586 parameters (202,122 in the head, 92%)
test accuracy 86.7%
```

Two readings again. The good: this VGG-style recipe reaches 86.7%, four points above LeNet in this run. Because depth, width, normalization, kernel sizes, parameter count, and training duration all changed together, this comparison does not isolate which ingredient earned the gain. The bad: 218,586 parameters, *triple* LeNet's total, and the head's share got worse — 92% of the network is a dense layer reading a flattened grid. The convolutional diet succeeded and the total

got fatter anyway. Which forces the lecture's third question: where do the parameters live, and — his phrasing — where can we do the most damage to the count?

9.4. Question 3: 1×1 convolutions, and firing the flatten head

The bloat has one address: `nn.Linear(1568, 128)`. To evict it we need one more tool, odd-looking at first sight: a convolution whose kernel is 1×1.

A 1×1 kernel does no spatial mixing at all — it looks at a single pixel. But across *channels* it does exactly what Equation 8.1 says: at each position, take the C_{in} channel values and form C_{out} weighted combinations plus bias. Pause on what that is: a **linear model across the channels, run separately at every pixel** — Chapter 1's machine, miniaturized and stamped across the image. Add a ReLU and each pixel is running a tiny MLP over its own feature vector. The lecture's summary: we *summarize* channels, we don't discard them — compressing 64 feature maps into 32 loses far less than pooling them away, and each compression injects another nonlinearity almost for free.

That tool enables the **Network-in-Network** design (Lin et al., 2013), and with it the clean head. A NiN block is one 3×3 (some spatial mixing) followed by two 1×1s (per-pixel channel mixing). And the classifier becomes:

1. Let the last block produce a stack of feature maps.
2. **Global average pooling (GAP)**: average each map over all positions — 64 maps in, 64 numbers out. No parameters at all.
3. One small linear layer to the logits.

```
def nin_block(c_in: int, c_out: int) -> list[nn.Module]:
    return [nn.Conv2d(c_in, c_out, 3, padding=1, bias=False),
            nn.BatchNorm2d(c_out), nn.ReLU(),
            nn.Conv2d(c_out, c_out, 1), nn.ReLU(),
            nn.Conv2d(c_out, c_out, 1), nn.ReLU()]

class NINSmall(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            *nin_block(1, 16), nn.MaxPool2d(2),
            *nin_block(16, 32), nn.MaxPool2d(2),
            *nin_block(32, 64),
        )
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Linear(64, 10))
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.head(self.features(x))

torch.manual_seed(6050)
```

```
nin = NINSmall()
nin_opt = torch.optim.Adam(nin.parameters(), lr=1e-3)
nin = train_existing(nin, nin_opt, epochs=75)
```

```
nin = train_existing(nin, nin_opt, epochs=75)
print(f"NINSmall: {count(nin):,} parameters "
      f"(head: {count(nin.head):,})")
print(f"test accuracy {accuracy(nin, X_te, y_te):.1%}")
```

```
NINSmall: 35,034 parameters (head: 650)
test accuracy 76.2%
```

The parameter story is a rout: 35,034 total, a 650-parameter head, six times smaller than VGGSmall. The accuracy is 76.2%, and that dip is worth more attention than the win, so let us not rush past it. First, though, collect the promised payoff. Chapter 8 said the remaining shift-cliff was the flatten head's fault: it reads the final grid *positionally*. GAP averages over positions, so the head no longer assigns a different weight to every location. The rematch of the rematch:

```
class LeNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, 5, padding=2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(400, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = F.max_pool2d(F.relu(self.conv1(x)), 2)
        x = F.max_pool2d(F.relu(self.conv2(x)), 2)
        x = x.flatten(1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x)

lenet = train_model(LeNet, epochs=150)          # Chapter 8's model, same recipe

def shift_right(X: torch.Tensor, px: int) -> torch.Tensor:
    if px == 0:
        return X.clone()
    out = torch.zeros_like(X)
    out[..., px:] = X[..., :-px]
    return out
```

```
shifts = list(range(5))
acc_lenet = [accuracy(lenet, shift_right(X_te, s), y_te) for s in shifts]
acc_nin = [accuracy(nin, shift_right(X_te, s), y_te) for s in shifts]
for s in shifts:
    print(f"shift {s}px:   LeNet {acc_lenet[s]:.1%}   NIN+GAP {acc_nin[s]:.1%}")

plt.figure(figsize=(5.5, 3.2))
plt.plot(shifts, acc_lenet, "s-", color="#232D4B", lw=2, ms=7, label="LeNet (flatten head)")
plt.plot(shifts, acc_nin, "^-", color="#E57200", lw=2, ms=7, label="NINSmall (GAP head)")
plt.axhline(0.1, ls=":", color="#B8B8A8")
plt.xlabel("shift (pixels right)"); plt.ylabel("test accuracy")
plt.ylim(0, 0.9); plt.xticks(shifts); plt.legend()
plt.tight_layout(); plt.show()
```

```
shift 0px:   LeNet 82.5%   NIN+GAP 76.2%
shift 1px:   LeNet 74.8%   NIN+GAP 70.3%
shift 2px:   LeNet 62.3%   NIN+GAP 69.3%
shift 3px:   LeNet 45.3%   NIN+GAP 67.5%
shift 4px:   LeNet 26.8%   NIN+GAP 66.5%
```

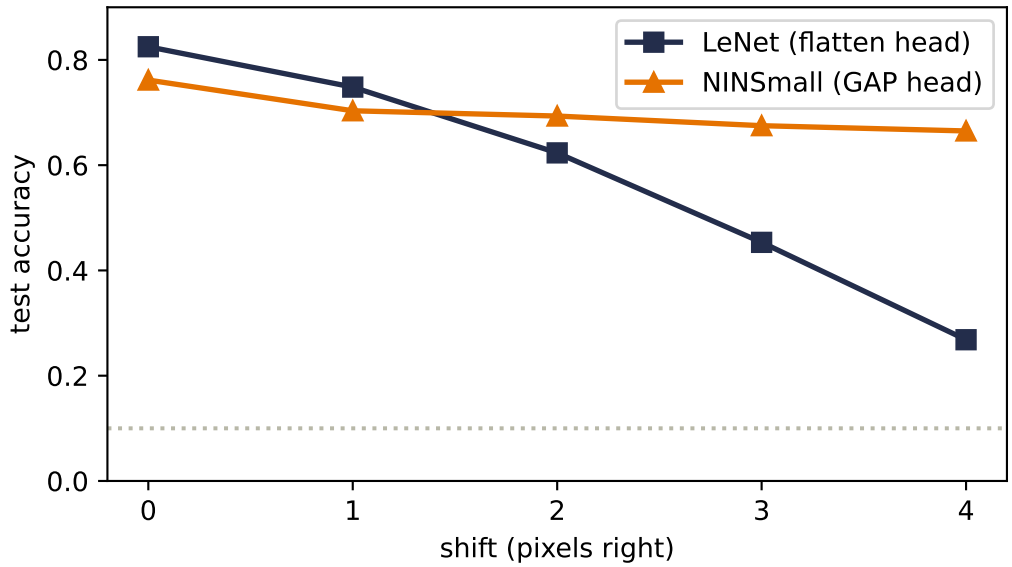

Figure 9.1.: Chapter 8's experiment, third round. LeNet's flatten head still slides below 30% by four pixels of shift. NINSmall with its global-average-pool head barely tilts: per-channel averages change much less when features move. GAP removes position-specific weights from the head, but padding, stride, pooling, and content clipped at the image edge keep the complete network from being exactly shift-invariant.

The curve that fell from 82% to 27% across this book's Part II is now a gentle slope from 76% to 67%. Position-specific weights are gone from NINSmall's head, and the result is consistent

with the promised robustness benefit. It is not a matched-head ablation: the trunks and training recipes differ too. The complete CNN is only approximately shift-tolerant; boundaries, strides, and pooling can still change its features.

Now the dip. On clean, centered test garments NINSmall trails LeNet by six points. That result is *consistent with* Chapter 8's warning: on centered data, position can carry information, and a GAP head cannot assign separate weights to separate locations. But GAP is not the only changed ingredient, so this run does not prove it caused the gap. With the full 60,000-image dataset the lecture's NiN reaches about 90%, near its LeNet. At our scale the comparison exposes a hypothesis worth testing with a matched-head ablation: shift tolerance may be bought with positional information.

> **i** Question 4's other famous block: Inception, in one breath
>
> GoogLeNet's Inception block (2014) answers "which kernel size?" with "all of them": parallel 1×1, 3×3, 5×5, and pooling branches, concatenated. Its enabling trick is the 1×1 *bottleneck*: compress channels before the expensive spatial kernels. The seed's arithmetic for one 5×5 branch at 256 channels: direct, $5^2\times256\times128\approx819$k parameters; with a 1×1 squeeze to 32 first, $256\times32+5^2\times32\times128\approx111$k — an 86% cut for the same nominal operation. We won't build Inception here (the principle, channel compression before spatial expense, is the transferable part), but Exercise 2 walks the arithmetic and the course assignment has you build the block itself.

9.5. Question 2: going deeper, and the wall you hit

VGG says depth is the direction. So push it: stack twenty of our two-conv blocks (with BN, per the recipe, on a 14×14 grid so the experiment fits a laptop) and compare against the *identical* stack with one change we'll reveal after the numbers.

```
class Block(nn.Module):
    def __init__(self, ch: int, residual: bool):
        super().__init__()
        self.c1 = nn.Conv2d(ch, ch, 3, padding=1, bias=False)
        self.b1 = nn.BatchNorm2d(ch)
        self.c2 = nn.Conv2d(ch, ch, 3, padding=1, bias=False)
        self.b2 = nn.BatchNorm2d(ch)
        self.residual = residual
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        y = self.b2(self.c2(F.relu(self.b1(self.c1(x)))))
        return F.relu(y + x) if self.residual else F.relu(y)

class DeepNet(nn.Module):
    def __init__(self, nblocks: int, residual: bool, ch: int = 12):
        super().__init__()
        self.stem = nn.Conv2d(1, ch, 3, padding=1)
```

```
        self.pool = nn.MaxPool2d(2)                          # 28 -> 14 early
        self.blocks = nn.Sequential(*[Block(ch, residual)
                                      for _ in range(nblocks)])
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Linear(ch, 10))
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.head(self.blocks(self.pool(F.relu(self.stem(x)))))

plain = train_model(lambda: DeepNet(20, residual=False), epochs=30)
print(f"plain-20     train {accuracy(plain, X_tr, y_tr):.1%}   "
      f"test {accuracy(plain, X_te, y_te):.1%}")
```

```
plain-20     train 60.8%   test 51.0%
```

```
resid = train_model(lambda: DeepNet(20, residual=True), epochs=30)
print(f"residual-20  train {accuracy(resid, X_tr, y_tr):.1%}   "
      f"test {accuracy(resid, X_te, y_te):.1%}")
```

```
residual-20  train 100.0%   test 77.8%
```

Look at the *training* column first, because it carries the whole lesson. The plain 40-conv-layer network cannot even fit the 1,200 images it sees every epoch: 61% train accuracy, while its residual twin memorizes them and generalizes twenty-seven points better. This is an **optimization failure** in the same family as the degradation problem He et al. reported in 2015, where their 56-layer plain net trained worse than their 20-layer one. Our experiment isolates the residual rescue at one depth; a strict demonstration that adding depth degrades a plain network would also require a shallower plain control.

The one change: each block computes $F(x)$ and outputs $F(x) + x$. A **residual connection** lets the input skip over the block and adds it back.

$$H(x) = F(x) + x \qquad \Longrightarrow \qquad \frac{\partial L}{\partial x} = \frac{\partial L}{\partial H}\left(\frac{\partial F}{\partial x} + I\right). \tag{9.2}$$

```
from matplotlib.patches import FancyBboxPatch, Circle

fig, ax = plt.subplots(figsize=(7.2, 2.5))
ax.set_xlim(0, 10); ax.set_ylim(0, 4); ax.axis("off")

def box(x: float, label: str, color: str) -> None:
    patch = FancyBboxPatch((x, 1.25), 1.65, 1.0,
                           boxstyle="round,pad=0.08", facecolor=color,
                           edgecolor="#232D4B", linewidth=1.3)
    ax.add_patch(patch)
```

```
    ax.text(x + 0.825, 1.75, label, ha="center", va="center", fontsize=9)

ax.text(0.35, 1.75, "$x$", ha="center", va="center", fontsize=12)
box(1.2, "conv-BN-ReLU", "#E8F0F8")
box(3.55, "conv-BN", "#E8F0F8")
add = Circle((6.45, 1.75), 0.28, facecolor="#F9DCBF",
             edgecolor="#E57200", linewidth=1.5)
ax.add_patch(add); ax.text(6.45, 1.75, "+", ha="center", va="center", fontsize=14)
box(7.25, "ReLU", "#F4F4EF")
ax.text(9.55, 1.75, "$H(x)$", ha="center", va="center", fontsize=12)

for start, end in [((0.55, 1.75), (1.2, 1.75)),
                   ((2.85, 1.75), (3.55, 1.75)),
                   ((5.2, 1.75), (6.15, 1.75)),
                   ((6.73, 1.75), (7.25, 1.75)),
                   ((8.9, 1.75), (9.3, 1.75))]:
    ax.annotate("", xy=end, xytext=start,
                arrowprops=dict(arrowstyle="->", color="#232D4B", lw=1.5))
ax.plot([0.55, 0.8, 0.8, 6.45, 6.45], [1.75, 1.75, 3.2, 3.2, 2.05],
        color="#E57200", lw=2)
ax.annotate("", xy=(6.45, 2.05), xytext=(6.45, 2.55),
            arrowprops=dict(arrowstyle="->", color="#E57200", lw=2))
ax.text(3.65, 3.45, "identity shortcut: $x$", color="#B45309",
        ha="center", fontsize=10)
ax.text(4.35, 0.75, "learned correction $F(x)$", color="#232D4B",
        ha="center", fontsize=10)
plt.tight_layout(); plt.show()
```

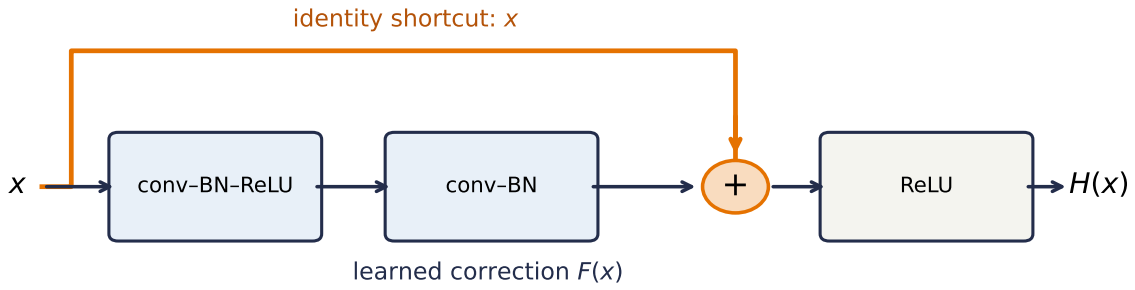

Figure 9.2.: A residual block has two routes. The learned branch computes a correction $F(x)$; the identity branch carries x unchanged to the addition. Even when the learned branch has a difficult Jacobian, the shortcut leaves a direct route for activations and gradients.

Read the right-hand side with Chapter 5 eyes. Blame arriving at a block's output reaches its input through two additive terms: the learned branch $\partial F/\partial x$ and the identity I. The identity contribution gives gradient flow a direct route that does not itself multiply by the learned weights. It is powerful, not magical: the learned Jacobian can still reinforce, distort, or even partly cancel that term. Chapter 5 called ReLU's open gate a *gradient superhighway* through a single unit; a skip connection builds a direct lane into every block. The block only needs to learn the *residual*, the correction on top of "pass it through," and doing nothing is easy to represent: $F = 0$.

Here is the highway visible at initialization, extending Chapter 5's depth experiment from dense layers to conv stacks:

```
class PlainNoBN(nn.Module):
    def __init__(self, nconvs: int, ch: int = 12):
        super().__init__()
        self.stem = nn.Conv2d(1, ch, 3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.layers = nn.Sequential(*[m for _ in range(nconvs)
                                      for m in (nn.Conv2d(ch, ch, 3, padding=1),
                                                nn.ReLU())])
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Linear(ch, 10))
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.head(self.layers(self.pool(F.relu(self.stem(x)))))

def stem_grad(make_net: Callable[[], nn.Module]) -> tuple[float, float, int, int]:
    torch.manual_seed(0)
    net = make_net()
    F.cross_entropy(net(X_tr[:128]), y_tr[:128]).backward()
    grad = net.stem.weight.grad
    return (grad.double().norm().item(), grad.abs().max().item(),
            torch.count_nonzero(grad).item(), grad.numel())

depths = [4, 12, 24]                                    # blocks (2 convs each)
diagnostics = {
    "plain, no BN": [stem_grad(lambda d=d: PlainNoBN(2 * d)) for d in depths],
    "plain + BN": [stem_grad(lambda d=d: DeepNet(d, False)) for d in depths],
    "residual + BN": [stem_grad(lambda d=d: DeepNet(d, True)) for d in depths],
}
curves = {name: [row[0] for row in rows]
          for name, rows in diagnostics.items()}
for name, ys in curves.items():
    print(f"{name:14s}: " + "   ".join(f"{v:.1e}" for v in ys))
norm64, maxabs, nonzero, total = diagnostics["plain, no BN"][-1]
print(f"48-layer no-BN check: max |component| {maxabs:.1e}; "
      f"nonzero {nonzero}/{total}; float64 norm {norm64:.1e}")

plt.figure(figsize=(6.2, 3.4))
```

```
for (name, ys), color in zip(curves.items(),
                             ["#B8B8A8", "#E57200", "#232D4B"]):
    xs = [2 * d for d in depths]
    plt.semilogy(xs, ys, "o-", ms=5, color=color, label=name)
plt.xlabel("conv layers"); plt.ylabel(r"$\|\partial L / \partial W^{(\mathrm{stem})}\|$")
plt.legend(); plt.tight_layout(); plt.show()
```

```
plain, no BN  : 9.0e-06   3.3e-13   2.2e-23
plain + BN    : 7.6e-02   5.0e-01   8.3e+01
residual + BN : 1.6e-01   4.9e-01   5.0e-01
48-layer no-BN check: max |component| 5.2e-24; nonzero 108/108; float64 norm 2.2e-
23
```

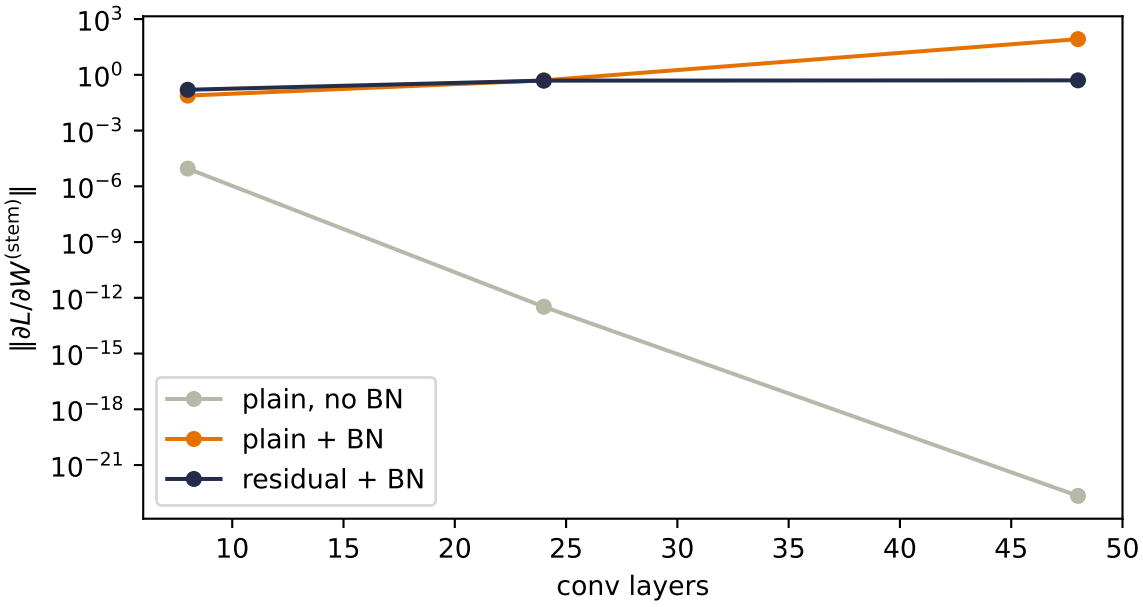

Figure 9.3.: Gradient magnitude reaching the stem (first layer) at initialization, versus depth, for three designs. In the 48-layer no-BN stack, the true norm is 2.2e-23; a float32 norm calculation reports zero because squaring these nonzero components underflows. BN fixes the vanishing but overshoots: in plain stacks the gradient grows with depth. Residual blocks keep the gradient within one order of magnitude across these depths.

The most important seed this chapter plants

Residual connections are not only a CNN trick. They became a central design device in many deep architectures; when we assemble a transformer in Chapter 14, every attention layer and every feed-forward layer will be wrapped as $x + F(x)$, and the freshly-met layer normalization will sit beside each skip. The picture to carry forward: a *residual stream* with direct additive routes through the network, each block reading from it and writing a

> correction back. Those routes improve conditioning without making the stream immune to learned-branch interactions. Attention will be one kind of correction. You now own both parts of that skeleton.

9.6. The scorecard: what a decade of design bought

The lecture closes its architecture tour with a scatter worth reproducing: every model of Part II on one canvas, accuracy against parameters, each trained to convergence under our shared recipe:

```
models_ = {
    "LeNet": (lenet, "#B8B8A8"),
    "VGGSmall": (vgg, "#E57200"),
    "NIN+GAP": (nin, "#232D4B"),
    "plain-20": (plain, "#DDA0DD"),
    "residual-20": (resid, "#5379AA"),
}
plt.figure(figsize=(6.2, 3.6))
for name, (m, color) in models_.items():
    p, a = count(m), accuracy(m, X_te, y_te)
    plt.scatter(p, a, s=70, color=color, zorder=3)
    plt.annotate(name, (p, a), textcoords="offset points",
                 xytext=(8, -3), fontsize=9)
plt.xscale("log")
plt.xlabel("parameters (log scale)"); plt.ylabel("test accuracy")
plt.ylim(0.45, 0.95)
plt.tight_layout(); plt.show()
```

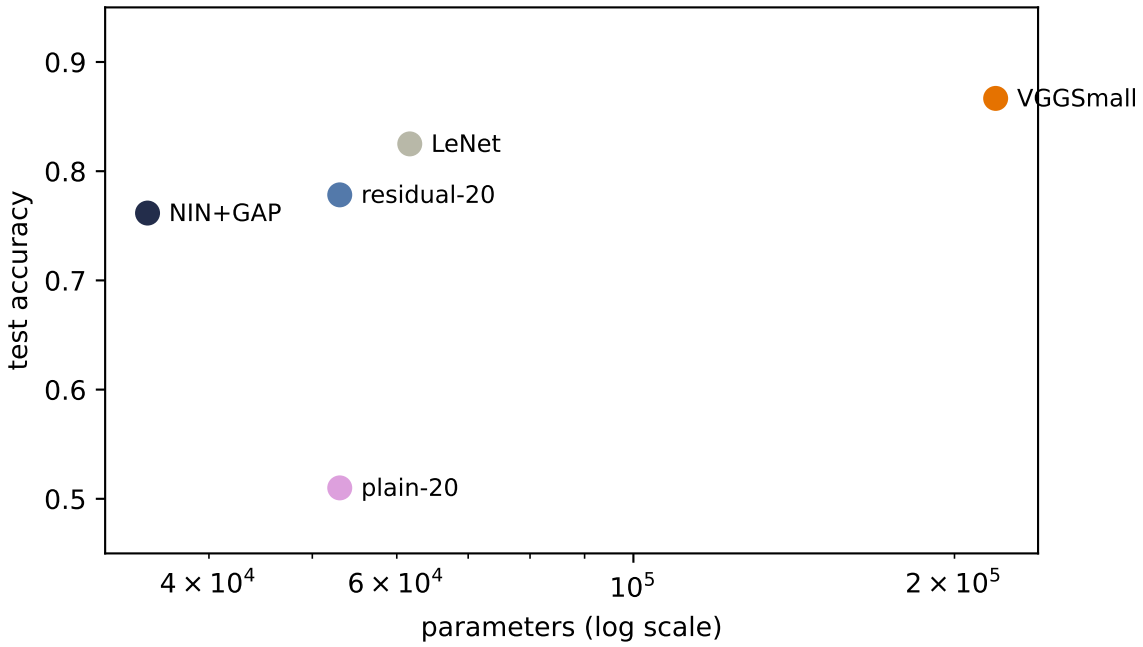

Figure 9.4.: Part II's models on one canvas (our 1,200-image subset; each model trained to convergence with the shared Adam recipe). VGG-style depth buys accuracy but hoards parameters in its flatten head; NiN+GAP evicts the head at a visible cost in clean accuracy at this data scale; residual blocks let a 53,050-parameter network go 40 layers deep. The lecture's full-data frontier, where the models exceed 89%, needs data our subset does not have; the shape of the trade-offs is already visible.

9.7. Transfer learning: the mechanics, and an honest experiment

Everything so far trains from scratch. The lecture's final move is the one that defines practice today: most image features are *shared* — edges, textures, parts transfer across tasks — so train one strong backbone once, on enormous data, and reuse it everywhere. The mechanics come in two strengths:

- **Linear probe**: freeze the pretrained trunk entirely (`requires_grad = False`), extract its features, train only a new linear head on your labels.
- **Fine-tuning**: additionally unfreeze the last block or two, training them at a *much* smaller learning rate than the fresh head, so the pretrained features adapt gently instead of being trampled.

We will test the idea properly. The task: classify the three shoe classes (sandal, sneaker, ankle boot) from **ten labeled examples each**. The 30-image regime is exactly where transfer should shine — too few labels to learn features, goes the story, so imported features should dominate. Three contenders:

1. **From scratch**: our VGG-style trunk trained on the 30 images alone.

2. **Our own pretrained trunk**: the same trunk pretrained on the seven *non-shoe* classes (863 images), frozen, linear probe.
3. **A real pretrained backbone**: SqueezeNet 1.1 trained on ImageNet — 1.2 million photographs, 1,000 classes — loaded from weights committed with this book, frozen, linear probe on its 512-dimensional features. (Fashion images get upsampled to 96×96 and repeated to three channels to fit its expectations. That awkwardness is part of the experiment, and part of the lesson.)

```
from torchvision.models import squeezenet1_1

shoe = torch.tensor([5, 7, 9])
new_label = {5: 0, 7: 1, 9: 2}
is_shoe_tr = (y_tr[:, None] == shoe).any(1)
is_shoe_te = (y_te[:, None] == shoe).any(1)
X_shoe_te = X_te[is_shoe_te]
y_shoe_te = torch.tensor([new_label[int(c)] for c in y_te[is_shoe_te]])

# our own trunk: pretrain on the 7 non-shoe classes
src = [0, 1, 2, 3, 4, 6, 8]
remap = {c: i for i, c in enumerate(src)}
X_src = X_tr[~is_shoe_tr]
y_src = torch.tensor([remap[int(c)] for c in y_tr[~is_shoe_tr]])

class Trunk(nn.Module):
    def __init__(self, n_classes: int):
        super().__init__()
        self.features = nn.Sequential(
            *atom(1, 16), *atom(16, 16), nn.MaxPool2d(2),
            *atom(16, 32), *atom(32, 32), nn.MaxPool2d(2), *atom(32, 64))
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Linear(64, n_classes))
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.head(self.features(x))

own_trunk = train_model(lambda: Trunk(7), epochs=100, data=(X_src, y_src))
X_src_te = X_te[~is_shoe_te]
y_src_te = torch.tensor([remap[int(c)] for c in y_te[~is_shoe_te]])
print(f"own trunk, source task (7 classes): "
      f"{accuracy(own_trunk, X_src_te, y_src_te):.1%}")

# the real thing: ImageNet SqueezeNet, from committed weights (no download)
sq = squeezenet1_1(weights=None)
sq.load_state_dict(torch.load("../../data/squeezenet1_1-imagenet.pt"))
sq.eval()
for p in sq.parameters():
    p.requires_grad = False
```

```
IMNET_MEAN = torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1)
IMNET_STD = torch.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1)

@torch.no_grad()
def squeeze_features(X: torch.Tensor) -> torch.Tensor:  # (N,1,28,28) -> (N,512)
    x = X.repeat(1, 3, 1, 1)
    x = F.interpolate(x, size=96, mode="bilinear", align_corners=False)
    f = sq.features((x - IMNET_MEAN) / IMNET_STD)
    return F.adaptive_avg_pool2d(f, 1).flatten(1)

@torch.no_grad()
def own_features(X: torch.Tensor) -> torch.Tensor:      # (N,1,28,28) -> (N,64)
    return F.adaptive_avg_pool2d(own_trunk.features(X), 1).flatten(1)

def linear_probe(Z_tr: torch.Tensor, y_f: torch.Tensor,
                 Z_te: torch.Tensor, seed: int) -> float:
    torch.manual_seed(seed)
    head = nn.Linear(Z_tr.shape[1], 3)
    opt = torch.optim.Adam(head.parameters(), lr=1e-2,
                           weight_decay=1e-3)   # built-in L2 penalty (ch. 1's ridge)
    for _ in range(400):
        loss = F.cross_entropy(head(Z_tr), y_f)
        opt.zero_grad(); loss.backward(); opt.step()
    return (head(Z_te).argmax(1) == y_shoe_te).float().mean().item()
```

```
own trunk, source task (7 classes): 79.7%
```

```
K = 10
rows = []
for seed in [0, 1, 6050]:
    torch.manual_seed(seed)
    idx = torch.cat([(y_tr == c).nonzero().squeeze(1)
                     [torch.randperm(int((y_tr == c).sum()))[:K]] for c in shoe])
    X_few = X_tr[idx]
    y_few = torch.tensor([new_label[int(c)] for c in y_tr[idx]])

    scratch = train_model(lambda: Trunk(3), epochs=100, seed=seed,
                          data=(X_few, y_few))
    rows.append((accuracy(scratch, X_shoe_te, y_shoe_te),
                 linear_probe(own_features(X_few), y_few,
                              own_features(X_shoe_te), seed),
                 linear_probe(squeeze_features(X_few), y_few,
                              squeeze_features(X_shoe_te), seed)))
```

```
print("seed     scratch   own-trunk probe   ImageNet probe")
for seed, r in zip([0, 1, 6050], rows):
    print(f"{seed:4d}     {r[0]:.1%}       {r[1]:.1%}           {r[2]:.1%}")
mean = [sum(c) / 3 for c in zip(*rows)]
print(f"mean     {mean[0]:.1%}       {mean[1]:.1%}           {mean[2]:.1%}")
```

```
seed     scratch   own-trunk probe   ImageNet probe
   0     86.4%       68.9%           88.1%
   1     85.3%       65.5%           85.9%
6050     88.1%       77.4%           87.6%
mean     86.6%       70.6%           87.2%
```

Read that table slowly, because it does *not* say what the textbook story predicts, and the discrepancy is the best lesson in this chapter. Training from scratch on thirty images fights the mighty ImageNet backbone to a dead heat — the means differ by less than a point, well inside seed noise. And our own pretrained trunk, perfectly competent on its source task per the printout above, transfers *worse than nothing*. Three reasons, each a general principle:

1. **A trunk can only donate what its data taught it.** Our own trunk saw 863 shirts, bags, and trousers. Nothing in that curriculum required foot-shaped detectors, so its 64 features flatten sandals, sneakers, and boots into nearly the same point. Pretraining is not magic; it is *curriculum*.
2. **Domain and resolution gaps tax the donation.** SqueezeNet's filters expect 224-pixel color photographs; we feed it 28-pixel grayscale icons inflated to 96. Its early layers hunt for detail that simply is not there. (The lecture's recipe — resize and normalize *to match the pretraining pipeline* — is doing real work; we complied as far as the data allows.)
3. **Transfer wins when the target is big relative to your labels — not small.** This is the quiet one. Three shoe silhouettes at 28×28 is a *small* problem: thirty images genuinely suffice, so the scratch baseline is strong and there is little room for imported knowledge to pay rent. The lecture's numbers show the other regime: linear-probing an ImageNet ResNet-18 on the *full* FashionMNIST task reaches about 93%, and fine-tuning about 94% — at 224 pixels, with a task rich enough that features matter more than labels.

> **When to reach for a pretrained backbone**
>
> The honest decision rule our experiment and the lecture's jointly support: transfer pays when (your labels are scarce) *and* (the target task is feature-hungry) *and* (the pretraining data plausibly covers the target's features at a matched scale). At 224 pixels and 18 landmark classes — the course assignment — all three hold, and transfer is the winning move by a wide margin. At 28 pixels and 3 silhouettes, the conditions fail and scratch fights the superpower to a draw. Knowing *which regime you are in* is the skill; the mechanics (`requires_grad`, learning-rate splits) are the easy part.

The mechanics, for completeness, since you will use them constantly from Part V onward — fine-tuning SqueezeNet's last block with the lecture's two-learning-rate recipe (fresh head fast, pretrained layers slow):

```
def prep(X: torch.Tensor) -> torch.Tensor:
    x = X.repeat(1, 3, 1, 1)
    x = F.interpolate(x, size=96, mode="bilinear", align_corners=False)
    return (x - IMNET_MEAN) / IMNET_STD

torch.manual_seed(6050)
idx = torch.cat([(y_tr == c).nonzero().squeeze(1)
                 [torch.randperm(int((y_tr == c).sum()))[:K]] for c in shoe])
X_few, y_few = prep(X_tr[idx]), torch.tensor([new_label[int(c)]
                                              for c in y_tr[idx]])

ft = squeezenet1_1(weights=None)
ft.load_state_dict(torch.load("../../data/squeezenet1_1-imagenet.pt"))
head = nn.Linear(512, 3)
for p in ft.parameters():
    p.requires_grad = False
for p in ft.features[-1].parameters():         # last Fire block only
    p.requires_grad = True

opt = torch.optim.Adam([
    {"params": head.parameters(), "lr": 1e-3},          # fresh head: normal speed
    {"params": ft.features[-1].parameters(), "lr": 1e-4},  # pretrained: gentle
], weight_decay=1e-3)

ft.train()
for _ in range(60):
    z = F.adaptive_avg_pool2d(ft.features(X_few), 1).flatten(1)
    loss = F.cross_entropy(head(z), y_few)
    opt.zero_grad(); loss.backward(); opt.step()
ft.eval()
with torch.no_grad():
    z = F.adaptive_avg_pool2d(ft.features(prep(X_shoe_te)), 1).flatten(1)
    print(f"fine-tuned (last block + head): "
          f"{(head(z).argmax(1) == y_shoe_te).float().mean():.1%}")
```

```
fine-tuned (last block + head): 88.7%
```

Fine-tuning edges the frozen probe on this seed and lands in the same tie with scratch. This is consistent with the regime analysis above: adaptation helps at the margin, but no amount of it makes a small target task need imported features.

Keep the pattern, though: *this* loop, scaled to real backbones and feature-hungry tasks, is how most of applied deep learning ships today. It is also this book's destination: Part V is what happened when the field noticed that "pretrain big, adapt cheaply" was not a vision trick but *the* recipe for language too. The BERT moment (Chapter 15) is this section's idea, taken seriously at scale.

9.8. Okay, so —

1. **Small kernels, stacked**: two 3×3s see what a 5×5 sees, for $18C^2$ against $25C^2$, with twice the nonlinearity — provided width holds constant through the stack. Design in blocks, repeat the block.
2. **BatchNorm** re-standardizes every channel over the batch and spatial axes, then hands back two knobs (γ, β); conv→BN→ReLU with `bias=False`; *train and eval are different machines*, so flip the switch. Its per-token cousin, LayerNorm, awaits in Chapter 14.
3. 1×1 **convolutions** run Chapter 1's linear model across channels at every pixel: summarize channels, don't discard them. With **global average pooling** they fire the flatten head: parameters collapse (218k → 35k) and position-specific weights leave the head, at a clean-accuracy price our small dataset makes visible. Boundaries and downsampling still prevent exact invariance.
4. **Depth can hit an optimization wall**: our plain 40-layer net couldn't fit its own training set. The residual connection $H(x) = F(x) + x$ adds an identity lane to the Jacobian, Chapter 5's gradient superhighway as infrastructure, and the same-depth twin trains to 100%. Transformer stacks will reuse this residual pattern around each attention and feed-forward sublayer.
5. **Transfer learning** = freeze + probe, or gently fine-tune. Our honest experiment found scratch and the ImageNet probe in a near tie across three seeds at 28-pixel scale. Pretraining is curriculum, gaps are taxed, and small targets may leave little room for imports. The lecture's full-scale runs (93–94% via ImageNet ResNet-18) illustrate a richer regime where transfer pays. Diagnosing the regime is the skill.

Sources and further reading

- Simonyan & Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*: the small-kernel, repeated-block argument behind VGG.
- Lin, Chen, & Yan, *Network In Network*: introduces the 1×1 channel mixer and global-average-pooling classifier.
- Ioffe & Szegedy, *Batch Normalization*: the original normalization method and its train/evaluation distinction.
- He et al., *Deep Residual Learning for Image Recognition*: the degradation experiment and residual-block solution.
- Yosinski et al., *How Transferable Are Features in Deep Neural Networks?*: a careful empirical account of when transferred features remain useful.

Exercises

1. **(Pencil.)** Generalize the kernel-stacking arithmetic: n stacked 3×3 layers versus one $(2n{+}1) \times (2n{+}1)$ kernel, at constant width C. Then redo it for a layer that grows channels $C \to 2C$: split into two 3×3s ($C \to C \to 2C$ and $C \to 2C \to 2C$) and find when the split stops saving parameters.

2. **(Pencil.)** Verify the Inception bottleneck arithmetic from the callout (819,200 versus 110,592), then design the cheapest 1×1-plus- 5×5 pipeline from 128 channels to 64 output channels that keeps the squeeze width at least a quarter of the input width.
3. **(Code.)** Make VGGSmall's blocks *depthwise separable*: replace each 3×3 conv with a per-channel 3×3 (`groups=c_in`) followed by a 1×1 mixer. Count parameters, retrain, and report the accuracy-per-parameter ratio against the original. (This is MobileNet's trick for phones, the lecture's fifth principle.)
4. **(Code.)** Ablate BatchNorm: retrain VGGSmall with the BN layers removed, same budget. Compare final accuracy *and* the first ten epochs' training accuracy. Then repeat the `bn-drift` cell's measurement on both trained networks. Does training itself fix the activation scales?
5. **(Code.)** Data augmentation as a third road: on the 30-image shoe task, train from scratch with random horizontal flips and ± 3-pixel shifts (Chapter 6's exercise, now as a tool). Does augmentation close the gap to the lecture's full-data numbers further than transfer did? Why might augmentation and transfer help in *different* regimes?

Part III.

III · Sequences

10. Sequences and Recurrence

Part II taught machines to see, and every trick in it leaned on one fact about images: an image arrives *all at once*, at a fixed size. This part is about data that refuses both conditions. Text, audio, sensor streams, stock prices — they come one piece at a time, they stop whenever they please, and their meaning lives in the *order* of the pieces.

Order deserves a moment, because the book has been here before with the opposite verdict. Chapter 6 shuffled every pixel with one fixed permutation and retrained the MLP to the same accuracy: because that coordinate map was invertible, the first-layer weights could absorb it. The trained MLP was still position-specific; it was not a bag model. Now shuffle the words of *this sentence* and read it back. For sequences, no single compensating remap preserves arbitrary order — order is most of the signal. We need models whose computation represents it explicitly.

10.1. Stretching the old toolkit

Before building anything new, the book's habit: try what we own.

A window. Predict the next element from the last k: slide a width-k window along the sequence and feed each snapshot to an MLP. This should feel familiar twice over: it is autoregression, and the *sliding* is Chapter 7's one-dimensional convolution — locality, transplanted from space to time. Windows are genuinely useful (they power real forecasting systems), but they inherit locality's blind spot: anything older than k steps ago does not exist. Widen k and the parameter count grows with it, yet no finite window survives the sentence "The company whose first-day price we recorded back in January finally …". Context has no fixed radius.

A function per timestep. Then let the model change over time: learn f_1 for step one, f_2 for step two, each passing a summary forward. The lecture builds this strawman carefully, because its failure names the cure. The parameter count now grows with sequence *length*; a length-500 essay needs 500 learned functions; and the model is **non-stationary** — it insists the rules of language at position 17 differ from the rules at position 18, when the whole point of language is that they do not.

You have watched this book face that exact configuration twice. A template for every image position was wasteful and brittle → share the kernel across space (Chapter 7). A detector per location was the wrong prior → share it (Chapter 8). The move is the same here, and it is the chapter's one big idea: **share the function across time**,

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t), \tag{10.1}$$

one learned rule f, applied at every step, carrying a running summary $\mathbf{h}_t$ — the **hidden state** — from each step to the next. This is the book's third weight sharing: across examples (every chapter since the first), across space (Part II), and now across *time*. Stationarity is to sequences what translation equivariance was to images: the same pattern means the same thing whenever it occurs.

> **i** The hidden state is a running memory — and a Markov claim
>
> $\mathbf{h}_t$ is whatever f chooses to remember about everything seen so far, compressed into one fixed-size vector. Feeding it back in makes a claim with a classical name, the **Markov property**: $\mathbf{h}_{t-1}$ and $\mathbf{x}_t$ together contain everything needed for the next step. Nothing else from the past gets a second look. That claim buys streaming (process tokens as they arrive), any sequence length (the loop doesn't care), and — later in this chapter — the license to train on chopped-up chunks. Its price is that the memory is *finite*, and this part of the book ends when we finally refuse to pay that price (Chapter 12).

10.2. A network with a loop

The simplest choice of f is Chapter 3's recipe with the state concatenated in: a linear map of $(\mathbf{h}_{t-1}, \mathbf{x}_t)$, squashed by a tanh:

$$\mathbf{h}_t \;=\; \tanh(\mathbf{W}_{hh}\,\mathbf{h}_{t-1} + \mathbf{W}_{xh}\,\mathbf{x}_t + \mathbf{b}_h)\,, \qquad \mathbf{o}_t \;=\; \mathbf{W}_{hq}\,\mathbf{h}_t + \mathbf{b}_q, \tag{10.2}$$

the **vanilla recurrent neural network**. Three weight matrices total, reused at every step, regardless of whether the sequence has five tokens or five thousand. The readout $\mathbf{o}_t$ produces logits — Chapter 2's machinery — and you can collect one at every step (predicting each next character, labeling each word) or only at the end (classifying the whole sequence); both designs appear below.

Per the book's habit, build the loop by hand, then verify against the framework:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

torch.manual_seed(6050)
V, H, T, B = 8, 16, 12, 4                          # vocab, hidden, time, batch
rnn = nn.RNN(V, H, batch_first=True)
x = torch.randn(B, T, V)
out_ref, _ = rnn(x)

W_xh, W_hh = rnn.weight_ih_l0, rnn.weight_hh_l0
b = rnn.bias_ih_l0 + rnn.bias_hh_l0
h = torch.zeros(B, H)                               # h_0 = 0, the standard start
```

```
outs = []
for t in range(T):                                  # the loop IS the architecture
    h = torch.tanh(x[:, t] @ W_xh.T + h @ W_hh.T + b)
    outs.append(h)
manual = torch.stack(outs, 1)
print(f"manual loop vs. nn.RNN: max |diff| = {(manual - out_ref).abs().max():.1e}")
```

```
manual loop vs. nn.RNN: max |diff| = 1.2e-07
```

Notice what the shapes say. `nn.RNN` eats `(batch, time, features)` — a third axis has joined Chapter 8's NCHW discipline — and the loop runs over that middle axis carrying `h` along. Unroll the loop on paper and the RNN is an ordinary feed-forward network that happens to be T layers deep with every layer's weights tied. Hold that thought; it is about to matter enormously.

10.3. Blame through time

Tied depth means Chapter 5's rules apply verbatim. To train on a sequence, run the loop forward, collect losses at the readouts, and backpropagate through the *unrolled* network, a procedure called **backpropagation through time** (BPTT). Blame that lands on $\mathbf{h}_T$ must travel back through every application of Equation 10.2 to reach an early input. Define $\mathbf{D}_t = \mathrm{diag}(\tanh'(\cdot))$ at step t. The forward Jacobian for one hop is $\mathbf{D}_t\mathbf{W}_{hh}$, so the full Jacobian is

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_1} = (\mathbf{D}_T\mathbf{W}_{hh})\cdots(\mathbf{D}_2\mathbf{W}_{hh}), \qquad \mathbf{g}_{t-1} = \mathbf{W}_{hh}^\top \mathbf{D}_t \mathbf{g}_t. \tag{10.3}$$

A product of T near-identical matrices. The second expression shows the reverse-mode order explicitly: the derivative gate acts on incoming blame, then $\mathbf{W}_{hh}^\top$ carries it to the previous state. Chapter 5 taught you to fear this shape. If the factors shrink the signal, the product dies exponentially; if they grow it, it explodes exponentially. Since $\tanh' \le 1$ and $\mathbf{W}_{hh}$ repeats at every step, keeping every relevant direction near unit gain over a long sequence is a fragile special case. Depth was measured in layers there; here it is measured in *time*, and a paragraph of text is a hundred layers deep. Watch the collapse, in the spirit of Chapter 5's depth experiment:

```
def grad_at_first_input(rec: nn.Module, lags: list[int],
                        vocab: int = 4) -> list[float]:
    norms = []
    for T in lags:
        generator = torch.Generator().manual_seed(100 + T)
        x = torch.randn(8, T, vocab, generator=generator, requires_grad=True)
        o, _ = rec(x)
        o[:, -1].sum().backward()
        norms.append(x.grad[:, 0].norm().item())    # blame on input #1
    return norms
```

```
lags = [1, 5, 10, 20, 40, 60]
torch.manual_seed(0)
vanilla = nn.RNN(4, 32, batch_first=True)
g_rnn = grad_at_first_input(vanilla, lags)
print("lag:      " + "   ".join(f"{l:>7d}" for l in lags))
print("|grad|:   " + "   ".join(f"{g:.1e}" for g in g_rnn))

plt.figure(figsize=(6.2, 3.2))
plt.semilogy(lags, g_rnn, "o-", color="#E57200", ms=5, label="vanilla RNN")
plt.xlabel("lag $T$ between output and first input")
plt.ylabel(r"$\|\partial L / \partial \mathbf{x}_1\|$")
plt.legend(); plt.tight_layout(); plt.show()
```

```
lag:            1         5        10        20        40        60
|grad|:   2.9e+00   6.8e-01   2.6e-02   7.2e-05   3.4e-10   1.1e-15
```

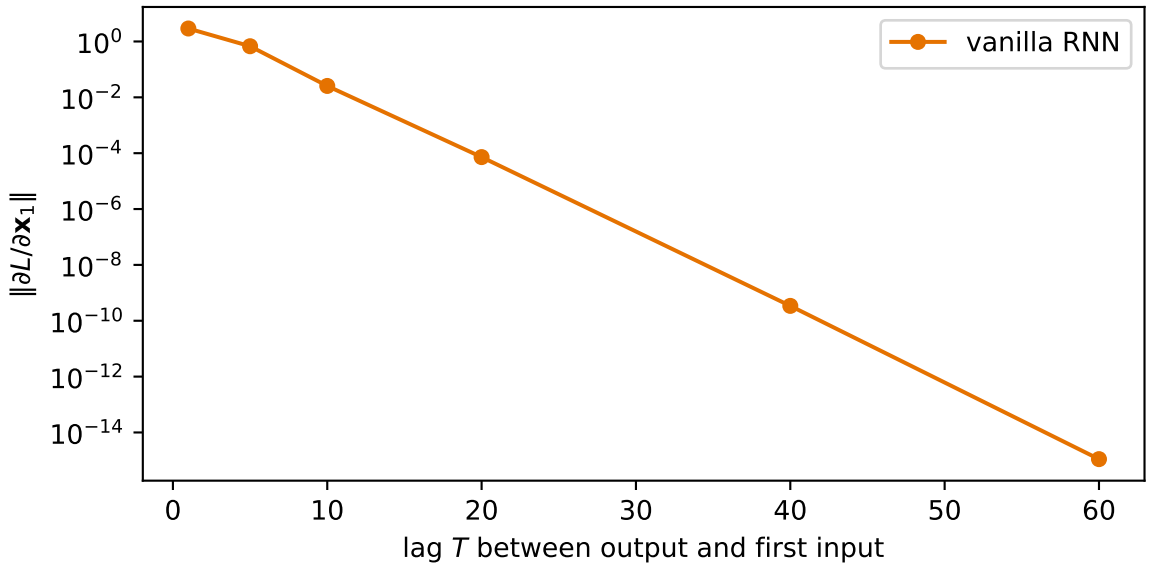

Figure 10.1.: How much a freshly initialized vanilla RNN's output can blame its first input, as the lag between them grows. Fifteen orders of magnitude gone by sixty steps: the same exponential decay Chapter 5 measured down a deep network, now measured across time — because an unrolled RNN *is* a deep network with tied weights.

By lag 60 the gradient reaching the first input is 10^{-15}: at initialization, whatever happened at the start supplies almost no usable learning signal. That makes success fragile rather than mathematically impossible — a later experiment catches one lucky seed. The mirror-image failure, explosion, is just as real — one large spike in Equation 10.3's product can make an update destabilize training.

> ⚠ Gradient clipping is a seatbelt, not a cure
>
> Practice handles explosion with **gradient clipping**: if the gradient's norm exceeds a threshold, rescale it down to the threshold before stepping (`nn.utils.clip_grad_norm_` — it appears in every training loop below). The lecture is blunt about what this buys: it tames the spikes so training survives, at the cost of damping legitimately large steps — and it does *nothing* for vanishing. You cannot clip your way to a longer memory.

10.3.1. Training with a finite horizon

BPTT has a second, blunter problem: unrolling a book-length sequence costs book-length memory and compute per step. The standard fix leans on the Markov claim from the callout above. In **truncated BPTT**, contiguous chunks carry the recurrent state forward but detach it at each boundary. The values cross the boundary; the gradient graph does not. This bounds memory while preserving running context.

The final experiment below uses a simpler finite-horizon recipe: it samples random 100-character windows and starts each from zero state. That is **fixed-window training**, not truncated BPTT, because no state is carried between windows. It is cheap and appropriate for this small demonstration, but the model can neither see nor learn dependencies beyond 100 characters. In production character models, contiguous chunks of roughly 20–50 steps with carried-and-detached state are common. In either recipe, clipping handles spikes; neither creates gradients beyond the truncation horizon.

10.4. Engineering a memory: the LSTM

So the vanilla RNN's memory dies of repeated multiplication. The seed notes stage the fix as a design-by-constraints exercise, and it is worth walking, because you already own its key move. We want a memory pathway that is:

1. **Preserved by default.** Repeated *multiplication* killed the signal → make the carried state **additive**: $\mathbf{c}_t = \mathbf{c}_{t-1} + \Delta_t$. A conveyor belt: information rides along untouched unless something deliberately intervenes. You met this exact maneuver one chapter ago — Chapter 9's residual connection, $H(x) = F(x) + x$, whose Jacobian's identity term let gradients cross forty layers. Same trick, laid across time.
2. **Forgettable on command.** Memory is finite; some of it should expire. Install a valve: $\mathbf{f}_t \in (0, 1)$ multiplying $\mathbf{c}_{t-1}$.
3. **Writable on command.** New information enters through a second valve $\mathbf{i}_t$ scaling a candidate $\tilde{\mathbf{c}}_t$.
4. **Readable on command.** Expose only what the current step needs, through a third valve $\mathbf{o}_t$.

Each valve is a small learned layer squashed by a sigmoid — Chapter 2's machine, reinterpreted: a number in $(0, 1)$ read as *what fraction passes*. The candidate content is shaped by tanh, keeping

it bounded and centered. Assembled, this is the **Long Short-Term Memory** cell (Hochreiter & Schmidhuber, 1997):

$$
\begin{aligned}
\mathbf{f}_t &= \sigma\big(\mathbf{W}_f\,[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f\big) & \mathbf{i}_t &= \sigma(\mathbf{W}_i\,[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \\
\tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c\,[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) & \mathbf{o}_t &= \sigma(\mathbf{W}_o\,[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} \;+\; \mathbf{i}_t \odot \tilde{\mathbf{c}}_t & \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t).
\end{aligned}
\tag{10.4}
$$

```
from matplotlib.patches import FancyBboxPatch, Circle

fig, ax = plt.subplots(figsize=(7.4, 3.3))
ax.set_xlim(0, 11); ax.set_ylim(0, 5); ax.axis("off")

def gate_box(x: float, y: float, label: str,
             color: str = "#F9DCBF") -> None:
    patch = FancyBboxPatch((x - 0.42, y - 0.32), 0.84, 0.64,
                           boxstyle="round,pad=0.06", facecolor=color,
                           edgecolor="#232D4B", linewidth=1.2)
    ax.add_patch(patch)
    ax.text(x, y, label, ha="center", va="center", fontsize=10)

def operation(x: float, y: float, symbol: str) -> None:
    patch = Circle((x, y), 0.25, facecolor="white",
                   edgecolor="#E57200", linewidth=1.5)
    ax.add_patch(patch)
    ax.text(x, y, symbol, ha="center", va="center", fontsize=12)

ax.text(0.45, 3.6, "$c_{t-1}$", ha="center", fontsize=11)
gate_box(2.1, 3.6, "$f_t$")
operation(3.2, 3.6, r"$\times$")
operation(6.8, 3.6, "+")
ax.text(10.45, 3.6, "$c_t$", ha="center", fontsize=11)
ax.annotate("", (1.62, 3.6), (0.85, 3.6), arrowprops=dict(arrowstyle="->", lw=2, color="#E572
ax.annotate("", (2.95, 3.6), (2.52, 3.6), arrowprops=dict(arrowstyle="->", lw=2, color="#E572
ax.annotate("", (6.55, 3.6), (3.45, 3.6), arrowprops=dict(arrowstyle="->", lw=2, color="#E572
ax.annotate("", (10.05, 3.6), (7.05, 3.6), arrowprops=dict(arrowstyle="->", lw=2, color="#E57

ax.text(0.9, 1.0, "$[h_{t-1},x_t]$", ha="center", fontsize=10)
gate_box(3.0, 1.55, "$i_t$")
gate_box(3.0, 0.55, r"$\tilde{c}_t$", "#E8F0F8")
operation(4.35, 1.05, r"$\times$")
ax.annotate("", (2.55, 1.55), (1.45, 1.12), arrowprops=dict(arrowstyle="->", lw=1.3, color="#
ax.annotate("", (2.55, 0.55), (1.45, 0.92), arrowprops=dict(arrowstyle="->", lw=1.3, color="#
ax.annotate("", (4.1, 1.15), (3.45, 1.55), arrowprops=dict(arrowstyle="->", lw=1.3, color="#2
ax.annotate("", (4.1, 0.95), (3.45, 0.55), arrowprops=dict(arrowstyle="->", lw=1.3, color="#2
ax.plot([4.6, 6.8, 6.8], [1.05, 1.05, 3.35], color="#232D4B", lw=1.5)
ax.annotate("", (6.8, 3.35), (6.8, 2.65), arrowprops=dict(arrowstyle="->", lw=1.5, color="#23
```

```
gate_box(8.55, 1.45, "$o_t$")
gate_box(8.55, 2.45, r"$\tanh$", "#E8F0F8")
operation(9.65, 1.95, r"$\times$")
ax.plot([8.15, 7.8, 7.8, 8.1], [2.45, 2.45, 3.6, 3.6], color="#232D4B", lw=1.3)
ax.annotate("", (8.1, 2.45), (7.8, 2.45), arrowprops=dict(arrowstyle="->", lw=1.3, color="#23
ax.annotate("", (9.4, 2.05), (9.0, 2.45), arrowprops=dict(arrowstyle="->", lw=1.3, color="#23
ax.annotate("", (9.4, 1.85), (9.0, 1.45), arrowprops=dict(arrowstyle="->", lw=1.3, color="#23
ax.annotate("", (10.45, 1.95), (9.9, 1.95), arrowprops=dict(arrowstyle="->", lw=1.5, color="#
ax.text(10.65, 1.95, "$h_t$", ha="left", va="center", fontsize=11)
ax.text(5.2, 4.25, "additive cell-state highway", color="#B45309",
        ha="center", fontsize=10)
plt.tight_layout(); plt.show()
```

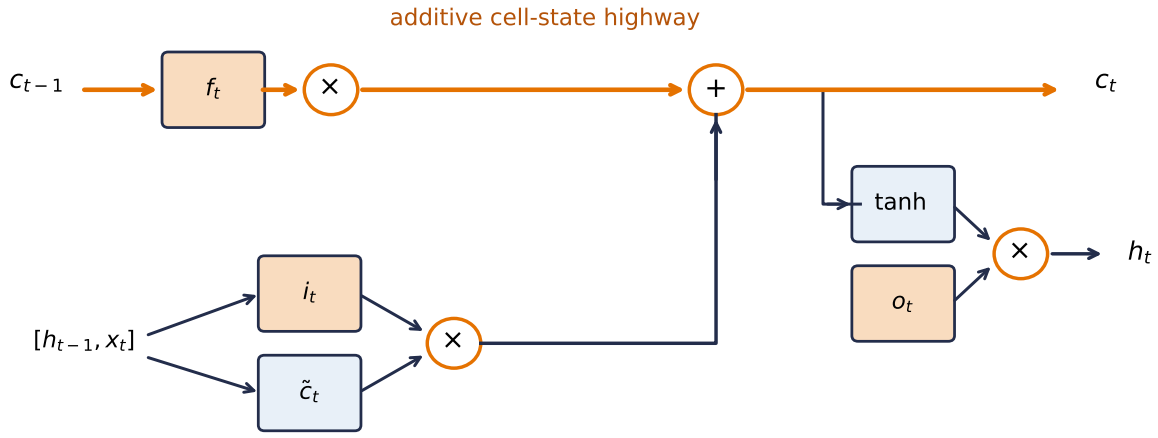

Figure 10.2.: The LSTM cell as a conveyor belt with valves. The forget gate f_t controls how much old cell state continues; the input gate i_t controls how much candidate content is added; the output gate o_t controls what becomes visible as h_t. The upper additive route is the long-memory highway.

Two states now travel: the cell state $\mathbf{c}_t$ (long-term memory, the conveyor belt) and the hidden state $\mathbf{h}_t$ (working memory, playing the vanilla RNN's old role). The line that changes everything is the $\mathbf{c}_t$ update — read its gradient the way the seed's notes do:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \mathbf{f}_t \qquad \Longrightarrow \qquad \frac{\partial L}{\partial \mathbf{c}_{t-k}} \approx \left(\prod_{j=t-k+1}^{t} \mathbf{f}_j \right) \odot \frac{\partial L}{\partial \mathbf{c}_t}. \tag{10.5}$$

Still a product — but of *learned, data-dependent gates*, not a fixed weight matrix. Where Chapter 9's residual highway kept an unconditional identity lane open, the LSTM's lane has a valve the network controls: hold $\mathbf{f}_j$ near 1 and gradients ride the conveyor belt across dozens of steps; drop it toward 0 and the memory (deliberately) expires. The practical corollary, straight from the

lecture notes: *initialize the forget-gate bias positive* (say +1), so the cell starts life remembering by default. PyTorch stores separate input-hidden and hidden-hidden bias vectors, then adds them. The code sets their **sum** to +1; setting both slices to +1 would silently create an effective bias of +2. Watch what that buys, on the same axes as before:

```
torch.manual_seed(0)
lstm = nn.LSTM(4, 32, batch_first=True)
with torch.no_grad():
    n = lstm.bias_ih_l0.shape[0] // 4
    lstm.bias_ih_l0[n:2 * n].fill_(1.0)
    lstm.bias_hh_l0[n:2 * n].zero_()              # the two bias slices sum to +1
g_lstm = grad_at_first_input(lstm, lags)

plt.figure(figsize=(6.2, 3.2))
plt.semilogy(lags, g_rnn, "o-", color="#E57200", ms=5, label="vanilla RNN")
plt.semilogy(lags, g_lstm, "s-", color="#232D4B", ms=5, label="LSTM (forget bias +1)")
plt.xlabel("lag $T$ between output and first input")
plt.ylabel(r"$\|\partial L / \partial \mathbf{x}_1\|$")
plt.legend(); plt.tight_layout(); plt.show()
```

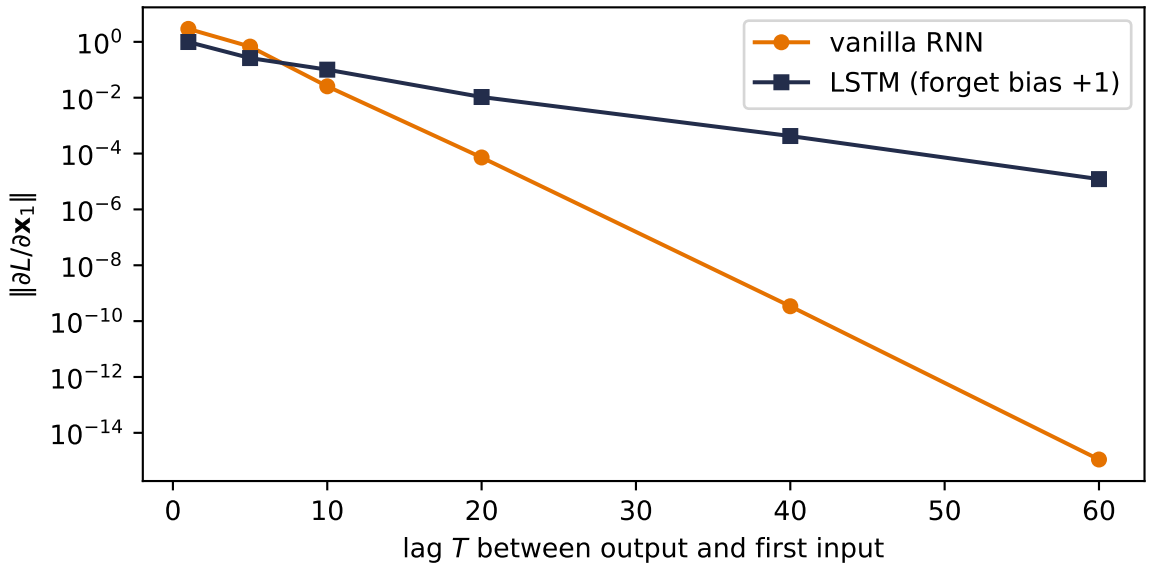

Figure 10.3.: The gradient-vs-lag experiment, rerun with an LSTM whose effective forget-gate bias is initialized to +1. The vanilla RNN's blame signal dies exponentially; the LSTM's rides the cell-state conveyor belt and arrives at lag 60 attenuated but alive, about ten orders of magnitude stronger. Chapter 9 built this highway across layers; the LSTM builds it across time, with a learned valve on the on-ramp.

10.4.1. The memory test

Gradients at initialization are promissory notes; training is where they get cashed. The seed notes pose the decisive task as a two-company story: Company A's prices open at 0, Company B's at 1, then both trade identically for days — and the correct day-5 prediction is whatever day 1 said. Everything hinges on carrying one early bit across a long stretch of identical noise. Here is that story as an executable task: sequences of 80 random tokens, where the label to predict at the end is simply *the first token*. Chance is 25%. Three contenders, three seeds each: the vanilla RNN, the LSTM as PyTorch hands it to you, and the LSTM with the lecture's remember-by-default initialization.

```
def make_recall(n: int, T: int, vocab: int = 4,
                seed: int = 0) -> tuple[torch.Tensor, torch.Tensor]:
    g = torch.Generator().manual_seed(seed)
    X = torch.randint(0, vocab, (n, T), generator=g)
    return F.one_hot(X, vocab).float(), X[:, 0]     # label = first token

class SeqClassifier(nn.Module):
    def __init__(self, cell: str, vocab=4, hidden=32, forget_bias=None):
        super().__init__()
        self.rec = {"rnn": nn.RNN, "lstm": nn.LSTM}[cell](vocab, hidden,
                                                          batch_first=True)
        if forget_bias is not None:
            with torch.no_grad():
                n = self.rec.bias_ih_l0.shape[0] // 4
                self.rec.bias_ih_l0[n:2 * n].fill_(forget_bias)
                self.rec.bias_hh_l0[n:2 * n].zero_()
        self.out = nn.Linear(hidden, vocab)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        o, _ = self.rec(x)
        return self.out(o[:, -1])                    # read out at the END only

def train_recall(cell: str, T: int, seed: int, forget_bias: float | None = None,
                 epochs: int = 80) -> float:
    torch.manual_seed(seed)
    net = SeqClassifier(cell, forget_bias=forget_bias)
    X_tr, y_tr = make_recall(2000, T, seed=1)
    X_te, y_te = make_recall(500, T, seed=2)
    opt = torch.optim.Adam(net.parameters(), lr=3e-3)
    for _ in range(epochs):
        perm = torch.randperm(len(X_tr))
        for i in range(0, len(X_tr), 128):
            idx = perm[i:i + 128]
            loss = F.cross_entropy(net(X_tr[idx]), y_tr[idx])
            opt.zero_grad(); loss.backward()
            nn.utils.clip_grad_norm_(net.parameters(), 1.0)
```

```
            opt.step()
    with torch.no_grad():
        return (net(X_te).argmax(1) == y_te).float().mean().item()

T = 80
configs = [("vanilla RNN", "rnn", None),
           ("LSTM, default init", "lstm", None),
           ("LSTM, forget bias +1", "lstm", 1.0)]
print(f"recall accuracy at lag {T} (chance 25%), seeds 0 / 1 / 6050:")
for label, cell, fb in configs:
    accs = [train_recall(cell, T, s, fb) for s in [0, 1, 6050]]
    print(f"  {label:22s} " + "   ".join(f"{a:.0%}" for a in accs))
```

```
recall accuracy at lag 80 (chance 25%), seeds 0 / 1 / 6050:
  vanilla RNN            25%   100%   25%
  LSTM, default init     24%   26%   26%
  LSTM, forget bias +1   100%   100%   100%
```

Three rows, one lesson each. The vanilla RNN is a *lottery*: two seeds sit at chance, one cracks the task. Figure 10.1 says the teaching gradient is astronomically attenuated, not exactly zero — so once in a while optimization stumbles onto a solution anyway, and you cannot build a system on once-in-a-while. The default-initialized LSTM loses on *every* seed — worth a long pause, because it says the architecture alone is not the cure. A fresh LSTM's forget gates hover near $\sigma(0) = \frac{1}{2}$, and $0.5^{80} \approx 10^{-24}$: a half-closed valve, compounded eighty times, is as fatal as no valve. The third row flips the single bias that opens the valve, and the task is solved outright, every seed. The highway was built by the architecture; *initialization opened it* — the same lesson Chapter 5 taught about signal scales at birth, now with a memory attached. And the deliverable is not raw capability but **reliability**: what the gates buy is that memory stops being a lottery. (For shorter stretches the drama disappears: at lag 20 all three rows solve the task under this budget. The gates earn their keep precisely where Equation 10.3 says the vanilla cell must fail.)

10.5. GRU: the streamlined cousin

The **Gated Recurrent Unit** (Cho et al., 2014) folds the same ideas into a smaller box: cell and hidden state merged into one $\mathbf{h}_t$, forget and input valves merged into a single **update gate** $\mathbf{z}_t$ (what you keep you don't overwrite, and vice versa), plus a **reset gate** $\mathbf{r}_t$ that can blank the past when a fresh start is called for:

$$\begin{aligned} \mathbf{z}_t &= \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]), \qquad \mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]), \\ \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]), \qquad \mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t. \end{aligned} \tag{10.6}$$

Two gates instead of three, one state instead of two, comparable performance in most settings. The lecture explicitly skips the derivation, and the book follows suit. With the equations on

record and Exercise 3 waiting, you can build one yourself and let it compete on the memory test.

One convention matters before you compare code. Equation 10.6 applies the reset gate to $\mathbf{h}_{t-1}$ **before** the candidate's hidden-state linear map. PyTorch's `nn.GRU` uses a reset-after form, $\tanh(W_{in}x + b_{in} + r \odot (W_{hn}h + b_{hn}))$, for implementation efficiency. The gates serve the same purpose, but the two cells are not algebraically identical for the same weights. An exact manual-loop verification must implement PyTorch's ordering rather than Equation 10.6's.

10.6. The book reads itself

Time to put fixed windows, clipping, gates, and Chapter 2's softmax over a vocabulary to work on real text. In keeping with a book whose every experiment runs on its own pages, the training corpus is *the nine chapters you have already read*: their prose, stripped of code cells, more than 130,000 characters. The task is the oldest one in language modeling: read a character, predict the next.

```
import glob, re

files = sorted(glob.glob("../part*/0*.qmd"))          # chapters 1-9, not this one
text = ""
for f in files:
    raw = open(f).read()
    raw = re.sub(r"```\{python\}.*?```", "", raw, flags=re.S)   # drop code cells
    raw = re.sub(r"<!--.*?-->", "", raw, flags=re.S)            # drop comments
    text += raw
chars = sorted(set(text))
stoi = {c: i for i, c in enumerate(chars)}
data = torch.tensor([stoi[c] for c in text])
split = int(0.9 * len(data))
train_data, valid_data = data[:split], data[split:]
print(f"corpus: {len(files)} chapters, {len(text):,} characters, vocab {len(chars)}")
print(f"deterministic split: {len(train_data):,} train / {len(valid_data):,} held out")

class CharLSTM(nn.Module):
    def __init__(self, vocab: int, hidden: int = 128):
        super().__init__()
        self.vocab = vocab
        self.lstm = nn.LSTM(vocab, hidden, batch_first=True)
        self.out = nn.Linear(hidden, vocab)
    def forward(
        self, x: torch.Tensor,
        state: tuple[torch.Tensor, torch.Tensor] | None = None,
    ) -> tuple[torch.Tensor, tuple[torch.Tensor, torch.Tensor]]:
        o, state = self.lstm(F.one_hot(x, self.vocab).float(), state)
        return self.out(o), state                      # logits at EVERY step
```

```
torch.manual_seed(6050)
model = CharLSTM(len(chars))
opt = torch.optim.Adam(model.parameters(), lr=2e-3)
chunk, batch = 100, 64                                  # random fixed-context windows
for step in range(2501):
    s = torch.randint(0, len(train_data) - chunk - 1, (batch,))
    xb = torch.stack([train_data[j:j + chunk] for j in s])
    yb = torch.stack([train_data[j + 1:j + chunk + 1] for j in s])
    logits, _ = model(xb)
    loss = F.cross_entropy(logits.reshape(-1, len(chars)), yb.reshape(-1))
    opt.zero_grad(); loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step()
    if step % 500 == 0:
        print(f"step {step:4d}   loss {loss.item():.2f}")

@torch.no_grad()
def fixed_window_loss(net: nn.Module, sequence: torch.Tensor,
                      window: int = 100) -> float:
    net.eval()
    total_loss, n_tokens = 0.0, 0
    for start in range(0, len(sequence) - 1, window):
        stop = min(start + window, len(sequence) - 1)
        xb = sequence[start:stop].unsqueeze(0)
        yb = sequence[start + 1:stop + 1]
        logits, _ = net(xb)                             # state resets at each window
        total_loss += F.cross_entropy(
            logits.reshape(-1, len(chars)), yb, reduction="sum").item()
        n_tokens += yb.numel()
    return total_loss / n_tokens

print(f"held-out fixed-window loss {fixed_window_loss(model, valid_data):.2f}")
```

```
corpus: 9 chapters, 148,594 characters, vocab 104
deterministic split: 133,734 train / 14,860 held out
step    0   loss 4.62
step  500   loss 2.28
step 1000   loss 1.93
step 1500   loss 1.75
step 2000   loss 1.57
step 2500   loss 1.42
held-out fixed-window loss 1.89
```

The loss falls from $\ln(V) \approx 4.6$ for the printed vocabulary size (uniform guessing) to 1.42 on the last sampled training minibatch. The deterministic held-out tail is harder, at 1.89, but both

are far below uniform guessing. The model has learned a great deal about what this book's next character tends to be. To *hear* what it learned, generate: feed a prompt, sample the next character from the softmax (divided by a **temperature** that sharpens or flattens it, Chapter 2's dial between hard max and uniform), feed that character back in, repeat. The hidden state streams; nothing is recomputed.

```
@torch.no_grad()
def sample(prompt: str, n: int = 300, temp: float = 0.8) -> str:
    model.eval()
    idx = torch.tensor([[stoi.get(c, 0) for c in prompt]])
    logits, state = model(idx)
    out = prompt
    for _ in range(n):
        p = F.softmax(logits[0, -1] / temp, -1)
        nxt = torch.multinomial(p, 1)[None]
        out += chars[nxt.item()]
        logits, state = model(nxt, state)              # consume each sample once
    return out

print(sample("The gradient ", n=300))
```

```
The gradient mupolition both
lates
chood each the burk's map into a mack from the bupfured way
   intene agmenten. Pyout nothing buith and why the dirge melies to exactly. What algien is

$$
\logit_i).
   ixsections.

## Exercise reseds the barkwarper from to twe optimized $\rightarrow$ and classitivations, in
```

Read the output honestly, because both halves of the verdict teach. What it *got*: words, spacing, sentence rhythm, markdown furniture (asterisks, pipes, colons), and the book's working vocabulary (training, batch, layer, convolution in various misspellings), all learned from raw characters by a one-layer recurrence reading 100-character chunks. What it *lacks*: meaning. Clauses trail off; equations open and never close; the prose has the book's accent with none of its intent. Some of that is scale (a 130k-character corpus and 128 hidden units is a *tiny* language model), and some of it is the finite state. By the end of a sentence, the beginning has been squeezed through the LSTM's two 128-dimensional states, $(\mathbf{h}, \mathbf{c})$: 256 scalars in all. Both diagnoses point down the road this part of the book is traveling.

10.7. What a fixed-size state cannot hold

The lecture closes with the bridge this part of the book now crosses. If a recurrent cell can read a sequence into a state, then two of them can translate: an **encoder** reads the source sentence into its final state, a **decoder** spins that state back out as words in another language. That architecture, plus its training tricks, is Chapter 11. But notice what the design asks of one fixed-size state: the *entire sentence*, compressed into $(\mathbf{h}, \mathbf{c})$, no matter how long the sentence runs. The char-LM's trailing clauses were this bottleneck in miniature. The honest fix is not a bigger vector; it is admitting that the decoder should be allowed to *look back at all of the encoder's states*, not just the last. Choosing where to look, on the fly, is a similarity computation you have been prepared for since Chapter 1's dot products. That is attention (Chapter 12, Chapter 13), and it is two chapters away.

10.8. Okay, so —

1. **Order is the signal.** Chapter 6's MLP could absorb one fixed pixel permutation when retrained; shuffle a sentence into arbitrary orders and its meaning changes. Sequences add variable length and streaming, and the fixed-size toolkit of Parts I–II has no honest answer.
2. **The third weight sharing**: one function f, applied at every timestep, carrying a hidden state (Equation 10.1, Equation 10.2). This is stationarity as inductive bias, exactly as sharing across space was in Part II. The unrolled RNN is a deep network with tied weights.
3. **Tied depth in time = Chapter 5's nightmare**: BPTT multiplies T near-identical Jacobians (Equation 10.3) $\rightarrow$ vanishing or exploding, unless their gains are carefully controlled. Clipping belts in the explosions; truncation caps the cost, but neither lengthens memory. Windows of roughly 20–50 steps are a common practical horizon, not a universal vanilla-RNN ceiling.
4. **The LSTM engineers the memory**: an additive cell-state conveyor belt, Chapter 9's residual highway laid across time, with three sigmoid valves (forget, input, output; Equation 10.4). Its gradient rides a product of *gates*, not weights (Equation 10.5).
5. **Architecture proposes, initialization disposes**: at lag 80 the default LSTM scores chance ($\sigma(0)^{80} \approx 10^{-24}$); set the forget bias to $+1$ and the task is solved on every seed. GRU packs the same ideas into two gates and one state (Equation 10.6).
6. **A one-layer character LSTM learns this book's accent** (words, rhythm, LaTeX tics) but not its meaning: part scale, part the finite-state bottleneck. Cramming a whole sequence into 256 recurrent-state scalars is the debt Part III carries forward, and attention is how the book repays it.

Sources and further reading

- Elman, *Finding Structure in Time*: the classic simple recurrent network and its view of hidden state as temporal context.
- Hochreiter & Schmidhuber, *Long Short-Term Memory*: the original gated-memory architecture and constant-error pathway.

- Cho et al., *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*: introduces the gated recurrent unit in its encoder–decoder setting.
- Pascanu, Mikolov, & Bengio, *On the Difficulty of Training Recurrent Neural Networks*: geometric analysis of vanishing/exploding gradients and gradient clipping.

Exercises

1. **(Pencil.)** Count parameters: a width-k window MLP mapping k one-hot tokens (vocabulary V) through hidden width H to V logits, versus the vanilla RNN of Equation 10.2 with the same V and H. Which count depends on how much context the model can use, and why is that the whole argument of this chapter in two numbers?
2. **(Pencil.)** From Equation 10.3, show that if tanh operated in its linear regime ($\tanh' \approx 1$), the gradient norm grows or decays like the spectral radius of $\mathbf{W}_{hh}$ raised to T. What does $\tanh'(0) = 1$ say about the moment right after initialization, when $\mathbf{h} \approx \mathbf{0}$?
3. **(Code.)** Implement the reset-before GRU in Equation 10.6 as a manual loop and enter it in the lag-80 memory test. Then change only the candidate update to PyTorch's reset-after convention and verify that version against `nn.GRU`. Does either need an initialization trick, and which bias would you set?
4. **(Code.)** The chapter starts every sequence with $\mathbf{h}_0 = \mathbf{0}$. Make $\mathbf{h}_0$ a learned `nn.Parameter` in the recall task and retrain. Where does its gradient come from, and when could a learned start plausibly matter?
5. **(Code.)** Sample the character model at temperatures 0.2, 0.8, and 1.5, and with the temperature near zero compare against picking the argmax at every step. Connect what you see to Chapter 2's hard-max-versus-softmax discussion: which failure mode of the hard max does greedy decoding reproduce?

11. Encoder–Decoder, Teacher Forcing, Beam Search

Chapter 10 ended with a promise: if one recurrent cell can read a sequence *into* a state, two cells can translate: one reads, one writes. This chapter builds that machine and the craft around it: how to train it fast (teacher forcing), what poisonous bookkeeping to avoid (padding), and how to get answers back out of it (greedy versus beam search). It ends where Part III must: staring at the fixed-size recurrent state in the middle.

The lecture frames the stakes well. Every model so far has lived on a **fixed-size contract**: 784 pixels in, 10 logits out; 28×28 garments in, one verdict out. The interesting world breaks that contract on both ends at once: translation maps a sentence of *any* length to a sentence of *some other* length, in a different vocabulary. The recurrent cell broke the contract on the input side; today we break it on both.

11.1. Two RNNs and a handoff

The architecture is one idea long. An **encoder** RNN reads the source sequence and keeps nothing but its final state, the entire input compressed into fixed-size memory, the **context**. A **decoder** RNN starts *from* that state and generates the output token by token, feeding each token back in, exactly like Chapter 10's character model. A language model, in other words, but one with something to say.

Our laboratory task keeps the core mechanics of translation and none of the downloads: **date normalization**. The source language is human-written dates in several dialects; the target language is ISO 8601:

```
march 5, 2021        -> 2021-03-05
26 september 2024    -> 2024-09-26
12/15/1950           -> 1950-12-15
```

Variable-length input, a different output alphabet, and real long-range structure (the year arrives *last* in most sources but must be emitted *first*). The target is deliberately fixed at ten characters, so this laboratory does **not** test variable-length target padding. It does include a dialect ambiguity that gives beam search something genuine to reveal: numeric dates are written `mm/dd/yyyy` by 70% of our imaginary writers (the US convention) and `dd/mm/yyyy` by 30%, and nothing in `03/05/2021` says which. Machine translation in miniature.

```
import random, torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

MONTHS = ["january", "february", "march", "april", "may", "june", "july",
          "august", "september", "october", "november", "december"]
DAYS = dict(zip(MONTHS, [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]))

def make_date(rng: random.Random) -> tuple[str, str]:
    mo = rng.randrange(12)
    d = rng.randrange(1, DAYS[MONTHS[mo]] + 1)
    y = rng.randrange(1950, 2026)
    iso = f"{y:04d}-{mo+1:02d}-{d:02d}"
    style = rng.random()
    if style < 0.35:
        src = f"{MONTHS[mo]} {d}, {y}"
    elif style < 0.6:
        src = f"{d} {MONTHS[mo]} {y}"
    elif rng.random() < 0.7:
        src = f"{mo+1:02d}/{d:02d}/{y}"            # US convention: mm/dd
    else:
        src = f"{d:02d}/{mo+1:02d}/{y}"            # EU convention: dd/mm
    return src, iso

rng = random.Random(6050)
pairs, seen_sources = [], set()
while len(pairs) < 9000:
    pair = make_date(rng)
    if pair[0] not in seen_sources:
        seen_sources.add(pair[0])
        pairs.append(pair)
train_pairs = pairs[:8000]
valid_pairs = pairs[8000:8500]
test_pairs = pairs[8500:]
print(*pairs[:4], sep="\n")
train_sources = {s for s, _ in train_pairs}
valid_sources = {s for s, _ in valid_pairs}
test_sources = {s for s, _ in test_pairs}
print("exact-source overlaps: "
      f"train/valid {len(train_sources & valid_sources)}, "
      f"train/test {len(train_sources & test_sources)}, "
      f"valid/test {len(valid_sources & test_sources)}")

PAD, BOS, EOS = 0, 1, 2                            # special tokens first
SRC = sorted(set("".join(s for s, _ in pairs)))
```

```
TGT = sorted(set("".join(t for _, t in pairs)))
src_stoi = {c: i + 3 for i, c in enumerate(SRC)}
tgt_stoi = {c: i + 3 for i, c in enumerate(TGT)}
tgt_itos = {i: c for c, i in tgt_stoi.items()}
V_src, V_tgt = len(SRC) + 3, len(TGT) + 3
print(f"source vocab {V_src}, target vocab {V_tgt}")
```

```
('26 september 2024', '2024-09-26')
('april 7, 1962', '1962-04-07')
('12/15/1950', '1950-12-15')
('14 february 1997', '1997-02-14')
exact-source overlaps: train/valid 0, train/test 0, valid/test 0
source vocab 37, target vocab 14
```

The rejection loop is evaluation hygiene, not decoration. It removes duplicate source strings **before** the 8,000/500/500 train/validation/test split. We use validation sources for every learning curve and reserve test sources for one final audit after the design is fixed. The same underlying date may appear in another written dialect, which is the structural generalization we want; exact-string memorization is no longer available.

Three special tokens run the show, the same conventions as the course's full translation pipeline: `<pad>` fills the short sequences of a batch out to a rectangle, `<bos>` tells the decoder "begin," and `<eos>` lets the *model* say "I'm done" — remember, the machine must be free to choose its output length.

One more new tool, and it earns a pause. Chapter 10 fed characters in as one-hot vectors. This chapter gives each token a learned vector instead: `nn.Embedding`, a table with one trainable row per vocabulary entry. There is no new math — an embedding is exactly a one-hot vector times a weight matrix, Chapter 1's linear map wearing a lookup table's clothes — but the reading matters: *the address is hard, the content is learned.* Row 17 is fetched by index, rigidly; what lives in row 17 is whatever training decides that token should mean. Keep the phrase "hard address, learned content" nearby: in Chapter 13 we soften the address too, and that will be the whole story.

```
class Seq2Seq(nn.Module):
    def __init__(self, v_src: int, v_tgt: int, embed: int = 32,
                 hidden: int = 128, packed: bool = True):
        super().__init__()
        self.packed = packed
        self.src_emb = nn.Embedding(v_src, embed, padding_idx=PAD)
        self.tgt_emb = nn.Embedding(v_tgt, embed, padding_idx=PAD)
        self.encoder = nn.LSTM(embed, hidden, batch_first=True)
        self.decoder = nn.LSTM(embed, hidden, batch_first=True)
        self.out = nn.Linear(hidden, v_tgt)

    def encode(
```

```
        self, S: torch.Tensor, lengths: torch.Tensor,
    ) -> tuple[torch.Tensor, torch.Tensor]:
        e = self.src_emb(S)                          # (B, T_src, embed)
        if self.packed:                              # stop at each true length
            e = nn.utils.rnn.pack_padded_sequence(
                e, lengths, batch_first=True, enforce_sorted=False)
        _, state = self.encoder(e)
        return state                                 # (h, c): the context

    def forward(self, S: torch.Tensor, lengths: torch.Tensor,
                T_in: torch.Tensor) -> torch.Tensor:  # teacher-forced pass
        o, _ = self.decoder(self.tgt_emb(T_in), self.encode(S, lengths))
        return self.out(o)                           # (B, T_tgt, V_tgt) logits
```

The whole architecture is that `encode` return value: the LSTM's final $(\mathbf{h}, \mathbf{c})$, handed to the decoder as its *initial* state. With hidden width 128, that bridge carries two 128-dimensional vectors, or **256 scalars**. Everything the decoder will ever know about the source crosses it.

```
from matplotlib.patches import FancyBboxPatch

fig, ax = plt.subplots(figsize=(7.6, 3.1))
ax.set_xlim(0, 12); ax.set_ylim(0, 5); ax.axis("off")

def token_box(x: float, y: float, label: str, color: str,
              alpha: float = 1.0) -> None:
    patch = FancyBboxPatch((x - 0.42, y - 0.35), 0.84, 0.7,
                           boxstyle="round,pad=0.04", facecolor=color,
                           edgecolor="#232D4B", linewidth=1.2, alpha=alpha)
    ax.add_patch(patch)
    ax.text(x, y, label, ha="center", va="center", fontsize=9, alpha=alpha)

source_labels = ["m", "a", "r", "5", "<pad>", "<pad>"]
for j, label in enumerate(source_labels):
    faded = label == "<pad>"
    token_box(0.75 + 1.05 * j, 3.15, label,
              "#F4F4EF" if faded else "#E8F0F8", 0.4 if faded else 1.0)
    if j < 3:
        ax.annotate("", (1.25 + 1.05 * j, 3.15), (1.05 + 1.05 * j, 3.15),
                    arrowprops=dict(arrowstyle="->", color="#5379AA", lw=1.2))
ax.annotate("", (6.55, 3.15), (4.32, 3.15),
            arrowprops=dict(arrowstyle="->", color="#E57200", lw=2,
                            connectionstyle="arc3,rad=-0.45"))
ax.text(5.4, 4.25, "packing bypasses padding", color="#B45309",
        ha="center", fontsize=9)

token_box(7.0, 3.15, "$(h,c)$", "#F9DCBF")
```

```
ax.text(7.0, 3.9, "256 scalars", color="#B45309", ha="center", fontsize=9)

for j, label in enumerate(["<bos>", "2", "0", "2", "1", "…"]):
    token_box(7.9 + 0.72 * j, 1.35, label, "#F4F4EF")
    if j < 5:
        ax.annotate("", (8.25 + 0.72 * j, 1.35), (8.15 + 0.72 * j, 1.35),
                    arrowprops=dict(arrowstyle="->", color="#232D4B", lw=1.0))
ax.annotate("", (7.9, 1.72), (7.15, 2.8),
            arrowprops=dict(arrowstyle="->", color="#E57200", lw=1.8))
ax.text(2.7, 2.25, "encoder", ha="center", color="#232D4B", fontsize=10)
ax.text(9.7, 0.55, "decoder", ha="center", color="#232D4B", fontsize=10)
plt.tight_layout(); plt.show()
```

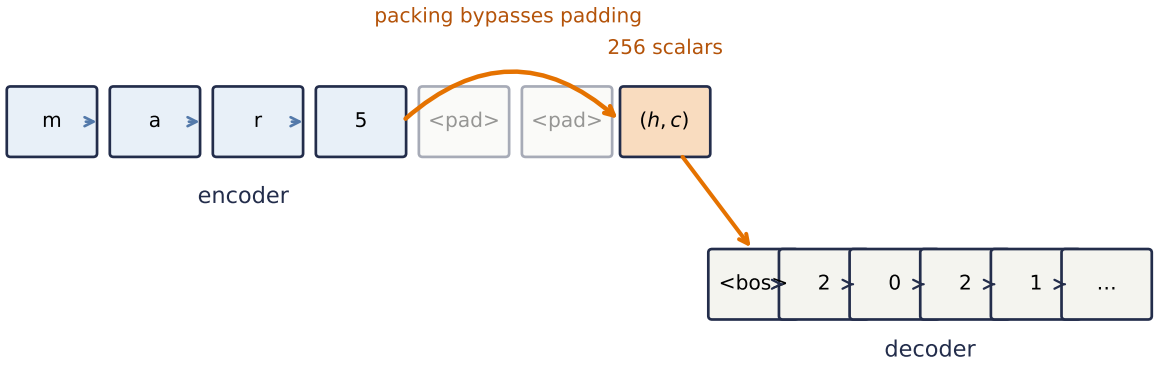

Figure 11.1.: The encoder–decoder handoff. Packing stops each encoder at its last real source token, rather than marching through right-padding. Only the final (h, c) pair crosses the bridge to initialize the decoder; the earlier encoder states are discarded. Part IV will keep those states and let the decoder look back.

11.2. The padding trap

Before the machinery works, an honest detour through the way it *fails*, because the seed notes devote a full section to this and experience says everyone hits it once. Batching variable-length sequences means padding them to a rectangle, and padding poisons two places:

1. **The loss.** Positions holding `<pad>` are not predictions to be graded. `F.cross_entropy(..., ignore_index=PAD)` excludes them. Our ISO targets all have the same length, so this branch is dormant here, but it makes the loop correct for genuinely variable-length targets.
2. **The encoder.** Subtler and far worse: an LSTM does not know your zeros are padding. It keeps stepping through them, and every pad step smears the final state, the context pair, for every sequence shorter than the batch's longest. Worse still, at inference you feed single sequences with *no* padding, so the model is tested in a regime it never trained in.

Watch what that costs. Same model, same data, same budget; the only difference is whether the encoder is told the true lengths (`pack_padded_sequence`, which runs the recurrence only through each sequence's real tokens):

```
def batchify(prs: list[tuple[str, str]], idx: list[int]) -> tuple[
    torch.Tensor, torch.Tensor, torch.Tensor
]:
    srcs = [torch.tensor([src_stoi[c] for c in prs[i][0]]) for i in idx]
    lengths = torch.tensor([len(s) for s in srcs])
    tgts = [torch.tensor([BOS] + [tgt_stoi[c] for c in prs[i][1]] + [EOS])
            for i in idx]
    S = nn.utils.rnn.pad_sequence(srcs, batch_first=True, padding_value=PAD)
    T = nn.utils.rnn.pad_sequence(tgts, batch_first=True, padding_value=PAD)
    return S, lengths, T

def train_seq2seq(mode: str = "tf", packed: bool = True, epochs: int = 25,
                  seed: int = 6050, batch: int = 128,
                  checkpoints: tuple[int, ...] = ()) -> tuple[
                      Seq2Seq, dict[int, float]
                  ]:
    torch.manual_seed(seed)
    model = Seq2Seq(V_src, V_tgt, packed=packed)
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    n, curve = len(train_pairs), {}
    for ep in range(1, epochs + 1):
        model.train()
        perm = torch.randperm(n)
        for i in range(0, n, batch):
            S, L, T = batchify(train_pairs, perm[i:i + batch].tolist())
            if mode == "tf":                          # gold rail: one fused call
                logits = model(S, L, T[:, :-1])
            else:                                     # free-running: its own rail
                state = model.encode(S, L)
                tok, outs = T[:, :1], []
                for t in range(T.shape[1] - 1):
                    o, state = model.decoder(model.tgt_emb(tok), state)
                    logit = model.out(o)
                    outs.append(logit)
                    tok = logit.argmax(-1)            # feed its own prediction
                logits = torch.cat(outs, 1)
            loss = F.cross_entropy(logits.reshape(-1, V_tgt),
                                   T[:, 1:].reshape(-1), ignore_index=PAD)
            opt.zero_grad(); loss.backward()
            nn.utils.clip_grad_norm_(model.parameters(), 5.0)
            opt.step()
        if ep in checkpoints:
            curve[ep] = exact_match(model, valid_unamb[:400])
```

```
    return model, curve

@torch.no_grad()
def translate(model: Seq2Seq, src: str, maxlen: int = 12) -> str:
    model.eval()
    S = torch.tensor([[src_stoi[c] for c in src]])
    state = model.encode(S, torch.tensor([len(src)]))
    tok, out = torch.tensor([[BOS]]), ""
    for _ in range(maxlen):
        o, state = model.decoder(model.tgt_emb(tok), state)
        tok = model.out(o).argmax(-1)                # greedy: best next char
        if tok.item() == EOS:
            break
        out += tgt_itos.get(tok.item(), "?")
    return out

@torch.no_grad()
def exact_match(model: Seq2Seq, prs: list[tuple[str, str]]) -> float:
    return sum(translate(model, s) == t for s, t in prs) / len(prs)

def unambiguous(prs: list[tuple[str, str]]) -> list[tuple[str, str]]:
    return [(s, t) for s, t in prs
            if "/" not in s or int(s[:2]) > 12 or int(s[3:5]) > 12]

valid_unamb = unambiguous(valid_pairs)
test_unamb = unambiguous(test_pairs)

cps = (2, 4, 6, 8, 12, 16, 20, 25)
naive, _ = train_seq2seq(packed=False)
packed, packed_curve = train_seq2seq(packed=True, checkpoints=cps)
print(f"naive right-padding:   validation exact match "
      f"{exact_match(naive, valid_unamb):.1%}")
print(f"packed (true lengths): validation exact match "
      f"{exact_match(packed, valid_unamb):.1%}")
```

```
naive right-padding:   validation exact match 37.3%
packed (true lengths): validation exact match 95.7%
```

Validation exact match jumps from 37.3% to 95.7%, a 58.4-point gap. Not from a better architecture, optimizer, or dataset; from telling the encoder where the sentences actually end. Let the gap sink in, because this is the cheapest lesson in the book: the two runs are identical except for bookkeeping, and the bookkeeping was worth more than every design decision in Part III so far. The sealed test set remains untouched until both training-mode designs below are fixed.

> ⚠ The model will happily read your padding
>
> Nothing in the tensor marks `<pad>` as meaningless; meaning is something *you* enforce, in two places: `ignore_index` for the loss, `pack_padded_sequence` (or an explicit mask) for the encoder. When a sequence model underperforms mysteriously, audit the padding before touching the architecture. And file the general shape of this tool away: telling a model *which positions it may not look at* is called **masking**, and it returns center-stage in Chapter 14, where a *causal* mask is what turns a transformer into a language model.

11.3. Teacher forcing: training on the gold rail

Look back at the two `mode` branches in the training loop; they are this section. When the decoder is trained, what should it see as *its own previous output*?

Free-running ("fr") is the honest answer: feed the model's own predictions back in, exactly as at inference. But its flaws are structural. Early in training those predictions are garbage → the decoder learns conditioned on garbage → every sequence must be generated token-by-token in a Python loop, because step t's input does not exist until step $t-1$ is computed. Slow, and unstable exactly when training is young.

Teacher forcing ("tf") conditions each step on the *ground-truth* previous token instead, the gold rail. Every conditioning input is then known in advance → the whole target can be submitted in **one fused decoder call** (the `model(S, L, T[:, :-1])` line), rather than a Python feedback loop. The LSTM still processes time recurrently inside that call; teacher forcing does not make its timesteps mathematically parallel. Every step nevertheless learns from a sensible context from epoch one. Watch both effects:

```
tf_model, tf_curve = packed, packed_curve
fr_model, fr_curve = train_seq2seq("fr", checkpoints=cps)
print("one final test audit (unambiguous sources):")
for label, candidate in [("naive padding", naive),
                         ("packed / teacher forced", tf_model),
                         ("packed / free-running", fr_model)]:
    print(f"  {label:25s} {exact_match(candidate, test_unamb):.1%}")

plt.figure(figsize=(6, 3.2))
plt.plot(cps, [tf_curve[c] for c in cps], "o-", color="#232D4B", lw=2,
         label="teacher forcing")
plt.plot(cps, [fr_curve[c] for c in cps], "s-", color="#E57200", lw=2,
         label="free-running")
plt.xlabel("epoch"); plt.ylabel("exact match")
plt.legend(); plt.tight_layout(); plt.show()
```

```
one final test audit (unambiguous sources):
  naive padding             34.3%
```

```
packed / teacher forced   93.1%
packed / free-running     99.1%
```

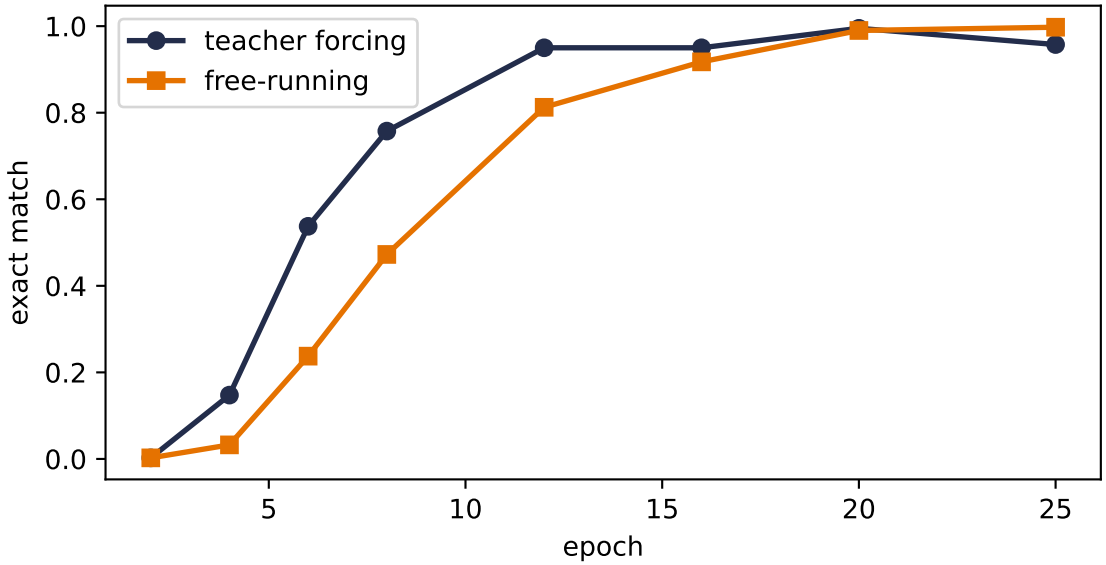

Figure 11.2.: Learning curves on a fixed subset of 400 unambiguous validation source strings, disjoint from both training and test. Teacher forcing leads through the early and middle epochs (53.8% vs. 23.8% at epoch 6; 95.0% vs. 81.2% at epoch 12) and uses one fused decoder call per batch where free-running needs a Python token loop. By epoch 20 the validation curves meet; the single final test audit favors free-running.

So teacher forcing is the pragmatic default: fused, stable, and fast out of the gate. What it costs has a name. A teacher-forced decoder has *only ever seen perfect histories*. At inference it must condition on its own imperfect predictions — a distribution it never trained on. This mismatch is **exposure bias**: the model never learned to recover from its own mistakes, so one early slip can send the rest of the sequence somewhere strange. You can see the shape of this failure in our model's rare residual errors:

```
errs = [(s, t, translate(tf_model, s))
        for s, t in test_unamb if translate(tf_model, s) != t]
print(f"{len(errs)} errors on {len(test_unamb)} unambiguous test dates; a sample:")
for s, t, g in errs[:3]:
    print(f"  {s!r} -> {g!r}   (truth {t})")
```

```
30 errors on 437 unambiguous test dates; a sample:
  '03/21/1958' -> '1958-03-22'   (truth 1958-03-21)
  '01/22/1996' -> '1996-02-12'   (truth 1996-01-22)
  '21 november 2013' -> '2013-11-22'   (truth 2013-11-21)
```

The errors are plausible-looking digit substitutions and order slips. The decoder has learned a date-shaped language, but no hard mechanism forces each output field to copy the corresponding source fact. Off the gold rail, one wrong character can alter the context for every character after it. Hold that thought two sections.

> **i** Scheduled sampling: renting the rail
>
> The engineered compromise (Bengio et al., 2015): during training, at *each timestep* flip a biased coin — probability ϵ of the gold token, $1-\epsilon$ of the model's own prediction — and anneal ϵ from 1 toward 0 as training matures. Early epochs enjoy the gold rail; late epochs practice recovery. One design detail from the original paper worth repeating: flip **per token, not per sequence** — committing a whole sequence to model predictions early in training lets consecutive errors amplify. Exercise 3 has you build it.

11.4. Getting answers out: search

Training is settled; now the neglected half. At inference the decoder defines a probability for every possible output sequence, and we want the best one. That is a *search problem* over a tree where every node branches V_{tgt} ways; exhaustive search is V^T, hopeless. Everything practical is a heuristic walk through that tree.

Greedy search, which `translate` above already performs, takes the single most probable token at each step and never looks back. Complexity $O(V \cdot T)$, quality: usually fine, occasionally myopic. The failure mode is structural: a token that looks best *right now* can commit the decoder to a mediocre continuation, when a slightly worse token now would have opened a much better path. Maximizing each step is not maximizing the sequence. You met this exact gap in Chapter 2, where the hard max's stepwise confidence hid the alternatives softmax kept alive.

Beam search widens the walk: carry the k best partial sequences (the *beam*), expand each by all V tokens, keep the top k by *cumulative* log-probability, repeat. Complexity $O(k \cdot V \cdot T)$, a small multiple of greedy, nothing like exhaustive.

```
@torch.no_grad()
def beam_search(model: Seq2Seq, src: str, k: int = 5,
                maxlen: int = 12) -> list[tuple[str, float]]:
    model.eval()
    S = torch.tensor([[src_stoi[c] for c in src]])
    state = model.encode(S, torch.tensor([len(src)]))
    beams = [([], 0.0, state, False)]              # (tokens, logp, state, done)
    for _ in range(maxlen):
        cand = []
        for seq, lp, st, done in beams:
            if done:
                cand.append((seq, lp, st, True)); continue
            tok = torch.tensor([[seq[-1] if seq else BOS]])
            o, st2 = model.decoder(model.tgt_emb(tok), st)
```

```
            logp = F.log_softmax(model.out(o)[0, -1], -1)
            top = torch.topk(logp, k)
            for lg, ix in zip(top.values, top.indices):
                cand.append((seq + [ix.item()], lp + lg.item(), st2,
                             ix.item() == EOS))
        beams = sorted(cand, key=lambda b: -b[1])[:k]
        if all(b[3] for b in beams):
            break
    return [("".join(tgt_itos.get(i, "?") for i in seq if i > 2), lp)
            for seq, lp, _, _ in beams]
```

Now the payoff, and it is exactly where we buried it: the ambiguous dates. For `03/05/2021`, "the one best answer" is the wrong question. The training world itself was split 70/30 between two readings. Greedy silently hands you one sequence; beam search shows which complete alternatives the model ranks highly:

```
amb = [(s, t) for s, t in test_pairs
       if "/" in s and int(s[:2]) <= 12 and int(s[3:5]) <= 12]
for s, t in amb[:4]:
    print(f"{s!r}   truth: {t!r}   greedy: {translate(tf_model, s)!r}")
    for text, lp in beam_search(tf_model, s, k=4)[:2]:
        print(f"   beam: {text!r}   joint log score = {lp:.2f}")
    print()
```

```
'09/02/1995'   truth: '1995-09-02'   greedy: '1995-09-02'
   beam: '1995-09-02'   joint log score = -0.23
   beam: '1995-09-20'   joint log score = -1.84

'12/06/2006'   truth: '2006-12-12'   greedy: '2006-12-06'
   beam: '2006-12-06'   joint log score = -0.21
   beam: '2006-12-12'   joint log score = -2.51

'08/01/1989'   truth: '1989-08-01'   greedy: '1989-08-01'
   beam: '1989-08-01'   joint log score = -0.14
   beam: '1989-01-08'   joint log score = -2.51

'01/03/1991'   truth: '1991-01-03'   greedy: '1991-01-03'
   beam: '1991-01-03'   joint log score = -0.54
   beam: '1991-03-01'   joint log score = -1.23
```

Each displayed score is the sum of token log-probabilities along that one sequence, including `<eos>` when it was reached. These are **raw joint log scores**, not a normalized distribution over the printed beam, so exponentiating two of them and calling the result a 70/30 posterior would be wrong. Some pairs correspond cleanly to the month/day and day/month readings. Others include a high-scoring hybrid that matches neither convention, an honest reminder that beam

search searches the model; it does not install a calendar constraint. The ranking still exposes alternatives that greedy search hides.

Honesty requires the other half of the report:

```
sub = random.Random(0).sample(test_pairs, 200)
n_diff = sum(beam_search(tf_model, s, k=5)[0][0] != translate(tf_model, s)
             for s, _ in sub)
print(f"beam-5 disagrees with greedy on {n_diff} of {len(sub)} test dates")
```

```
beam-5 disagrees with greedy on 1 of 200 test dates
```

On this small, well-trained model, beam search changes almost nothing. Beam search earns its keep at the margins, but real translation, with vocabularies of tens of thousands and long sentences, lives entirely at those margins, which is why beam decoding became a standard tool of the seq2seq era.

> **i** Length normalization
>
> Summed log-probabilities shrink as sequences grow → a raw beam quietly favors short outputs. The standard correction divides the score by L^{α} with $\alpha \approx 0.6$–0.75. Our dates all have the same length, so the bias is largely invisible for correct outputs here (Exercise 4 makes you check the remaining error cases); in free-length generation it matters enormously.

11.5. The fixed-size state in the middle

Time to stare at the handoff. Every property of the source that the decoder will ever use crossed one bridge: $(\mathbf{h}, \mathbf{c})$, two 128-dimensional vectors and therefore 256 scalars. Our task fits because a date is three facts, yet its residual digit and order errors are consistent with a decoder reconstructing from a compressed summary rather than consulting source positions directly. They do not, by themselves, prove that compression caused each error. Now scale the ambition: a forty-word sentence, clause structure, names, negations, all crammed through the same fixed pipe, *regardless of length*. Chapter 10 called this the price of the Markov claim; here the bill arrives itemized.

And notice what the architecture throws away: the encoder *had* a state at every step, a whole trajectory of summaries, one per source position, and we kept only the last. The fix that defines Part IV is almost embarrassingly direct: keep them all, and let the decoder *look back* at whichever ones matter for the token it is writing right now. Choosing where to look, by similarity, on the fly: you have owned the primitive since Chapter 1's dot product, and Chapter 2 promised you the "soft lookup" that makes it differentiable. Chapter 12 builds the classical version; Chapter 13 makes it learnable and comes back for this very date task with an alignment map to show where the decoder looks.

11.6. Okay, so —

1. **Encoder–decoder = two RNNs and a handoff**: read the source into a final state, hand it over, generate from it. `<bos>` starts the decoder, `<eos>` lets it choose its length, embeddings replace one-hots (hard address, learned content).
2. **Padding poisons twice**: the loss (`ignore_index`) and the encoder (`pack_padded_sequence`). On never-seen source strings, the same model and budget score 34.3% naive versus 93.1% packed. Target padding is included for reuse but not exercised by these fixed-length ISO targets. Audit bookkeeping before architecture.
3. **Teacher forcing** trains the decoder on the gold rail: one fused call with no Python feedback loop and stable early learning, at the cost of **exposure bias**, the train/inference mismatch that free-running avoids by paying with speed and early instability. Scheduled sampling anneals between them, flipping per token.
4. **Decoding is search.** Greedy maximizes steps, not sequences; beam search carries k rails for $O(kVT)$ and surfaces high-scoring alternatives. Its raw joint log scores are rankings, not a normalized posterior, and a beam can contain invalid hybrids. Length normalization (L^α) keeps long outputs competitive.
5. **The bottleneck is now the story.** One fixed-size $(\mathbf{h}, \mathbf{c})$ pair, 256 scalars here, connects reader and writer regardless of input length. The encoder's per-step states were there all along. Part IV is what happens when we stop throwing them away.

Sources and further reading

- Sutskever, Vinyals, & Le, *Sequence to Sequence Learning with Neural Networks*: the canonical fixed-state LSTM encoder–decoder and source-reversal result.
- Bengio et al., *Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks*: the original exposure-bias intervention used in Exercise 3.
- Bahdanau, Cho, & Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate*: removes the single-state bottleneck by learning where to look; Chapter 13 performs the full harvest.

Exercises

1. **(Pencil.)** Count this chapter's parameters: two embeddings, two LSTMs, one output layer ($V_{\text{src}} = 37$, $V_{\text{tgt}} = 14$, embed 32, hidden 128; an LSTM holds $4\,[(e + h)h + 2h]$ — derive that first). Where do most parameters live, and why is the answer different from Chapter 8's LeNet audit?
2. **(Pencil.)** With vocabulary 10^4 and output length 10, compare the number of sequences examined by exhaustive search, greedy, and beam-5. Then explain why beam search with $k = V^T$ is exact and with $k = 1$ is greedy — and why neither endpoint is usable.
3. **(Code.)** Implement scheduled sampling: per token, use the gold input with probability ϵ, the model's own prediction otherwise, annealing ϵ linearly $1 \to 0$ over 25 epochs. Compare its learning curve with both curves in Figure 11.2. Does it inherit teacher forcing's fast start *and* free-running's finish?

4. **(Code.)** Add length normalization ($/L^{\alpha}$) to `beam_search` and sweep $\alpha \in \{0, 0.7, 1\}$ on the date task. Explain why correct ten-character outputs are unaffected, inspect whether any erroneous beams change, and design the smallest change to the *task* that would make α matter.
5. **(Code.)** Sutskever et al. (2014) famously improved translation by feeding the source sentence *reversed*. Try it here. Whatever you observe, explain it with this chapter's tools: which source-target dependencies does reversal shorten in their setting, and which does it shorten — or lengthen — for dates, where the year sits at the end of the source but the start of the target?

Part IV.

IV · Attention: Learning the Similarity

12. Kernel Regression: Attention Before It Was Learnable

Chapter 11 ended with a machine that could translate, and one uncomfortable picture: every source fact had to cross a single fixed-size bridge before the decoder wrote its first token. The encoder had computed a state at every source position. Then we threw all but the last one away.

The lecture's repair begins with an almost embarrassing question: why throw them away? Keep the whole sequence of encoder states as a **memory bank**. At each decoding step, ask what the decoder needs, compare that query with every stored state, and retrieve a weighted mixture of the relevant content. Keeping a bank does not make its entries lossless, but it removes the rule that *all* source information must pass through the last entry alone.

One problem remains: how should a query decide the mixture? A nearest-neighbor rule changes its chosen slot abruptly. We first need a smooth similarity-weighted rule, and statistics had one in 1964, half a century before modern neural attention.

Let us start with three observed pairs:

observation	location x_i	response y_i
1	1	1.5
2	3	2.8
3	5	1.8

For a query $q = 3.5$, a Gaussian similarity with bandwidth $h = 0.6$ assigns weights $(0.0002, 0.9413, 0.0585)$. Their weighted mixture is 2.7412. The second observation dominates because its location is closest, the third contributes a little, and the distant first observation is nearly silent. No similarity function was learned; the chosen metric and bandwidth did the routing.

12.1. Chapter 1 already mixed old answers

There is a piece of unfinished business from Chapter 1. Linear regression makes a prediction by mixing the observed targets. It hid that fact inside the normal equation.

Let the augmented design matrix $\widetilde{\mathbf{X}} \in \mathbb{R}^{n\times(d+1)}$ include the intercept column, let $\tilde{\mathbf{q}} \in \mathbb{R}^{d+1}$ be a new query with its leading 1, and let $\mathbf{y} \in \mathbb{R}^n$ contain the observed targets. Assuming full column rank,

$$\hat{} = (\widetilde{\mathbf{X}}^\top \widetilde{\mathbf{X}})^{-1} \widetilde{\mathbf{X}}^\top \mathbf{y}.$$

The prediction at the query is therefore

$$\hat{f}(\mathbf{q}) = \tilde{\mathbf{q}}^{\hat{}} = \underbrace{\left[\widetilde{\mathbf{X}}(\widetilde{\mathbf{X}}^\top \widetilde{\mathbf{X}})^{-1} \tilde{\mathbf{q}}\right]^\top}_{(\mathbf{q})} \mathbf{y}. \tag{12.1}$$

So Chapter 1's promise was exact: $\hat{f}(\mathbf{q})$ is a weighted combination of the training targets. With an intercept, the weights sum to one because OLS reproduces a constant response. But **weighted combination** is not the same as **weighted average**. The entries of may be negative.

For keys $(-2, -1, 0, 1, 2)$ and query $q = 1.5$, the OLS weights are

$$(-0.10, 0.05, 0.20, 0.35, 0.50).$$

They sum to one, yet the first is negative. If the targets are $(1, 0, 0, 0, 0)$, OLS predicts -0.10, outside the observed range $[0, 1]$. Linear regression is allowed to leave the observed targets' convex hull even when the query lies inside the observed key range. For local smoothing, we want a stricter contract: nearby evidence should receive more weight, every weight should be nonnegative, and the weights should sum to one.

12.2. The magnifying glass becomes a kernel

A **smoothing kernel** is a similarity rule. It looks at a query and a stored key and returns a nonnegative affinity: large when they are close, small when they are far. To avoid confusing the kernel function with the matrix of keys later, we write it as κ_h. The Gaussian choice is

$$\kappa_h(\mathbf{q}, \mathbf{k}) = \exp\left(-\frac{\|\mathbf{q} - \mathbf{k}\|^2}{2h^2}\right), \qquad h > 0. \tag{12.2}$$

The bandwidth h sets the width of the magnifying glass. A small value sees only the closest keys; a large value blends a broad neighborhood.

Given observed pairs $(\mathbf{x}_i, y_i)$, normalize the affinities,

$$\alpha_i(\mathbf{q}) = \frac{\kappa_h(\mathbf{q}, \mathbf{x}_i)}{\sum_{j=1}^n \kappa_h(\mathbf{q}, \mathbf{x}_j)}, \qquad \hat{f}_h(\mathbf{q}) = \sum_{i=1}^n \alpha_i(\mathbf{q}) y_i. \tag{12.3}$$

This is the **Nadaraya–Watson estimator**, proposed independently by Nadaraya and Watson in 1964. For the strictly positive Gaussian, each $\alpha_i > 0$ and $\sum_i \alpha_i = 1$. Scalar predictions remain between the smallest and largest observed values. Unlike OLS, this operator smooths locally while keeping every prediction inside the observed values' convex hull.

The endpoint behavior is worth stating precisely. As $h \to 0^+$, the weight concentrates on the unique nearest key; exact nearest-key ties split the mass. At $h = 0$ the formula is undefined. As $h \to \infty$, every finite distance looks alike and the weights approach $1/n$.

Now let us make the lecture's change of names. Locations $\mathbf{x}_i$ become **keys** $\mathbf{k}_i$, observed responses become **values** $\mathbf{v}_i$, and $\mathbf{q}$ remains the **query**. The formula does not change. Here is the whole operator in NumPy. The shapes come before the multiplication: queries are (m, d), keys are (n, d), values are (n, p), and the returned mixture is (m, p).

```
from __future__ import annotations

import matplotlib.pyplot as plt
import numpy as np
import torch

torch.manual_seed(6050)
np.set_printoptions(precision=4, suppress=True)

def row_softmax(scores: np.ndarray) -> np.ndarray:
    """Normalize each row after a stability-preserving shift."""
    shifted = scores - scores.max(axis=1, keepdims=True)
    weights = np.exp(shifted)
    return weights / weights.sum(axis=1, keepdims=True)

def gaussian_attention(
    queries: np.ndarray,
    keys: np.ndarray,
    values: np.ndarray,
    bandwidth: float,
) -> tuple[np.ndarray, np.ndarray]:
    """Fixed Gaussian attention for Q:(m,d), K:(n,d), V:(n,p)."""
    if not np.isfinite(bandwidth) or bandwidth <= 0:
        raise ValueError("bandwidth must be positive and finite")
    if queries.ndim != 2 or keys.ndim != 2 or values.ndim != 2:
        raise ValueError("queries, keys, and values must be matrices")
    if queries.shape[1] != keys.shape[1] or keys.shape[0] != values.shape[0]:
        raise ValueError("Q/K feature dimensions and K/V row counts must agree")

    distances2 = ((queries[:, None, :] - keys[None, :, :]) ** 2).sum(axis=2)
    log_scores = -distances2 / (2.0 * bandwidth**2)  # (m, n)
    weights = row_softmax(log_scores)                # (m, n)
    return weights @ values, weights                 # (m, p), (m, n)

keys3 = np.array([[1.0], [3.0], [5.0]])
values3 = np.array([[1.5], [2.8], [1.8]])
query3 = np.array([[3.5]])
pred3, weights3 = gaussian_attention(query3, keys3, values3, bandwidth=0.6)
```

```
print("weights:", weights3[0])
print(f"prediction: {pred3.item():.4f}")
```

```
weights: [0.0002 0.9413 0.0585]
prediction: 2.7412
```

```
fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.0), constrained_layout=True)
axes[0].scatter(keys3[:, 0], values3[:, 0], s=55, color="#232D4B", zorder=3)
axes[0].axvline(query3.item(), color="#E57200", ls="--", lw=1.4)
axes[0].scatter(query3.item(), pred3.item(), marker="*", s=130,
                color="#E57200", zorder=4)
for key, value, weight in zip(keys3[:, 0], values3[:, 0], weights3[0]):
    axes[0].plot([query3.item(), key], [pred3.item(), value],
                 color="#B45309", lw=0.8 + 5.0 * weight,
                 alpha=0.25 + 0.7 * weight)
axes[0].set(xlabel="location", ylabel="value", title="Query nearby evidence")
axes[0].set_xticks([1, 3, 3.5, 5], ["1", "3", "q=3.5", "5"])
axes[0].annotate(r"$\hat f(q)=2.74$", (query3.item(), pred3.item()),
                 xytext=(8, -22), textcoords="offset points", color="#B45309")

axes[1].bar(["$k_1$", "$k_2$", "$k_3$"], weights3[0],
            color=["#B0B7C3", "#E57200", "#B45309"])
for i, weight in enumerate(weights3[0]):
    axes[1].text(i, weight + 0.025, f"{weight:.4f}", ha="center")
axes[1].set(ylabel="normalized weight", ylim=(0, 1.05),
            title="Similarity decides the mix")
plt.show()
```

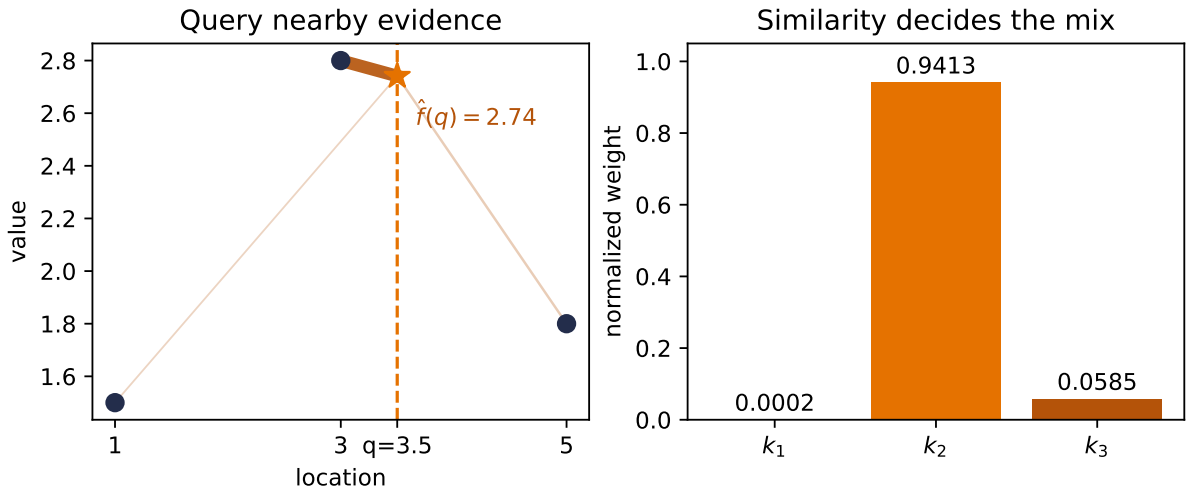

Figure 12.1.: A fixed Gaussian lookup. The bars give the normalized weights, and the connecting lines show which observations feed the query. The query lands close to the second location, so its response dominates the mixture; no similarity function was learned.

The same code makes the OLS distinction concrete:

```
ols_keys = np.array([-2.0, -1.0, 0.0, 1.0, 2.0])
ols_query = 1.5
design = np.column_stack([np.ones_like(ols_keys), ols_keys])
query_design = np.array([1.0, ols_query])
ols_weights = query_design @ np.linalg.solve(design.T @ design, design.T)

witness_values = np.array([[1.0], [0.0], [0.0], [0.0], [0.0]])
nw_pred, nw_weights = gaussian_attention(
    np.array([[ols_query]]), ols_keys[:, None], witness_values, bandwidth=1.0
)
print("OLS weights:", ols_weights, "sum:", ols_weights.sum())
print("NW  weights:", nw_weights[0], "sum:", nw_weights.sum())
print(f"witness predictions: OLS {ols_weights @ witness_values[:, 0]:.4f}, "
      f"NW {nw_pred.item():.4f}; target range [0, 1]")
```

```
OLS weights: [-0.1   0.05  0.2   0.35  0.5 ] sum: 1.0
NW  weights: [0.001  0.0206 0.152  0.4132 0.4132] sum: 1.0
witness predictions: OLS -0.1000, NW 0.0010; target range [0, 1]
```

Which kernels make a convex average?

Only nonnegative smoothing kernels make this convex average. A Gaussian gives positive weights; a compact-support kernel may give zeros. Other objects also bear the name *kernel*. Equation 12.3 needs nonnegative numerators and a positive denominator.

12.3. Chapter 2's scores-to-weights machine

Now harvest the second promise. Chapter 2 introduced softmax as the machine that turns arbitrary scores into nonnegative weights summing to one. Nadaraya–Watson has exactly that form. For any strictly positive kernel,

$$\alpha_i(\mathbf{q}) = \frac{\kappa_h(\mathbf{q}, \mathbf{k}_i)}{\sum_j \kappa_h(\mathbf{q}, \mathbf{k}_j)} = \mathrm{softmax}_i(\log \kappa_h(\mathbf{q}, \mathbf{k}_i)) \,. \tag{12.4}$$

The **score** is the log-kernel. With the Gaussian,

$$a_i(\mathbf{q}) = -\frac{\|\mathbf{q} - \mathbf{k}_i\|^2}{2h^2}.$$

This also makes the temperature connection exact. If the unscaled score is $s_i = -\|\mathbf{q} - \mathbf{k}_i\|^2$, then

$$(\mathbf{q}) = \mathrm{softmax}\Big(\frac{\mathbf{s}}{\tau}\Big), \qquad \tau = 2h^2. \tag{12.5}$$

Small bandwidth → low temperature → sharp lookup. Large bandwidth → high temperature → diffuse averaging. The factor 2 belongs to this score convention; if we had defined $s_i = -\|\mathbf{q} - \mathbf{k}_i\|^2/2$, the temperature would be h^2 instead.

Notice the implementation never computes κ_h and then takes its logarithm. It computes the log-score directly, subtracts the largest score in each row, and only then exponentiates. That is Chapter 2's stable-softmax hygiene, now paying rent again.

12.4. Bandwidth: the honesty gate

The three-point example makes a narrow bandwidth look wonderful. That is not evidence that narrower is better. A tiny neighborhood follows every noisy bump; a huge neighborhood washes real structure away. Chapter 1 called this bias versus variance. Here the bandwidth controls the trade.

Because this is a synthetic laboratory, we can do something ordinary observed data does not permit: hold 60 key locations fixed, redraw the response noise 1,000 times, and compare every prediction with the known function

$$f(x) = \sin(1.5x) + 0.3\cos(4x), \qquad y_i = f(x_i) + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, 0.25^2).$$

The key locations are seeded uniform draws on $[-3, 3]$. The 241 query locations lie on $[-2.9, 2.9]$ and share no exact coordinate with a key. Holding the design fixed lets us separate squared bias from variance over independent noise draws. The population identity

$$\text{MSE} = \text{bias}^2 + \text{variance}$$

has no irreducible-noise term because we evaluate against the noiseless f; predicting a fresh noisy response would add the irreducible variance 0.25^2 to every row. In our finite 1,000-world ensemble, the analogous empirical decomposition closes to floating-point roundoff.

```
def truth(x: np.ndarray) -> np.ndarray:
    return np.sin(1.5 * x) + 0.3 * np.cos(4.0 * x)

data_rng = np.random.default_rng(605012)
keys = np.sort(data_rng.uniform(-3.0, 3.0, size=60))
observed_rng = np.random.default_rng(605013)
observed_values = truth(keys) + observed_rng.normal(0.0, 0.25, size=keys.size)
queries = np.linspace(-2.9, 2.9, 241)

mc_rng = np.random.default_rng(605014)
responses = truth(keys)[None, :] + mc_rng.normal(
    0.0, 0.25, size=(1_000, keys.size)
)
bandwidths = np.array([0.05, 0.08, 0.12, 0.18, 0.28, 0.42, 0.65, 1.00])

rows = []
for bandwidth in bandwidths:
    _, weights = gaussian_attention(
        queries[:, None], keys[:, None], observed_values[:, None], bandwidth
    )
    predictions = responses @ weights.T                 # (world, query)
    mean_prediction = predictions.mean(axis=0)
    squared_bias = np.mean((mean_prediction - truth(queries)) ** 2)
    variance = np.mean(np.var(predictions, axis=0))
    mse = np.mean((predictions - truth(queries)[None, :]) ** 2)
    rows.append((bandwidth, 2 * bandwidth**2, squared_bias, variance, mse))

print(" h     2h^2    bias^2  variance     MSE")
for row in rows:
    print(f"{row[0]:.2f}   {row[1]:.4f}   {row[2]:.4f}   "
          f"{row[3]:.4f}   {row[4]:.4f}")
best = rows[int(np.argmin([row[-1] for row in rows]))]
print(f"grid minimum: h={best[0]:.2f}, MSE={best[-1]:.4f}")
```

```
 h     2h^2    bias^2  variance     MSE
0.05   0.0050   0.0119   0.0343   0.0462
0.08   0.0128   0.0101   0.0247   0.0348
0.12   0.0288   0.0090   0.0169   0.0259
0.18   0.0648   0.0127   0.0109   0.0236
```

```
0.28   0.1568   0.0304   0.0069   0.0373
0.42   0.3528   0.0747   0.0047   0.0794
0.65   0.8450   0.1721   0.0031   0.1753
1.00   2.0000   0.3142   0.0022   0.3164
grid minimum: h=0.18, MSE=0.0236
```

The variance falls at every step, from 0.0343 at $h = 0.05$ to 0.0022 at $h = 1.00$. Bias squared is not perfectly monotone at the sharp end of this sparse, random design; it bottoms at 0.0090 for $h = 0.12$, then rises to 0.3142 at $h = 1.00$. Their sum is the U-shaped quantity we care about. On this predeclared grid, $h = 0.18$ has the smallest MSE, 0.0236.

One noisy world shows what those numbers mean:

```
fig, ax = plt.subplots(figsize=(7.2, 4.1), constrained_layout=True)
dense = np.linspace(-3.0, 3.0, 601)
ax.plot(dense, truth(dense), color="black", lw=2.0, label="truth")
ax.scatter(keys, observed_values, s=15, color="#555555", alpha=0.72,
           label="one noisy sample", zorder=3)
for bandwidth, color in [(0.05, "#D55E00"),
                         (0.18, "#0072B2"),
                         (0.65, "#009E73")]:
    predictions, _ = gaussian_attention(
        dense[:, None], keys[:, None], observed_values[:, None], bandwidth
    )
    ax.plot(dense, predictions[:, 0], color=color, lw=1.8,
            label=fr"$h={bandwidth:.2f}$")
ax.set(xlabel="query $q$", ylabel="prediction",
       title="Bandwidth is a smoothness dial")
ax.legend(ncol=2, frameon=False, fontsize=9)
plt.show()
```

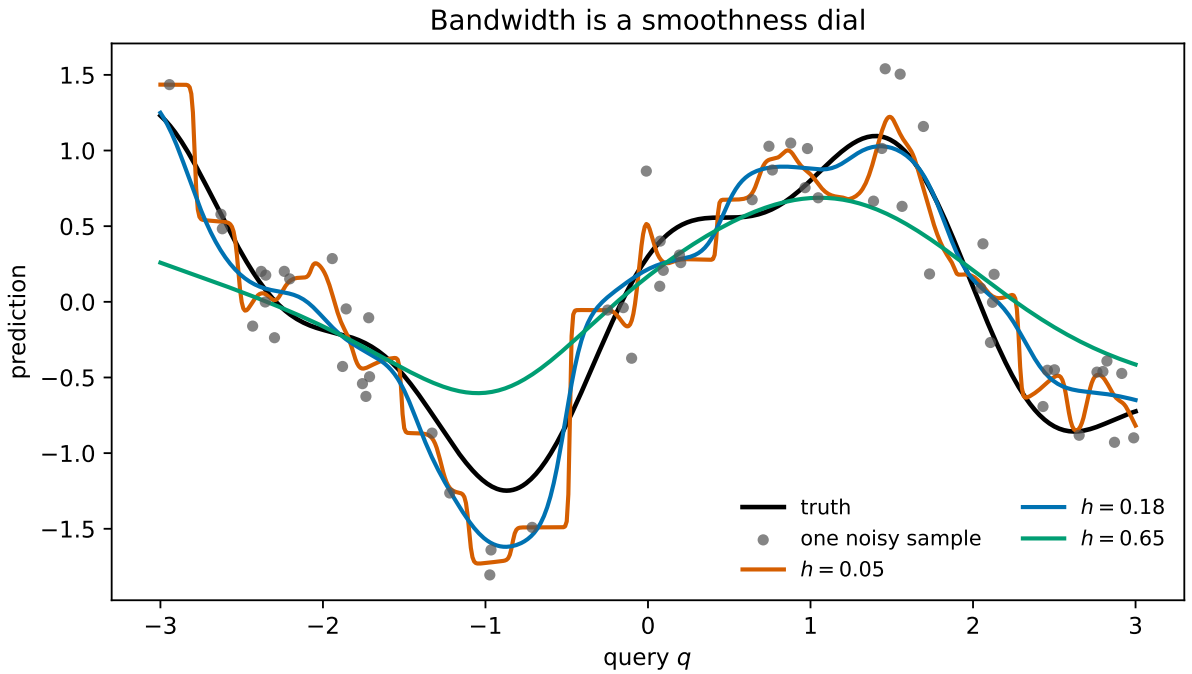

Figure 12.2.: One fixed noisy sample under three bandwidths. The narrow rule follows local noise, the middle rule tracks the signal, and the broad rule erases the function's smaller-scale structure. The blue bandwidth is the grid minimum in the 1,000-world audit, not a hand-picked attractive curve.

Selecting h when the truth is hidden

The known black curve is a laboratory privilege. On real data, choose the bandwidth using a validation set, then report once on a sealed test set. Training error alone will usually reward an extremely narrow kernel for copying each noisy target back to itself. The generalization protocol from Chapter 6 still applies even when nothing is trained by gradient descent.

12.5. From one query to an attention matrix

Now write the operator in the vocabulary that the rest of Part IV will keep. Let

$$\mathbf{Q} \in \mathbb{R}^{m \times d}, \qquad \mathbf{K} \in \mathbb{R}^{n \times d}, \qquad \mathbf{V} \in \mathbb{R}^{n \times p}$$

hold m queries, n keys, and one p-dimensional value per key. The score and weight matrices are

$$S_{ri} = -\frac{\|\mathbf{q}_r - \mathbf{k}_i\|^2}{2h^2}, \qquad \mathbf{A} = \operatorname{rowsoftmax}(\mathbf{S}) \in \mathbb{R}^{m \times n}. \tag{12.6}$$

Every row of $\mathbf{A}$ sums to one; columns generally do not. The pooled outputs are one matrix multiplication,

$$\text{Attention}_h(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{AV} \in \mathbb{R}^{m \times p}. \tag{12.7}$$

This is the book's first **attention-weight matrix**. It is not a learned alignment. It is the allocation imposed by Euclidean distance and the chosen bandwidth.

```
fig, axes = plt.subplots(1, 2, figsize=(7.4, 3.25), constrained_layout=True)
row_array = np.asarray(rows)
axes[0].plot(row_array[:, 0], row_array[:, 2], "o-", label=r"bias$^2$")
axes[0].plot(row_array[:, 0], row_array[:, 3], "o-", label="variance")
axes[0].plot(row_array[:, 0], row_array[:, 4], "o-", lw=2.2, label="MSE")
axes[0].axvline(best[0], color="#888888", ls="--", lw=1)
axes[0].set_xscale("log")
axes[0].set(xlabel="bandwidth $h$", ylabel="error against true $f$",
            title="The trade-off is measurable")
axes[0].legend(frameon=False, fontsize=8)

map_queries = np.linspace(-2.9, 2.9, 31)
map_predictions, attention_map = gaussian_attention(
    map_queries[:, None], keys[:, None], observed_values[:, None], best[0]
)
image = axes[1].imshow(
    attention_map, origin="lower", aspect="auto", cmap="magma",
    extent=(-0.5, len(keys) - 0.5, map_queries[0], map_queries[-1]),
)
key_index_at_query = np.interp(map_queries, keys, np.arange(len(keys)))
axes[1].plot(key_index_at_query, map_queries, color="white", lw=1.0, alpha=0.8)
ticks = np.array([0, 15, 30, 45, 59])
axes[1].set_xticks(ticks, [f"{keys[i]:.1f}" for i in ticks])
axes[1].set(xlabel="key location $k_i$ (sorted)", ylabel="query $q$",
            title=fr"Fixed attention matrix, $h={best[0]:.2f}$")
fig.colorbar(image, ax=axes[1], label="weight", fraction=0.05, pad=0.04)
plt.show()

print(f"Q {map_queries[:, None].shape}, K {keys[:, None].shape}, "
      f"V {observed_values[:, None].shape}, A {attention_map.shape}, "
      f"AV {map_predictions.shape}")
print("maximum row-sum error:",
      f"{np.max(np.abs(attention_map.sum(axis=1) - 1)):.2e}")
```

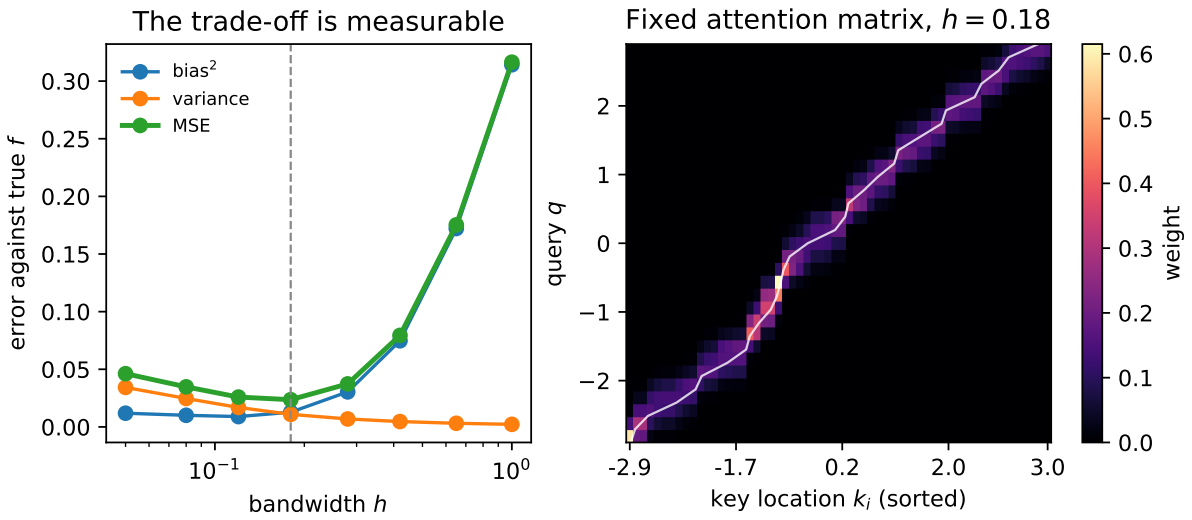

Figure 12.3.: Left: the fixed-design bias–variance audit. Right: the same Gaussian rule written as a 31-query by 60-key attention matrix at $h = 0.18$. Bright cells receive more of a row's unit mass; the white trace marks the $k = q$ locus in sorted-key coordinates. The map is local and interpretable, but it was specified rather than learned.

```
Q (31, 1), K (60, 1), V (60, 1), A (31, 60), AV (31, 1)
maximum row-sum error: 2.22e-16
```

Read the bright band as a moving magnifying glass. Each query row reallocates one unit of weight across the stored keys. Where keys are dense, that unit is divided among several neighbors. Where keys are sparse, one or two slots carry most of it. Changing h changes the width of the band, exactly as changing temperature changes the spread of a softmax distribution.

There is one numerical trap. A Gaussian is mathematically positive everywhere, but a distant query and small bandwidth can make every directly exponentiated kernel value underflow to zero. Normalizing those zeros produces `0/0`. The log-score version still has a largest entry, so the shifted softmax remains finite:

```
identity_query, identity_h = 0.37, 0.28
log_kernel = -0.5 * ((identity_query - keys) / identity_h) ** 2
direct_kernel = np.exp(log_kernel)
direct_weights = direct_kernel / direct_kernel.sum()
softmax_weights = row_softmax(log_kernel[None, :])[0]

far_log_kernel = -0.5 * ((1_000.0 - keys) / 0.1) ** 2
far_kernel = np.exp(far_log_kernel)
with np.errstate(divide="ignore", invalid="ignore"):
    naive_far_weights = far_kernel / far_kernel.sum()
stable_far_weights = row_softmax(far_log_kernel[None, :])[0]
```

```
print("max |normalize(K)-softmax(log K)|:",
      f"{np.max(np.abs(direct_weights - softmax_weights)):.2e}")
print(f"far query: naive denominator={far_kernel.sum():.1f}, "
      f"all finite={np.all(np.isfinite(naive_far_weights))}")
print(f"stable: sum={stable_far_weights.sum():.1f}, "
      f"nearest key={keys[np.argmax(stable_far_weights)]:.4f}")
```

```
max |normalize(K)-softmax(log K)|: 2.78e-17
far query: naive denominator=0.0, all finite=False
stable: sum=1.0, nearest key=2.9886
```

> ⚠ Similarity weights are not uncertainty probabilities
>
> The entries of $\mathbf{A}$ are probability-like allocations: nonnegative and row-normalized. They are not calibrated probabilities that the corresponding key is "correct," and a sharp row is not automatically confidence. It may simply mean the bandwidth is small.

12.6. Return the fixed operator to the memory bank

The regression names now return to Chapter 11's memory bank. Each one names a different job:

job	kernel regression	sequence memory preview
query	location $\mathbf{q}$ where we need an answer	what the decoder needs now
keys	stored coordinates $\mathbf{k}_i$	descriptors used to match encoder memory slots
values	observed responses $\mathbf{v}_i$	content retrieved from those slots
weights	normalized fixed similarities $\alpha_i(\mathbf{q})$	how much each slot contributes
output	$\sum_i \alpha_i \mathbf{v}_i$	a query-specific context vector

Queries specify what is needed; keys provide what to compare against; values provide what to retrieve. In the simplest sequence model, the same encoder state can play both roles, just as our one-dimensional example stores a location and its response together. Later, learned projections will let keys and values carry different representations.

Chapter 1 also planted one more similarity primitive, the dot product:

$$\mathbf{q}^\top \mathbf{k} = \|\mathbf{q}\| \, \|\mathbf{k}\| \cos\theta.$$

It is pure angular similarity only when the norms are controlled. If $\mathbf{q} = (1, 0)$, $\mathbf{k}_1 = (1, 0)$, and $\mathbf{k}_2 = (2, 0)$, both keys point in the same direction as the query, but their dot products are 1 and 2. Magnitude participates. That is not a defect; it is a reminder that the score defines what "similar" means.

Our score today is a hand-written negative squared Euclidean distance, and h is selected rather than learned. The operator stores the data, computes a fixed map, and pools the values. It shares attention's normalized similarity-pooling algebra, but it has no learned compatibility function.

Now the Part II rhyme returns. Chapter 7 ended with a fixed image kernel and asked what would happen if the filter became learnable. Chapter 8 answered with a CNN. We end Part IV's opening chapter with the same question:

> What if the similarity itself were learnable?
>
> The memory bank is ready. Query, keys, values, scores, row-wise softmax, and weighted pooling are ready. Only one hand-designed piece remains: the rule that decides whether a query and key belong together. Chapter 13 makes that rule learnable, then returns to Chapter 11's date task so the decoder can show us where it looked.

12.7. Okay, so —

1. **Chapter 1's linear prediction already mixed training targets.** OLS gives $\hat{f}(\mathbf{q}) = (\mathbf{q})^\top \mathbf{y}$, but its weights may be signed, so the prediction can leave the targets' convex hull.
2. **Nadaraya–Watson turns locality into a convex mixture.** A nonnegative kernel scores query–key similarity; normalization makes the weights sum to one; their weighted values form the prediction.
3. **Bandwidth is temperature in geometric clothing.** For Gaussian distance scores, $\tau = 2h^2$: narrow is sharp and variable, wide is diffuse and biased. Our fixed audit reached its grid-minimum MSE of 0.0236 at $h = 0.18$.
4. **Chapter 2's softmax is the normalization exactly.** Positive-kernel weights equal `softmax(log kernel)`. Computing log-scores directly and shifting each row avoids the distant-query `0/0` trap.
5. **Fixed attention is AV.** Queries score keys, row-softmax produces an $m \times n$ allocation matrix, and that matrix pools the values. The algebra is complete; learning the score is the next chapter's move.

Sources and further reading

- Nadaraya, *On Estimating Regression*: the 1964 two-page paper introducing one side of the estimator now bearing Nadaraya and Watson's names.
- Watson, *Smooth Regression Analysis*: the independent 1964 development of smooth regression by local weighting.

- Bahdanau, Cho, & Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate*: the neural-attention step that replaces one fixed context with learned soft alignment; Chapter 13 performs that harvest.

Exercises

1. **(Pencil.)** Prove that Equation 12.3 is a convex combination when $\kappa_h \geq 0$ and its denominator is positive. Then prove that a scalar prediction lies in $[\min_i v_i, \max_i v_i]$. Which step fails for the OLS weights in Equation 12.1?
2. **(Pencil.)** For keys $(-2, -1, 0, 1, 2)$, derive the five OLS target weights at $q = 1.5$ without using code. Verify that they sum to one, identify the negative weight, and construct a second target vector for which OLS leaves the target range.
3. **(Code.)** Split a fresh noisy sample from this chapter's function into training, validation, and sealed test sets. Select h on validation data, report the test MSE once, and compare your selected curve with the synthetic-truth grid minimum above. Explain why the two procedures need not select the same bandwidth.
4. **(Pencil + code.)** Derive Equation 12.5. Then use keys $[0, 0, 1]$, query 0, and successively smaller positive bandwidths. What weights does stable softmax assign to the exact nearest-key tie, and why is $h = 0$ still invalid?
5. **(Code.)** Extend `gaussian_attention` to two-dimensional keys whose coordinates have very different units. Compare the attention map before and after standardizing each feature using training-set statistics. What does this reveal about the metric hidden inside a fixed kernel?

13. Attention: Making the Kernel Learnable

Chapter 12 built the whole attention operator except for one piece. It had a query, a bank of keys and values, a score for every query–key pair, softmax weights, and a weighted read. But Euclidean distance decided what counted as similar. The task could tune the bandwidth; it could not invent a better notion of relevance.

Chapter 11 left us a more pointed promise: **hard address, learned content**. An embedding can learn what lives at row 17, but fetching row 17 is still a rigid indexing operation. Chapter 2 told us how to soften that hard choice. Replace the single address with a differentiable distribution over addresses, let the loss send blame through every score, and learn which memory locations deserve weight. A differentiable lookup, Chapter 2 promised, is precisely attention. The promise comes due here.

The result is visible in the date task before we name all its parts. For the fixed validation source

```
may 17, 1971  ->  1971-05-17
```

the trained decoder emits the year first even though those four characters sit at the end of the source. Across the fixed 400-example validation audit, its first four attention rows place **97.5%** of their mass on the four contextual encoder states indexed by the source-year characters. When the example above emits its second character, **9**, the largest weight is 0.856 on the state indexed by the source's 9; when it emits **7**, the largest weight is 0.985 on the state indexed by the 7 in the year region. Those weights were not supervised as alignments. The date loss taught the model where useful information lived.

13.1. The scores-to-weights machine, one more time

Let the encoder retain its full memory

$$\mathbf{H} = [\mathbf{h}_1, \dots, \mathbf{h}_n]^\top \in \mathbb{R}^{n \times d_e}.$$

At decoder step t, use the previous decoder hidden state $\mathbf{s}_{t-1} \in \mathbb{R}^{d_d}$ as the query. A learned compatibility function a_θ produces one score per source position:

$$e_{t,i} = a_\theta(\mathbf{s}_{t-1}, \mathbf{h}_i).$$

For each padded batch item b, let $M_{b,i} = 0$ at real source positions and $M_{b,i} = -\infty$ at padding. Suppress the batch index below. Then

$$\alpha_{t,i} = \mathrm{softmax}_i(e_{t,i} + M_i), \qquad \mathbf{c}_t = \sum_{i=1}^{n} \alpha_{t,i}\mathbf{h}_i. \tag{13.1}$$

This is Chapter 12's fixed matrix **AV** with a learned score. It is also Chapter 2's **scores** → **weights** machine in exactly the form that chapter advertised. A hard source-position choice would use argmax and one-hot indexing. The soft weights move continuously when the scores move, so backpropagation can tell the scorer how each valid address affected the loss. We have **softened the hard**.

Because the query comes from the decoder and the memory comes from the encoder, this is **cross-attention**. It bridges two sequences. The model still initializes the decoder from the encoder's final state, but that fixed bridge is no longer the only path from source to output: every decoder step receives its own context $\mathbf{c}_t$. Chapters 10 and 11 called the old handoff a **finite-state bottleneck**. Chapter 12 kept the memory bank; this chapter completes that relay by learning its access rule.

Chapter 8 made one other promise. A CNN buys global sight by stacking local operations until its receptive field spans the input. One cross-attention read can score every encoder position in a single layer. It buys global access directly, paying for all query–key comparisons and, for now, for the RNNs that created those states sequentially.

13.2. Additive attention: learn the comparison itself

The most literal answer to "make similarity learnable" is a small neural network. Choose an alignment width d_a. For one decoder query and one encoder state,

$$e_{t,i} = \mathbf{v}_a^\top \tanh\big(\mathbf{W}_q\mathbf{s}_{t-1} + \mathbf{W}_k\mathbf{h}_i\big)\,, \tag{13.2}$$

where

$$\mathbf{W}_q \in \mathbb{R}^{d_a \times d_d}, \qquad \mathbf{W}_k \in \mathbb{R}^{d_a \times d_e}, \qquad \mathbf{v}_a \in \mathbb{R}^{d_a}.$$

The two projections put objects of different native sizes into one alignment space. The tanh creates match features, and $\mathbf{v}_a$ selects which of those features matter for the scalar score. The same parameters are reused for every decoder step t and source position i. Remember Chapter 7's sliding filter: one learned rule, applied everywhere. Attention shares a compatibility rule across pairs.

For a batch, let $\mathbf{S}_{t-1} \in \mathbb{R}^{B\times 1\times d_d}$ collect the queries. The row-major shapes make the broadcasting explicit:

$$\mathbf{S}_{t-1}\mathbf{W}_q^\top : (B, 1, d_a), \qquad \mathbf{H}\mathbf{W}_k^\top : (B, n, d_a).$$

Their sum produces (B, n, d_a) match features. Reduction by $\mathbf{v}_a$ produces (B, n) scores, softmax runs over the n source positions, and the weighted read returns a (B, d_e) context. That last axis choice is not bookkeeping: softmax over the wrong dimension answers the wrong question.

The lecture's scene freezes one simple parameter snapshot so we can watch the arithmetic. With identity projections and an all-ones selector, three source states receive scores $(1.015, 1.562, 1.124)$ and therefore weights $(0.260, 0.450, 0.290)$:

```
from __future__ import annotations

import math
import random

import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(6050)

query_snapshot = np.array([0.5, 0.5, 0.2, -0.1])
key_snapshot = np.array([
    [-0.3, 0.1, 0.2, 0.0],
    [0.6, 0.5, 0.1, -0.2],
    [0.0, -0.4, 0.3, 0.2],
])
snapshot_scores = np.tanh(query_snapshot + key_snapshot).sum(axis=1)
snapshot_weights = np.exp(snapshot_scores - snapshot_scores.max())
snapshot_weights /= snapshot_weights.sum()

fig, axes = plt.subplots(1, 2, figsize=(7.5, 3.0),
                         gridspec_kw={"width_ratios": [1.35, 1]})
ax = axes[0]
ax.set_xlim(0, 10); ax.set_ylim(0, 5); ax.axis("off")

def stage_box(x: float, y: float, label: str, color: str,
              width: float = 1.25) -> None:
    box = FancyBboxPatch(
        (x - width / 2, y - 0.36), width, 0.72,
        boxstyle="round,pad=0.06", facecolor=color,
        edgecolor="#232D4B", linewidth=1.1,
    )
    ax.add_patch(box)
    ax.text(x, y, label, ha="center", va="center", fontsize=9)
```

```
stage_box(0.9, 3.7, r"$s_{t-1}$", "#F9DCBF")
stage_box(0.9, 1.5, r"$h_i$", "#DCEAF7")
stage_box(3.0, 3.7, r"$W_q$", "#F9DCBF")
stage_box(3.0, 1.5, r"$W_k$", "#DCEAF7")
stage_box(5.0, 2.6, "+", "#F4F4EF", 0.8)
stage_box(6.7, 2.6, "tanh", "#E8E3F2")
stage_box(8.3, 2.6, r"$v_a^\top$", "#FCE8B2")
ax.text(9.6, 2.6, r"$e_{t,i}$", ha="center", va="center",
        color="#B45309", fontsize=11)
for start, end in [
    ((1.55, 3.7), (2.35, 3.7)), ((1.55, 1.5), (2.35, 1.5)),
    ((3.65, 3.7), (4.55, 2.85)), ((3.65, 1.5), (4.55, 2.35)),
    ((5.4, 2.6), (6.08, 2.6)), ((7.32, 2.6), (7.68, 2.6)),
    ((8.95, 2.6), (9.25, 2.6)),
]:
    ax.annotate("", end, start,
                arrowprops={"arrowstyle": "->", "color": "#4A5568", "lw": 1.2})
ax.set_title("One shared learned scorer", fontsize=10)

labels = [r"$h_1$", r"$h_2$", r"$h_3$"]
axes[1].bar(labels, snapshot_weights,
            color=["#A9B7C9", "#E57200", "#C99456"])
for i, (score, weight) in enumerate(zip(snapshot_scores, snapshot_weights)):
    axes[1].text(i, weight + 0.018, f"{weight:.3f}",
                 ha="center", fontsize=9)
    axes[1].text(i, 0.025, f"$e={score:.3f}$",
                 ha="center", va="bottom", fontsize=8, rotation=90, color="white")
axes[1].set(ylabel="softmax weight", ylim=(0, 0.53),
            title="Scores decide the read")
plt.tight_layout()
plt.show()
```

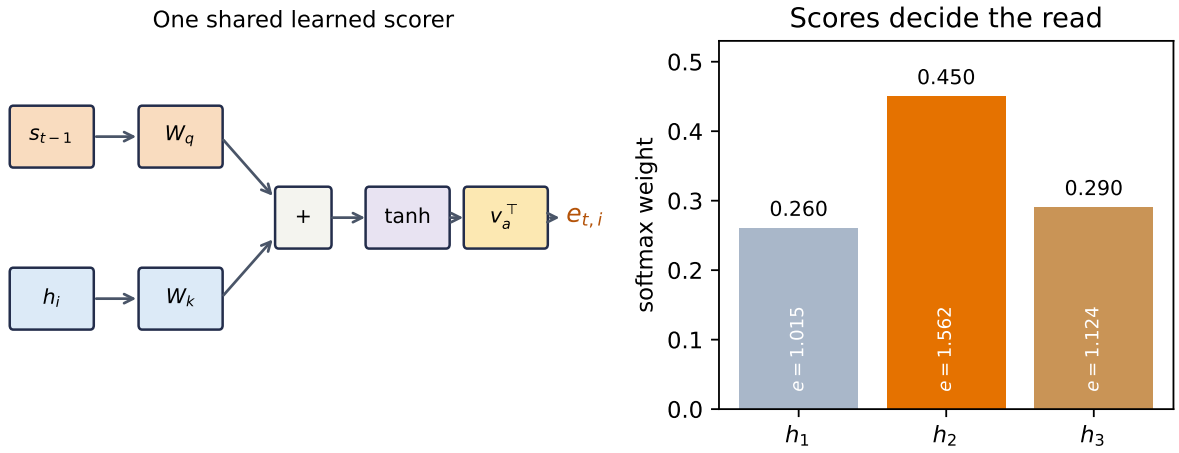

Figure 13.1.: Additive attention learns a compatibility function. Query and key enter separate projections, their alignment features interact through tanh, and a learned selector reduces those features to one score. The right panel freezes one parameter setting to show the resulting scores-to-weights calculation; it is arithmetic, not a trained result.

> **i** "Learned kernel" is an analogy
>
> Exponentiating a learned score and normalizing it reproduces the Nadaraya–Watson algebra. It does not make every attention score a classical kernel. Separate query and key projections can make $a_\theta(q, k)$ asymmetric and not positive semidefinite. What survives exactly is the operational pattern: compare → normalize → mix.

13.3. Scaled dot product: learn the spaces, simplify the score

For m decoder queries, additive attention is vectorized, but it forms a (B, m, n, d_a) intermediate, applies an elementwise tanh, and reduces it. Dense matrix multiplication has a simpler computational path. Chapter 1 introduced the dot product as a similarity score against a template. Here the key is the template, and training learns the space in which that comparison is useful.

The lecture first walks one four-dimensional query past three keys. Its raw dot products are $(1.10, -0.60, 0.15)$. Since $\sqrt{4} = 2$, scaling gives $(0.55, -0.30, 0.075)$, and softmax gives $(0.488, 0.209, 0.303)$:

```
query_walk = torch.tensor([0.9, -0.3, 0.2, 0.6])
key_walk = torch.tensor([
    [0.8, -0.2, 0.1, 0.5],
    [-0.5, 0.9, 0.0, 0.2],
    [0.3, 0.0, 0.6, -0.4],
])
raw_walk = key_walk @ query_walk
```

```
scaled_walk = raw_walk / math.sqrt(query_walk.numel())
weight_walk = torch.softmax(scaled_walk, dim=0)
print("raw scores:", raw_walk.numpy())
print("scaled scores:", scaled_walk.numpy())
print("weights:", weight_walk.numpy(), "sum:", weight_walk.sum().item())
```

```
raw scores: [ 1.0999999 -0.6        0.15     ]
scaled scores: [ 0.54999995 -0.3         0.075     ]
weights: [0.4879715  0.20856632 0.3034622 ] sum: 1.0
```

Now let raw decoder features $\mathbf{D} \in \mathbb{R}^{B \times m \times d_d}$ and encoder memory $\mathbf{H} \in \mathbb{R}^{B \times n \times d_e}$ enter learned projections:

$$\begin{aligned}\mathbf{Q} &= \mathbf{D}\mathbf{W}_Q, \quad \mathbf{W}_Q \in \mathbb{R}^{d_d \times d_k},\\ \mathbf{K} &= \mathbf{H}\mathbf{W}_K, \quad \mathbf{W}_K \in \mathbb{R}^{d_e \times d_k},\\ \mathbf{V} &= \mathbf{H}\mathbf{W}_V, \quad \mathbf{W}_V \in \mathbb{R}^{d_e \times d_v}.\end{aligned}$$

The resulting tensors have shapes

$$\mathbf{Q} \in \mathbb{R}^{B \times m \times d_k}, \qquad \mathbf{K} \in \mathbb{R}^{B \times n \times d_k}, \qquad \mathbf{V} \in \mathbb{R}^{B \times n \times d_v}.$$

Only the projected query and key widths must agree. The raw encoder and decoder widths need not. Let $\mathbf{M} \in \mathbb{R}^{B \times 1 \times n}$ broadcast across queries. **Scaled dot-product attention** is

$$\operatorname{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \operatorname{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}, \tag{13.3}$$

where $\mathbf{K}^\top$ transposes the last two axes within each batch. Softmax runs over the last, key-position axis. The hot path is two dense products: $\mathbf{Q}\mathbf{K}^\top$ and $\mathbf{A}\mathbf{V}$. Additive attention remains useful and accelerator-compatible; scaled dot product simply maps more directly to the matrix operations modern numerical libraries optimize heavily.

Luong and colleagues compared unscaled dot, general, and concat scores in recurrent translation. Vaswani and colleagues later introduced the $1/\sqrt{d_k}$ scaling in the Transformer formulation. The mechanisms belong in one conceptual family, not a single inevitable historical derivation.

The dot product is not magically semantic. $\mathbf{W}_Q$ and $\mathbf{W}_K$ learn which features to compare, including their magnitudes, while the surrounding RNN has already built nonlinear sequence features. In Chapter 14, a feed-forward network will help provide that nonlinear feature transformation after recurrence disappears.

13.3.1. Why divide by $\sqrt{d_k}$?

Suppose, for the moment, that projected coordinates q_ℓ and k_ℓ are independent, mean zero, and variance one, and that their products are independent across coordinates. Then

$$\mathbb{E}[\mathbf{q}^\top \mathbf{k}] = 0, \qquad \mathrm{Var}(\mathbf{q}^\top \mathbf{k}) = \sum_{\ell=1}^{d_k} \mathrm{Var}(q_\ell k_\ell) = d_k.$$

The score's standard deviation grows as $\sqrt{d_k}$. Dividing by that quantity returns the idealized variance to one. This is a variance-control rationale, not a promise that trained projections remain independent or unit variance.

Let us test both the variance and what softmax sees. For each width, draw 20,000 queries and 16 independent keys per query:

```
scaling_gen = torch.Generator().manual_seed(6050132)
scaling_rows = []
for width in (8, 32, 128, 512):
    raw_parts, raw_max_parts, scaled_max_parts = [], [], []
    for _ in range(20_000 // 500):
        q = torch.randn(500, 1, width, generator=scaling_gen)
        k = torch.randn(500, 16, width, generator=scaling_gen)
        raw = (q * k).sum(-1)
        raw_parts.append(raw.reshape(-1))
        raw_max_parts.append(torch.softmax(raw, dim=-1).max(-1).values)
        scaled_max_parts.append(
            torch.softmax(raw / math.sqrt(width), dim=-1).max(-1).values
        )
    raw_scores = torch.cat(raw_parts)
    scaling_rows.append((
        width,
        raw_scores.var(unbiased=True).item(),
        (raw_scores / math.sqrt(width)).var(unbiased=True).item(),
        torch.cat(raw_max_parts).mean().item(),
        torch.cat(scaled_max_parts).mean().item(),
    ))

print("d_k   raw var   scaled var   raw max-w   scaled max-w")
for row in scaling_rows:
    print(f"{row[0]:3d}   {row[1]:8.3f}   {row[2]:10.3f}   "
          f"{row[3]:9.3f}   {row[4]:12.3f}")

scaling_array = np.asarray(scaling_rows)
fig, axes = plt.subplots(1, 2, figsize=(7.3, 3.0), constrained_layout=True)
axes[0].plot(scaling_array[:, 0], scaling_array[:, 1], "o-",
             color="#E57200", label="raw")
```

```
axes[0].plot(scaling_array[:, 0], scaling_array[:, 2], "s-",
             color="#232D4B", label=r"divided by $\sqrt{d_k}$")
axes[0].set(xscale="log", yscale="log", xlabel=r"projected width $d_k$",
            ylabel="empirical score variance", title="Control the score scale")
axes[0].legend(frameon=False, fontsize=8)
axes[1].plot(scaling_array[:, 0], scaling_array[:, 3], "o-",
             color="#E57200", label="raw")
axes[1].plot(scaling_array[:, 0], scaling_array[:, 4], "s-",
             color="#232D4B", label="scaled")
axes[1].set(xscale="log", ylim=(0, 1), xlabel=r"projected width $d_k$",
            ylabel="mean largest weight", title="Control softmax concentration")
axes[1].legend(frameon=False, fontsize=8)
plt.show()
```

```
d_k   raw var   scaled var   raw max-w   scaled max-w
  8     8.023        1.003       0.564          0.241
 32    32.142        1.004       0.779          0.246
128   127.985        1.000       0.889          0.246
512   510.347        0.997       0.944          0.247
```

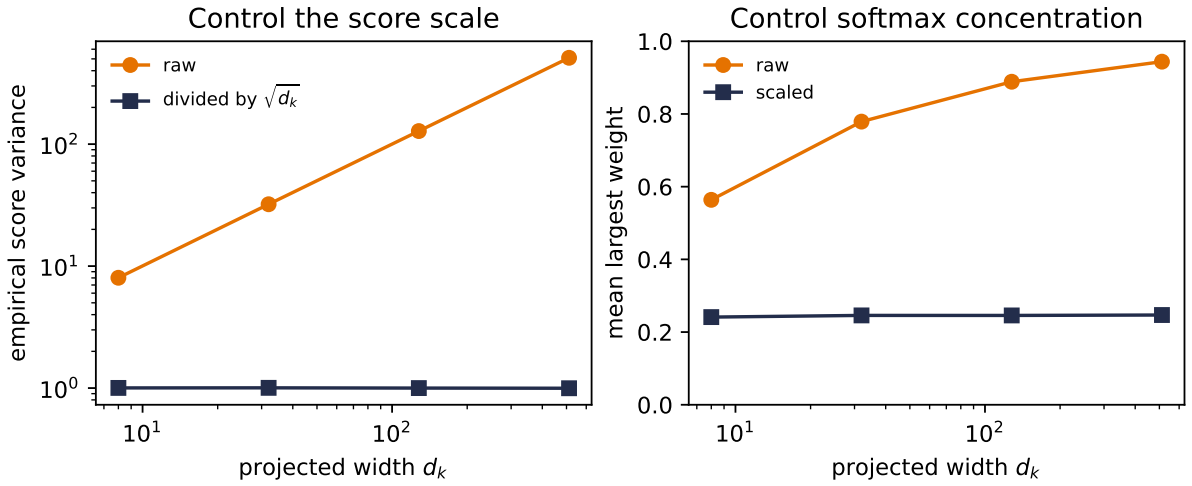

Figure 13.2.: The scaling argument under its stated random-feature assumptions. Left: raw dot-product variance grows with width, while division by the square root of the width keeps it near one. Right: without scaling, a 16-key softmax becomes nearly one-hot as width grows; scaling keeps its mean largest weight near 0.25. These are seeded synthetic draws about concentration, not a claim about every trained layer's gradients.

Raw variance rises from 8.02 to 510.35 as d_k grows from 8 to 512. Scaled variance stays between 0.997 and 1.004. Over the same range, the average largest unscaled softmax weight climbs from 0.564 to 0.944; the scaled value stays near 0.24. This is Chapter 2's **temperature dial** returning

with a dimension-dependent setting. Scaling corrects the dimension-induced spread; it does not normalize the vectors.

13.4. Mask before softmax, and mask the right thing

Chapter 11 showed that padding can poison an encoder's final state. Attention creates a third place to enforce sequence lengths: the score matrix. A padded key must not receive a merely mediocre score. It must be excluded before normalization. Setting its score to zero is wrong because $\exp(0) = 1$ still earns weight.

Here is an independently written scaled-dot operator. It accepts already projected queries, keys, and values, then audits a batch whose second example has two padded keys:

```
def scaled_dot_attention(
    queries: torch.Tensor,
    keys: torch.Tensor,
    values: torch.Tensor,
    key_is_valid: torch.Tensor,
) -> tuple[torch.Tensor, torch.Tensor]:
    """Attend with Q:(B,m,d_k), K:(B,n,d_k), V:(B,n,d_v)."""
    if queries.ndim != 3 or keys.ndim != 3 or values.ndim != 3:
        raise ValueError("Q, K, and V must be three-dimensional")
    if queries.shape[-1] != keys.shape[-1]:
        raise ValueError("projected Q and K widths must match")
    if keys.shape[:2] != values.shape[:2]:
        raise ValueError("K and V must have the same batch and key axes")
    if key_is_valid.shape != keys.shape[:2] or not key_is_valid.any(dim=1).all():
        raise ValueError("every example needs at least one valid key")

    scores = queries @ keys.transpose(-2, -1) / math.sqrt(keys.shape[-1])
    scores = scores.masked_fill(~key_is_valid[:, None, :], -torch.inf)
    weights = torch.softmax(scores, dim=-1)
    return weights @ values, weights

mask_gen = torch.Generator().manual_seed(6050133)
Q_demo = torch.randn(2, 2, 4, generator=mask_gen)
K_demo = torch.randn(2, 4, 4, generator=mask_gen)
V_demo = torch.randn(2, 4, 3, generator=mask_gen)
valid_demo = torch.tensor([[True, True, True, True],
                           [True, True, False, False]])
context_demo, mask_weights = scaled_dot_attention(
    Q_demo, K_demo, V_demo, valid_demo
)
assert torch.allclose(
    mask_weights.sum(-1), torch.ones_like(mask_weights[..., 0])
)
```

```
assert torch.count_nonzero(mask_weights[1, :, 2:]) == 0
print("Q", tuple(Q_demo.shape), "K", tuple(K_demo.shape),
      "V", tuple(V_demo.shape), "A", tuple(mask_weights.shape),
      "AV", tuple(context_demo.shape))
print("maximum row-sum error:",
      f"{(mask_weights.sum(-1) - 1).abs().max().item():.2e}")
print("maximum padded weight:",
      f"{mask_weights[1, :, 2:].max().item():.1f}")

fig, ax = plt.subplots(figsize=(5.8, 2.5), constrained_layout=True)
shown = mask_weights[1].detach().numpy()
image = ax.imshow(shown, vmin=0, vmax=shown.max(), cmap="magma", aspect="auto")
for row in range(shown.shape[0]):
    for col in range(shown.shape[1]):
        label = "PAD" if col >= 2 else f"{shown[row, col]:.2f}"
        ax.text(col, row, label, ha="center", va="center",
                color="white" if shown[row, col] < 0.65 * shown.max() else "black")
ax.set(xticks=range(4), xticklabels=["key 1", "key 2", "pad", "pad"],
       yticks=range(2), yticklabels=["query 1", "query 2"],
       xlabel="source/key position", title="Mask source padding before softmax")
fig.colorbar(image, ax=ax, label="attention weight", fraction=0.05, pad=0.04)
plt.show()
```

```
Q (2, 2, 4) K (2, 4, 4) V (2, 4, 3) A (2, 2, 4) AV (2, 2, 3)
maximum row-sum error: 0.00e+00
maximum padded weight: 0.0
```

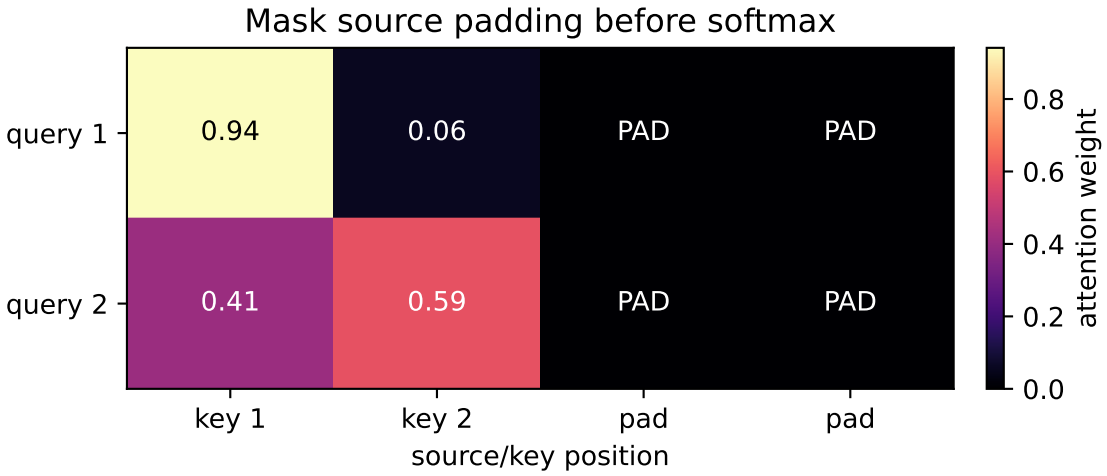

Figure 13.3.: A source-padding mask acts before softmax. The second example has only two real keys; its last two score cells are excluded and receive exactly zero weight. Cross-attention may read every real source position, so no causal triangle belongs here.

> ⚠ Three masks are not one mask
>
> The target loss mask says which output positions should not be graded; it remains good general hygiene but is dormant for this fixed-length ISO target. The cross-attention padding mask says which source positions do not exist. A causal mask says which future target positions may not be seen. Our recurrent decoder generates one step at a time and the entire source is already available, so the source-padding attention mask is the one exercised here. Chapter 14 will use causal masking inside decoder self-attention.
> An all-masked row has no sensible distribution and would make softmax of all $-\infty$ undefined. Good hygiene is to assert that every example has at least one real key, as the function does above.

13.5. Return to Chapter 11's date task

The abstraction has earned a real rematch. We will change one architectural idea and keep the experimental contract fixed:

- the same date generator and duplicate-source rejection;
- the same 8,000/500/500 train/validation/test split;
- the same embedding width 32 and LSTM hidden width 128;
- the same Adam learning rate 10^{-3}, batch size 128, 25 epochs, teacher forcing, gradient clip 5, and seed 6050;
- the same first 400 unambiguous validation sources for every learning curve;
- one final scalar audit on the same 437 unambiguous test sources.

The generator and split below are deliberately byte-for-byte copies of Chapter 11. Changing them would end the running benchmark:

```
import random, torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

MONTHS = ["january", "february", "march", "april", "may", "june", "july",
          "august", "september", "october", "november", "december"]
DAYS = dict(zip(MONTHS, [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]))

def make_date(rng: random.Random) -> tuple[str, str]:
    mo = rng.randrange(12)
    d = rng.randrange(1, DAYS[MONTHS[mo]] + 1)
    y = rng.randrange(1950, 2026)
    iso = f"{y:04d}-{mo+1:02d}-{d:02d}"
    style = rng.random()
    if style < 0.35:
        src = f"{MONTHS[mo]} {d}, {y}"
```

```
    elif style < 0.6:
        src = f"{d} {MONTHS[mo]} {y}"
    elif rng.random() < 0.7:
        src = f"{mo+1:02d}/{d:02d}/{y}"            # US convention: mm/dd
    else:
        src = f"{d:02d}/{mo+1:02d}/{y}"            # EU convention: dd/mm
    return src, iso

rng = random.Random(6050)
pairs, seen_sources = [], set()
while len(pairs) < 9000:
    pair = make_date(rng)
    if pair[0] not in seen_sources:
        seen_sources.add(pair[0])
        pairs.append(pair)
train_pairs = pairs[:8000]
valid_pairs = pairs[8000:8500]
test_pairs = pairs[8500:]
print(*pairs[:4], sep="\n")
train_sources = {s for s, _ in train_pairs}
valid_sources = {s for s, _ in valid_pairs}
test_sources = {s for s, _ in test_pairs}
print("exact-source overlaps: "
      f"train/valid {len(train_sources & valid_sources)}, "
      f"train/test {len(train_sources & test_sources)}, "
      f"valid/test {len(valid_sources & test_sources)}")

PAD, BOS, EOS = 0, 1, 2                            # special tokens first
SRC = sorted(set("".join(s for s, _ in pairs)))
TGT = sorted(set("".join(t for _, t in pairs)))
src_stoi = {c: i + 3 for i, c in enumerate(SRC)}
tgt_stoi = {c: i + 3 for i, c in enumerate(TGT)}
tgt_itos = {i: c for c, i in tgt_stoi.items()}
V_src, V_tgt = len(SRC) + 3, len(TGT) + 3
print(f"source vocab {V_src}, target vocab {V_tgt}")
```

```
('26 september 2024', '2024-09-26')
('april 7, 1962', '1962-04-07')
('12/15/1950', '1950-12-15')
('14 february 1997', '1997-02-14')
exact-source overlaps: train/valid 0, train/test 0, valid/test 0
source vocab 37, target vocab 14
```

Numeric dates can be ambiguous, so we retain the same unambiguous subsets as Chapter 11. Design decisions and the heatmap use validation only. The test set remains a single number at the end.

13.5.1. Put the learned read inside the recurrent decoder

The packed encoder now returns two things: its final $(\mathbf{h}, \mathbf{c})$ state, which still initializes the decoder, and a padded memory tensor containing every real encoder output plus a validity mask that excludes the padded slots. At step t:

1. use decoder hidden state $\mathbf{s}_{t-1}$ as the query;
2. compute additive scores over the encoder memory and mask padding;
3. form the context $\mathbf{c}_t$;
4. concatenate $\mathbf{c}_t$ with the embedding of the previous target token;
5. advance one decoder LSTM step, then predict from $[\mathbf{s}_t; \mathbf{c}_t]$.

This timing follows the Bahdanau-style formulation. Luong-style variants often score with the current decoder state instead; both are valid, but silently mixing their indices makes an implementation impossible to audit.

For this date model, the shape ledger is concrete:

quantity	shape
previous hidden query	$(B, 128)$
encoder memory	$(B, T_{\text{src}}, 128)$
attention weights	(B, T_{src})
context	$(B, 128)$
token embedding joined with context	$(B, 1, 160)$
decoder output	$(B, 1, 128)$
output-head input $[\mathbf{s}_t; \mathbf{c}_t]$	$(B, 256)$

```
def batchify(prs: list[tuple[str, str]], idx: list[int]) -> tuple[
    torch.Tensor, torch.Tensor, torch.Tensor
]:
    srcs = [torch.tensor([src_stoi[c] for c in prs[i][0]]) for i in idx]
    lengths = torch.tensor([len(s) for s in srcs])
    tgts = [torch.tensor([BOS] + [tgt_stoi[c] for c in prs[i][1]] + [EOS])
            for i in idx]
    S = nn.utils.rnn.pad_sequence(srcs, batch_first=True, padding_value=PAD)
    T = nn.utils.rnn.pad_sequence(tgts, batch_first=True, padding_value=PAD)
    return S, lengths, T

def unambiguous(prs: list[tuple[str, str]]) -> list[tuple[str, str]]:
    return [(s, t) for s, t in prs
            if "/" not in s or int(s[:2]) > 12 or int(s[3:5]) > 12]

valid_unamb = unambiguous(valid_pairs)
test_unamb = unambiguous(test_pairs)
valid_fixed = valid_unamb[:400]
print(f"fixed validation {len(valid_fixed)}; final test {len(test_unamb)}")
```

```
class LearnedAlignment(nn.Module):
    def __init__(self, hidden: int = 128, alignment: int = 128):
        super().__init__()
        self.key_map = nn.Linear(hidden, alignment, bias=False)
        self.query_map = nn.Linear(hidden, alignment, bias=False)
        self.selector = nn.Linear(alignment, 1, bias=False)

    def forward(
        self,
        memory: torch.Tensor,
        query: torch.Tensor,
        valid: torch.Tensor,
    ) -> tuple[torch.Tensor, torch.Tensor]:
        match_features = torch.tanh(
            self.key_map(memory) + self.query_map(query).unsqueeze(1)
        )                                                    # (B, T_src, d_a)
        scores = self.selector(match_features).squeeze(-1)  # (B, T_src)
        scores = scores.masked_fill(~valid, -torch.inf)
        weights = torch.softmax(scores, dim=-1)
        context = torch.bmm(weights.unsqueeze(1), memory).squeeze(1)
        return context, weights

class AttentiveSeq2Seq(nn.Module):
    def __init__(self, v_src: int, v_tgt: int, embed: int = 32,
                 hidden: int = 128, alignment: int = 128):
        super().__init__()
        self.src_emb = nn.Embedding(v_src, embed, padding_idx=PAD)
        self.tgt_emb = nn.Embedding(v_tgt, embed, padding_idx=PAD)
        self.encoder = nn.LSTM(embed, hidden, batch_first=True)
        self.attention = LearnedAlignment(hidden, alignment)
        self.decoder = nn.LSTM(embed + hidden, hidden, batch_first=True)
        self.out = nn.Linear(2 * hidden, v_tgt)

    def encode(
        self, S: torch.Tensor, lengths: torch.Tensor,
    ) -> tuple[torch.Tensor, tuple[torch.Tensor, torch.Tensor], torch.Tensor]:
        packed = nn.utils.rnn.pack_padded_sequence(
            self.src_emb(S), lengths, batch_first=True, enforce_sorted=False)
        packed_memory, state = self.encoder(packed)
        memory, _ = nn.utils.rnn.pad_packed_sequence(
            packed_memory, batch_first=True, total_length=S.shape[1])
        valid = torch.arange(S.shape[1])[None, :] < lengths[:, None]
        return memory, state, valid

    def decode_step(
```

```
        self,
        token: torch.Tensor,
        state: tuple[torch.Tensor, torch.Tensor],
        memory: torch.Tensor,
        valid: torch.Tensor,
    ) -> tuple[torch.Tensor, tuple[torch.Tensor, torch.Tensor], torch.Tensor]:
        query = state[0][-1]                              # previous hidden state
        context, weights = self.attention(memory, query, valid)
        decoder_input = torch.cat(
            (self.tgt_emb(token), context.unsqueeze(1)), dim=-1
        )
        decoder_output, state = self.decoder(decoder_input, state)
        logits = self.out(torch.cat((decoder_output[:, 0], context), dim=-1))
        return logits, state, weights

    def forward(
        self, S: torch.Tensor, lengths: torch.Tensor, T_in: torch.Tensor,
    ) -> torch.Tensor:
        memory, state, valid = self.encode(S, lengths)
        logits = []
        for t in range(T_in.shape[1]):
            logit, state, _ = self.decode_step(
                T_in[:, t:t + 1], state, memory, valid)
            logits.append(logit)
        return torch.stack(logits, dim=1)                 # (B, T_tgt, V_tgt)

@torch.inference_mode()
def greedy_batch(
    model: AttentiveSeq2Seq,
    prs: list[tuple[str, str]],
    batch: int = 128,
    maxlen: int = 12,
) -> list[tuple[str, str, torch.Tensor]]:
    model.eval()
    outputs = []
    for start in range(0, len(prs), batch):
        subset = prs[start:start + batch]
        S, lengths, _ = batchify(subset, list(range(len(subset))))
        memory, state, valid = model.encode(S, lengths)
        token = torch.full((len(subset), 1), BOS)
        token_steps, weight_steps = [], []
        for _ in range(maxlen):
            logits, state, weights = model.decode_step(
                token, state, memory, valid)
            token = logits.argmax(-1, keepdim=True)
            token_steps.append(token[:, 0])
```

```
            weight_steps.append(weights)
        predicted = torch.stack(token_steps, dim=1)
        attention = torch.stack(weight_steps, dim=1)
        for row, (source, _) in enumerate(subset):
            chars = []
            for index in predicted[row].tolist():
                if index == EOS:
                    break
                chars.append(tgt_itos.get(index, "?"))
            outputs.append((
                source,
                "".join(chars),
                attention[row, :, :lengths[row]].clone(),
            ))
    return outputs

@torch.inference_mode()
def exact_match(
    model: AttentiveSeq2Seq, prs: list[tuple[str, str]],
) -> float:
    generated = greedy_batch(model, prs)
    return sum(
        guess == truth
        for (_, truth), (_, guess, _) in zip(prs, generated)
    ) / len(prs)
```

```
fixed validation 400; final test 437
```

Notice what changed relative to Chapter 11. The embeddings and encoder width are untouched. Values are the contextual encoder states themselves; learned projections create match features for keys and queries. The decoder must now advance one target step at a time because its next context depends on its previous state. Teacher forcing still supplies the true previous token, but it no longer permits one fused decoder call.

13.5.2. The matched-schedule rematch

The full Chapter 11 validation curve below is frozen from a byte-identical re-execution of that chapter's published baseline. We do not retrain it here. We train the attentive model once, inspect only validation alignments, and then request one final test scalar:

```
checkpoints = (2, 4, 6, 8, 12, 16, 20, 25)
baseline_curve = {
    2: 0.0025, 4: 0.1475, 6: 0.5375, 8: 0.7575,
    12: 0.9500, 16: 0.9500, 20: 0.9950, 25: 0.9575,
```

```
}

torch.manual_seed(6050)
attention_model = AttentiveSeq2Seq(V_src, V_tgt)
optimizer = torch.optim.Adam(attention_model.parameters(), lr=1e-3)
attention_curve = {}

for epoch in range(1, 26):
    attention_model.train()
    permutation = torch.randperm(len(train_pairs))
    for start in range(0, len(train_pairs), 128):
        ids = permutation[start:start + 128].tolist()
        S, lengths, T = batchify(train_pairs, ids)
        logits = attention_model(S, lengths, T[:, :-1])
        loss = F.cross_entropy(
            logits.reshape(-1, V_tgt),
            T[:, 1:].reshape(-1),
            ignore_index=PAD,
        )
        optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(attention_model.parameters(), 5.0)
        optimizer.step()
    if epoch in checkpoints:
        attention_curve[epoch] = exact_match(
            attention_model, valid_fixed)

print("epoch   fixed-state   attention")
for epoch in checkpoints:
    print(f"{epoch:2d}       {baseline_curve[epoch]:6.2%}       "
          f"{attention_curve[epoch]:6.2%}")

validation_outputs = greedy_batch(attention_model, valid_fixed)
source_example, truth_example = valid_fixed[0]           # fixed before training
_, generated_example, alignment_example = validation_outputs[0]

year_mass, year_top1, row_errors = [], [], []
for (source, _), (_, _, weights) in zip(valid_fixed, validation_outputs):
    year_positions = list(range(len(source) - 4, len(source)))
    for step in range(4):
        year_mass.append(weights[step, year_positions].sum().item())
        year_top1.append(int(weights[step].argmax().item() in year_positions))
    row_errors.append((weights[:10].sum(1) - 1).abs().max().item())

attention_params = sum(p.numel() for p in attention_model.parameters())
baseline_params = 169_326
```

```
print(f"parameters: baseline {baseline_params:,}; "
      f"attention {attention_params:,} "
      f"(+{(attention_params / baseline_params - 1):.1%})")
print(f"validation example: {source_example!r} -> "
      f"{generated_example!r} (truth {truth_example})")
print(f"validation year-region mass: {np.mean(year_mass):.3%}; "
      f"top key in region: {np.mean(year_top1):.3%}")
print("maximum validation row-sum error:", f"{max(row_errors):.2e}")

test_exact = exact_match(attention_model, test_unamb)     # one final test audit
print(f"one final test audit ({len(test_unamb)} sources): "
      f"baseline 93.1%; attention {test_exact:.1%}")
```

```
epoch   fixed-state   attention
 2        0.25%        9.50%
 4       14.75%       74.50%
 6       53.75%       93.25%
 8       75.75%       99.00%
12       95.00%       99.75%
16       95.00%       99.75%
20       99.50%       99.75%
25       95.75%       99.75%
parameters: baseline 169,326; attention 269,550 (+59.2%)
validation example: 'may 17, 1971' -> '1971-05-17' (truth 1971-05-17)
validation year-region mass: 97.469%; top key in region: 99.688%
maximum validation row-sum error: 2.38e-07
one final test audit (437 sources): baseline 93.1%; attention 100.0%
```

```
from matplotlib.patches import Rectangle

fig, axes = plt.subplots(
    1, 2, figsize=(9.8, 3.45),
    gridspec_kw={"width_ratios": [1.0, 1.35]},
    constrained_layout=True,
)
axes[0].plot(
    checkpoints, [baseline_curve[c] for c in checkpoints],
    "o-", color="#8D99AE", lw=2, label="fixed-state baseline",
)
axes[0].plot(
    checkpoints, [attention_curve[c] for c in checkpoints],
    "s-", color="#E57200", lw=2, label="learned attention",
)
axes[0].set(
    xlabel="epoch", ylabel="validation exact match",
```

```
    ylim=(-0.02, 1.03), title="Same schedule, learned access",
)
axes[0].legend(frameon=False, fontsize=8)

heatmap = alignment_example[:len(generated_example)].numpy()
image = axes[1].imshow(
    heatmap, cmap="Blues", aspect="auto", vmin=0, vmax=1,
)
source_labels = ["·" if char == " " else char for char in source_example]
axes[1].set_xticks(range(len(source_example)), source_labels)
axes[1].set_yticks(range(len(generated_example)), list(generated_example))
axes[1].set(
    xlabel="source character / encoder state",
    ylabel="emitted character",
    title=f"{source_example}  ->  {generated_example}",
)
axes[1].add_patch(Rectangle(
    (len(source_example) - 4.5, -0.5), 4, 4,
    fill=False, edgecolor="#E57200", linewidth=2,
))
fig.colorbar(
    image, ax=axes[1], fraction=0.047, pad=0.03,
    label="attention weight",
)
plt.show()
```

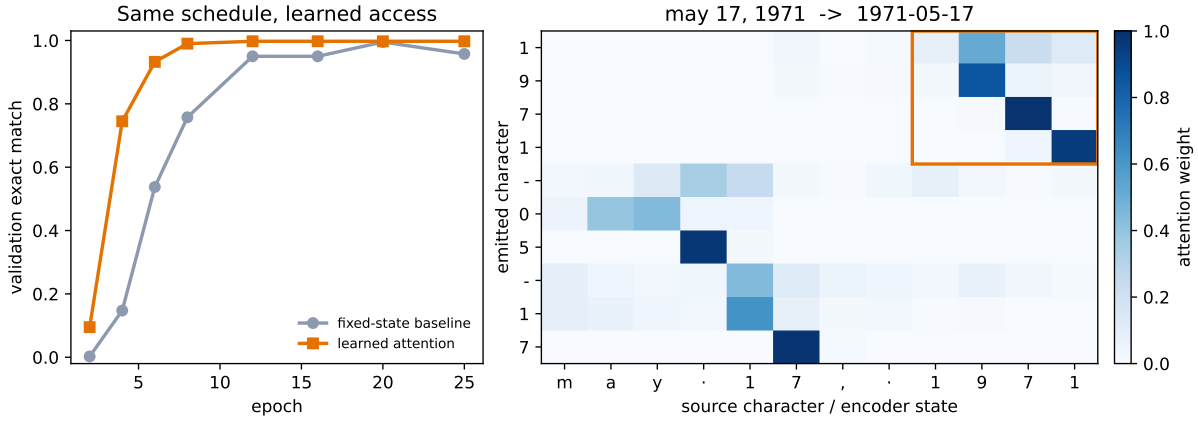

Figure 13.4.: Left: exact-sequence accuracy on Chapter 11's fixed 400-source validation subset. Both models use the same data, embedding and recurrent hidden widths, optimizer, top-level seed, and epoch schedule; attention adds parameters and a stepwise decoder, so this is not a parameter- or compute-matched ablation. Right: the predeclared first validation example. Rows are emitted ISO characters, columns are contextual encoder states indexed by source character, and the orange box marks the source year. The first four rows concentrate there.

At epoch 6, the fixed-state baseline is at 53.8% and the attentive model is at 93.25%, a 39.5-point lead. At epoch 12 the comparison is 95.0% versus 99.75%. The one final test audit is 93.1% versus 100.0% on 437 unambiguous sources.

> ⚠ Faster convergence is not a one-variable proof
>
> We matched examples, embedding and recurrent hidden widths, optimizer, top-level seed, number of updates, and epochs. We did **not** match parameter count, computation, or minibatch order: constructing models with different parameter counts consumes different random draws before their first permutation. The attentive model has 269,550 parameters versus 169,326, an increase of 59.2%, and its input-feeding decoder uses a Python step loop where Chapter 11's teacher-forced baseline uses one fused target-sequence call. This one seeded synthetic rematch shows that the attention-equipped model reaches higher accuracy in fewer epochs and optimizer updates, not in less wall-clock time. It does not isolate bottleneck removal as the only possible cause.

The heatmap answers a narrower question. On the fixed validation example, the year must travel from the last four source positions to the first four outputs. The orange block catches that movement, and across all 400 fixed validation examples, 97.5% of those four rows' mass lands on states indexed by the source-year region. The top-weighted state is indexed inside that region 99.7% of the time.

Columns are indexed by source characters, but their values are encoder states. Each state summarizes a recurrent prefix, so a delimiter can legitimately carry information about a field that just ended. Attention weights are internal allocation weights, not calibrated confidence and not a causal explanation of the model. The quantitative audit and the visual alignment complement each other; neither substitutes for the other.

13.6. One learned view, for now

One attention head learns one set of query, key, and value projections. Multi-head attention repeats that operator with several projection sets, concatenates the reads, and learns an output projection. It works for cross-attention as well as self-attention, and different heads *can* learn different relationships without any guarantee that they will. We keep one head in the date rematch so its alignment stays easy to inspect. Chapter 14 derives the multi-head shapes and implements the full operator in its self-attention setting.

13.7. The bottleneck that remains

Cross-attention has repaired the fixed handoff. A decoder output can read any encoder state through one learned allocation. Yet both surrounding LSTMs remain recurrent: $\mathbf{h}_i$ waits for $\mathbf{h}_{i-1}$, and $\mathbf{s}_t$ waits for $\mathbf{s}_{t-1}$. Within either sequence, training cannot create all recurrent states at once.

The Q/K/V operator itself does not care where its three inputs came from. What if every position in one sequence supplied queries, keys, and values to every other position? Using that operator in place of the RNN can remove recurrence within each layer, but it also removes the RNN's built-in sense of order. Chapter 8 warned that throwing away *where* incurs a debt; Chapter 14 pays it back with positional information. Chapter 11's masking warning also returns there: a causal mask will decide which future positions an autoregressive model may not see.

13.8. Okay, so —

1. **Attention softens the address.** Chapter 11 learned embedding content while keeping lookup indices hard. Chapter 2's differentiable-lookup promise is now paid: learned scores become a distribution over memory locations.
2. **Additive attention learns the compatibility function.** Query and key enter a shared alignment space, tanh creates match features, and one selector produces each score. The same scorer is reused across all decoder/source pairs.
3. **Scaled dot product learns comparison spaces.** Projected $\mathbf{QK}^\top/\sqrt{d_k}$ maps directly to dense matrix products. Under the idealized independent unit-variance model, scaling holds score variance near one instead of letting it grow with d_k.
4. **Masking is part of the operator.** Padding is excluded before softmax; zero is not a mask. Cross-attention sees all real source positions, while causal masking belongs to the self-attentive decoder in Chapter 14.
5. **The date rematch exposes learned access.** Under the shared schedule, attention reaches 93.25% validation exact match at epoch 6 and 100.0% on the final test audit. Its year rows route 97.5% of validation weight to states indexed by the source-year region, with the parameter/compute caveat stated in full.

Sources and further reading

- Bahdanau, Cho, & Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate*: conjectures that the fixed-length encoder vector is a bottleneck, then tests differentiable soft search with an additive scorer.
- Luong, Pham, & Manning, *Effective Approaches to Attention-based Neural Machine Translation*: compares dot, general, and concat scoring functions and clarifies a second recurrent-attention timing convention.
- Vaswani et al., *Attention Is All You Need*: introduces scaled dot-product and multi-head attention in the architecture Chapter 14 builds.

Exercises

1. **(Pencil.)** Starting from Equation 13.2, write every batched shape for $B = 32$, source length 17, $d_e = 128$, $d_d = 96$, and $d_a = 64$. Which axis must softmax normalize, and why can the raw encoder and decoder widths differ?

2. **(Code.)** Extend the scaled-dot function with a deliberately all-false mask row. Confirm the guard catches it. Remove the guard and inspect the result of softmax over all $-\infty$. Then replace the mask with zero scores and explain why padding receives weight.
3. **(Pencil + code.)** Generalize the scaling derivation to independent coordinates with variances σ_q^2 and σ_k^2. Modify the seeded audit and check which divisor returns score variance near one.
4. **(Code.)** Replace the date model's additive scorer with learned projected scaled-dot attention. Keep the split, schedule, and validation subset fixed. Select any design choices on validation, report the sealed test once, and compare both convergence and year-region alignment. Explain why a large weight is neither calibrated confidence nor a causal explanation.
5. **(Pencil.)** The heatmap labels columns by source characters even though each value is a recurrent prefix state. Give two reasons a delimiter state might carry useful month or day information. Then state one conclusion the heatmap supports and one conclusion it cannot support.

14. Self-Attention and the Transformer

Chapter 13 ended with a question that the attention operator itself had already answered. A query, key, and value need not come from an encoder or a decoder. They need not come from a recurrent network at all. If all three come from the same sequence, each token can ask every token—including itself—what matters and route the answers directly. The recurrent state—the last serial bottleneck left standing—becomes optional.

That substitution is the core of a *Transformer*. It is also easy to state too quickly. Removing recurrence creates a position problem. Splitting one attention operation into heads creates a bookkeeping problem. Stacking the resulting operations creates an optimization problem. This chapter solves each one—from the equations outward—then returns to the book-corpus benchmark from Chapter 10 with a deliberately small causal Transformer.

Here is the result we have to explain on the same 14,860-character held-out tail. In one exactly matched seed, positional encoding improves the Transformer's loss from 2.3405 to 1.9190. Yet Chapter 10's compact LSTM remains better at 1.8881. The new architecture has learned, and position matters in this run—but at this scale, the LSTM still wins.

14.1. Let the sequence query itself

Suppose the sentence contains the words *bank*, *river*, and *boats*. A useful representation of *bank* should route information from *river* and *boats*, while another occurrence—now near *loan* and *interest*—should route different information. Chapter 13 built exactly this kind of content-dependent routing. Self-attention changes only the source of the three inputs.

The lecture's three questions still fit: a query asks "what am I looking for?", a key advertises "what do I offer?", and a value carries "what information do I contain?"

Let a batch of token representations be

$$X \in \mathbb{R}^{B\times n\times d},$$

where B is batch size, n is sequence length, and d is model width. Learned projections produce

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

with $W_Q, W_K, W_V \in \mathbb{R}^{d\times d}$. Scaled dot-product attention is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V.$$

For one sequence, $QK^\top$ has shape $n \times n$. Row i contains the scores from query position i to every key position j. The row-wise softmax makes each row a distribution, and multiplying by V returns one weighted sum for each query:

$$(n, d)(d, n) \to (n, n), \qquad (n, n)(n, d) \to (n, d).$$

Chapter 10 named time as the book's third sharing axis. An RNN shares one transition rule across time. Self-attention keeps that stationarity and extends the shared rule across every ordered pair of positions. The weights in a particular attention row still depend on the content; what is shared is the machinery that produces them.

All query positions can be evaluated together during training because none waits for a hidden state from the preceding position. That is a structural statement—not a timing result. Dense self-attention still forms n^2 pairwise scores, and autoregressive generation still emits one new token at a time.

14.1.1. The position debt

Bare self-attention knows content but not slot number. A precise proof is more useful than the slogan.

Let P be an $n \times n$ permutation matrix, so PX reorders the tokens. Then

$$Q' = PQ, \quad K' = PK, \quad V' = PV$$

and therefore

$$Q'K'^\top = P(QK^\top)P^\top.$$

Write $S = QK^\top/\sqrt{d}$; then $S' = PSP^\top$. Row-wise softmax respects the same simultaneous row-and-column permutation:

$$\text{softmax}(PSP^\top) = P\,\text{softmax}(S)P^\top.$$

Putting the pieces together gives

$$\text{SA}(PX) = P\,\text{SA}(X).$$

Self-attention is *permutation equivariant*: permute the inputs and the outputs permute in the same way. It is not permutation invariant—the token outputs do not all become identical. But

without another signal, the operator has no basis for distinguishing "first" from "fifth." A pooled sequence summary can consequently become invariant to order.

Chapter 8 planted this debt when convolution received locality "for free." A convolution knows which values are neighbors because its kernel is tied to nearby offsets. Global attention abandons that built-in geometry—so we must pay to put position back in.

```
import math
import numpy as np
import matplotlib.pyplot as plt

def numpy_attention(x: np.ndarray) -> np.ndarray:
    scores = x @ x.T / math.sqrt(x.shape[-1])
    scores = scores - scores.max(axis=-1, keepdims=True)
    weights = np.exp(scores)
    weights = weights / weights.sum(axis=-1, keepdims=True)
    return weights @ x

def numpy_positions(length: int, width: int) -> np.ndarray:
    position = np.arange(length)[:, None]
    frequency = np.exp(
        np.arange(0, width, 2) * (-math.log(10_000.0) / width)
    )
    pe = np.zeros((length, width))
    pe[:, 0::2] = np.sin(position * frequency)
    pe[:, 1::2] = np.cos(position * frequency)
    return pe

rng = np.random.default_rng(6050)
x = rng.normal(size=(5, 8))
permutation = np.array([2, 4, 0, 1, 3])
bare_error = np.abs(
    numpy_attention(x[permutation]) - numpy_attention(x)[permutation]
).max()

pe = numpy_positions(5, 8)
with_position = numpy_attention(x + pe)
fixed_slot_change = np.abs(
    numpy_attention(x[permutation] + pe) - with_position[permutation]
).max()

fig, axes = plt.subplots(1, 2)
token_ids = np.arange(5)
tokens = np.array(["bank", "by", "the", "river", "."])
axes[0].imshow(
    np.stack([token_ids, token_ids[permutation]]),
    cmap="tab10", aspect="auto", vmin=0, vmax=9,
```

```
)
for row, order in enumerate([token_ids, token_ids[permutation]]):
    for column, token_id in enumerate(order):
        axes[0].text(column, row, tokens[token_id], ha="center", va="center")
axes[0].set_yticks([0, 1], ["$X$", "$PX$"])
axes[0].set_xticks(range(5), [f"slot {i}" for i in range(5)])
axes[0].set_title("content moves with the permutation")

axes[1].bar(
    ["bare equivariance\nerror", "+ fixed position\ndifference"],
    [bare_error, fixed_slot_change],
    color=["#6b7280", "#d37b29"],
)
axes[1].set_yscale("log")
axes[1].set_ylabel("maximum rematched difference")
axes[1].set_title("position breaks the rematch")
plt.tight_layout()
plt.show()

print(f"bare permutation-equivariance error: {bare_error:.2e}")
print(f"fixed-slot difference after adding position: {fixed_slot_change:.3f}")
```

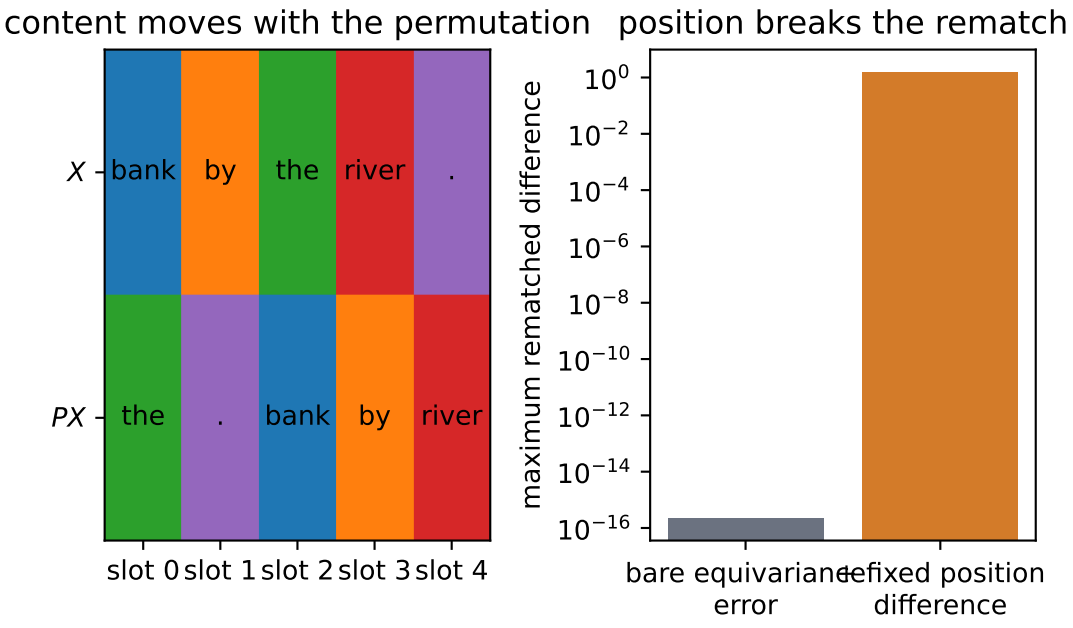

Figure 14.1.: The same content is permuted across slots (left). The audit (right) shows bare self-attention rematches after the same output permutation, while fixed positional signals break that rematch.

```
bare permutation-equivariance error: 2.22e-16
```

```
fixed-slot difference after adding position: 1.581
```

The first error is numerical roundoff, about 2.2×10^{-16}. Adding a fixed signal to each slot changes the rematched outputs by about 1.581 in this example. Position has not been inferred from content; it has been supplied.

14.2. Position as a bank of clocks

The simplest additive code repeats $i/(n-1)$ in every coordinate. It supplies order, but every position vector lies on the same ray and differs only in magnitude; attention receives no multidirectional geometry. A binary code supplies more coordinates, yet adjacent integers can flip many bits and have poor locality. These are not impossible designs—they are weak inductive biases for a model that must reason about shifts and distances.

Sinusoidal encoding gives every position a bank of clocks. For even model width d, define

$$\begin{aligned} \mathrm{PE}(i, 2j) &= \sin\left(i\, 10000^{-2j/d}\right), \\ \mathrm{PE}(i, 2j+1) &= \cos\left(i\, 10000^{-2j/d}\right). \end{aligned}$$

Small j gives a fast clock; large j gives a slow one. Every sine coordinate is paired with a cosine at the same frequency. The model receives

$$Z_i = \sqrt{d}\, E_i + \mathrm{PE}(i),$$

where E_i is the token embedding. Scaling the embedding keeps its initial magnitude commensurate with the position vector.

Why pair sine and cosine? For frequency ω, advancing by offset δ is a rotation—an algebraic answer to a geometric question:

$$\begin{bmatrix} \sin((i+\delta)\omega) \\ \cos((i+\delta)\omega) \end{bmatrix} = \begin{bmatrix} \cos(\delta\omega) & \sin(\delta\omega) \\ -\sin(\delta\omega) & \cos(\delta\omega) \end{bmatrix} \begin{bmatrix} \sin(i\omega) \\ \cos(i\omega) \end{bmatrix}.$$

The rotation depends on the offset, not the absolute position. This gives learned linear maps a convenient raw material for representing relative shifts. It is a property of the positional pair itself—not a guarantee that arbitrary learned W_Q and W_K will preserve or use it.

```python
pe = numpy_positions(100, 32)

delta = 7
j = 5
omega = 10_000 ** (-2 * j / 32)
rotation = np.array([
    [np.cos(delta * omega), np.sin(delta * omega)],
    [-np.sin(delta * omega), np.cos(delta * omega)],
```

```
])
pair_i = pe[13, 2 * j : 2 * j + 2]
pair_shifted = pe[13 + delta, 2 * j : 2 * j + 2]
rotation_error = np.abs(rotation @ pair_i - pair_shifted).max()
norm_error = np.abs(np.linalg.norm(pe, axis=1) - 4.0).max()

fig, axes = plt.subplots(
    2, 2, figsize=(10, 5), gridspec_kw={"height_ratios": [1, 2]}
)
axes[0, 0].plot(pe[:, 0], label="sin")
axes[0, 0].plot(pe[:, 1], label="cos")
axes[0, 0].set_title("fast clock: coordinates 0-1")
axes[0, 0].legend(frameon=False, ncol=2)
axes[0, 1].plot(pe[:, 20], label="sin")
axes[0, 1].plot(pe[:, 21], label="cos")
axes[0, 1].set_title("slow clock: coordinates 20-21")
axes[0, 1].legend(frameon=False, ncol=2)

heat = axes[1, 0].imshow(pe.T, aspect="auto", cmap="coolwarm", vmin=-1, vmax=1)
axes[1, 0].set_xlabel("position")
axes[1, 0].set_ylabel("coordinate")
axes[1, 0].set_title("the complete position code")
fig.colorbar(heat, ax=axes[1, 0], fraction=0.05)

from matplotlib.patches import FancyArrowPatch

angles = np.arange(0, 2 * np.pi, 0.01)
axes[1, 1].plot(np.sin(angles), np.cos(angles), color="0.8")
axes[1, 1].scatter(*pair_i, label="position 13")
axes[1, 1].scatter(*pair_shifted, label=f"position {13 + delta}")
axes[1, 1].plot([0, pair_i[0]], [0, pair_i[1]], color="0.4")
axes[1, 1].plot([0, pair_shifted[0]], [0, pair_shifted[1]], color="0.4")
axes[1, 1].add_patch(
    FancyArrowPatch(
        pair_i,
        pair_shifted,
        connectionstyle="arc3,rad=0.25",
        arrowstyle="->",
        mutation_scale=12,
        color="black",
    )
)
axes[1, 1].axis("equal")
axes[1, 1].set_title(f"offset {delta} is a rotation")
axes[1, 1].legend(frameon=False)
plt.tight_layout()
```

```
plt.show()

print(f"rotation identity maximum error: {rotation_error:.2e}")
print(f"constant position-vector norm maximum error: {norm_error:.2e}")
```

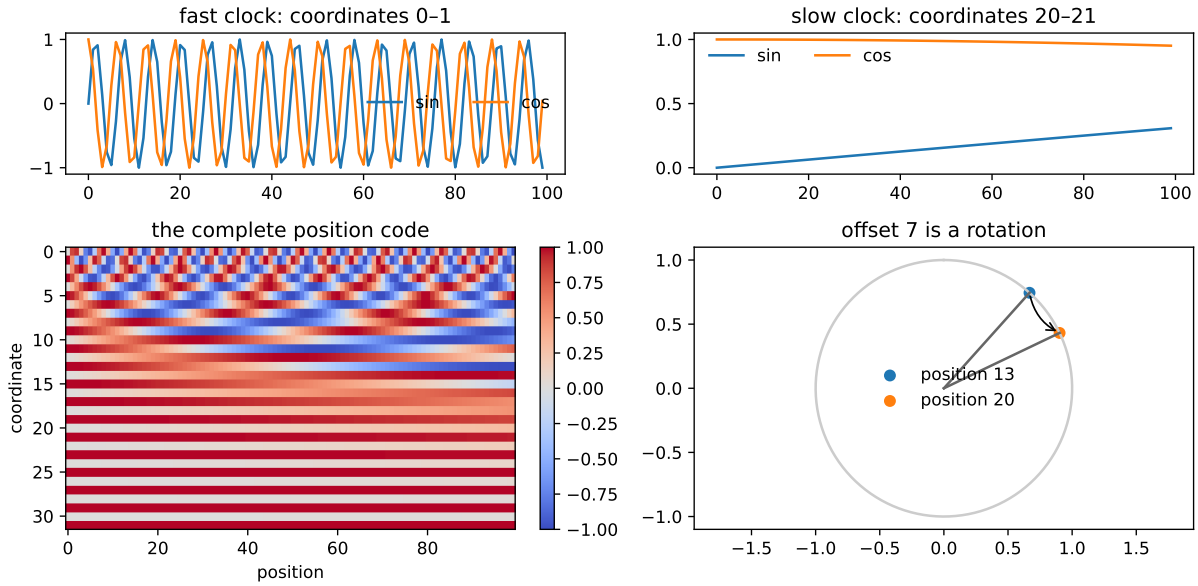

Figure 14.2.: Sinusoidal position is a bank of clocks. Fast coordinates track local offsets; slow coordinates vary across longer ranges. Each sine/cosine pair acts like a clock, and a fixed offset rotates that pair by a start-independent angle.

```
rotation identity maximum error: 1.11e-16
constant position-vector norm maximum error: 0.00e+00
```

For $d = 32$, each position vector has norm $\sqrt{d/2} = 4$ because every $\sin^2 + \cos^2$ pair contributes one. Adding this vector does *not* preserve the norm of the combined token representation—the embedding and position vector can reinforce or cancel one another. Layer normalization will soon manage scale at the block level.

14.3. One routing operation, many heads

A single attention distribution must make one compromise about what to retrieve. Multi-head attention lets several smaller routing systems—separate views of the same residual stream—operate in parallel. Choose a number of heads h that divides d, and let $d_h = d/h$. For each head r,

$$Q_r = XW_Q^{(r)}, \quad K_r = XW_K^{(r)}, \quad V_r = XW_V^{(r)},$$

where $W_Q^{(r)}, W_K^{(r)}, W_V^{(r)} \in \mathbb{R}^{d \times d_h}$. The head output is

$$H_r = \text{softmax}\left(\frac{Q_r K_r^\top}{\sqrt{d_h}} + M\right) V_r.$$

Here M is the visibility mask defined next; $M = 0$ for unmasked attention.

The complete operation concatenates the heads and mixes them:

$$\text{MHA}(X) = \text{Concat}(H_1, \dots, H_h) W_O, \qquad W_O \in \mathbb{R}^{d \times d}.$$

The shape ledger is the safest implementation guide:

$$(B, n, d) \to (B, h, n, d_h) \to (B, h, n, n) \to (B, h, n, d_h) \to (B, n, d).$$

Scaling uses $\sqrt{d_h}$, not $\sqrt{d}$, because a head's dot product sums d_h terms. With fixed d, splitting into heads does not multiply the leading projection count: W_Q, W_K, W_V, W_O together contribute about $4d^2$ weights regardless of h. Heads divide the representational subspace—they do not each receive a fresh width-d model.

14.3.1. The mask is part of the model

Chapter 11 called masking the rule that tells a model *which positions it may not look at.* Here that rule is causal: position i may use tokens at positions $j \le i$ but must not inspect future tokens $j > i$. Rows are queries and columns are keys, so the forbidden region is the strict upper triangle. We add

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i \end{cases}$$

before softmax. Replacing forbidden probabilities with zero afterward would leave the remaining row unnormalized. Masking every key in a row is invalid—softmax would receive only negative infinity. Here the diagonal remains visible: the representation at a character may use that character while predicting the next one.

```
import torch
from torch import nn
from matplotlib.patches import FancyBboxPatch

class CausalMultiHeadAttention(nn.Module):
    def __init__(self, width: int, heads: int) -> None:
        super().__init__()
        if width % heads != 0:
            raise ValueError("width must be divisible by heads")
```

```
        self.heads = heads
        self.head_width = width // heads
        self.qkv = nn.Linear(width, 3 * width)
        self.project = nn.Linear(width, width)
        nn.init.xavier_uniform_(self.qkv.weight)
        nn.init.zeros_(self.qkv.bias)

    def forward(self, x: torch.Tensor, return_weights: bool = False):
        batch, length, width = x.shape
        qkv = self.qkv(x)
        qkv = qkv.reshape(
            batch, length, 3, self.heads, self.head_width
        ).permute(2, 0, 3, 1, 4)
        q, k, v = qkv.unbind(dim=0)
        scores = q @ k.transpose(-2, -1) / math.sqrt(self.head_width)
        forbidden = torch.triu(
            torch.ones(length, length, dtype=torch.bool, device=x.device),
            diagonal=1,
        )
        scores = scores.masked_fill(forbidden, float("-inf"))
        weights = torch.softmax(scores, dim=-1)
        routed = weights @ v
        routed = routed.transpose(1, 2).reshape(batch, length, width)
        output = self.project(routed)
        if return_weights:
            return output, weights
        return output

torch.manual_seed(6050)
demo_attention = CausalMultiHeadAttention(width=32, heads=4)
demo_x = torch.randn(1, 12, 32)
_, demo_weights = demo_attention(demo_x, return_weights=True)

future = torch.triu(torch.ones(12, 12, dtype=torch.bool), diagonal=1)
assert demo_weights[0, :, future].max().item() == 0.0
assert torch.allclose(
    demo_weights.sum(dim=-1),
    torch.ones_like(demo_weights.sum(dim=-1)),
    atol=1e-6,
)

fig = plt.figure(figsize=(10, 7), layout="constrained")
grid = fig.add_gridspec(2, 4, height_ratios=[1, 3])
flow_axis = fig.add_subplot(grid[0, :])
flow_axis.set_xlim(0, 10)
flow_axis.set_ylim(0, 1)
```

```
flow_axis.axis("off")
flow = [
    (0.1, "$X$", "#d8ecff"),
    (2.1, "split $Q,K,V$\ninto $h$ heads", "#f9dfb5"),
    (4.5, "$h$ attention\nroutes", "#f9dfb5"),
    (6.9, "concatenate", "#d9f2d0"),
    (8.7, "$W_O$", "#d9f2d0"),
]
for index, (left, label, color) in enumerate(flow):
    width = 1.2 if index in [0, 4] else 1.7
    flow_axis.add_patch(FancyBboxPatch(
        (left, 0.2), width, 0.6,
        boxstyle="round,pad=0.05",
        facecolor=color, edgecolor="0.25",
    ))
    flow_axis.text(left + width / 2, 0.5, label, ha="center", va="center")
    if index < len(flow) - 1:
        next_left = flow[index + 1][0]
        flow_axis.annotate(
            "", xy=(next_left, 0.5), xytext=(left + width, 0.5),
            arrowprops={"arrowstyle": "->", "color": "0.25"},
        )

axes = [fig.add_subplot(grid[1, head]) for head in range(4)]
for head, axis in enumerate(axes):
    image = axis.imshow(
        demo_weights[0, head].detach().numpy(),
        cmap="viridis", vmin=0, vmax=1,
    )
    axis.set_title(f"head {head + 1}")
    axis.set_xlabel("key position")
    if head == 0:
        axis.set_ylabel("query position")
fig.colorbar(image, ax=axes, fraction=0.025)
plt.show()

print("maximum forbidden attention:", demo_weights[0, :, future].max().item())
print(
    "maximum row-sum error:",
    (demo_weights.sum(dim=-1) - 1).abs().max().item(),
)
```

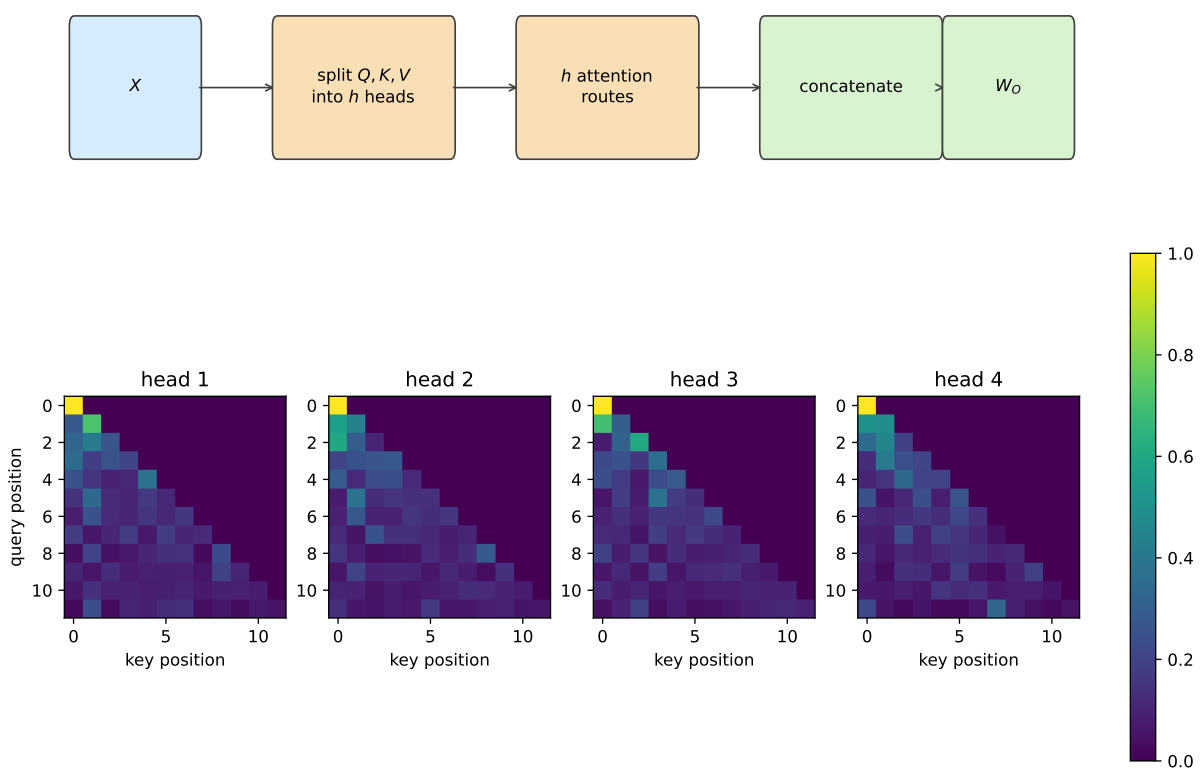

Figure 14.3.: Multi-head attention splits, routes in parallel, concatenates, and projects (top). In the untrained causal heads below, every query row sums to one and every future key has exactly zero weight.

```
maximum forbidden attention: 0.0
maximum row-sum error: 1.1920928955078125e-07
```

The assertions are part of the derivation. A visually plausible triangle can still be transposed, shifted by one, or applied after softmax. Executable shape and normalization checks catch those errors—before training can disguise them.

The mask changes visibility, not dense computation. The implementation still forms all n^2 scores and stores Bhn^2 attention weights, alongside roughly Bnd^2 projection work and $Bndd_{ff}$ work in the FFN. Self-attention shortens the graph path between distant tokens to one routing step—but "one step away" is not "constant runtime."

14.4. Build a Transformer block

Attention routes information between token positions. A Transformer block needs two more ingredients—a stable path through depth and a nonlinear computation at each position.

14.4.1. The residual stream

Chapter 9 introduced the residual stream with one durable sentence: every block reads the stream and writes a correction back. If a sublayer computes $F(x)$, the residual update is

$$x \leftarrow x + F(x).$$

The identity branch provides a direct route for representations and gradients while the learned branch proposes a correction. It makes deep optimization more tractable—it does not guarantee that gradients can never vanish or explode.

Layer normalization controls each token's feature scale. For one token vector $x \in \mathbb{R}^d$,

$$\mu = \frac{1}{d}\sum_{k=1}^{d} x_k, \qquad \sigma^2 = \frac{1}{d}\sum_{k=1}^{d}(x_k - \mu)^2,$$

$$\mathrm{LN}(x)_k = \gamma_k \frac{x_k - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_k, \qquad \gamma, \beta \in \mathbb{R}^d.$$

This is Chapter 9's normalization equation applied along a different axis—the same equation, different axis. Chapter 9's BatchNorm2d used batch and spatial axes for each channel. LayerNorm flips to the feature axis of each token: it computes statistics across the features within one token, without aggregating statistics across other tokens or examples. Its calculation is the same at training and evaluation time. Before the learned γ and β, the vector is approximately zero mean and unit variance. After applying them, that claim need not remain true.

```
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch

fig, axes = plt.subplots(1, 2, figsize=(9, 4.5))
axis = axes[0]
axis.set_xlim(0, 10)
axis.set_ylim(0, 10)
axis.axis("off")

def box(
    axis: plt.Axes,
    x: float,
    y: float,
    width: float,
    height: float,
    text: str,
    color: str,
) -> None:
    patch = FancyBboxPatch(
        (x, y), width, height,
        boxstyle="round,pad=0.08",
```

```
        facecolor=color, edgecolor="0.25",
    )
    axis.add_patch(patch)
    axis.text(x + width / 2, y + height / 2, text, ha="center", va="center")

axis.plot([2, 2], [0.7, 9.3], color="0.25", linewidth=3)
axis.text(0.2, 0.25, "residual stream", color="0.25")
box(axis, 3.5, 1.0, 2.5, 1.0, "LayerNorm", "#d8ecff")
box(axis, 3.5, 2.6, 2.5, 1.0, "causal MHA", "#f9dfb5")
box(axis, 3.5, 5.7, 2.5, 1.0, "LayerNorm", "#d8ecff")
box(axis, 3.5, 7.3, 2.5, 1.0, "FFN", "#d9f2d0")

for stream_y in [1.5, 6.2]:
    axis.add_patch(
        FancyArrowPatch((2, stream_y), (3.5, stream_y), arrowstyle="->")
    )
for box_top, stream_y in [(3.6, 4.3), (8.3, 9.0)]:
    axis.add_patch(
        FancyArrowPatch((4.75, box_top), (2, stream_y), arrowstyle="->")
    )
axis.add_patch(FancyArrowPatch((4.75, 2.0), (4.75, 2.6), arrowstyle="->"))
axis.add_patch(FancyArrowPatch((4.75, 6.7), (4.75, 7.3), arrowstyle="->"))
axis.text(2.25, 4.05, "+", fontsize=18)
axis.text(2.25, 8.75, "+", fontsize=18)
axis.set_title("reads, computes, writes")

audit = torch.tensor([
    [[1.0, 3.0, 5.0, 7.0], [40.0, 50.0, 60.0, 70.0]],
    [[-3.0, 1.0, 5.0, 9.0], [2.0, 2.5, 3.0, 3.5]],
])
normalized = nn.functional.layer_norm(audit, (4,))
token_means = normalized.mean(dim=-1)
token_vars = normalized.var(dim=-1, unbiased=False)
assert token_means.abs().max().item() < 1e-6
assert (token_vars - 1).abs().max().item() < 1e-4

for token, values in enumerate(normalized.reshape(-1, 4)):
    axes[1].plot(
        range(1, 5), values.numpy(), marker="o", label=f"token {token + 1}"
    )
axes[1].axhline(0, color="0.7", linestyle="--")
axes[1].set_xticks(range(1, 5))
axes[1].set_xlabel("feature within token")
axes[1].set_ylabel("normalized activation")
axes[1].set_title("each token centers and scales itself")
axes[1].legend(frameon=False, ncol=2)
```

```
plt.tight_layout()
plt.show()

print("maximum absolute token mean:", token_means.abs().max().item())
print("maximum token variance error:", (token_vars - 1).abs().max().item())
```

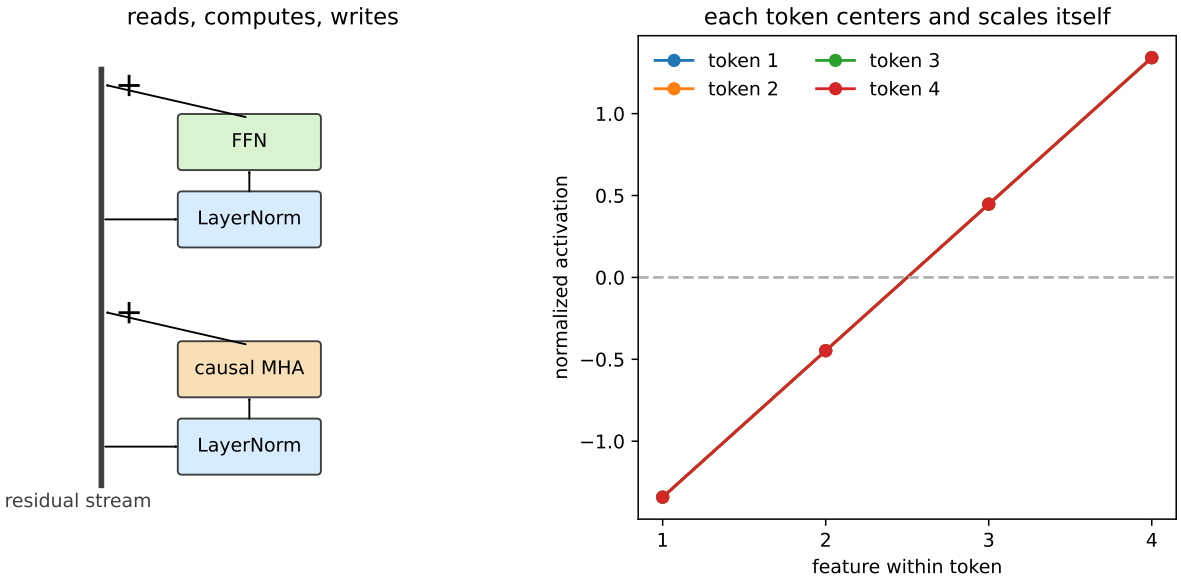

Figure 14.4.: A pre-LayerNorm Transformer block (left): each sublayer reads a normalized view of the residual stream and writes a correction back. LayerNorm centers and scales every token across its own feature axis (right).

```
maximum absolute token mean: 0.0
maximum token variance error: 3.2067298889160156e-05
```

The original Transformer normalized *after* each residual addition:

$$x \leftarrow \mathrm{LN}(x + \mathrm{MHA}(x)), \qquad x \leftarrow \mathrm{LN}(x + \mathrm{FFN}(x)).$$

Our small experiment uses the now-common pre-LayerNorm arrangement:

$$x \leftarrow x + \mathrm{MHA}(\mathrm{LN}(x)),$$
$$x \leftarrow x + \mathrm{FFN}(\mathrm{LN}(x)).$$

Pre-LayerNorm leaves an unobstructed identity route along the residual stream and often improves optimization at initialization. This is an implementation choice, not a universal theorem that post-LayerNorm fails or that pre-LayerNorm eliminates every need for careful optimization.

14.4.2. The position-wise feed-forward network

After tokens communicate through attention, the same two-layer network processes each position independently:

$$\mathrm{FFN}(z) = W_2\,\mathrm{ReLU}(W_1 z + b_1) + b_2.$$

Here $W_1 \in \mathbb{R}^{d_{ff}\times d}$, $b_1 \in \mathbb{R}^{d_{ff}}$, $W_2 \in \mathbb{R}^{d\times d_{ff}}$, and $b_2 \in \mathbb{R}^d$.

If $w_{1j}^\top$ is row j of W_1 and v_j is column j of W_2, the same operation can be written

$$\mathrm{FFN}(z) = \sum_j [w_{1j}^\top z + b_{1j}]_+ \, v_j + b_2.$$

This form motivates an associative-memory lens. Each first-layer row tests whether the current representation lies in a learned half-space; its positive activation controls how much of the corresponding output vector is written. Empirically, Transformer FFNs can behave like key-value memories. The analogy has limits—the selectors are half-spaces rather than point anchors, the gates do not sum to one, and the operation is not literally kernel regression or a normalized lookup.

The complete block therefore alternates two kinds of computation:

1. causal multi-head attention moves information *between positions*; and
2. the FFN transforms information *within each position*.

Residual additions keep both results in one shared stream—a common workspace that survives the full stack.

14.5. From the original Transformer to this experiment

The 2017 Transformer was an encoder–decoder system for translation. Its encoder stack uses unmasked self-attention to build a representation of the source sentence. Its decoder stack uses causal self-attention over the partial target, then cross-attention whose queries come from the decoder while keys and values come from the encoder. In compact form:

$$\begin{aligned}&\text{encoder:} && X_s \to \text{self-attention} \to \text{FFN},\\ &\text{decoder:} && X_t \to \text{causal self-attention} \to \text{cross-attention to encoder} \to \text{FFN}.\end{aligned}$$

The book-corpus task predicts the next character from earlier characters. It has no separate source sentence, so the encoder and cross-attention are unnecessary. What remains—a **decoder-only causal Transformer**—has token plus position embeddings, two pre-LayerNorm blocks, a final LayerNorm, and a linear map to vocabulary logits. The logits go directly to cross-entropy. Softmax belongs inside attention and, later, inside sampling—not between the output layer and the loss.

14.6. The book reads itself, again

A fair rematch begins by preserving what can be preserved. Chapter 10 trained on the code-stripped text of Chapters 1–9: 148,594 characters, vocabulary 104, a contiguous 90/10 split, 100-character windows, batch size 64, and 2,501 updates with Adam at learning rate 0.002 and gradient clipping at norm 1. The held-out metric resets state at each fixed window.

The two architectures cannot be made identical—one has recurrent gates and the other has attention blocks—but their opportunity to see training targets can be. The code below reconstructs Chapter 10's exact random-window schedule, then gives both Transformer variants the same initial weights and all 16,006,400 target characters in the same order. The held-out tail has been inspected repeatedly across chapters, so treat it as a stable validation benchmark—not a newly sealed test set.

```
import glob
import hashlib
import re
import torch.nn.functional as F

torch.set_num_threads(4)
torch.use_deterministic_algorithms(True)

SEED = 6050
CONTEXT = 100
BATCH = 64
UPDATES = 2501
WIDTH = 84
HEADS = 4
FF_WIDTH = 168
BLOCKS = 2

fence = chr(96) * 3
files = sorted(glob.glob("../part*/0*.qmd"))
text = ""
for filename in files:
    raw = open(filename).read()
    code_pattern = (
        re.escape(fence) + r"\{python\}.*?" + re.escape(fence)
    )
    raw = re.sub(code_pattern, "", raw, flags=re.S)
    raw = re.sub(r"<!--.*?-->", "", raw, flags=re.S)
    text += raw

chars = sorted(set(text))
stoi = {char: index for index, char in enumerate(chars)}
data = torch.tensor([stoi[char] for char in text])
split = int(0.9 * len(data))
```

```
train_data, valid_data = data[:split], data[split:]

def sinusoidal_positions(length: int, width: int) -> torch.Tensor:
    position = torch.arange(length, dtype=torch.float32).unsqueeze(1)
    frequency = torch.exp(
        torch.arange(0, width, 2, dtype=torch.float32)
        * (-math.log(10_000.0) / width)
    )
    table = torch.zeros(length, width)
    table[:, 0::2] = torch.sin(position * frequency)
    table[:, 1::2] = torch.cos(position * frequency)
    return table

class TransformerBlock(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.norm1 = nn.LayerNorm(WIDTH)
        self.attention = CausalMultiHeadAttention(WIDTH, HEADS)
        self.norm2 = nn.LayerNorm(WIDTH)
        self.ff = nn.Sequential(
            nn.Linear(WIDTH, FF_WIDTH),
            nn.ReLU(),
            nn.Linear(FF_WIDTH, WIDTH),
        )

    def forward(self, x: torch.Tensor, return_weights: bool = False):
        if return_weights:
            attended, weights = self.attention(self.norm1(x), True)
            x = x + attended
            return x + self.ff(self.norm2(x)), weights
        x = x + self.attention(self.norm1(x))
        return x + self.ff(self.norm2(x))

class TinyTransformerLM(nn.Module):
    def __init__(self, vocab: int, positions: bool) -> None:
        super().__init__()
        self.positions = positions
        self.token_embedding = nn.Embedding(vocab, WIDTH)
        nn.init.normal_(
            self.token_embedding.weight,
            mean=0.0,
            std=1 / math.sqrt(WIDTH),
        )
        self.blocks = nn.ModuleList(
            [TransformerBlock() for _ in range(BLOCKS)]
        )
```

```
        self.final_norm = nn.LayerNorm(WIDTH)
        self.output = nn.Linear(WIDTH, vocab)
        self.register_buffer(
            "position_table",
            sinusoidal_positions(CONTEXT, WIDTH),
            persistent=False,
        )

    def forward(self, tokens: torch.Tensor, return_weights: bool = False):
        length = tokens.shape[1]
        x = self.token_embedding(tokens) * math.sqrt(WIDTH)
        if self.positions:
            x = x + self.position_table[:length]
        maps = []
        for block in self.blocks:
            if return_weights:
                x, weights = block(x, True)
                maps.append(weights)
            else:
                x = block(x)
        logits = self.output(self.final_norm(x))
        return (logits, maps) if return_weights else logits

class HistoricalCharLSTM(nn.Module):
    def __init__(self, vocab: int, hidden: int = 128) -> None:
        super().__init__()
        self.vocab = vocab
        self.lstm = nn.LSTM(vocab, hidden, batch_first=True)
        self.out = nn.Linear(hidden, vocab)

def state_digest(state: dict[str, torch.Tensor]) -> str:
    digest = hashlib.sha256()
    for name, value in state.items():
        digest.update(name.encode())
        digest.update(value.detach().cpu().numpy().tobytes())
    return digest.hexdigest()

# Reconstruct the random-number state immediately after Chapter 10 created
# its LSTM, then draw that chapter's complete minibatch-start schedule.
torch.manual_seed(SEED)
_ = HistoricalCharLSTM(len(chars))
schedule_generator = torch.Generator()
schedule_generator.set_state(torch.random.get_rng_state())
starts = torch.randint(
    0,
    len(train_data) - CONTEXT - 1,
```

```
    (UPDATES, BATCH),
    generator=schedule_generator,
)

# The two Transformer variants begin with exactly the same tensors.
torch.manual_seed(SEED)
base_model = TinyTransformerLM(len(chars), positions=True)
initial_state = {
    name: value.clone() for name, value in base_model.state_dict().items()
}
initial_hash = state_digest(initial_state)
schedule_hash = hashlib.sha256(starts.numpy().tobytes()).hexdigest()

print(
    f"corpus: {len(files)} chapters, {len(text):,} characters, "
    f"vocab {len(chars)}"
)
print(
    f"deterministic split: {len(train_data):,} train / "
    f"{len(valid_data):,} held out"
)
print(
    f"Transformer parameters: "
    f"{sum(parameter.numel() for parameter in base_model.parameters()):,}"
)
print(f"first eight window starts: {starts[0, :8].tolist()}")
print(f"initial tensors: {initial_hash[:12]}…")
print(f"window schedule: {schedule_hash[:12]}…")
```

```
corpus: 9 chapters, 148,594 characters, vocab 104
deterministic split: 133,734 train / 14,860 held out
Transformer parameters: 132,488
first eight window starts: [24368, 94128, 60074, 121547, 118001, 4396, 17011, 8495]
initial tensors: 4d2f7f434cb5…
window schedule: e470a091bd50…
```

The model has 132,488 trainable parameters—736 fewer than Chapter 10's 133,224. That difference is only 0.55%—close enough to remove model size as an easy explanation. The model width is 84, split into four 21-dimensional heads; each of the two FFNs expands to 168 features. Token embeddings are initialized with coordinate standard deviation $1/\sqrt{d}$ before the displayed $\sqrt{d}$ scaling, so their coordinates and the sinusoidal coordinates begin on comparable scales. `ModuleList` registers the two repeated blocks with the model. The nonpersistent position buffer moves with the model but is neither optimized nor included in the state-dictionary fingerprint; its contents follow deterministically from the displayed formula. There is no dropout, so resetting the trainable/state-dictionary tensors and window schedule makes the positional ablation exact and repeatable.

One seed call was not enough

Calling the same random seed before both training runs would not, by itself, prove that they saw the same windows. Model construction consumes random numbers, and different construction paths can silently shift every later draw. The experiment precomputes the full start-index tensor with an explicit generator and copies one saved initialization into both models. It also prints fingerprints for both artifacts.

```
def train_transformer(
    positions: bool,
) -> tuple[nn.Module, list[tuple[int, float]]]:
    net = TinyTransformerLM(len(chars), positions=positions)
    net.load_state_dict(initial_state, strict=True)
    assert state_digest(net.state_dict()) == initial_hash
    optimizer = torch.optim.Adam(net.parameters(), lr=2e-3)
    curve = []
    for step, batch_starts in enumerate(starts):
        xb = torch.stack([
            train_data[j : j + CONTEXT] for j in batch_starts
        ])
        yb = torch.stack([
            train_data[j + 1 : j + CONTEXT + 1] for j in batch_starts
        ])
        logits = net(xb)
        loss = F.cross_entropy(
            logits.reshape(-1, len(chars)), yb.reshape(-1)
        )
        optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(net.parameters(), 1.0)
        optimizer.step()
        if step % 250 == 0:
            curve.append((step, loss.item()))
            print(f"step {step:4d}   loss {loss.item():.4f}")
    return net, curve

@torch.no_grad()
def fixed_window_loss(
    net: nn.Module,
    sequence: torch.Tensor,
    window: int = CONTEXT,
) -> float:
    net.eval()
    total_loss, n_tokens = 0.0, 0
    for start in range(0, len(sequence) - 1, window):
        stop = min(start + window, len(sequence) - 1)
```

```
        xb = sequence[start:stop].unsqueeze(0)
        yb = sequence[start + 1 : stop + 1]
        logits = net(xb)
        total_loss += F.cross_entropy(
            logits.reshape(-1, len(chars)),
            yb,
            reduction="sum",
        ).item()
        n_tokens += yb.numel()
    return total_loss / n_tokens

@torch.no_grad()
def transformer_sample(
    net: nn.Module,
    prompt: str,
    n: int = 300,
    temperature: float = 0.8,
) -> str:
    net.eval()
    generator = torch.Generator().manual_seed(1_406_050)
    tokens = [stoi.get(char, 0) for char in prompt]
    for _ in range(n):
        context = torch.tensor(tokens[-CONTEXT:]).unsqueeze(0)
        probabilities = F.softmax(
            net(context)[0, -1] / temperature, dim=-1
        )
        next_token = torch.multinomial(
            probabilities, 1, generator=generator
        ).item()
        tokens.append(next_token)
    return "".join(chars[index] for index in tokens)
```

The training function is defined once, then called in separate cells so each substantial run stays within the chapter's execution budget.

```
position_model, position_curve = train_transformer(positions=True)
position_train_loss = fixed_window_loss(position_model, train_data)
position_valid_loss = fixed_window_loss(position_model, valid_data)
print(f"fixed-window train loss: {position_train_loss:.4f}")
print(f"fixed-window held-out loss: {position_valid_loss:.4f}")
```

```
step    0   loss 4.7550
step  250   loss 2.1850
step  500   loss 1.7159
step  750   loss 1.5176
```

```
step 1000   loss 1.3955
step 1250   loss 1.3444
step 1500   loss 1.2813
step 1750   loss 1.2386
step 2000   loss 1.1708
step 2250   loss 1.1525
step 2500   loss 1.1083
fixed-window train loss: 1.1132
fixed-window held-out loss: 1.9190
```

```
no_position_model, no_position_curve = train_transformer(positions=False)
no_position_train_loss = fixed_window_loss(no_position_model, train_data)
no_position_valid_loss = fixed_window_loss(no_position_model, valid_data)
print(f"fixed-window train loss: {no_position_train_loss:.4f}")
print(f"fixed-window held-out loss: {no_position_valid_loss:.4f}")
```

```
step    0   loss 4.7289
step  250   loss 2.4955
step  500   loss 2.3612
step  750   loss 2.2436
step 1000   loss 2.2153
step 1250   loss 2.1826
step 1500   loss 2.0792
step 1750   loss 1.9876
step 2000   loss 1.9527
step 2250   loss 1.8986
step 2500   loss 1.8435
fixed-window train loss: 1.8672
fixed-window held-out loss: 2.3405
```

```
lstm_valid_loss = 1.888110429

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for curve, label, color in [
    (position_curve, "Transformer + position", "#3566a8"),
    (no_position_curve, "Transformer, no position", "#d37b29"),
]:
    steps, losses = zip(*curve)
    axes[0].plot(steps, losses, marker="o", ms=3, label=label, color=color)
axes[0].set_xlabel("update")
axes[0].set_ylabel("sampled training-minibatch loss")
axes[0].set_title("same windows, same initialization")
axes[0].legend(frameon=False)

labels = ["Ch. 10\nLSTM", "Transformer\n+ position", "Transformer\nno position"]
```

```
values = [lstm_valid_loss, position_valid_loss, no_position_valid_loss]
bars = axes[1].bar(labels, values, color=["#6b7280", "#3566a8", "#d37b29"])
axes[1].set_ylim(1.5, 2.7)
axes[1].set_ylabel("held-out loss ↓")
axes[1].set_title("fixed-window rematch")
axes[1].bar_label(bars, fmt="%.4f", padding=3)
plt.tight_layout()
plt.show()

improvement = no_position_valid_loss - position_valid_loss
relative = improvement / no_position_valid_loss
lstm_gap = position_valid_loss - lstm_valid_loss
print(
    f"position improvement: {improvement:.4f} loss "
    f"({relative:.1%} relative)"
)
print(f"positional Transformer minus LSTM: {lstm_gap:.4f} loss")
```

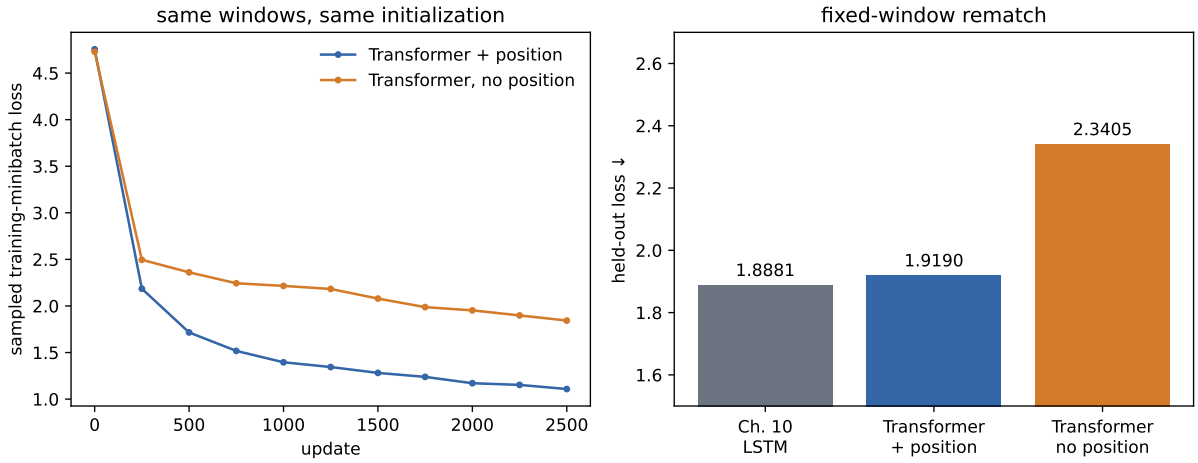

Figure 14.5.: Matched training curves (left) and fixed-window held-out loss (right). Position helps in this seed, while the Chapter 10 LSTM remains the strongest small model.

```
position improvement: 0.4214 loss (18.0% relative)
positional Transformer minus LSTM: 0.0309 loss
```

The positional model finishes at 1.1132 on a deterministic pass over the training text and 1.9190 on the held-out tail. Removing position raises those values to 1.8672 and 2.3405. Position lowers held-out loss by 0.4214, or 18.0% relative to the no-position variant.

⚠ What this matched run can—and cannot—show

The causal mask already gives a position some weak structural information: row i sees a prefix of length $i + 1$, and stacked layers can propagate clues from those nested prefixes. That is why the no-position model does better than a purely orderless caricature might suggest—fixed sinusoidal encoding still produces a large improvement in this run.
But Chapter 10's LSTM remains ahead by 0.0309 held-out loss, or 1.64%. At 100 characters of context and roughly 133,000 parameters, the LSTM wins this regime. This comparison matches corpus, parameter scale, updates, minibatch order, optimizer, clipping, and evaluation. It does not match internal architecture—or promise that either model has been tuned to its own optimum. We did not repeat the comparison across initialization seeds, so the 0.4214 gap is a controlled case study, not an estimate of an average effect.

14.6.1. What did the heads route?

An attention map records routing weights, not reasons, confidence, or a guaranteed explanation of a prediction. Still, inspecting real learned maps can verify the mask and reveal the kinds of patterns a trained model happened to use. The next cell sends the prompt *The gradient* through the positional model and plots the second block's four heads.

```
prompt = "The gradient "
prompt_tokens = torch.tensor([[stoi[char] for char in prompt]])
position_model.eval()
with torch.no_grad():
    _, learned_maps = position_model(prompt_tokens, return_weights=True)
learned = learned_maps[-1][0]

future = torch.triu(
    torch.ones(len(prompt), len(prompt), dtype=torch.bool), diagonal=1
)
assert learned[:, future].max().item() == 0.0
assert torch.allclose(
    learned.sum(dim=-1),
    torch.ones_like(learned.sum(dim=-1)),
    atol=1e-6,
)

display_tokens = ["·" if char == " " else char for char in prompt]
fig, axes = plt.subplots(2, 2, figsize=(10, 6), layout="constrained")
for head, axis in enumerate(axes.flat):
    image = axis.imshow(
        learned[head].numpy(), cmap="viridis", vmin=0, vmax=1
    )
    axis.set_title(f"block 2, head {head + 1}")
    axis.set_xticks(range(len(prompt)), display_tokens)
```

```
    axis.set_yticks(range(len(prompt)), display_tokens)
    axis.set_xlabel("key")
    axis.set_ylabel("query")
fig.colorbar(image, ax=axes, fraction=0.025, shrink=0.8, pad=0.02)
plt.show()

print("maximum future attention:", learned[:, future].max().item())
print(
    "maximum row-sum error:",
    (learned.sum(dim=-1) - 1).abs().max().item(),
)
```

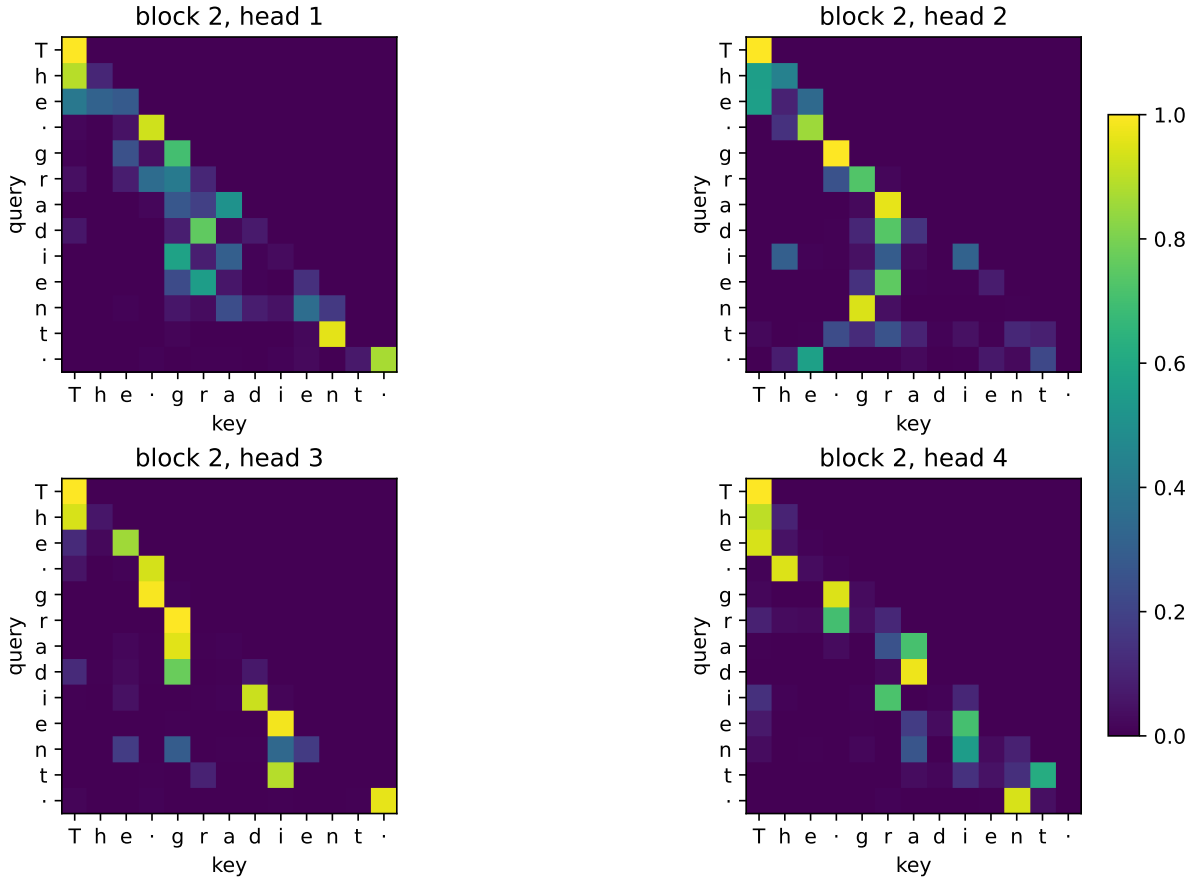

Figure 14.6.: Learned attention weights in the second block for the prompt “The gradient ”. The maps are real routing distributions, not explanations or named head roles.

```
maximum future attention: 0.0
maximum row-sum error: 1.1920928955078125e-07
```

The maps obey the contract exactly: no future key receives weight and every row sums to one. The heads also differ from one another, but naming one “the previous character head” or another

"the word head" from this single prompt would outrun the evidence. Attention shows where values were mixed—the rest of the network can transform, cancel, or ignore what was retrieved.

14.6.2. Listen, but do not grade by fluency

Both Transformer variants can now generate by repeatedly truncating to the most recent 100 characters, predicting a distribution for the next one, sampling, and feeding the result back. Unlike Chapter 10's LSTM, this simple implementation does not carry a recurrent state; it recomputes the current context on every step.

```
print("WITH POSITION\n")
print(transformer_sample(position_model, "The gradient "))
print("\n\nWITHOUT POSITION\n")
print(transformer_sample(no_position_model, "The gradient "))
```

```
WITH POSITION

The gradient choing fine on autograd it by tweight the clue larges of calling functions in Wh
normalized of the argument builds give if the five is the funit or principled update a thresh


WITHOUT POSITION

The gradient explens ine. The beveuind or clos? we the canlay. Wet orurle
this moples doing ther diest the the contive losio thes despunction erbiveng or of a the
corno isy la prenod in thrathe s to derngexperevive. With the chate is the compfathes bl. Thi
tat ur is $L$\vect{w}^\vect{w} = \mectrimes \vect
```

The positional sample has the book's punctuation, fragments of mathematical vocabulary, and stretches that resemble prose. It still loses its argument. The no-position sample is visibly rougher. Neither qualitative judgment replaces the held-out loss—a sample is one stochastic path, while the loss evaluates every held-out target under the model's full probability distribution.

14.7. What the architecture bought

The rematch is not a referendum on all Transformers—it isolates what this small one bought and what it paid.

Direct content routing. Any earlier visible token can contribute to the next representation in one attention sublayer. The recurrent state no longer has to compress the entire past before the current token can consult it.

Short paths, dense work. The graph distance between visible tokens is short, but the routing matrix is quadratic in context length. Global access is a different tradeoff, not free memory.

Position becomes a modeling choice. Convolution and recurrence bake in local order. Attention asks the designer to supply position—and decide visibility. Chapter 15 will change visibility itself; Chapter 16 will revisit what happens when global routing trades away an image model's locality bias.

A modular residual stream. Attention moves information, the FFN transforms it, and residual additions let both write into a shared representation. LayerNorm keeps each token's feature scale manageable along the way.

The causal mask is therefore more than an implementation detail. It says which information a representation is allowed to use. "Visibility is a modeling decision" is the seed this chapter hands forward.

14.8. Okay, so—

1. **Self-attention reuses Chapter 13's operator within one sequence.** Q, K, and V all come from X; one learned comparison rule is shared over every ordered pair. Training positions can be evaluated together, while autoregressive generation remains sequential.
2. **Bare attention is permutation equivariant, not invariant.** $\mathrm{SA}(PX) = P\,\mathrm{SA}(X)$. Sinusoidal clocks repay Chapter 8's position debt, and their sine/cosine pairs turn relative shifts into rotations.
3. **Multi-head attention is shape discipline.** Split $(B, n, d) \to (B, h, n, d_h)$, scale by $\sqrt{d_h}$, form (B, h, n, n) routing weights, concatenate, and project. At fixed d, the four leading projections remain about $4d^2$ parameters.
4. **The causal mask defines visibility.** Future scores become negative infinity before softmax. The dense operator still performs quadratic work; the mask does not make the forbidden half computationally disappear.
5. **The block alternates routing and computation.** Multi-head attention mixes positions, the FFN transforms each position, and both write corrections into Chapter 9's residual stream. LayerNorm is the same normalization equation over a token's feature axis.
6. **Position wins this seed's controlled ablation.** It lowers held-out loss from 2.3405 to 1.9190. **The LSTM narrowly wins the small-model rematch**, 1.8881 versus 1.9190. Architecture changes inductive bias; it does not guarantee victory in every data and scale regime.

Sources and further reading

- Vaswani et al., *Attention Is All You Need*: multi-head attention, sinusoidal position, and the original post-LayerNorm encoder–decoder.
- Ba, Kiros, and Hinton, *Layer Normalization*: normalization within one example and identical train/evaluation computation.
- Xiong et al., *On Layer Normalization in the Transformer Architecture*: a qualified comparison of pre- and post-LayerNorm optimization.
- Geva et al., *Transformer Feed-Forward Layers Are Key-Value Memories*: empirical evidence behind the associative-memory interpretation of FFNs.

- Jain and Wallace, *Attention Is Not Explanation*: an important limit on reading attention maps as explanations.

Exercises

1. **(Pencil.)** Starting from $Q' = PQ$, $K' = PK$, and $V' = PV$, prove each step of $\mathrm{SA}(PX) = P\,\mathrm{SA}(X)$. Then show that mean-pooling the token outputs produces a permutation-*invariant* sequence representation.
2. **(Pencil.)** For width $d = 512$ and $h = 8$, write every tensor shape in multi-head attention for $B = 32$ and $n = 100$. Count the weights in W_Q, W_K, W_V, W_O. Repeat for $h = 16$ and explain which dimensions change while the leading parameter count does not.
3. **(Code.)** Deliberately transpose the causal mask in Figure 14.3, then design an assertion that identifies the first leaked query-key pair. Next set forbidden probabilities to zero *after* softmax and measure the row-sum error.
4. **(Code.)** Replace the fixed sinusoidal table with a learned $\mathrm{Embedding}(100, d)$ while preserving the exact initialization and window schedule protocol. Compare held-out loss. Then extend the sinusoidal buffer by evaluating its formula beyond position 99, and propose an explicit extension rule for the learned table before testing either model on a longer context. Which design has a natural extension, and which has to invent one?
5. **(Code and interpretation.)** Aggregate learned attention by relative offset across the held-out tail rather than naming heads from one prompt. Report a distribution for every head and block. Then perturb the highest-weight key's value vector and measure the logit change. Explain why routing weight and causal influence need not rank tokens identically.

15. The BERT Moment: Pretraining as the New Regime

Chapter 9 left a decision rule unfinished. Transfer should pay when **labels are scarce, the target is feature-hungry, and source coverage is useful at a matched scale**. Images supplied a natural source task—learn visual features from many labeled photographs, then adapt them. What should play the same role for language, where task labels are expensive but raw text is everywhere?

Text can write its own questions. Hide *bank* in "the boat reached the bank," and the surrounding words contain the answer. Hide the same spelling in "the bank approved the loan," and a different neighborhood supplies a different answer. No person had to label either sense. The answer is already inside the sequence; corruption turns it into a question.

That move depends on the seed from Chapter 14: **visibility is a modeling decision**. A next-token model used a causal triangle because looking right would reveal its target. A representation model can instead let every nonpadding token consult both sides—provided selected targets are treated so copying cannot solve the whole task. This chapter separates those controls, derives **masked language modeling (MLM)**, and shows how pretraining changed the default workflow from "initialize and train" to "pretrain, then adapt."

15.1. Separate controls, not one mask

Bidirectional Encoder Representations from Transformers—**BERT**—uses a few special tokens. `[CLS]` is a learned summary placeholder, `[SEP]` marks a boundary, and `[PAD]` fills unused batch slots. Consider six positions:

[CLS], the, bank, rose, [SEP], [PAD].

There are four distinct decisions in play.

1. **Attention visibility** decides which key positions each query may use. A causal decoder blocks future keys; a BERT encoder permits all nonpadding keys.
2. **The MLM target selector** chooses which ordinary positions contribute loss.
3. **The corruption branch** replaces some selected tokens by `[MASK]`, replaces some by random tokens, and deliberately leaves some unchanged.
4. **The padding mask** prevents real tokens from routing information from placeholder slots used only to make batch shapes rectangular.

```
import copy
import itertools

import matplotlib.pyplot as plt
import numpy as np
import torch
from torch import nn
import torch.nn.functional as F

tokens = ["[CLS]", "the", "bank", "rose", "[SEP]", "[PAD]"]
nonpadding = np.array([1, 1, 1, 1, 1, 0], dtype=bool)
causal = np.tril(np.ones((6, 6), dtype=int)) * nonpadding[None, :]
full = np.ones((6, 6), dtype=int) * nonpadding[None, :]
selector = np.zeros((1, 6), dtype=int)
selector[0, 2] = 1

fig, axes = plt.subplots(
    1, 3, figsize=(10, 3.2), gridspec_kw={"width_ratios": [1, 1, 0.45]}
)
for axis, matrix, title in [
    (axes[0], causal, "causal visibility"),
    (axes[1], full, "full nonpadding visibility"),
]:
    axis.imshow(matrix, cmap="Blues", vmin=0, vmax=1)
    axis.set_xticks(range(6), tokens, rotation=55, ha="right")
    axis.set_yticks(range(6), tokens)
    axis.set_xlabel("key")
    axis.set_ylabel("query")
    axis.set_title(title)
axes[2].imshow(selector.T, cmap="Oranges", vmin=0, vmax=1)
axes[2].set_yticks(range(6), tokens)
axes[2].set_xticks([0], ["loss"])
axes[2].set_title("MLM selector")
plt.tight_layout()
plt.show()
```

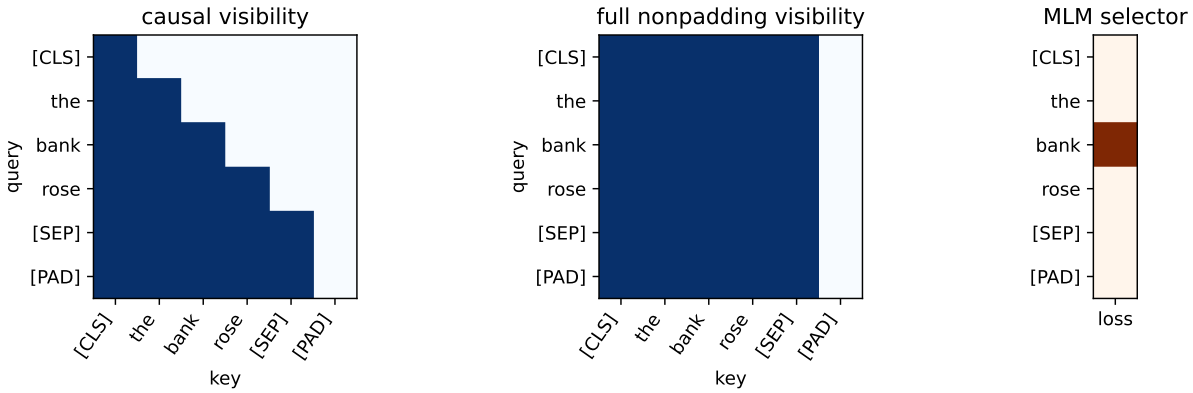

Figure 15.1.: Visibility and supervision are separate controls. A causal decoder sees a lower triangle (left); BERT-style attention sees every nonpadding key (center). The MLM selector marks one selected target position (right).

The center matrix contains no future-blocking triangle. Every nonpadding query can route from every nonpadding key—including itself. That is what *bidirectional* means here: a token's representation can jointly use left and right context in every layer. It does not mean running two separate recurrent networks, and it does not define a left-to-right probability for the whole sentence.

Only padding *keys* are blocked in the drawing. A `[PAD]` query row can still produce an output, but downstream computation ignores that row; the important guarantee is that real queries cannot retrieve padding as information.

Now suppose *bank* is selected and replaced:

[CLS] the [MASK] rose [SEP].

Every ordinary token may attend to `[MASK]`, because `[MASK]` is an input token—not an attention prohibition. Only the selector says that position 2 contributes MLM loss. The padding mask separately blocks the final slot as a key. Reusing *mask* for visibility, selection, corruption, and padding is convenient shorthand—and a reliable source of bugs.

The copy trap

If full attention received the original token at *every* selected position, the model could copy answers through the residual stream. BERT's policy corrupts 90% of selected sites, so copying cannot solve the whole task; the deliberate 10% unchanged branch has a separate purpose below. Conversely, replacing a token by `[MASK]` does not make it invisible—its neighbors still see the placeholder and one another. Always audit visibility, selection, corruption, and padding separately.

15.2. Text supplies supervision

In ordinary supervised learning, an external label y accompanies input x. Masked language modeling derives both from one unlabeled sequence: the corrupted sequence is the input, and the original selected tokens are the targets. This is *self-supervised learning*—the supervision is constructed from the data rather than supplied by a human annotator.

> **Self-supervised does not mean target-free**
>
> The method still has labels and a loss. What changes is their origin. The clean text supplies a target for every selected position, so a large corpus can create many training examples without a task-specific annotation campaign.

This idea was part of a broader 2018 convergence, not a solitary invention.

Work	Pretraining signal	What moved downstream
ELMo	Forward and backward LSTM language models	Fixed contextual features
ULMFiT	Causal language modeling	The entire adapted language model
GPT	Causal Transformer language modeling	A pretrained Transformer plus task head
BERT	Masked tokens in full context, plus NSP	A deeply bidirectional Transformer encoder plus task head

ELMo demonstrated that a word's useful vector should depend on its sentence. ULMFiT supplied a robust recipe for adapting a pretrained language model rather than freezing it. GPT paired Transformer pretraining with supervised fine-tuning. BERT's decisive change was to pretrain every encoder layer with joint left-and-right context. The workflow mattered as much as the architecture: learn broadly from raw text first, then spend scarce labels on adaptation.

15.3. The masked-language-model objective

Let us pin the idea to shapes before writing code. Let a tokenized sequence be

$$x = (x_1, \dots, x_T), \qquad x_t \in \{1, \dots, |V|\},$$

where T is sequence length and V is the vocabulary. Choose a random set M of eligible positions, then apply a corruption operator c to obtain $\tilde{x} = c(x, M)$. A bidirectional encoder with parameters θ produces

$$H = f_\theta(\tilde{x}) \in \mathbb{R}^{T \times d}.$$

An output head maps each selected hidden state to vocabulary logits. The MLM loss is

$$\mathcal{L}_{\text{MLM}}(\theta) = -\frac{1}{|M|} \sum_{i \in M} \log p_\theta(x_i \mid \tilde{x}) \,.$$

The condition is $\tilde{x}$, the one jointly corrupted sequence—not a different clean sequence with only x_i removed for every term. With batches, token IDs have shape (B, T), hidden states (B, T, d), and dense vocabulary logits $(B, T, |V|)$. Boolean selection gathers $|M|$ rows, so cross-entropy receives selected logits $(|M|, |V|)$ and targets $(|M|,)$. Unselected positions can affect the representations but contribute no direct MLM loss.

15.3.1. Fifteen percent, then 80/10/10

Original BERT selects 15% of eligible tokens. Conditional on selection:

- 80% are replaced by `[MASK]`;
- 10% are replaced by a random vocabulary token;
- 10% remain unchanged.

For 1,000 eligible positions, the expected flow is therefore

$$1000 \longrightarrow 150 \longrightarrow \begin{cases} 120 & \text{[MASK]}, \\ 15 & \text{random token}, \\ 15 & \text{unchanged}. \end{cases}$$

All 150 selected positions are scored against their original token. The 120 mask replacements are 12% of all eligible positions, not the complete supervision rate. Random ordinary-looking replacements force the encoder to cross-check a token against its context rather than trusting the visible identity. The unchanged branch prevents the opposite shortcut—the assumption that every selected ordinary-looking token must be wrong. It also reduces the mismatch between pretraining, where `[MASK]` exists, and downstream inputs, where it ordinarily does not. An unchanged selected token exposes its answer on purpose; only about 1.5% of all eligible positions follow that branch, and the encoder is not told which normal-looking tokens were selected.

```
eligible_count = 1_000
selected_count = round(0.15 * eligible_count)
branch_counts = np.array([120, 15, 15])
branch_labels = ["[MASK]", "random", "unchanged"]
branch_colors = ["#3566a8", "#d37b29", "#6b7280"]

fig, axes = plt.subplots(1, 2, figsize=(9, 3.2))
axes[0].bar(
    ["not selected", "selected"],
    [eligible_count - selected_count, selected_count],
    color=["#d9dde3", "#3566a8"],
```

```
)
axes[0].set_ylabel("eligible positions")
axes[0].set_title("first draw: 15%")
axes[0].bar_label(axes[0].containers[0])
bars = axes[1].bar(branch_labels, branch_counts, color=branch_colors)
axes[1].set_ylabel("selected positions")
axes[1].set_title("conditional branch: 80 / 10 / 10")
axes[1].bar_label(bars)
plt.tight_layout()
plt.show()

print(f"direct loss targets: {branch_counts.sum()}")
print(f"[MASK] share of all eligible positions: {branch_counts[0] / 1000:.1%}")
```

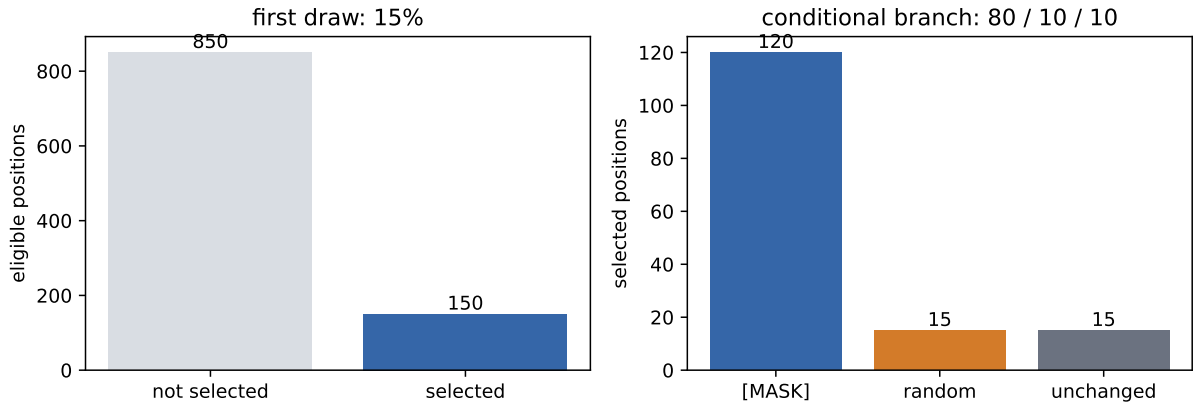

Figure 15.2.: BERT's 80/10/10 policy is conditional on the selected 15%. All three branches retain the original token as the prediction target.

```
direct loss targets: 150
[MASK] share of all eligible positions: 12.0%
```

Original BERT pre-generated several masked versions of its training examples; RoBERTa later redrew masks as examples were presented. Our controlled lab uses the latter *dynamic corruption* choice—the same sentence can ask different questions on later passes. Reproducibility therefore requires a separate random generator. Seeding model initialization while leaving the corruption stream implicit is not an exact experiment.

> **Audit the selector before the loss**
>
> Print the number of eligible and selected positions, verify that special and padding tokens are excluded, and inspect the realized 80/10/10 proportions over many batches. Then assert that the selected logit and target shapes agree. A loss can decrease while a selector silently trains on padding or on the wrong tokens.

15.4. From a Transformer encoder to BERT

The original BERT input at position i adds three learned vectors:

$$z_i = e_{\text{token}(i)} + e_{\text{position}(i)} + e_{\text{segment}(i)}.$$

The token comes from a **WordPiece** vocabulary: frequent character sequences are stored whole, while rarer words can be assembled from reusable subword pieces. Learned absolute position embeddings replace Chapter 14's fixed sinusoidal table. Segment A/B embeddings mark which of two packed text spans supplied a token; they do not block cross-segment attention. `[CLS]` begins the sequence, and `[SEP]` ends each span:

$$\text{[CLS]}\ A\ \text{[SEP]}\ B\ \text{[SEP]}.$$

The resulting sequence passes through a stack of full-context Transformer encoder blocks. Original BERT uses the post-LayerNorm order introduced as the historical Transformer in Chapter 14. Its two published sizes were:

Model	Blocks L	Width d	Heads	Parameters
BERT Base	12	768	12	about 110 million
BERT Large	24	1,024	16	about 340 million

Pretraining used BooksCorpus and English Wikipedia—about 3.3 billion words in the paper's accounting. The scale is historically important—but the reusable idea is the separation of stages:

$$\text{raw text} \xrightarrow{\text{self-supervised pretraining}} \text{general encoder} \xrightarrow{\text{labeled adaptation}} \text{task model}.$$

Original BERT combined MLM with **next sentence prediction** (NSP). Half the training pairs used the actual following segment and half used a random segment; the `[CLS]` state predicted which kind it saw—its direct pretraining assignment. MLM alone gives `[CLS]` no token-reconstruction target, although information can still route through it. Later recipes such as RoBERTa showed that strong BERT-style training did not require NSP in their settings. That is a recipe result, not proof that a sentence-relation objective can never help. This chapter studies the MLM half because it supplies the cleanest bridge from visibility to representation learning.

15.4.1. [CLS] is a learned meeting place

[CLS] is an ordinary learned token with an unusual assignment: place it first, allow it to attend through every layer, then give its final hidden state to a sequence-level head. For K classes,

$$\ell = W_{\text{cls}} h_{\text{CLS}} + b, \qquad W_{\text{cls}} \in \mathbb{R}^{K \times d}.$$

For token labeling, apply a head to every h_i. For extractive question answering, score candidate start and end positions. The encoder interface stays the same while the small task head changes.

[CLS] is not born with sentence meaning. Pretraining and downstream gradients teach it what information to gather. Chapter 16 will reuse this learned-summary pattern after replacing words by image patches.

15.5. Fine-tuning means the backbone moves

Let θ_0 denote pretrained encoder weights and ϕ a newly initialized task head. Standard BERT fine-tuning solves

$$(\theta^*, \phi^*) = \arg\min_{\theta,\phi} \sum_{(x,y)\in\mathcal{D}_{\text{task}}} \mathcal{L}_{\text{task}}(g_\phi(f_\theta(x)), y),$$

starting at $\theta = \theta_0$. Gradients update both θ and ϕ. Freezing θ_0 and training only ϕ is a useful *linear probe* or feature probe, but it is not the standard fine-tuning procedure. This distinction matters when the task needs the representation itself to move.

```
fig, axis = plt.subplots(figsize=(10, 3.8))
axis.set_xlim(0, 10)
axis.set_ylim(0, 6)
axis.axis("off")

def draw_box(
    x: float,
    y: float,
    width: float,
    height: float,
    text: str,
    color: str,
) -> None:
    rectangle = plt.Rectangle(
        (x, y), width, height,
        facecolor=color, edgecolor="#263238", linewidth=1.2,
    )
    axis.add_patch(rectangle)
    axis.text(
```

```
        x + width / 2, y + height / 2, text,
        ha="center", va="center", fontsize=10,
    )

draw_box(0.25, 2.2, 1.7, 1.3, "tokenized input\n[CLS] … [SEP]", "#eef2f7")
draw_box(2.8, 1.0, 2.5, 3.7, "pretrained BERT\nencoder\n\nall layers update", "#c9dcf5")
head_specs = [
    (4.45, "sequence head\n$h_{CLS} \\to$ class"),
    (2.55, "token head\n$h_1,\\ldots,h_T \\to$ tags"),
    (0.65, "span head\n$h_i \\to$ start / end"),
]
for y, label in head_specs:
    draw_box(6.4, y, 2.7, 1.05, label, "#f6dfc5")
    axis.annotate(
        "", xy=(6.35, y + 0.52), xytext=(5.35, 2.85),
        arrowprops={"arrowstyle": "->", "color": "#546e7a", "lw": 1.3},
    )
axis.annotate(
    "", xy=(2.75, 2.85), xytext=(2.0, 2.85),
    arrowprops={"arrowstyle": "->", "color": "#546e7a", "lw": 1.5},
)
axis.annotate(
    "task-loss gradients update the backbone",
    xy=(2.85, 0.55), xytext=(8.9, 0.15),
    ha="right", color="#a33a2b",
    arrowprops={"arrowstyle": "->", "color": "#a33a2b", "lw": 2},
)
plt.tight_layout()
plt.show()
```

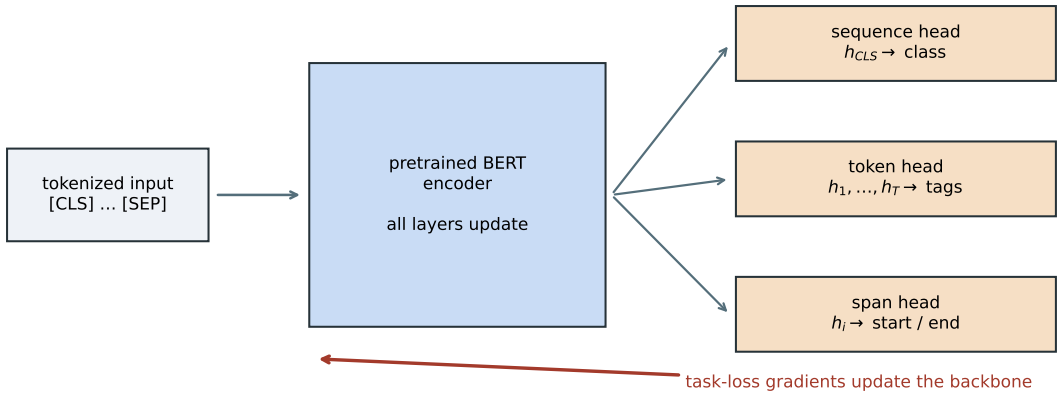

Figure 15.3.: One pretrained encoder supports several task interfaces. In standard fine-tuning, task-loss gradients pass through the new head and continue through the encoder; stopping them at the head would make the run a frozen probe.

Fine-tuning also creates experimental obligations. The scratch and pretrained arms must share labeled examples, head initialization, minibatch order, update count, evaluation, and downstream hyperparameters. The pretrained arm has already received extra unlabeled-data compute, so the comparison estimates the value of pretraining—not a total-compute advantage.

15.6. A controlled transfer lab

Chapter 9's three gates are easy to repeat and surprisingly hard to test. A first pre-test asked a tiny encoder to distinguish true continuations in this book from distant splices. The five-seed MLM gain was small, faded as labels increased, and lost decisively to a weighted lexical-overlap baseline. We rejected that task as the chapter's affirmative experiment—the shallow baseline solved more of the problem than the representation story did.

The replacement lab is synthetic by design. It removes spelling, frequency, and downstream-context shortcuts so that source coverage is a controllable variable.

15.6.1. Build a latent regularity

Eighty randomly generated four-letter token strings are divided into two latent families. Family assignment is independent of spelling. Forty *covered* strings—20 per family—each appear eight times in an unlabeled MLM corpus. Forty more—20 per family—live in the input vocabulary but are withheld from MLM input sequences, random replacements, and prediction targets.

Each covered word receives eight source sentences. Six contain cues from its own family; two deliberately contain cues from the other family. One family appears near words such as *rain*, *soil*, *roots*, and *garden*; the other appears near *power*, *metal*, *gears*, and *workshop*. The 25% cue noise gives every covered string a mixture of supporting and opposing contexts—the aggregate family signal is statistical rather than pure.

The downstream input is always the neutral, previously unseen template

"we found the PSEUDOWORD today."

No family cue remains. A classifier reads the generated token's hidden state and receives only 1, 2, or 4 labeled word types from each family. Evaluation uses the remaining covered types, never the labeled types. The 40 uncovered controls use the same neutral sentence. Source and target use the same atomic token granularity and similarly short sequences, making "matched scale" explicit rather than assuming it from a shared domain.

```
SEED = 6050
WIDTH = 48
HEADS = 4
FF_WIDTH = 96
BLOCKS = 2
MAX_LEN = 10
PRETRAIN_STEPS = 600
BATCH_SIZE = 64
LABEL_COUNTS = (1, 2, 4)
REPLICATE_SEEDS = tuple(range(6050, 6055))

torch.set_num_threads(4)
torch.use_deterministic_algorithms(True)

forms = [
    "".join(letters)
    for letters in itertools.product("bglmprstvz", "aeiou", "dknrst", "aeiou")
]
form_order = torch.randperm(
    len(forms), generator=torch.Generator().manual_seed(SEED)
)[:80]
token_strings = [forms[index] for index in form_order]
family_words = [token_strings[:40], token_strings[40:]]
covered_words = [words[:20] for words in family_words]
uncovered_words = [words[20:40] for words in family_words]

context_pairs = [
    ("rain", "power"),
    ("soil", "metal"),
    ("roots", "gears"),
    ("water", "fuel"),
    ("garden", "workshop"),
    ("grows", "moves"),
    ("blooms", "turns"),
    ("sunlight", "current"),
]
```

```
def source_sentences(word: str, family: int) -> list[list[str]]:
    cue = [pair[family] for pair in context_pairs]
    other = [pair[1 - family] for pair in context_pairs]
    return [
        ["after", cue[0], "the", word, cue[5]],
        ["the", word, "needs", cue[3], "near", cue[1]],
        [cue[2], "support", "the", word],
        ["in", "the", cue[4], "the", word, cue[6]],
        ["we", "found", "the", word, "beside", cue[1]],
        [cue[0], "and", cue[7], "help", "the", word],
        ["the", word, "rests", "near", other[4]],
        ["today", "the", word, "follows", other[2]],
    ]

sentences: list[list[str]] = []
for family, words in enumerate(covered_words):
    for word in words:
        sentences.extend(source_sentences(word, family))

specials = ["[PAD]", "[UNK]", "[CLS]", "[SEP]", "[MASK]"]
ordinary = sorted(
    {token for sentence in sentences for token in sentence} | set(token_strings)
)
vocabulary = specials + [token for token in ordinary if token not in specials]
stoi = {token: index for index, token in enumerate(vocabulary)}
PAD, UNK, CLS, SEP, MASK = [stoi[token] for token in specials]

def encode(sentence: list[str]) -> torch.Tensor:
    ids = [CLS] + [stoi.get(token, UNK) for token in sentence] + [SEP]
    ids += [PAD] * (MAX_LEN - len(ids))
    return torch.tensor(ids[:MAX_LEN])

source_ids = torch.stack([encode(sentence) for sentence in sentences])
uncovered_ids = torch.tensor([
    stoi[word] for family in uncovered_words for word in family
])
source_token_ids = torch.unique(source_ids)
random_pool = source_token_ids[
    ~torch.isin(source_token_ids, torch.tensor([PAD, CLS, SEP]))
]

assert len(sentences) == 320
assert len(vocabulary) == 115
assert not torch.isin(source_ids, uncovered_ids).any()
print(f"source: {len(sentences)} sentences; vocabulary: {len(vocabulary)} tokens")
```

```
print(f"covered/uncovered token strings: 40 / {len(uncovered_ids)}")
```

```
source: 320 sentences; vocabulary: 115 tokens
covered/uncovered token strings: 40 / 40
```

This is an atomic word-level vocabulary, not WordPiece. That simplification is intentional—splitting the generated strings into subwords could let spelling leak family identity. The source generator—not natural language—defines the latent regularity, so the experiment can demonstrate a mechanism but cannot estimate real-world BERT performance.

15.6.2. A small BERT-style encoder

The encoder has two post-LayerNorm blocks, width 48, four heads, learned absolute positions, full nonpadding attention, and GELU feed-forward layers. Here $\mathrm{GELU}(x) = x\Phi(x)$, with Φ the standard-normal cumulative distribution—a smooth, input-dependent gate used by original BERT. The lab omits segment embeddings, NSP, and dropout. Its 43,920 encoder parameters make the study fast enough to repeat end to end, while preserving the relevant path from bidirectional attention to MLM to fine-tuning.

```
class FullAttention(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.qkv = nn.Linear(WIDTH, 3 * WIDTH)
        self.output = nn.Linear(WIDTH, WIDTH)

    def forward(
        self, x: torch.Tensor, visible: torch.Tensor
    ) -> torch.Tensor:
        batch, length, _ = x.shape
        head_width = WIDTH // HEADS
        qkv = self.qkv(x).view(
            batch, length, 3, HEADS, head_width
        )
        q, k, v = qkv.permute(2, 0, 3, 1, 4)
        scores = q @ k.transpose(-2, -1) / head_width**0.5
        scores = scores.masked_fill(
            ~visible[:, None, None, :], float("-inf")
        )
        weights = F.softmax(scores, dim=-1)
        mixed = (weights @ v).transpose(1, 2).reshape(
            batch, length, WIDTH
        )
        return self.output(mixed)
```

```
class EncoderBlock(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.attention = FullAttention()
        self.norm1 = nn.LayerNorm(WIDTH)
        self.ff = nn.Sequential(
            nn.Linear(WIDTH, FF_WIDTH),
            nn.GELU(),
            nn.Linear(FF_WIDTH, WIDTH),
        )
        self.norm2 = nn.LayerNorm(WIDTH)

    def forward(
        self, x: torch.Tensor, visible: torch.Tensor
    ) -> torch.Tensor:
        x = self.norm1(x + self.attention(x, visible))
        return self.norm2(x + self.ff(x))

class TinyBertEncoder(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.token_embedding = nn.Embedding(
            len(vocabulary), WIDTH, padding_idx=PAD
        )
        self.position_embedding = nn.Embedding(MAX_LEN, WIDTH)
        self.blocks = nn.ModuleList([
            EncoderBlock() for _ in range(BLOCKS)
        ])
        nn.init.normal_(self.token_embedding.weight, std=0.02)
        nn.init.normal_(self.position_embedding.weight, std=0.02)
        with torch.no_grad():
            self.token_embedding.weight[PAD].zero_()

    def forward(self, ids: torch.Tensor) -> torch.Tensor:
        positions = torch.arange(ids.shape[1], device=ids.device)
        hidden = (
            self.token_embedding(ids)
            + self.position_embedding(positions)
        )
        visible = ids != PAD
        for block in self.blocks:
            hidden = block(hidden, visible)
        return hidden

class MaskedLanguageModel(nn.Module):
    def __init__(self, encoder: TinyBertEncoder) -> None:
```

```
        super().__init__()
        self.encoder = encoder
        self.dense = nn.Linear(WIDTH, WIDTH)
        self.norm = nn.LayerNorm(WIDTH)
        self.decoder = nn.Linear(WIDTH, len(vocabulary))

    def forward(
        self, ids: torch.Tensor, selected: torch.Tensor
    ) -> torch.Tensor:
        hidden = self.encoder(ids)[selected]
        hidden = self.norm(F.gelu(self.dense(hidden)))
        return self.decoder(hidden)

def corrupt(
    ids: torch.Tensor,
    generator: torch.Generator,
) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
    eligible = (ids != PAD) & (ids != CLS) & (ids != SEP)
    selected = eligible & (
        torch.rand(ids.shape, generator=generator) < 0.15
    )
    if not selected.any():
        rows, columns = torch.where(eligible)
        choice = torch.randint(
            len(rows), (1,), generator=generator
        )
        selected[rows[choice], columns[choice]] = True

    draw = torch.rand(ids.shape, generator=generator)
    mask_sites = selected & (draw < 0.8)
    random_sites = selected & (draw >= 0.8) & (draw < 0.9)
    corrupted = ids.clone()
    corrupted[mask_sites] = MASK
    random_choices = torch.randint(
        len(random_pool), ids.shape, generator=generator
    )
    random_ids = random_pool[random_choices]
    corrupted[random_sites] = random_ids[random_sites]

    unchanged_sites = selected & ~mask_sites & ~random_sites
    counts = torch.tensor([
        mask_sites.sum().item(),
        random_sites.sum().item(),
        unchanged_sites.sum().item(),
    ])
    assert not (selected & ~eligible).any()
```

```
    assert not torch.isin(corrupted, uncovered_ids).any()
    return corrupted, selected, counts

encoder_parameters = sum(
    parameter.numel() for parameter in TinyBertEncoder().parameters()
)
print(f"encoder parameters: {encoder_parameters:,}")
```

```
encoder parameters: 43,920
```

The MLM decoder is deliberately *untied* from the input embedding table. The original released BERT code explicitly tied those weights, but tying would update an uncovered input row through the vocabulary softmax even when that token never appeared. The custom control is stricter—random replacements are drawn only from source tokens, and the code later asserts that every uncovered input embedding remains bitwise unchanged through pretraining. Untied decoder rows for the controls still participate as negative softmax classes, but they are discarded downstream and never expose a control token's context or identity to the encoder input.

15.6.3. Pretrain, then adapt

For each seed from 6050 through 6054, one encoder is saved before training and the same encoder receives 600 MLM updates. The two downstream arms then receive the same labeled word types, identical classifier-head tensors, 160 full-batch updates, and the same learning rates. All encoder weights move in both arms. Each seed therefore repeats initialization, corruption, MLM, label selection, and fine-tuning—not merely the last classifier fit.

```
TensorState = dict[str, torch.Tensor]

def pretrain(
    seed: int,
) -> tuple[TensorState, TensorState, torch.Tensor, int]:
    torch.manual_seed(seed)
    encoder = TinyBertEncoder()
    initial = copy.deepcopy(encoder.state_dict())
    mlm = MaskedLanguageModel(encoder)
    optimizer = torch.optim.Adam(mlm.parameters(), lr=1e-3)
    index_generator = torch.Generator().manual_seed(seed + 1)
    mask_generator = torch.Generator().manual_seed(seed + 2)
    schedule = torch.randint(
        len(source_ids),
        (PRETRAIN_STEPS, BATCH_SIZE),
        generator=index_generator,
    )
    branch_counts = torch.zeros(3, dtype=torch.long)
```

```
    eligible_count = 0

    for indices in schedule:
        original = source_ids[indices]
        eligible_count += (
            (original != PAD) & (original != CLS) & (original != SEP)
        ).sum().item()
        corrupted, selected, counts = corrupt(
            original, mask_generator
        )
        branch_counts += counts
        logits = mlm(corrupted, selected)
        targets = original[selected]
        assert logits.shape == (targets.numel(), len(vocabulary))
        loss = F.cross_entropy(logits, targets)
        optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(mlm.parameters(), 1.0)
        optimizer.step()

    pretrained = copy.deepcopy(mlm.encoder.state_dict())
    initial_control = initial["token_embedding.weight"][uncovered_ids]
    final_control = pretrained["token_embedding.weight"][uncovered_ids]
    assert torch.equal(initial_control, final_control)
    return initial, pretrained, branch_counts, eligible_count

def neutral_sentence(word: str) -> torch.Tensor:
    return encode(["we", "found", "the", word, "today"])

class WordClassifier(nn.Module):
    def __init__(
        self, encoder_state: TensorState, head_state: TensorState
    ) -> None:
        super().__init__()
        self.encoder = TinyBertEncoder()
        self.encoder.load_state_dict(encoder_state)
        self.head = nn.Linear(WIDTH, 2)
        self.head.load_state_dict(head_state)

    def forward(self, ids: torch.Tensor) -> torch.Tensor:
        # [CLS] we found the WORD today [SEP] -> WORD is position 4.
        return self.head(self.encoder(ids)[:, 4])

def fit_classifier(
    encoder_state: TensorState,
    head_state: TensorState,
```

```
    family_zero: list[str],
    family_one: list[str],
) -> tuple[WordClassifier, float]:
    inputs = torch.stack([
        neutral_sentence(word)
        for word in family_zero + family_one
    ])
    targets = torch.tensor(
        [0] * len(family_zero) + [1] * len(family_one)
    )
    model = WordClassifier(encoder_state, head_state)
    optimizer = torch.optim.Adam([
        {"params": model.encoder.parameters(), "lr": 2e-4},
        {"params": model.head.parameters(), "lr": 2e-3},
    ])
    for _ in range(160):
        loss = F.cross_entropy(model(inputs), targets)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    train_accuracy = (
        (model(inputs).argmax(1) == targets).float().mean().item()
    )
    return model, train_accuracy

@torch.no_grad()
def classifier_accuracy(
    model: WordClassifier,
    family_zero: list[str],
    family_one: list[str],
) -> float:
    inputs = torch.stack([
        neutral_sentence(word)
        for word in family_zero + family_one
    ])
    targets = torch.tensor(
        [0] * len(family_zero) + [1] * len(family_one)
    )
    return (
        (model(inputs).argmax(1) == targets).float().mean().item()
    )

paired_results: dict[int, list[tuple[float, float]]] = {
    count: [] for count in LABEL_COUNTS
}
uncovered_results: dict[int, list[tuple[float, float]]] = {
```

```
    count: [] for count in LABEL_COUNTS
}
training_results: dict[int, list[tuple[float, float]]] = {
    count: [] for count in LABEL_COUNTS
}
all_branches = torch.zeros(3, dtype=torch.long)
all_eligible = 0
first_states: tuple[TensorState, TensorState] | None = None

for replicate_seed in REPLICATE_SEEDS:
    initial_state, pretrained_state, counts, eligible_count = pretrain(
        replicate_seed
    )
    if first_states is None:
        first_states = (initial_state, pretrained_state)
    all_branches += counts
    all_eligible += eligible_count

    label_generator = torch.Generator().manual_seed(replicate_seed)
    order_zero = torch.randperm(
        len(covered_words[0]), generator=label_generator
    ).tolist()
    order_one = torch.randperm(
        len(covered_words[1]), generator=label_generator
    ).tolist()
    torch.manual_seed(replicate_seed + 10_000)
    head_state = copy.deepcopy(nn.Linear(WIDTH, 2).state_dict())

    for count in LABEL_COUNTS:
        label_zero = [covered_words[0][i] for i in order_zero[:count]]
        label_one = [covered_words[1][i] for i in order_one[:count]]
        test_zero = [covered_words[0][i] for i in order_zero[count:]]
        test_one = [covered_words[1][i] for i in order_one[count:]]

        scratch, scratch_train = fit_classifier(
            initial_state, head_state, label_zero, label_one
        )
        pretrained, pretrained_train = fit_classifier(
            pretrained_state, head_state, label_zero, label_one
        )
        scratch_score = classifier_accuracy(
            scratch, test_zero, test_one
        )
        pretrained_score = classifier_accuracy(
            pretrained, test_zero, test_one
        )
```

```
        scratch_uncovered = classifier_accuracy(
            scratch, uncovered_words[0], uncovered_words[1]
        )
        pretrained_uncovered = classifier_accuracy(
            pretrained, uncovered_words[0], uncovered_words[1]
        )
        paired_results[count].append(
            (scratch_score, pretrained_score)
        )
        uncovered_results[count].append(
            (scratch_uncovered, pretrained_uncovered)
        )
        training_results[count].append(
            (scratch_train, pretrained_train)
        )

branch_shares = all_branches / all_branches.sum()
print(
    "selected targets across five runs: "
    f"{all_branches.sum().item():,}"
)
print(f"eligible positions: {all_eligible:,}")
print(
    "realized selection rate: "
    f"{all_branches.sum().item() / all_eligible:.3%}"
)
print("realized mask/random/unchanged shares:", branch_shares.tolist())
for count in LABEL_COUNTS:
    pairs = np.array(paired_results[count])
    control_pairs = np.array(uncovered_results[count])
    train_pairs = np.array(training_results[count])
    deltas = pairs[:, 1] - pairs[:, 0]
    print(
        f"{count}/family: scratch={pairs[:, 0].mean():.3f}, "
        f"pretrained={pairs[:, 1].mean():.3f}, "
        f"delta={deltas.mean():+.3f}, wins={(deltas > 0).sum()}/5, "
        f"uncovered={control_pairs[:, 0].mean():.3f}/"
        f"{control_pairs[:, 1].mean():.3f}, "
        f"train={train_pairs[:, 0].mean():.3f}/"
        f"{train_pairs[:, 1].mean():.3f}"
    )
```

```
selected targets across five runs: 155,019
eligible positions: 1,032,096
realized selection rate: 15.020%
realized mask/random/unchanged shares: [0.8003599643707275, 0.09977486729621887, 0.0998651757
```

```
1/family: scratch=0.468, pretrained=0.995, delta=+0.526, wins=5/5, uncovered=0.475/0.425, tra
2/family: scratch=0.489, pretrained=1.000, delta=+0.511, wins=5/5, uncovered=0.500/0.440, tra
4/family: scratch=0.494, pretrained=1.000, delta=+0.506, wins=5/5, uncovered=0.550/0.430, tra
```

Across 1,032,096 eligible positions, the five runs select 155,019—15.020%. The realized branches are 80.04% mask, 9.98% random, and 9.99% unchanged. Both scratch and pretrained models reach 100% on their tiny labeled training sets in every run; failure to fit the labels therefore does not explain the scratch result.

15.6.4. What transferred?

Remember the question: did pretraining organize covered representations in a way that scarce labels could reuse—or did the classifier merely learn the tiny label set? The task, control, and geometry answer different parts.

```
@torch.no_grad()
def representation_geometry(
    encoder_state: TensorState,
) -> tuple[float, float]:
    model = TinyBertEncoder()
    model.load_state_dict(encoder_state)
    words = covered_words[0] + covered_words[1]
    vectors = model(torch.stack([
        neutral_sentence(word) for word in words
    ]))[:, 4]
    vectors = vectors - vectors.mean(dim=0, keepdim=True)
    vectors = F.normalize(vectors, dim=1)
    similarities = vectors @ vectors.T
    labels = torch.tensor(
        [0] * len(covered_words[0]) + [1] * len(covered_words[1])
    )
    off_diagonal = ~torch.eye(len(words), dtype=torch.bool)
    within = similarities[
        (labels[:, None] == labels[None, :]) & off_diagonal
    ]
    between = similarities[labels[:, None] != labels[None, :]]
    return within.mean().item(), between.mean().item()

assert first_states is not None
initial_geometry = representation_geometry(first_states[0])
pretrained_geometry = representation_geometry(first_states[1])

budgets = np.array(LABEL_COUNTS)
scratch_means = np.array([
    np.mean(paired_results[count], axis=0)[0]
```

```
    for count in LABEL_COUNTS
])
pretrained_means = np.array([
    np.mean(paired_results[count], axis=0)[1]
    for count in LABEL_COUNTS
])
scratch_uncovered_means = np.array([
    np.mean(uncovered_results[count], axis=0)[0]
    for count in LABEL_COUNTS
])
pretrained_uncovered_means = np.array([
    np.mean(uncovered_results[count], axis=0)[1]
    for count in LABEL_COUNTS
])

fig, axes = plt.subplots(1, 3, figsize=(11, 3.7))
for values, label, color, marker in [
    (scratch_means, "scratch, covered", "#6b7280", "o"),
    (pretrained_means, "MLM, covered", "#3566a8", "o"),
    (
        scratch_uncovered_means,
        "scratch, unexposed",
        "#9ca3af",
        "s",
    ),
    (
        pretrained_uncovered_means,
        "MLM, unexposed",
        "#d37b29",
        "s",
    ),
]:
    axes[0].plot(budgets, values, label=label, color=color, marker=marker)
axes[0].axhline(0.5, color="black", lw=1, ls="--", alpha=0.5)
axes[0].set_xticks(budgets)
axes[0].set_ylim(0.25, 1.05)
axes[0].set_xlabel("labeled words per family")
axes[0].set_ylabel("accuracy")
axes[0].set_title("only covered strings benefit")
axes[0].legend(frameon=False, fontsize=8)

for column, count in enumerate(LABEL_COUNTS):
    pairs = np.array(paired_results[count])
    deltas = pairs[:, 1] - pairs[:, 0]
    jitter = np.linspace(-0.08, 0.08, len(deltas))
    axes[1].scatter(
```

```
        np.full(len(deltas), column) + jitter,
        deltas,
        color="#3566a8",
    )
axes[1].axhline(0, color="black", lw=1)
axes[1].set_xticks(range(3), ["1", "2", "4"])
axes[1].set_xlabel("labeled words per family")
axes[1].set_ylabel("pretrained - scratch")
axes[1].set_title("all 15 paired gains")

geometry = np.array([initial_geometry, pretrained_geometry])
x = np.arange(2)
axes[2].bar(
    x - 0.18, geometry[:, 0], width=0.36,
    label="within family", color="#3566a8",
)
axes[2].bar(
    x + 0.18, geometry[:, 1], width=0.36,
    label="between family", color="#d37b29",
)
axes[2].axhline(0, color="black", lw=1)
axes[2].set_xticks(x, ["initial", "after MLM"])
axes[2].set_ylabel("centered cosine similarity")
axes[2].set_title("seed 6050 geometry")
axes[2].legend(frameon=False, fontsize=8)
plt.tight_layout()
plt.show()

print(f"initial within/between: {initial_geometry}")
print(f"after MLM within/between: {pretrained_geometry}")
```

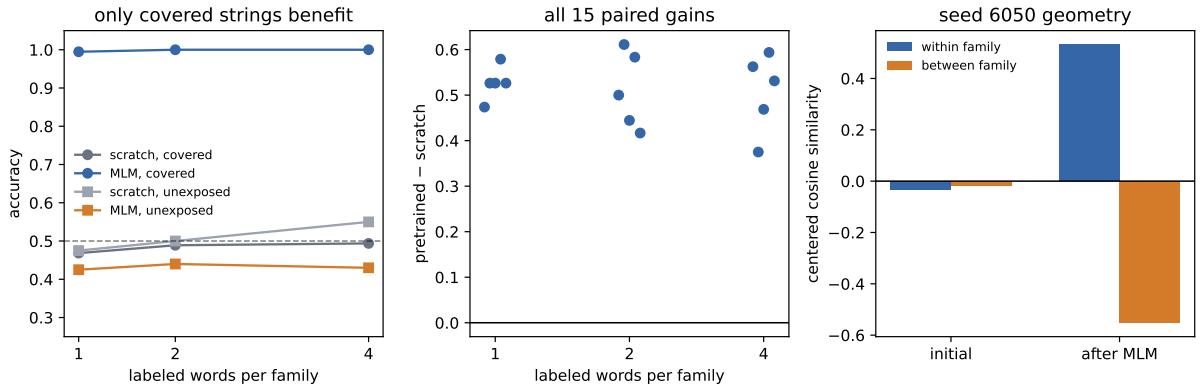

Figure 15.4.: Five paired end-to-end seeds in the synthetic transfer lab. MLM initialization separates covered latent families and improves every covered-type comparison (left and center); both arms stay near chance on input-vocabulary controls withheld from MLM inputs, replacements, and targets (left). Centered cosine similarity is a representation diagnostic, not a task metric (right).

```
initial within/between: (-0.03466804325580597, -0.0160280279815197)
after MLM within/between: (0.5322557687759399, -0.5517237186431885)
```

Labels / family	Scratch, covered	MLM, covered	Gain	Scratch, unexposed	MLM, unexposed
1	0.468	0.995	+0.526	0.475	0.425
2	0.489	1.000	+0.511	0.500	0.440
4	0.494	1.000	+0.506	0.550	0.430

MLM initialization wins all 15 paired covered-type comparisons. At two labels per family, for example, mean covered accuracy moves from 0.489 to 1.000. On the 40 unexposed controls, scratch and MLM-initialized means stay around chance at every budget. MLM therefore helps types whose source contexts contain the latent regularity—not arbitrary rows that merely exist in the input vocabulary.

The labeled sets are nested within a seed, but increasing the budget removes those additional types from that budget's test pool. The within-budget scratch-versus-MLM comparison is exact; movement across budgets is not a controlled learning curve.

The representation diagnostic agrees with the task result. In seed 6050, mean centered cosine similarity starts at −0.0347 within a family and −0.0160 between families—no useful separation. After MLM, it is 0.5323 within and −0.5517 between. *Centered cosine* first subtracts the mean representation, then unit-normalizes each vector and takes a dot product: positive values point in similar residual directions, negative values in opposing directions. Pretraining has organized neutral-context token states around the latent source regularity, and a few labels recover that organization for unseen covered types.

This closes Chapter 9's contract at toy scale:

- **Labels are scarce:** the task supplies only 2, 4, or 8 labeled words total.
- **The target is representation-hungry:** spelling, frequency, and task-context cues are neutralized; the label must transfer through the token representation.
- **Source coverage is useful at a matched scale:** source and target use atomic tokens and similarly short sequences; covered strings transfer, while balanced vocabulary residents withheld from MLM inputs, replacements, and targets remain near chance in both arms.

The result is an existence proof for a mechanism. A co-occurrence method could also recover the synthetic families; the experiment does not establish Transformer superiority. The limits are concrete—the data generator builds the regularity, the pretrained arm receives extra compute, and five toy seeds do not estimate natural-language performance. The justified conclusion is narrower: **in this controlled construction, MLM organizes covered token representations around a latent regularity, and a few labels recover it for unseen covered types.**

15.7. Three pretraining families

Pretraining is a regime, not an architecture. The visibility rule and pretraining objective determine what a model can learn—and how it will later be used.

Family	Visibility during pretraining	Objective	Characteristic use
BERT-style encoder	Full nonpadding context	Predict selected original tokens after 80/10/10 treatment	Contextual representations and task heads
GPT-style decoder	Causal prefix	Predict the next token	Autoregressive generation and adaptation
T5-style encoder–decoder	Full encoder; causal decoder with cross-attention	Generate removed spans separated by sentinel tokens	Text-to-text conditional generation

A GPT-style decoder can train all next-token positions in parallel under teacher forcing because the causal mask exposes only each prefix. Generation is still sequential: the model must emit a token before that token can join the next prefix. Training and inference share a factorization and visibility rule, not an execution schedule.

Once those next-token logits exist, Chapter 11's greedy and beam-search machinery can rank continuations—the architecture changes, but decoding remains a separate decision layer. Chapter 17 will return to how prompts alter the context supplied to that decoder.

T5 turns tasks into text-to-text problems and marks removed spans with learned sentinel tokens. Its encoder produces a *sequence* of memory states under full visibility; its causal decoder cross-attends to that sequence while generating the target. It does not squeeze the input into one fixed bottleneck vector. These families trade representation, generation, and input–output structure differently—they are not a ranking from old to new.

15.8. What changed after 2018

Before the pretrained era, a new labeled dataset often implied a new model trained from a random initialization. After BERT and its contemporaries, the default question changed: which pretrained model, which adaptation, and which data interface fit the task?

Three consequences follow.

Optimization starts elsewhere. The downstream run begins in a parameter region already shaped by a broad source objective. The task is not asking a few labels to invent syntax and semantics from scratch.

Data work moves upstream. Corpus choice, tokenization, corruption, and source coverage become part of model design. Pretraining is curriculum at dataset scale.

One backbone serves many interfaces. A sequence head, token head, or span head can reuse the same encoder. That modularity makes comparison easier—but also makes pretraining assumptions easy to inherit without inspecting them.

The regime introduces new failure modes. Source bias can transfer with useful features. A vocabulary can represent a token without supplying useful exposure. Full fine-tuning can forget source behavior or become too expensive as the backbone grows. Those are not footnotes to transfer; they are the agenda of the pretrained era.

The chapter closes with three forward seeds. A learned summary token can gather a sequence for a downstream head; Chapter 16 will give that same pattern a sequence of image patches. Pretraining is a regime, not an architecture; Chapter 16 will ask what changes when an encoder leaves text for images and scaling takes center stage. Fine-tuning changed every encoder weight here; Chapter 17 will ask what adaptation remains when most, or all, of those weights must stay fixed.

15.9. Okay, so—

1. **Visibility, selection, corruption, and padding are separate.** BERT removes the causal triangle, selects loss targets, then applies the 80/10/10 treatment so copying cannot solve most of them. `[MASK]` is a token—not an attention ban.
2. **MLM turns raw text into supervision.** Select 15% of eligible positions; conditional on selection, use 80% mask, 10% random, and 10% unchanged. All selected positions predict their original tokens.
3. **BERT is a full-context Transformer encoder.** WordPiece tokens, learned token/position/segment embeddings, post-LayerNorm blocks, MLM, and historical NSP define the original recipe.
4. **Fine-tuning updates the encoder.** Training only a new head over frozen features is a probe. Standard BERT adaptation moves the pretrained backbone and task head together.
5. **The controlled lab closes the transfer loop at toy scale.** Across five full seeds, MLM initialization wins every covered-type comparison under 2–8 total labels; input-vocabulary controls withheld from MLM inputs, replacements, and targets remain near chance in both arms.

6. **Pretraining changed the unit of work.** The field moved from one random model per task toward reusable backbones and specialized adaptation. Corpus, objective, visibility, and adaptation now belong to one design decision.

Sources and further reading

- Peters et al., *Deep Contextualized Word Representations*: ELMo and fixed contextual features from forward and backward language models.
- Howard and Ruder, *Universal Language Model Fine-tuning for Text Classification*: ULMFiT's full-model adaptation recipe.
- Radford et al., *Improving Language Understanding by Generative Pre-Training*: causal Transformer pretraining followed by supervised adaptation.
- Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*: the original BERT architecture, MLM/NSP objectives, and fine-tuning results.
- Google Research, *BERT pretraining implementation*: the released implementation documents the tied input/output embedding weights.
- Liu et al., *RoBERTa: A Robustly Optimized BERT Pretraining Approach*: a later study of BERT training choices, including removal of NSP.
- Raffel et al., *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*: the T5 text-to-text framework and span-corruption recipe.

Exercises

1. **(Pencil.)** For `[CLS] the wug is calm [SEP] [PAD] [PAD]`, suppose *wug* is selected and replaced by `[MASK]`. Draw Chapter 14's causal visibility matrix and BERT's full nonpadding visibility matrix, using queries as rows and keys as columns. Separately mark the one position that contributes MLM loss. Why may neighboring tokens attend to `[MASK]`, and why would full visibility leak information in a next-token generator?
2. **(Pencil.)** A batch contains 2,000 eligible ordinary tokens. Under BERT's policy, how many positions are expected to be selected, replaced by `[MASK]`, replaced by a random token, and left unchanged? How many contribute direct MLM loss? Explain why 12% is not the supervision rate.
3. **(Code.)** For token IDs (B, T), width d, and vocabulary size $|V|$, add assertions for hidden states (B, T, d), dense logits $(B, T, |V|)$, selected logits $(|M|, |V|)$, and selected targets $(|M|,)$. Assert that `[PAD]`, `[CLS]`, and `[SEP]` are never selected. Show numerically that selected-only cross-entropy matches dense cross-entropy when unselected targets use `ignore_index`.
4. **(Code.)** Starting from the same pretrained checkpoint and split, compare a frozen encoder probe with full fine-tuning in the generated-token lab. Match head tensors, labeled types, updates, and seeds. Plot all five paired results rather than only their means. Which regime wins here, and what broader claim would outrun the experiment?
5. **(Code and interpretation.)** Reproduce paired gains for covered and control strings at all three budgets. Map the evidence to Chapter 9's gates. Explain the unchanged-row assertion, name two unsupported conclusions, and propose one predeclared coverage test.

16. Vision Transformers and Scaling Laws

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Patches as tokens: the transformer eats images. Harvest Chapter 14's handoff by name—global routing trades away locality bias—then rematch the convolutional toolkit on Fashion images without pretending a small-data result settles the architecture. Harvest Chapter 15's learned summary-token pattern and its claim that pretraining is a regime, not an architecture. From there, scaling laws and Chinchilla's tokens-per-parameter arithmetic make the data side of the trade explicit.

Part V.

V · The Pretrained Era

17. Adapting Pretrained Models: Prompting, PEFT, Quantization

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

When the model is too big to fine-tune, get surgical: prompting and in-context learning, LoRA's low-rank updates, quantization and QLoRA.

18. Alignment and RL Fine-Tuning

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

From next-token prediction to helpfulness: instruction tuning, preference learning, and RLHF in outline.

19. Generative Models: From PCA to Diffusion

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

One more make-it-learnable arc: PCA → autoencoder → VAE → diffusion (and GANs as the adversarial detour). Latents, ELBO, noise schedules.

A. Linear Algebra and the SVD

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

The geometry the book leans on: matrices as transformations, eigenvectors, and the SVD as the master decomposition.

B. Tensors in Practice

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Shapes, broadcasting, indexing, and the layer-algebra habits that prevent 90% of PyTorch bugs.

C. Notation

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Symbols used throughout the book.

D. Floating Point and Machine Precision

> ⚠ Warning
>
> **Appendix stub.** Planned (author request, July 2026). See `docs/backlog.md` §1.

Every number in this book is a lie of finite precision. What a float actually is (sign, exponent, mantissa), machine epsilon, why $0.1 + 0.2 \neq 0.3$*, catastrophic cancellation, and the patterns deep learning uses to survive it: log-sum-exp (the trick inside every softmax), solving instead of inverting, and mixed-precision training (fp16/bf16) — the bridge to quantization in Chapter 17.*